MESHCOM WATCH

Technisches Handbuch

Warum diese Uhr so gebaut ist, wie sie gebaut ist

Gerät	LilyGo T-Watch-S3-Plus (ESP32-S3)
Firmware	1.0.5.48
Basis	SMashCom42 42.0.3.1
Stand	02.11.2026
Verfasser	Christian Raith, OE3LCR
Grundlage	Wolfgang Zelinka, OE3WAS

Dieses Buch richtet sich an Techniker, die den Quelltext übernehmen oder erweitern. Es beschreibt nicht, *was* der Quelltext tut — das steht dort selbst —, sondern *warum* er es so tut, und welche naheliegenden Wege bereits begangen und verworfen wurden.

Inhalt

1 Gegenstand, Rechtsrahmen und Lizenz

- 1.1 Was dieses Gerät ist
- 1.2 ⚠ Rechtsrahmen: Der Betrieb setzt eine Amateurfunkgenehmigung voraus
- 1.3 Lizenz
- 1.4 Zielgruppe und Aufbau dieses Buches

2 Zwei Autoren, zwei Präfixe

- 2.1 Warum das Präfix über die Zukunft entscheidet
- 2.2 Wenn es sich doch nicht vermeiden lässt
- 2.3 ⚠ Die Stelle, die zweimal verlorenging
- 2.4 Die Regel für neue Module

2b Übersetzen und Flashen

- 2b.1 Werkzeugkette
- 2b.2 Die drei Übersetzungsumgebungen
- 2b.3 Die USB-Schnittstelle
- 2b.4 Zwei Abbilder, zwei Verwendungen
- 2b.5 Herstellerunterlagen

3 Das Sechs-Ebenen-Modell

- 3.1 Die sechs Ebenen
- 3.2 Am Beispiel GPS durchgespielt
- 3.3 Warum keine Ebene entbehrlich ist
- 3.4 ⚠ Was das Modell *nicht* leistet
- 3.5 Die gemessene Ebene `hw_GPS` — ein Abbild für zwei Uhren
- 3.6 Die Regel für neue Module

4 Zwei Versionsnummern statt einer

- 4.1 Der Fehler
- 4.2 Die Trennung
- 4.3 ⚠ Das Format ist streng
- 4.4 ⚠ Die Zählregel — und warum sie nicht rückgängig zu machen ist
- 4.5 Der verbindliche Ablauf nach jedem Flashen
- 4.6 Was daraus allgemein folgt

5 Die Stromschienen des AXP2101

- 5.1 Die Belegung
- 5.2 ⚠ Die Schiene muss eingeschaltet werden — ALDO4
- 5.3 Die Reihenfolge ist nicht beliebig

6 ⚠ Die Schienen-Falle: ein Modul „aus” ist nicht stromlos

- 6.1 Der Fehler
- 6.2 Warum das so lange unentdeckt blieb
- 6.3 Wie es jetzt aussieht
- 6.4 Wiedereinschalten ist nicht das Gegenteil von Ausschalten
- 6.5 Die Regel für alles Weitere

7 SPI: Display und Funkmodul dürfen nicht auf derselben Peripherie liegen

- 7.1 Der Ausgangszustand
- 7.2 Warum es trotzdem lief
- 7.3 Wie er aufbrach
- 7.4 Die Behebung
- 7.5 ⚠ Was daraus für alles Weitere folgt

8 Der Touchcontroller FT6336U

- 8.1 Zwei Gesten-Mechanismen — und der naheliegende ist tot
- 8.2 ⚠ Der Hauptbefund: dem Chip fehlt die Panelgröße
- 8.3 Die Reihenfolge ist zwingend — der Chip überlebt den Reset
- 8.4 Die Engine wird vom Host getaktet
- 8.5 ⚠ Drei Eingriffe, die den Chip beschädigen
- 8.6 Fehlschläge sind nicht tödlich

9 Das Display-Panel: BGR und INVON
9.1 Das Fehlerbild
9.2 Der 5-Farben-Boot-Test
9.3 Befund 1: Die Inversion war aktiv abgeschaltet worden
9.4 Befund 2: `TFT_RGB_ORDER=1` bedeutet RGB
9.5 ⚠️ Der Altzustand war keine gültige Referenz
9.6 Der Stand
10 Die Pins, die etwas anderes sind als sie scheinen
10.1 Der Fall Pin 44: GPS empfing Rauschen
10.2 Nachtrag: die feste Baudrate
10.3 Namen aus anderen Boards, die im Header stehengeblieben sind
10.4 Pins, die es nicht gibt
10.5 Die Prüfliste für einen neuen Pin
11 Zwei Kerne, zwei Aufgaben
11.1 Die Aufteilung
11.2 ⚠️ Der Kernpunkt: die Trennung hängt an **zwei** Stellen
11.3 Warum der Fehler so lange unbemerkt blieb
11.4 Die Regel für Nachfolger
11.5 Was aus der Trennung folgt
12 Warum LVGL einen eigenen Task bekam
12.1 Der Task
12.2 ⚠️ Kern 1 ist nur die halbe Miete
12.3 Der Preis: jeder Zugriff braucht den Mutex
12.4 Wo bewusst *nicht* gesperrt wird
12.5 ⚠️ Was im Main-Loop nichts zu suchen hat
12.6 Der Watchdog
12b Die Bedienoberfläche entsteht im Quelltext
12b.1 Der gewählte Weg
12b.2 Der Bildschirmverbund: `CR_ScreenMgr`
12b.3 `CR_EEZShim` : die Anbindung an die gemeinsame Grundlage
12b.4 Aufbau eines Bildschirms
12b.4b Die vier Kachelfarben — eine Regel, zwei Aussagen
12b.5 ⚠️ Die Trennung der Aufgabenbereiche
12b.6 Was beim Hinzufügen eines Bildschirms zu tun ist
12b.7 Drei Zifferblätter, eine Screen-ID: die Weiche `CR_FaceSel`
12b.8 Warum die Scheibenanimation aus `_Loop()` läuft, nicht aus einem `lv_timer`
12b.8b Das Zählerrad: zwei Ziffern je Rolle, und warum ein halber Pixel ruckelt
12b.9 Die 3-Sekunden-Druckfläche: warum LVGLs eigener Langdruck nicht reicht
12b.10 Die Auswahl übernimmt Wisch-Navigation und Sprung-Wünsche
12b.11 Die Funkraumuhr: ein Bild, drei Polygone
12b.12 Doppeltipp für alle Zifferblätter — und die Klick-nach-Wisch-Falle
12b.13 Der Schreibschirm `CR_Compose` : T9 auf 240 Pixeln
12b.14 Ein Ziel für „Senden“: Ebenenliste, Kopfzeile, Schleife (seit 1.0.5.26)
12b.15 Wecken zeigt die Meldung (seit 1.0.5.27)
12b.16 Der Wecker: `CR_Alarm` und `CR_AlarmScreen` (seit 1.0.5.28)
12c Die Schriften
12c.1 Der Überblick
12c.2 ⚠️ Warum es eigene Schriften braucht: die eingebauten können kein „ä”
12c.3 Wie eine eigene Schrift erzeugt wird
12c.4 Die Ziffernblatt-Schriften
12c.5 Emojis sind keine Schrift, sondern Bilder
12c.6 ⚠️ Ein Sparhebel, der bisher nicht gezogen wurde
12c.7 Wo man nachsieht
13 Was im Hauptablauf verboten ist
13.1 Kein `lv_*` außerhalb des Zeichenablaufs
13.2 Kein `lv_lock()` im Hauptablauf

13.3 Kein `delay()`, kein Blockieren
13.4 Warum der Takt trotzdem zählt
13.5 Die Prüfliste
14 Der Fall REND-03: drei Fehler, von denen der erste die anderen verdeckte
14.1 Das Symptom
14.2 Die Erklärung, die alle glaubten
14.3 Was tatsächlich los war
14.4 Der Nebenbefund, den erst die Gleichzeitigkeit hervorbrachte
14.5 Ergebnis
14.6 Was daraus zu lernen ist
15 Warum ein zu *schneller* Loop ebenfalls schadet
15.1 Der Zustand nach der Entfesselung
15.2 Der gemessene Schaden: der andere Kern leidet mit
15.3 Die Bremse
15.4 ⚠️ Die Ausnahme — und der Fehler darin
15.5 Was daraus folgt
16 ⚠️ Die Bedienauswertung hängt an der Abtastrate
16.1 Der Mechanismus
16.2 Wischgesten: `CR_Gesture`
16.3 Schieberegler: nur der Knopf nimmt den Finger an
16.4 Kacheln: Tippen gegen Wischen gegen Langdruck
16.5 Regeln für Eingriffe in diesen Bereich
17 Die Schirm-Topologie
17.1 Jeder Schirm trägt seine Nachbarn selbst ein
17.2 Zwei Ringe, ein Kreuzungspunkt
17.3 Sackgassen sind erlaubt — aber nur nach unten
17.4 Der Ringschluss beim Blättern
17.5 Eine Namensfalle, die ins Handbuch durchschlug
18 Der MeshCom-Paketaufbau
18.1 Der Aufbau im Überblick
18.2 Ein echtes Paket, vollständig zerlegt
18.3 Byte 0 — die vier Typen
18.4 Bytes 1–4 — die Nachrichten-ID
18.5 Byte 5 — die Flags
18.6 Der ASCII-Teil
18.7 Der Positionsteil
18.8 Die Quittung — ein Paket, das die Regeln bricht
18.9 Die Prüfsumme
18.10 Mindestlängen und Puffergrößen
19 Senden: die Fallstricke
19.1 Der systematische Denkfehler
19.2 `fw_version = 0` — der Fehler, der alles erklärte
19.3 Das fehlende Pflicht-Bit im `last_hw`
19.4 Die Unterversion an der falschen Stelle
19.5 Ein Bit, das eine Unwahrheit behauptete
19.6 Die Prüfsumme aus einem fremden Paket
19.7 Grün beweist weniger, als es aussieht
19.8 Position: schweigen ist besser als raten
19.9 Die Sperren beim Senden
19.10 Wo gesendet werden darf
19.11 Was NICHT das Problem war
19b Kollisionserkennung: CSMA/CA mit Hardware-CAD
19b.1 Warum eine Kollision doppelt weh tut
19b.2 Woher das Verfahren kommt
19b.3 CAD — die Hardware horcht selbst
19b.4 Wenn der Kanal belegt ist: gestaffelt warten

19b.5 Ruhe nach einem fremden Paket
19b.6 Bei uns: Zustandsautomat, keine Warteschleife
19b.7 Zwei Dinge, die der Einbau bei uns erzwungen hat
19b.8 Warum eine eingereihte Aussendung nie storniert wird
19b.9 Was noch offen ist
19c Wartelisten statt Sendering — und die Seite /lora
Der Node hat einen Ring, die Uhr hat Wartelisten
Warum der Weiterleitungs-Ring nur 3 Plätze hat
Sendedruck: RUHIG, DICHT, VOLL
Das 5-Minuten-Fenster (`CR_Funklog`)
Der Paketstrom
Schnittstellen
20 Empfang und Zerlegung
20.1 Die Prüfkette
20.2 Wo der Text aufhört
20.3 Entdopplung — die wichtigste Funktion überhaupt
20.4 Die eigene Aussendung, zurückgehört
20.5 Zeittelegramme
20.6 Positionen
20.7 Der Weg vom Interrupt bis auf den Schirm
20.8 Was im Protokoll steht
21 Funkparameter: Sync-Wort und Präambel
21.1 Die Parameter
21.2 Das Sync-Wort ist ein Türsteher
21.3 Die Präambel
21.4 Was SF11 für alles andere bedeutet
21.5 Ein eigener SPI-Bus
21.6 Die Stromschiene
21.7 Prüfreihenfolge, wenn nichts empfangen wird
21b Bluetooth: die zweite Funkschnittstelle
21b.1 Warum in zwei Stufen
21b.2 Stufe A: der Speicher
21b.3 ✅ Der Befund, der Stufe B klein machte
21b.4 Der Dienstaufbau
21b.5 Der Empfangsweg und die Task-Grenze
21b.6 Die Anmeldung und der Konfigurations-Rundlauf
21b.7 🔬 Der Textkanal ist der Steuerkanal
21b.8 Zeit und Position vom Telefon
21b.9 Der Weg Telefon → Funk: seit 1.0.3.64 offen (E-13)
21b.10 ⚠️ Strom
21b.11 Der Stand
22 Der nichtflüchtige Speicher (NVS)
22.1 Zwei Funktionen, dieselbe Reihenfolge
22.2 Schlüssel sind höchstens 15 Zeichen lang
22.3 Die Default-Falle
22.4 Die Schemaversion — Aufräumen, nicht Wegwerfen
22.5 Einen einzelnen Wert löschen
22.6 Das Akku-Logbuch im NVS
23 Firmware-Update über Funk
23.1 Die fünf Grundentscheidungen
23.2 Zwei Wege
23.3 Das Server-Schema
23.4 ⚠️ Zwei Dateien, ein Stand
23.5 Was der Ablauf voraussetzt
24 Der Akku: was gemessen ist — und was nicht
24.1 Drei Größen, die auseinanderzuhalten sind

24.2 Das Logbuch (`CR_BattLog`)
24.3 Die Schutzabschaltung
24.4 Was gemessen ist
24.5 ⚠️ Was NICHT gemessen ist
24.6 🔴 Ein offener Befund: das Logbuch bricht bei 180 Einträgen ab
24.8 Der Lade-Wächter (1.0.3.113)
24.7 Merksätze für Messungen an diesem Gerät
24b Die Stromsparstufen: Ziel, Wirkung, Mechanik
24b.1 Der GPS-ECO-Automat in CR_Power
24b.2 ECOCPU — Prozessortakt (Vorgabe: an)
24b.3 ECOGPS — Telefon liefert (Vorgabe: an)
24b.4 ECOIMU — Ruhe-Erkennung (Vorgabe: an, seit 1.0.3.70)
24b.5 ECOFIX — Sparpause bei erfolgloser Suche (Vorgabe: an, seit 1.0.3.80er)
24b.6 ECOWLAN — Modem-Schlaf + Unterwegs-Pause (Vorgabe: an, seit 1.0.4.31)
24b.7 ECOBAKE — GPS auf Bestellung (Vorgabe: **aus**, seit 1.0.4.32)
24b.8 ECOSLEEP — Nachtschlaf des Prozessors (Vorgabe: **aus**, seit 1.0.4.105)
24b.9 ECODIGI — Digipeater-Pause in der Nacht (Vorgabe: **aus**, seit 1.0.4.145)
24b.10 Zusammenspiel und offene Messungen
25 Zugangsdaten und das öffentliche Abbild
25.1 Was im Entwicklungsabbild steckt
25.2 Die Auswahl beim Übersetzen
25.3 Die zweite Verteidigungslinie
25.4 Wo Geheimnisse sonst noch austreten
25.5 Die Regel
26 Der Emulator — und wo er *nicht* als Nachweis taugt
26.1 Was er ist
26.2 ⚠️ Wo er als Nachweis versagt
26.3 ⚠️ Ein Unterschied, der schweigend zuschlägt
26.4 Auch die Bedienung ist nachgebildet, nicht gleich
26.4b Aufnahmen ohne Fenster
26.5 Die Regel
27 Die eingebauten Messpunkte
27.1 `[LOOPHZ]` — der Takt des Main-Loops
27.2 `[CORE]` — die tatsächliche Kernzuordnung
27.3 `GUI_LAG` — Aussetzer des Render-Tasks
27.4 `[SLGUARD]` — der Schiebereglerschutz
27.5 Das Akku-Logbuch
27.6 Der LoRa-Selbsttest
27.7 `[LoRa TXHDR]` — der Paketkopf im Klartext
27.8 Die Startprotokoll-Wachposten
27.9 Die Regel hinter allen Messpunkten
28 Warum der GPS-Fix Minuten dauerte — und was die Uhr dem Empfänger seit 1.0.3.104 mitgibt
28.1 Die 28 Sekunden sind ein Laborwert
28.2 Die Platine: Backup-Zelle ja, Quarz nein
28.3 Was `CR_GpsAid` tut
28.4 Messen statt glauben: die TTFF-Zeile
28.5 Die Messung vom 27.08.2026 (Uhr am Kabel vor dem Haus, JN87ct)
28.6 Was bewusst NICHT gemacht wurde
28.7 Bahnvorhersagen per WLAN: AssistNow Predictive Orbits (1.0.4.83–.88)
28.8 Der interne Vorverstärker: drei Modi, ein Zähler zum Messen (1.0.4.112)
29 Absturzspuren: Reset-Journal, Brotkrumen und der Panic-Fanghaken (1.0.4.105–.113)
29.1 Das Problem: die Uhr stürzt nachts ab, und niemand war dabei
29.2 Brotkrumen: wer zuletzt gesprochen hat
29.3 Der Panic-Fanghaken und die Linker-Falle
29.4 Adressen in Zeilen verwandeln
29.5 Der Loop-Wächter (1.0.4.116)

29.6 Nachtschlaf: LoRa-Warten statt Phasenende (1.0.4.132)
Anhang A — Stromschienen
A.1 Geschaltete Schienen des AXP2101
A.2 Die drei Regeln
A.3 Wachposten im Startprotokoll
A.4 Diagnose-Merksatz
Anhang B — Pinbelegung T-Watch-S3-Plus
B.1 Display (ST7789V3, 240 × 240)
B.2 Funkmodul (SX1262)
B.3 Touch (FT6336U)
B.4 I²C-Hauptbus (AXP2101, PCF8563, BMA423, DRV2605L)
B.5 GPS (u-blox GNSS)
B.6 Nicht übersetzte Bauteile
B.7 ⚠ Doppelt belegte und irreführende Namen
B.8 Die Hardwarekennung steht *nicht* im Variant-Header
Anhang C — Was bereits versucht und verworfen wurde
C.1 Hardware und Treiber
C.2 Display und Farben
C.3 Touchcontroller FT6336U
C.4 Nebenläufigkeit und Bedienung
C.5 Angenommen, nie gemessen — und falsch
C.6 Was hier bewusst **nicht** steht
Anhang D — Quellenverzeichnis
D.1 Kapitel → Belegstellen
D.2 Quelldateien → wo sie erklärt werden
D.3 Fremddokumentation
D.4 Projektinterne Ablagen
D.5 Messmittel
D.6 u-blox-Dokumente zum GNSS-Modul (Kap. 28)
D.7 Hinweis zu den Zeilenverweisen
Abgleiche mit der MeshCom-Firmware

1 Gegenstand, Rechtsrahmen und Lizenz

1.1 Was dieses Gerät ist

Die MeshCom Watch ist eine Firmware für die **LilyGo T-Watch-S3-Plus** (ESP32-S3), die das Gerät zu einer tragbaren MeshCom-Station macht: LoRa-Funkverkehr auf 433 MHz, Positionsauswertung über GNSS, Anbindung an das MeshCom-Netz über WLAN und MQTT — bedient wie eine Armbanduhr.

Sie entstand auf Grundlage der SMashCom42-Firmware von **Wolfgang Zelinka, OE3WAS**, und wird von **Christian Raith, OE3LCR**, weiterentwickelt.

Das oberste Entwurfsziel steht über allen anderen:

> **100 % MeshCom-Kompatibilität bei LoRa-Empfang und -Aussendung.** Fällt eine andere Funktion aus, ist das hinnehmbar. Fällt der MeshCom-Verkehr aus, ist das Gerät zweckfrei.

Diese Rangordnung erklärt zahlreiche Entscheidungen in den folgenden Kapiteln — etwa die eigene SPI-Schnittstelle für das Funkmodul (Kapitel 7) oder die Sperre gegen Bildschirmwechsel während einer Aussendung.

1.2 ⚠️ Rechtsrahmen: Der Betrieb setzt eine Amateurfunkgenehmigung voraus

> ⚠️ **Die Aussendung auf 433 MHz ist ausschließlich lizenzierten Funkamateuren gestattet.**
>
> Der Betrieb dieses Geräts im Sendebetrieb ohne gültige Amateurfunkgenehmigung ist unzulässig.

Die Firmware arbeitet auf **433,175 MHz** (`variants/t-watch-S3-plus/pins_arduino.h:88`). Diese Frequenz liegt im **70-cm-Amateurfunkband** (430–440 MHz), dessen Nutzung dem Amateurfunkdienst zugewiesen ist. Eine Amateurfunkgenehmigung samt zugeteiltem Rufzeichen ist Voraussetzung.

Warum der Hinweis nicht durch den Verweis auf SRD/ISM entfällt

Im Bereich 433,05–434,79 MHz ist zusätzlich eine genehmigungsfreie Kurzstreckenfunk-Nutzung (SRD) zulässig — allerdings mit einer Strahlungsleistung von höchstens **10 mW ERP**. Die Sendestufen dieser Firmware reichen darüber deutlich hinaus (`CR_LoRaCtl.cpp:25-29`):

Stufe	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
dBm	—	2	5	8	11	**14**	17	20	**22**
mW	0	2	3	6	13	**25**	50	100	**158**

Die Auslieferungsstufe beträgt 14 dBm (25 mW), die Höchststufe 22 dBm (**158 mW**) — das Fünfzehnfache der SRD-Grenze. Ein Betrieb dieses Geräts als SRD-Anwendung scheidet damit aus. Hinzu kommt, dass MeshCom-Telegramme das Rufzeichen der aussendenden Station führen, was den Verkehr seiner Natur nach dem Amateurfunkdienst zuordnet.

Die Firmware setzt die Rufzeichenpflicht selbst durch

Die Genehmigungspflicht ist im Programmablauf verankert, nicht bloß in dieser Dokumentation. Ohne hinterlegtes gültiges Rufzeichen verweigert die Firmware jede Aussendung (`WZ_LoRa.cpp:281`):

```
[LoRa_ERR] Senden abgelehnt: kein gueltiges Rufzeichen im NVS (--setCALL)
```

Der Empfangsbetrieb bleibt davon unberührt: Das Abhören des Amateurfunkverkehrs unterliegt keiner Genehmigungspflicht. Ausgesendet wird jedoch erst nach Hinterlegung eines Rufzeichens.

⚠️ Diese Prüfung ist eine **Bedienhilfe, kein Rechtsersatz**. Sie prüft das Format der Zeichenfolge, nicht deren Zuteilung. Die Verantwortung für den genehmigungskonformen Betrieb — einschließlich der zulässigen Sendeleistung am jeweiligen Standort — trägt ausschließlich der Betreiber.

Weitere Stufe: Stufe 0 sperrt die Aussendung vollständig

Reglerstufe 0 schaltet den Sendebetrieb ab und belässt das Gerät im reinen Empfang (`CR_LoRaCtl.cpp:91` , Anzeige „SENDEN GESPERRT"). Für einen Betrieb ohne Genehmigung — etwa zu Beobachtungszwecken — ist dies die zu wählende Einstellung.

1.3 Lizenz

Die Firmware steht unter der **MIT-Lizenz**, gemeinsam gehalten von beiden Autoren:

```
MIT License
Copyright (c) 2026 Wolfgang Zelinka (OE3WAS)
Copyright (c) 2026 Christian Raith (OE3LCR)
```

Die MIT-Lizenz gestattet Verwendung, Vervielfältigung, Änderung und Weitergabe — auch zu gewerblichen Zwecken — unter der Bedingung, dass Lizenztext und Urheberrechtsvermerk erhalten bleiben. Eine Gewährleistung ist ausgeschlossen.

⚠️ Haftungsausschluss zum Funkbetrieb

Die MIT-Lizenz schließt eine Gewährleistung für die **Software** aus. Davon zu unterscheiden ist die Verantwortung für den **Funkbetrieb**, die von keiner Softwarelizenz berührt wird:

> **Der genehmigungskonforme Betrieb der Funkanlage liegt ausschließlich beim Betreiber.**
>
> Das betrifft insbesondere: das Vorliegen einer gültigen Amateurfunkgenehmigung, die am jeweiligen Standort zulässige Sendeleistung, die verwendete Antenne samt Anpassung, die Einhaltung der Bandpläne sowie die ordnungsgemäße Kennzeichnung der Aussendungen mit dem zugeteilten Rufzeichen.
>
> Weder die Autoren noch die Firmware prüfen oder gewährleisten die Zulässigkeit einer Aussendung. Die im Gerät hinterlegten Voreinstellungen — Frequenz, Sendeleistung, Sync-Wort — sind auf das MeshCom-Netz abgestimmt und stellen **keine Zusicherung** dar, dass ihr Einsatz an einem bestimmten Standort zulässig ist.

⚠️ Zu beachten ist ferner, dass ein Betrieb **ohne angepasste Antenne** das Sendeteil beschädigen kann. Dies ist ein Hardware-Sachverhalt und wird von der Firmware nicht erkannt.

Mitgelieferte Fremdbestandteile

Die Firmware bindet Bestandteile unter abweichenden Lizenzen ein. Zwei davon erfordern Beachtung bei der Weitergabe fertiger Abbilder:

Bestandteil	Lizenz	Auflage
`lib/tls_mini` (4 Dateien)	**LGPL-2.1**	Abschnitt 6: Empfänger eines Abbilds müssen die LGPL-Bestandteile austauschen können
Twemoji-Bildschrift	CC-BY 4.0	Namensnennung

> ⚠️ **Ein weitergegebenes `.bin` ohne den beigelegten LGPL-Quelltext erfüllt die Lizenz nicht.** Das Ausspielwerkzeug `tools/ota_release.sh` legt deshalb bei jedem Release automatisch ein Archiv `lgpl-sources.src` neben die Abbilddatei, versehen mit dem Commit-Stand, aus dem das Abbild erzeugt wurde.

Die LGPL wirkt hier **nicht** viral auf die übrige Firmware: Die betroffenen Dateien sind abgrenzbar, und die Austauschbarkeit ist durch die Beilage gewahrt.

1.4 Zielgruppe und Aufbau dieses Buches

Dieses Buch richtet sich an Techniker, die den Quelltext übernehmen, prüfen oder erweitern. Es beschreibt nicht, **was** der Quelltext tut — das ist ihm zu entnehmen —, sondern **warum** er es auf diese Weise tut, und welche naheliegenden Lösungswege bereits beschritten und verworfen wurden.

Für die Bedienung des Geräts existiert ein eigenes Benutzerhandbuch (`doku/handbuch/`).

Durchgängig gilt folgende Darstellungsregel:

> **Jede Aussage führt ihren Beleg mit** — Dateiname samt Zeilennummer, Messwert oder Prüfprotokoll. Wo eine Angabe nicht durch Messung gesichert ist, wird dies ausdrücklich vermerkt.

Der Grund dafür ist selbst ein Projektbefund: Die Dokumentation behauptete über Monate hinweg eine Trennung der Rechenkerne, die tatsächlich nicht bestand. Drei Zeilen Prüfcode hätten dies jederzeit widerlegt — eine Messung war nie erfolgt (Kapitel 11 und 14). Eine Dokumentation, die ungeprüfte Annahmen weiterreicht, richtet größeren Schaden an als gar keine.

2 Zwei Autoren, zwei Präfixe

Jede Quelldatei in `src/` trägt ein Präfix:

Präfix	Autor
`WZ_`	Wolfgang Zelinka, **OE3WAS**
`CR_`	Christian Raith, **OE3LCR**

Das sieht nach Kosmetik aus. Es ist die Voraussetzung dafür, dass dieser Zweig weiterhin Änderungen aus Wolfgangs Zweig übernehmen kann.

2.1 Warum das Präfix über die Zukunft entscheidet

Dieses Projekt ist kein abgespaltenes Eigenleben, sondern ein Zweig mit laufendem Abgleich: Der MeshCom-Kern wird von `wolfgang/main` weiter verfolgt, während Bedienung und Architektur hier eigene Wege gehen.

Solange neue Arbeit in **eigenen Dateien** mit `CR_`-Präfix liegt, kann ein Zusammenführen diese Dateien nicht anfassen — sie existieren im anderen Zweig gar nicht. Das Zusammenführen beschränkt sich damit auf die `WZ_`-Dateien, und dort auf die Stellen, an denen tatsächlich beide Seiten gearbeitet haben.

> **Wer eine neue Fähigkeit in eine bestehende `WZ_`-Datei schreibt, statt eine eigene `CR_`-Datei anzulegen, erzeugt für jede künftige Übernahme einen Konflikt** — und zwar dauerhaft, nicht einmalig.

Das gilt in beide Richtungen: Ein Beitrag, der an Wolfgangs Zweig zurückgehen soll, hat umgekehrt möglichst wenig `CR_`-Eigenheiten mitzubringen.

2.2 Wenn es sich doch nicht vermeiden lässt

Manche Änderungen müssen in eine `WZ_`-Datei, weil dort die betreffende Zeile steht. Für diesen Fall gilt eine ausdrückliche Markierung. Beispiel aus `WZ_LoRa.cpp:33`:

```
/// ⚠ CR-ABWEICHUNG von wolfgang/main (2026-08-04, Debug-Sitzung rend03-loop-1hz):
///    Hier stand `SPIClass radioSPI(HSPI)`. …
```

Drei solcher Markierungen bestehen derzeit:

Stelle	Worum es geht
`WZ_LoRa.cpp:33`	`FSPI` statt `HSPI` — ohne diese Zeile kehren die Watchdog-Neustarts zurück (Kapitel 7)
`WZ_GPS.cpp:512`	Abweichender Wächter in der GPS-Schleife
`WZ_GPS.cpp:581`	Zeile, die in Wolfgangs Zweig auskommentiert ist

Die Markierung leistet zweierlei: Beim nächsten Zusammenführen ist sofort erkennbar, dass eine Rückkehr zum Ursprungszustand **kein** neutrales Aufräumen wäre. Und sie nennt den Grund, damit die Entscheidung nicht erneut hergeleitet werden muss.

2.3 ⚠ Die Stelle, die zweimal verlorenging

Nicht jede Regression kündigt sich an. Der Auswahlblock für die persönliche Konfiguration in `globals.h:59–65` ist bei Zusammenführungen **zweimal** verschwunden — er existiert in Wolfgangs Zweig so nicht und wurde jedes Mal stillschweigend überschrieben.

Die Folge ist unangenehm, weil sie nicht wie ein Konfigurationsfehler aussieht: Das Übersetzen scheitert an einer fehlenden Datei, oder — schlimmer — es gelingt mit den falschen Zugangsdaten.

> **Nach jedem Zusammenführen mit `wolfgang/main` ist der `__has_include`-Block in `globals.h` zu prüfen.** Er ist bislang die einzige Stelle, an der das zweimal nachweislich schiefging.

2.4 Die Regel für neue Module

1. **Neue Fähigkeit → neue Datei mit dem Präfix des Autors.** Nicht in eine bestehende `WZ_`-Datei hineinschreiben.
2. **Unvermeidbare Eingriffe in `WZ_`-Dateien markieren** — mit Datum, Grund und der Folge eines Rückbaus.
3. **Der Modulaufbau bleibt gleich**, unabhängig vom Präfix: `XXX_Init()`, `XXX_Loop()` (nicht blockierend), Zustand als dateilokale Globale, Gate per `#ifdef ENABLE_XXX`.
4. **Kein `CR_`-Modul in einem Beitrag an Wolfgang**, der es nicht ausdrücklich anfordert.

Dass diese Trennung trägt, zeigt der praktische Fall: Der Emulator (Kapitel 26) besteht ausschließlich aus eigenen Dateien in einem eigenen Verzeichnis — `SMashCom42/` bleibt davon vollständig unberührt, und damit ist er für jede künftige Übernahme unsichtbar.

2b Übersetzen und Flashen

2b.1 Werkzeugkette

Die Firmware wird mit **PlatformIO** übersetzt, allerdings nicht mit dessen offizieller Plattformunterstützung, sondern mit **pioarduino** — einer Abspaltung, die entstand, weil PlatformIO den Arduino-Core 3.x für den ESP32-S3 nicht unterstützte.

Bestandteil	Fassung	Festgelegt in
Plattform	pioarduino `platform-espressif32` **55.03.312-1** (seit 1.0.5.0; davor 55.03.311 ab 1.0.3.134, davor 55.03.39)	`platformio.ini` (Zip-Adresse, fest verdrahtet)
Framework	Arduino-Core **3.3.12** auf ESP-IDF **5.5.5**	ergibt sich aus der Plattform
Grafik	LVGL **9.6.0** (seit 1.0.5.1; davor 9.5.0)	`platformio.ini`, Einstellungen in `src/lv_conf.h`
Sprachstand	C++17	`platformio.ini`
Speicheraufteilung	8 MB Anwendung + 8 MB OTA	`gen4esp32_8MBapp_8MBota.csv`

Die Fassungsangabe ist bewusst festgeschrieben und **nicht ohne Anlass zu ändern**.

Alle Bibliotheken sind auf die Fassung genau festgelegt (seit 1.0.4.174)

Bis 1.0.4.173 standen die Bibliotheken mit `^` in der `platformio.ini`, zum Beispiel `^1.6.6`. Das bedeutet „1.6.6 oder jede neuere 1.x". Welcher Stand tatsächlich gebaut wurde, hing damit vom Zeitpunkt des letzten Paket-Updates ab und nicht vom Projekt. Genau das war schon eingetreten: Arduino_GFX stand als `^1.6.6` in der Datei, installiert war ein Commit **nach** dem Tag v1.6.7. Mit der exakten Angabe fiel das Abbild um 16 Byte kleiner aus, bei gleichem Quelltext.

Bibliothek	Fassung	Lizenz
`lvgl/lvgl`	9.6.0	MIT
`bodmer/TFT_eSPI`	2.5.43	MIT
`moononournation/Arduino_GFX`	Git-Tag `v1.6.8`	BSD
`jgromes/RadioLib`	7.8.1	MIT
`mikalhart/TinyGPSPlus`	1.1.0	⚖ **LGPL-2.1**
`lewisxhe/XPowersLib`	0.3.3	MIT
`lewisxhe/SensorLib`	0.5.0	MIT
`h2zero/NimBLE-Arduino`	2.5.1	Apache-2.0
`knolleary/PubSubClient`	2.8	MIT
`mathertel/OneButton`	2.6.2	BSD-3-Clause
`mcxiaoke/ESPDateTime`	1.0.4	Apache-2.0

Stand 1.0.5.5 (30.09.2026): Alle Fassungen gegen die PlatformIO-Registry und die GitHub-Releases abgeglichen. Nachgezogen wurden Arduino_GFX 1.6.7 → 1.6.8, RadioLib 7.7.1 → 7.8.1 (unter anderem Korrekturen am SX126x: Standby-Oszillator vor `begin()`, Auswertung des Status-Bytes) und SensorLib 0.4.1 → 0.5.0. SensorLib 0.5 hat ihre Header neu geordnet; die alten `SensorBMA423.hpp` und `SensorPCF8563.hpp` sind nur noch Übergangshüllen, eingebunden werden jetzt `AccelerometerDrv.hpp` (`CR_Imu.cpp`) und `RtcDrv.hpp` (`CR_RTC.cpp`). Die Plattform 55.03.312-1 ist die neueste freigegebene; 61.04.00 lag nur als Vorabfassung (RC1) vor.

⚠ **An einer Git-Adresse ist nur `#<tag>` eindeutig.** Eine Angabe wie `@^1.6.6` hinter einer URL löst PlatformIO nicht als Fassung auf.

Ein Bibliotheks-Update ist damit eine bewusste Zeile im Diff. Danach gilt dieselbe Regel wie bei der Plattform: bauen, flashen, abnehmen, **LoRa-Empfang und -Senden zuerst**. Der Git-Tag `v1.0.4.173` ist der Rückfallpunkt vor der neuen Grundlage; seine Anmerkung listet alle damals verwendeten Fassungen.

⚖ **TinyGPSPlus steht unter LGPL-2.1** und fehlte bis 1.0.4.174 im beigelegten Quelltextarchiv. Seit 1.0.5.3 legt `tools/ota_release.sh` es bei und prüft alle nachgeladenen Bibliotheken auf LGPL-Lizenztexte. Eine neue LGPL-Bibliothek, die dort nicht eingetragen ist, bricht das Ausspielen ab. Der Wächter prüft sich selbst: Findet er nicht einmal TinyGPSPlus, meldet er die Suche als defekt.

Der Umstieg auf die neue Grundlage (1.0.5.0 bis 1.0.5.2)

Plattform, LVGL und die LVGL-Einstellungsdatei wurden in drei getrennten Schritten gewechselt, jeder mit eigener Nummer und eigener Abnahme am Gerät. Tritt später ein Fehler auf, lässt er sich so einem Schritt zuordnen.

Stand	Schritt	Stolperstelle
1.0.5.0	Plattform 55.03.311 → 55.03.312-1	keine
1.0.5.1	LVGL 9.5.0 → 9.6.0	kein Bruch der Programmierschnittstelle; Emoji-Bildschrift und Speichervorrat am Gerät nachgewiesen
1.0.5.2	`lv_conf.h` beider Ziele (Uhr und Emulator) neu aus der 9.6-Vorlage	siehe unten

Die 9.6-Vorlage hat einige Einstellungen umbenannt oder neu gegliedert: `LV_COLOR_DEPTH` heißt jetzt `LV_COLOR_FORMAT_DEFAULT`, der eigene Assert-Haken wird über `LV_ASSERT_CUSTOM_INCLUDE` eingebunden (`CR_LvAssert.h`), die Kalendernamen über `LV_MONDAY_STR` …, und der Anzeigetreiber braucht `LV_USE_GENERIC_MIPI 1` . Alle Abweichungen von der Vorlage stehen als Liste im Kopf der Datei, 60 bei der Uhr und 55 beim Emulator.

⚠️ **Drei Fallen beim Heben auf eine neue Vorlage:**

1. **Die Vorlage beginnt mit `#if 0` .** Wird das nicht auf `#if 1` gesetzt, ist die ganze Datei wirkungslos, und LVGL baut still mit seinen Voreinstellungen.
2. **Der Speichervorrat im PSRAM ist eine eigene Einstellung** (`LV_MEM_POOL_INCLUDE` / `LV_MEM_POOL_ALLOC` mit `heap_caps_malloc(…, MALLOC_CAP_SPIRAM)`). Fehlt sie, landen die 96 KB im internen Speicher, und der Linker bricht ab.
3. **Ein Kommentar darf kein `*/` enthalten.** Eine Liste im Dateikopf, die ein `*/` zitiert, beendet den Kommentarblock mitten im Text.

⚠️ Konflikt mit der PlatformIO-Erweiterung für VS Code

> **Symptom:** Der Übersetzungslauf bricht nach etwa zwei Sekunden ab mit `TypeError: argument should be a str or an os.PathLike object … not 'NoneType'` in `platform-espressif32/builder/frameworks/arduino.py` . Das Paket `framework-arduinoespressif32` fehlt dann in der Paketübersicht.

Ursache: Ein parallel laufendes VS Code mit der offiziellen **PlatformIO**-Erweiterung. Beide Werkzeuge beanspruchen ein Paket desselben Namens im selben Verzeichnis:

Zustand	Eigentümer laut `.piopm`	Fassung	Herkunft
brauchbar	`espressif`	3.3.9	GitHub-Adresse, von pioarduino
defekt	`platformio`	3.20017.241212 (= Core 2.0.17)	offizielle PlatformIO-Registrierung

Die VS-Code-Erweiterung zieht beim Indizieren die Registrierungsfassung und überschreibt damit die von pioarduino eingetragene. Die `platformio.ini` bleibt dabei unangetastet — die Fehlersuche führt dort also ins Leere.

Abhilfe vor jedem Übersetzungslauf:

```
~/.platformio/penv/bin/pio pkg install -d SMashCom42 -e t-watch-s3-plus
```

Dauerhaft: Die PlatformIO-Erweiterung für dieses Arbeitsverzeichnis abschalten oder durch die pioarduino-Erweiterung ersetzen.

> ⚠️ **Die Ausgabe von `pio` niemals in eine Pipe leiten.** Ein SIGPIPE mitten in einer Paketinstallation hinterlässt dasselbe defekte Mischverzeichnis. Statt `pio run … | tail` : in eine Datei umleiten und diese anschließend auswerten.

2b.2 Die drei Übersetzungsumgebungen

```
# Entwicklungsstand (enthält die persönlichen Zugangsdaten)
~/.platformio/penv/bin/pio run -d SMashCom42 -e t-watch-s3-plus

# Auslieferungsstand (Platzhalter statt Zugangsdaten)
~/.platformio/penv/bin/pio run -d SMashCom42 -e t-watch-s3-plus-release

# auf das Gerät schreiben
~/.platformio/penv/bin/pio run -d SMashCom42 -e t-watch-s3-plus -t upload \
    --upload-port /dev/cu.usbmodem-N>
```

> ⚠️ **Der Entwicklungsstand enthält sechs Geheimnisse im Klartext** — Rufzeichen, zwei WLAN-Namen, ein WLAN-Kennwort, die Broker-Adresse und das OTA-Kennwort. **Er darf niemals veröffentlicht werden.** Der Auslieferungsstand enthält davon nichts; das Ausspielwerkzeug `tools/ota_release.sh` weist das bei jedem Lauf über eine Zeichenkettensuche nach und bricht ab, wenn ein Treffer auftritt (Kapitel 25).

2b.3 Die USB-Schnittstelle

⚠️ Die T-Watch-S3-Plus meldet sich über den **nativen USB-Anschluss des ESP32-S3**, nicht über einen CP2102-Wandler:

USB-Kennung	`303A:1001`
Gerätedatei (macOS)	`/dev/cu.usbmodem*` — **nicht** `/dev/cu.SLAB_USBtoUART`
Übertragungsrate Konsole	115200
Übertragungsrate Flashen	921600

Ein Kabel **ohne Datenadern** (reines Ladekabel) erzeugt keine Gerätedatei. Tritt keine `/dev/cu.usbmodem*` auf, ist zuerst das Kabel zu prüfen.

2b.4 Zwei Abbilder, zwei Verwendungen

Datei	Enthält	Ziel	Verwendung
`firmware.factory.bin`	Bootloader, Partitionstabelle, `boot_app0`, Anwendung	Offset `0x0`	Flashen über Kabel
`firmware.bin`	nur die Anwendung	—	Update über Funk (OTA)

> ⚠️ **Die `firmware.bin` gehört nicht an `0x0`.** Ihr fehlen Bootloader und Partitionstabelle; an dieser Position geschrieben ergibt sie ein nicht startfähiges Gerät. Der Web-Flasher meldet in diesem Fall „Flash Read Failed".

2b.5 Herstellerunterlagen

Die Hardware-Unterlagen des Herstellers sind maßgeblich für Schaltplan, Bestückung und Pinbelegung. Anhang B dieses Buches gibt die für die Firmware belegten Anschlüsse wieder, ersetzt die Unterlagen aber nicht:

- **Produktseite T-Watch-S3:** lilygo.cc/products/t-watch-s3
- **Quelltext- und Unterlagenablage (Schaltpläne als PDF im Ordner `schematic`):** github.com/Xinyuan-LilyGO/TTGO_TWatch_Library
- **Leistungsregler AXP2101, Treiberbibliothek und Datenblatt:** github.com/lewisxhe/XPowersLib
- **Funkbaustein SX1262, verwendete Funkbibliothek:** github.com/jgromes/RadioLib

⚠️ **Beim Heranziehen fremder Beispiele auf die Modellbezeichnung achten: T-Watch-S3** und **T-Watch-S3-Plus** teilen sich dasselbe Motherboard und dieselbe Belegung der Stromschienen (Richtigstellung 02.09.2026, Kapitel 5.2); der Unterschied ist die aufsteckbare GNSS-Platine an BLDO1 und der halb so große Akku. Abweichen kann dagegen die Anzeigeansteuerung, und älterer Beispielcode meint oft die T-Watch **2020** mit anderem Prozessor.

3 Das Sechs-Ebenen-Modell

Ob das GPS-Modul dieser Uhr gerade Positionen liefert, hängt an sechs voneinander unabhängigen Bedingungen. Das wirkt beim ersten Lesen überbestimmt — bis man bemerkt, dass **jede Ebene eine andere Frage beantwortet**, und dass diese Fragen tatsächlich getrennt auftreten.

Der Entwurf stammt von Wolfgang Zelinka (OE3WAS) und gilt für jedes Modul.

3.1 Die sechs Ebenen

#	Ausdruck	Frage	Ort	Art
1	`HAS_XXX`	Ist die Hardware **verbaut**?	`variants/<board>/pins_arduino.h`	Übersetzungszeit
1	`USE_XXX`	Ist der Treiber für dieses Board **gewollt**?	ebenda	Übersetzungszeit
2	`ENABLE_XXX`	Wird der Code **mitübersetzt**?	`globals.h` (abgeleitet)	Übersetzungszeit
3–4	`en_XXX`	Hat der **Benutzer** es eingeschaltet?	`WZ_XXX.cpp` , persistiert im NVS	Laufzeit
5	`Read_Prefs()`	Was war beim letzten Mal eingestellt?	`prefs.cpp`	Start
6	`on_XXX`	Hat sich das Bauteil **tatsächlich gemeldet**?	`WZ_XXX_Init()`	Laufzeit

Die Ebenen sind eine Kette: Jede setzt die vorige voraus. `en_GPS` kann nur wahr sein, wenn `ENABLE_GPS` übersetzt wurde; `on_GPS` nur, wenn das Modul beim Start geantwortet hat.

3.2 Am Beispiel GPS durchgespielt

Ebene 1 — `variants/t-watch-S3-plus/pins_arduino.h:147` :

```
#define USE_GPS
```

Ebene 2 — `globals.h:248-256` leitet daraus ab und zieht das Modul herein:

```
#ifdef USE_GPS
  #define ENABLE_GPS
  #include "WZ_GPS.h"
  extern bool en_GPS;
  extern bool on_GPS;
  …
#endif
```

Ebene 3–4 — `WZ_GPS.cpp:58-59` definiert die beiden Zustandsgrößen.

Ebene 5 — `prefs.cpp:122` holt die Benutzereinstellung aus dem NVS:

```
en_GPS = preferences.getBool("enGPS", false);
```

Ebene 6 — `WZ_GPS_Init()` setzt `on_GPS` **nur bei erfolgreicher Antwort** (`WZ_GPS.cpp:205`), und zwar nach dem Grundsatz aus `:515` :

> Das Modul muss `false` zurückkommen, sonst läuft `on_GPS` / `on_UBLOX` **auf einer Lüge**.

Anwendungscode fragt anschließend beide Laufzeitgrößen ab — die Freigabe und die Bestätigung.

3.3 Warum keine Ebene entbehrlich ist

Der Reiz, das zu vereinfachen, ist groß. Die folgenden Fälle zeigen, warum es die Unterscheidungen wirklich braucht:

Situation	Welche Ebenen sich unterscheiden
Board ohne GNSS-Bestückung	`HAS_` / `USE_` fehlt — der Code ist gar nicht im Abbild
Board mit GNSS, Treiber bewusst aus	`HAS_` ja, `USE_` nein
Benutzer hat GPS ausgeschaltet	`ENABLE_` ja, `en_` nein
Benutzer will GPS, Modul antwortet nicht	`en_` ja, `on_` **nein**
Antenne abgeschattet, Modul in Ordnung	`on_` ja, aber kein Fix — eine **siebte** Frage, die keine dieser Ebenen beantwortet

Der vorletzte Fall ist der Grund, warum `on_XXX` existiert: Ein Benutzerwunsch ist kein Gerätezustand. Ohne diese Trennung müsste jeder Aufrufer selbst herausfinden, ob das Modul antwortet — und würde es unterschiedlich tun.

3.4 ⚠️ Was das Modell *nicht* leistet

Hier liegt die wichtigste Einschränkung des ganzen Kapitels:

> **Keine dieser sechs Ebenen sagt etwas über den Stromverbrauch.**

`en_GPS = false` beantwortet die Frage „will der Benutzer Positionen sehen?" — nicht die Frage „zieht das Bauteil Strom?". Bis zum 3. August 2026 wurde beides gleichgesetzt, mit dem Ergebnis, dass ein „abgeschaltetes" GNSS-Modul unverändert seine rund 30 mA zog und **jede Sparmessung wertlos** war.

Die vollständige Geschichte steht in Kapitel 6. Für dieses Kapitel genügt die Folgerung:

> Ein Modul abzuschalten heißt, **zwei** Dinge zu tun — das Flag setzen *und* die Stromschiene kappen. Das Ebenenmodell deckt nur das erste ab.

3.5 Die gemessene Ebene `hw_GPS` — ein Abbild für zwei Uhren

Seit 1.0.4.45 (Entscheidung E-32) trägt **ein** Abbild sowohl die T-Watch S3 Plus (GNSS an BLDO1) als auch die T-Watch S3 ohne GNSS. Dafür ist eine Ebene dazugekommen, die zwischen `ENABLE_GPS` und `on_GPS` liegt:

Ausdruck	Frage	Ort	Art
`ENABLE_GPS`	Ist der Code **mitübersetzt**?	`globals.h`	Übersetzungszeit
`hw_GPS`	**Hat *diese* Uhr überhaupt ein Modul?**	`WZ_GPS.cpp`, NVS-Schlüssel `hwGPS`	**Laufzeit, einmal gemessen**
`en_GPS`	Hat der Benutzer es eingeschaltet?	NVS `enGPS`	Laufzeit
`on_GPS`	Läuft es gerade?	`WZ_GPS_Init()`	Laufzeit, flüchtig

Werte: `CR_HWGPS_UNBEKANNT 0` (noch nie gemessen), `CR_HWGPS_VORHANDEN 1`, `CR_HWGPS_KEINS 2`.

Warum eine eigene Ebene und nicht einfach `on_GPS`? Weil `on_GPS` diese Frage seit `GPS_FIXED_BAUD 38400` nicht mehr beantworten *kann*. Mit fester Baudrate entfällt die Erkennung, und `WZ_GPS_Init()` setzt `on_GPS` **bedingungslos** auf `true`, bevor die Probe überhaupt läuft — sie braucht die geöffnete UART. Der Punkt aus Abschnitt 3.2 („`on_XXX` nur nach bestätigter Antwort") gilt an dieser Stelle also gerade nicht mehr für den Zwischenstand. Ausgewertet wird deshalb der **Rückgabewert von `GPSprobe()`**, also die echte `$PCAS06`-Antwort des Empfängers:

```
on_GPS = true;                    // Pflicht: die Probe braucht die UART
if (GPSprobe()) {
  hw_GPS = CR_HWGPS_VORHANDEN;
} else {
  hw_GPS = CR_HWGPS_KEINS;        // gemessen, nicht vermutet
  on_GPS = false;
  GPSSerial.end();
  pmu.disableBLDO1();
}
```

Der Wert landet im NVS, damit der **nächste** Start die Suche überspringen kann — eine Uhr ohne Modul soll nicht bei jedem Boot in denselben Timeout laufen.

Die Gegenprobe bleibt immer erreichbar. `hw_GPS` ist eine Messung, und Messungen können falsch im Flash stehen. Deshalb blendet **kein** Codepfad die GPS-Kachel aus: ein Tipp darauf setzt `hw_GPS` erst auf `UNBEKANNT` zurück und sucht dann über genau denselben Weg wie der Start (dieselbe `WAIT_DURATION`, keine

eigene Zeitgrenze — sonst könnte ein Modul „beim Booten weg, beim Tippen da" sein). Bleibt die Suche erfolglos, fällt `en_GPS` zurück, und die Kachel wird binnen einer Sekunde von selbst wieder rot, weil `cr_modulescreens_gps_tile_loop()` im Sekundentakt `en_GPS` liest — **ohne** `lv_*`-Aufruf aus dem Main-Task (Kapitel 13). Von der Konsole aus leisten `--gps suche` und `--gps keins` dasselbe.

3.6 Die Regel für neue Module

1. `HAS_` / `USE_` gehören **ausschließlich** in den Variant-Header, nie in `globals.h`.
2. `ENABLE_` wird in `globals.h` **abgeleitet**, nie von Hand gesetzt.
3. `en_XXX` wird in `Read_Prefs()` gelesen **und** in `Save_Prefs()` geschrieben — beide Funktionen spiegeln einander. ⚠️ Zur Falle dabei siehe Kapitel 22.
4. `on_XXX` wird **nur nach einer bestätigten Antwort** des Bauteils gesetzt, nie vorsorglich.
5. Anwendungscode prüft `on_XXX`, nicht `en_XXX` — der Benutzerwunsch allein liefert keine Daten.

4 Zwei Versionsnummern statt einer

Die Firmware führt zwei Versionsnummern nebeneinander. Das ist keine Redundanz, sondern die Behebung eines Fehlers, der **zweimal an zwei aufeinanderfolgenden Tagen je einen ganzen Arbeitstag vernichtet hat**.

4.1 Der Fehler

Bis zum 3. August 2026 gab es nur `FWVERSION`. Sie stammt aus Wolfgangs Nummernkreis und sagt aus, auf welchem SMashCom42-Stand dieser Zweig aufsetzt — sie wird folglich **nicht** hochgezählt, wenn hier gearbeitet wird.

Zugleich entschied genau diese Nummer über Firmware-Updates. Uhr und Server trugen beide `42.0.2.9`, die Prüfung meldete „aktuell" — und lud trotzdem die ältere Datei, weil hinter der gleichen Nummer verschiedene Stände lagen.

Am 2. und am 3. August 2026 hat ein Update auf diese Weise je einen Tagesstand vom Gerät gelöscht (`globals.h:37–39`).

> **Der Schaden entstand nicht durch eine falsche Nummer, sondern durch eine Nummer, die zwei Fragen gleichzeitig beantworten sollte:** „auf welchem Fremdstand setzen wir auf?" und „ist dieses Abbild neuer als jenes?"

4.2 Die Trennung

Nummer	Bedeutung	Wer zählt sie hoch
`FWVERSION`	Der SMashCom42-Stand, auf dem dieser Zweig aufsetzt	**niemand hier** — sie gehört Wolfgang
`CR_VERSION`	Der eigene Stand dieses Zweigs. **Nur sie entscheidet über Updates**	dieser Zweig, bei jedem Flashen

Beide stehen in `globals.h:32` und `:50`. Auf dem Info-Schirm erscheinen sie getrennt, dazu die Protokollfassung — drei Zeilen, drei verschiedene Fragen:

```
SMC42 | FW 1.0.3.0      ← unser Stand
Basis 42.0.2.9          ← worauf er aufsetzt
MeshCom 4.40A           ← welche Sprache die Uhr im Netz spricht
```

4.3 ⚠ Das Format ist streng

`cr_ota_parse_version()` verlangt **genau vier rein numerische Felder** und weist alles andere ab (`globals.h:41–42`):

Wert	Ergebnis
`1.0.3.0`	gültig
`1.0.3`	abgewiesen (drittes Feld fehlt)
`1.0.0.0-cr3`	abgewiesen (Suffix)
`1.0.3.0.1`	abgewiesen (fünftes Feld)

Verglichen wird numerisch Feld für Feld, das erste zuerst. Dieselbe Form gilt für die `version.json` auf dem Server (Kapitel 23).

4.4 ⚠ Die Zählregel — und warum sie nicht rückgängig zu machen ist

Festlegung Christian (OE3LCR) vom 5. August 2026 (`globals.h:43–48`):

> **Immer nur das letzte Feld hochzählen** — `1.0.2.0` → `1.0.2.1` → `1.0.2.2` … — unabhängig davon, ob es eine Korrektur oder eine neue Fähigkeit ist. Über die vorderen Felder entscheidet ausschließlich Christian.

Die Regel entstand aus einem Fehlgriff: Zuvor stand dort „drittes Feld = Funktionsstand", woraufhin von `1.0.1.0` auf `1.0.2.0` gesprungen wurde. Das war nicht gewollt — und ein Zurückdrehen war **nicht mehr möglich**, weil eine kleinere Nummer das Update blockiert hätte.

> **Eine Versionsnummer ist eine Einbahnstraße.** Ein zu großer Schritt lässt sich nicht zurücknehmen, ohne die Update-Kette zu unterbrechen.

4.5 Der verbindliche Ablauf nach jedem Flashen

Die getrennten Nummern beheben den Fehler nur zur Hälfte. Die andere Hälfte ist ein Ablauf, der nicht ausgelassen werden darf:

```
# 1. CR_VERSION in globals.h hochzählen (letztes Feld)
# 2. bauen und flashen
./tools/ota_release.sh                                  # 3. Release-Abbild + version.json
cd ~/Documents/esptool/espflash && CFT_SYNC=0 ./deploy.sh   # 4. ausspielen
```

⚠️ **Schritt 3 und 4 gehören zum Flashen, nicht zur Veröffentlichung.** Bleibt der Server zurück, trägt er ein älteres Abbild unter einer Nummer, die die Uhr für aktuell hält — und genau das war der ursprüngliche Schaden. `ota_release.sh` liest `CR_VERSION` deshalb selbst aus `globals.h` und trägt sie in `version.json` **und** ins Web-Flasher-Manifest ein; von Hand abgeschrieben wird sie nirgends.

4.6 Was daraus allgemein folgt

> **Eine Nummer, die zwei Fragen beantwortet, beantwortet früher oder später eine davon falsch** — und der Fehler tritt dort auf, wo niemand ihn vermutet, weil beide Fragen für sich genommen richtig beantwortet aussehen.

Dasselbe Muster in kleinerem Maßstab findet sich in Kapitel 15.4, wo ein Bezeichner (`CR_OTA_IsRunning()`) eine andere Frage beantwortete als die, die an der Aufrufstelle gestellt wurde.

5 Die Stromschienen des AXP2101

Die T-Watch-S3-Plus versorgt ihre Bauteile nicht direkt aus dem Akku, sondern über die programmierbaren Spannungsregler des **AXP2101**-Leistungsreglers. Jedes größere Bauteil hängt an einer eigenen, einzeln schaltbaren Schiene.

Diese Aufteilung ist der wichtigste Hebel für die Akkulaufzeit — und gleichzeitig die Quelle der hartnäckigsten Fehler in diesem Projekt. Drei der vier Schienen haben je einen eigenen Arbeitstag gekostet, bevor sie gefunden waren.

5.1 Die Belegung

Schiene	Versorgt	Spannung	Eingeschaltet in	Belegstelle
BLDO1	GNSS-Modul (GPS)	3300 mV	`WZ_GPS_Init()`	`WZ_GPS.cpp:527`
BLDO2	DRV2605 Haptik-Treiber (Vibration)	3300 mV	`CR_Power_Init()`	`CR_Power.cpp:183`
ALDO4	SX1262 Funkmodul (LoRa)	3300 mV	`WZ_LoRa_Init()`	`WZ_LoRa.cpp:95`
VBACKUP (Knopfzellen-Lader, kein LDO)	RTC PCF8563 + Pufferzelle MS412FE (Schaltplan: VBACKUP → R7 0 Ω → RTC_3_3V1 → D14 1N4148 → VCC_RTC)	3300 mV	`CR_Power_Init()`	`CR_Power.cpp` (seit 1.0.3.116)

> ⚠️ **Die RTC war bis 1.0.3.115 stromlos — und der erste Fix (.114) traf die falsche Schiene.** Der Bootlog meldete bei jedem Start `[RTC_ERR] PCF8563 nicht gefunden (0x51)`, 0x51 stand nie im I2C-Scan. Folge: ohne WLAN beim Start lief die Uhr mit 12:00, denn NTP kam nur beim Boot. Entdeckt am 28.08.2026 über die Frage „warum sync die Zeit nicht?" — nicht über die RTC selbst (Lehre: eine Fehlerzeile, die bei **jedem** Boot kommt, wird unsichtbar). 1.0.3.114 schaltete daraufhin **ALDO1** ein — LilyGos Tabelle führt die Schiene als „Unused", und der Schaltplan zeigt warum: ALDO1 führt über **R6 (NC, nicht bestückt)** ins Leere. Gemessen mit .115: `ALDO1 eingeschaltet` und 1,2 s später trotzdem `nicht gefunden`. Erst das Lesen von Seite 1 und 3 des Schaltplans zeigte den wahren Weg: die RTC hängt am **VBACKUP-Pin** des AXP2101, dem Lader für die Knopfzelle. Solange der aus ist, liegt an VCC_RTC nichts an, und die MS412FE ist längst leer. Seit 1.0.3.116 wird der Lader mit 3300 mV eingeschaltet (so macht es LilyGos eigenes Beispiel `AXP2101_Example.ino`) — damit bekommt der Chip Spannung **und** die Zelle lädt, die Uhrzeit übersteht dann auch ein PMU-Aus. Eine tief entladene Zelle kann das Netz beim ersten Boot noch herunterziehen; der Beweis ist der Boot danach. | PWM Pin 45 | Hintergrundbeleuchtung | 5000 Hz, 8 Bit | `CR_Power_Init()` | — |

Jede dieser Zeilen schreibt beim Start eine Bestätigung ins Log. Diese Zeilen sind nicht Zierrat, sondern **Wachposten** — sie lesen den Zustand nach dem Schalten aus dem Regler zurück:

```
[PWR] BLDO1 (GPS-GNSS): eingeschaltet
[PWR] BLDO2 (DRV2605/Vibration): eingeschaltet
[PWR] ALDO4 (LoRa SX1262): eingeschaltet
```

Steht dort `FEHLER`, hat der Regler den Befehl nicht angenommen. Alles, was danach an diesem Bauteil scheitert, ist Folgefehler — und sieht dabei aus wie ein Treiberproblem.

5.2 ⚠️ Die Schiene muss eingeschaltet werden — ALDO4

Daran ist die LoRa-Inbetriebnahme zunächst gescheitert (`WZ_LoRa.cpp:9`: *„Stromschiene ALDO4 wurde nie eingeschaltet"*). Beispielcode aus dem Netz beginnt oft erst beim Funkbaustein und setzt voraus, dass die Versorgung bereits steht; auf dieser Uhr steht sie nicht.

Richtigstellung (02.09.2026). Bis zu diesem Tag stand hier die Behauptung, beim normalen T-Watch-S3 hänge das Funkmodul an **ALDO3** und nur beim Plus-Modell an ALDO4. **Das ist falsch.** LilyGos eigener Bibliothekscode für das Grundmodell — `documents_original/src_Lib/LilyGoWatchS3.cpp`, intern `LilyGoWatch2022.cpp`, Klasse `LilyGoWatch2022`, gebunden an `#ifdef ARDUINO_T_WATCH_S3` — führt in seiner Rail-Tabelle (Zeilen 414–418) und im Startcode (Zeilen 444–445, 463–464) dieselbe Zuordnung wie das Plus-Modell:

Schiene	T-Watch-S3 (2022)	T-Watch-S3-Plus
ALDO1	unbenutzt	unbenutzt
ALDO2	Hintergrundbeleuchtung	Hintergrundbeleuchtung
ALDO3	Anzeige und Touch	Anzeige und Touch
ALDO4	**Funkmodul**	**Funkmodul**
BLDO1	**unbenutzt**	**GNSS**

Der einzige Unterschied der beiden Modelle in dieser Tabelle ist **BLDO1**: beim Grundmodell ohne Verbraucher, beim Plus die Versorgung des GNSS-Moduls. Die ursprüngliche Behauptung nannte als Quelle die Herstellerdoku zum Plus-Modell, die über das Grundmodell aber gar keine Aussage trifft; sie war eine Vermutung, die als Beleg gelesen wurde. Für eine gemeinsame Firmware beider Modelle ist das eine gute Nachricht: Es braucht **keine** Fallunterscheidung bei den Stromschienen.

Die Lehre ist allgemeiner als der Einzelfall: Ein stromloses Bauteil meldet sich nicht als „stromlos", sondern als „antwortet nicht". Diese beiden Zustände sind von der Software aus nicht unterscheidbar. Deshalb steht in diesem Projekt vor jedem Bauteil-Start das Einschalten der Schiene **samt Rücklesen** — und nicht die Annahme, sie sei schon an.

5.3 Die Reihenfolge ist nicht beliebig

`WZ_LoRa_Init()` und `WZ_GPS_Init()` schalten ihre Schiene **selbst** ein, als erste Handlung. Das ist Absicht: Es gibt keinen zentralen Ort, an dem alle Schienen hochgefahren werden, und es soll auch keinen geben.

Der Grund ist die Wiedereinschaltbarkeit. Beide Module lassen sich im Betrieb abschalten — und dann wieder an. Läge das Einschalten der Schiene in einer zentralen Startroutine, wäre der zweite Einschaltvorgang ein anderer Ablauf als der erste. Genau solche Unterschiede zwischen „beim Start" und „später" sind die Fehler, die sich erst nach Wochen zeigen.

So dagegen gilt: **Die startende Routine versorgt das Modul auch.** Ein Aufruf, immer derselbe Weg.

---

6 ⚠️ Die Schienen-Falle: ein Modul „aus” ist nicht stromlos

Dies ist das Kapitel, das bei der Akkulaufzeit den Unterschied macht.

6.1 Der Fehler

Das Sechs-Ebenen-Modell (Kapitel 3) kennt ein Benutzer-Freigabeflag `en_XXX` und ein Betriebsflag `on_XXX`. Ein Modul abzuschalten hieß bis zum 3. August 2026: `en_GPS = false` setzen. Die Schleife `WZ_GPS_Loop()` prüft dieses Flag und steigt dann sofort aus.

Das Ergebnis sah vollkommen richtig aus. Die Kachel wurde rot, die Anzeige zeigte keine Position mehr, der Datenstrom hörte auf. Alles deutete darauf hin, dass GPS aus war.

Es war nicht aus. Der Quelltextkommentar an der Stelle, an der das behoben wurde, sagt es schonungslos (`CR_ModuleToggle.cpp:56–59`):

> ⚠️ Der WICHTIGE Teil ist die Stromschiene, nicht das Flag. Bis zum 2026-08-03 setzte die Kachel nur `en_GPS = false` ; `WZ_GPS_Loop()` stieg daraufhin aus, das GNSS-Modul lief an BLDO1 aber unverändert weiter und zog seinen Strom (~30 mA — der größte Einzelposten der Uhr). **Ein Sparlauf mit „GPS aus” hätte exakt denselben Verbrauch gezeigt wie einer mit GPS an.**

Der letzte Satz ist der eigentliche Schaden. Es ging nicht nur Strom verloren — es ging die **Messbarkeit** verloren. Jede Sparmessung, die vor diesem Datum mit „GPS aus” gemacht wurde, ist wertlos, weil sie in Wahrheit mit GPS an gemessen wurde.

6.2 Warum das so lange unentdeckt blieb

Weil die Software sich völlig schlüssig verhielt. Es gab keinen erkennbaren Widerspruch: Das Flag war aus, die Schleife lief nicht, die Anzeige war leer. Nur das Bauteil selbst wusste es besser — und Bauteile schreiben keine Logzeilen.

Sichtbar wurde es erst bei einer Messung der **Laufzeit**, deren Ergebnis nicht zum erwarteten Wert passte.

6.3 Wie es jetzt aussieht

`CR_ModuleToggle.cpp:72–87` — das Abschalten in voller Länge:

```
} else if (reqGps == 0) {
  CR_GpsFix_Flush();        // letzte Position sichern, SOLANGE das Modul noch läuft
  en_GPS = false;
  Save_Prefs();
  on_GPS = false;
  pmu.disableBLDO1();       // ← das ist der eigentliche Sparhebel
  dbLOG("[MODTOGGLE] GPS aus, BLDO1 %s\n",
        pmu.isEnableBLDO1() ? "NOCH AN (Fehler)" : "abgeschaltet");
}
```

Drei Dinge daran verdienen Beachtung:

Erstens wird die zuletzt bekannte Position gesichert, *bevor* der Strom fällt. Das Logbuch schreibt nur alle 15 Minuten in den Flash-Speicher; ohne diesen Aufruf stünde die Uhr nach dem Abschalten ohne Standort da, obwohl sie ihn Sekunden vorher noch kannte.

Zweitens liest die Logzeile den Zustand zurück und schreibt bei Misserfolg ausdrücklich `NOCH AN (Fehler)` . Ein Abschaltbefehl, der nicht ankommt, meldet sich damit selbst — statt sich als unerklärlich kurze Akkulaufzeit zu tarnen.

Drittens steht dieser Code im **Haupt-Task**, nicht im Kachel-Callback. Der Grund steht in Kapitel 13: Der Kachel-Callback läuft im Anzeige-Task, und das Wiedereinschalten blockiert bis zu mehreren Sekunden.

6.4 Wiedereinschalten ist nicht das Gegenteil von Ausschalten

Beim Einschalten genügt es **nicht**, die Schiene wieder anzulegen (`CR_ModuleToggle.cpp:61–64`):

> Nach dem Abschalten der Schiene ist das Modul in seinem Auslieferungszustand, die Baudraten-Erkennung und die UBLOX-Konfiguration müssen neu durch.

Ein Bauteil ohne Strom vergisst alles. Es kommt nicht in dem Zustand zurück, in dem es war, sondern in dem, in dem es das Werk verlassen hat. Deshalb ruft der Einschaltweg die **vollständige** Startroutine `WZ_GPS_Init()` auf und nicht etwa nur `pmu.enableBLDO1()`.

Dasselbe gilt beim Funkmodul: `CR_LoRaCtl` ruft beim Einschalten `WZ_LoRa_Init()`, das Sync-Wort und Präambel neu setzt (Kapitel 21) — Werte, ohne die die Uhr im selben Netz taub wäre.

6.5 Die Regel für alles Weitere

> **Ein Modul abzuschalten heißt: die Schiene kappen.** Ein Flag zu setzen ist nur die halbe Handlung — es bringt die Software zum Schweigen, nicht die Hardware.
>
> Und umgekehrt: **Nach dem Kappen der Schiene ist beim Einschalten die vollständige Startroutine zu durchlaufen**, nicht nur die Spannung wieder anzulegen.

Diese Regel gilt für jedes Bauteil an einer schaltbaren Schiene. Beim Funkmodul ist sie umgesetzt (`CR_LoRaCtl_PowerOff()` kappt ALDO4), beim GPS seit dem 3. August. Der Vibrationsmotor hängt dauerhaft an BLDO2 — ob sich ein Abschalten dort lohnt, wurde bislang nicht gemessen. Das ist eine offene Frage, keine getroffene Entscheidung.

7 SPI: Display und Funkmodul dürfen nicht auf derselben Peripherie liegen

Dieses Kapitel beschreibt einen Fehler, der **jahrelang im Quelltext stand, ohne sich zu zeigen** — und der genau in dem Moment aufbrach, in dem an einer ganz anderen Stelle etwas richtig gemacht wurde. Es ist damit weniger ein SPI-Kapitel als ein Lehrstück darüber, wie Nebenläufigkeit alte Fehler freilegt.

7.1 Der Ausgangszustand

Der ESP32-S3 stellt zwei frei nutzbare SPI-Peripherien bereit. Die Arduino-Schicht spricht sie über zwei Namen an, die nicht das bedeuten, wonach sie aussehen:

Arduino-Name	Arduino-Bus	Peripherie des S3
`FSPI`	0	**SPI2**
`HSPI`	1	**SPI3**

Die Zuordnung steht in `esp32-hal-spi.h:36`. Sie ist die erste Falle: Die Namen stammen aus der Zeit des ursprünglichen ESP32, wo `HSPI` und `VSPI` andere Nummern trugen.

In der Firmware trafen nun zwei Bauteile aufeinander:

- **Das Display.** `platformio.ini:42` setzt `-D USE_HSPI_PORT`. In `TFT_eSPI_ESP32_S3.h:73` wird daraus `#define SPI_PORT 3` — das Display liegt auf **SPI3**.
- **Das Funkmodul.** In `WZ_LoRa.cpp` stand `SPIClass radioSPI(HSPI)` — und `HSPI` ist ebenfalls **SPI3**.

Beide Bauteile benutzten dieselbe SPI-Peripherie. Der Kommentar über der Zeile behauptete dabei ausdrücklich, es handle sich um einen *eigenen* Bus für das Funkmodul; er beschrieb also einen Zustand, den der Code darunter nicht herstellte.

7.2 Warum es trotzdem lief

Zwei Bauteile an einer Peripherie sind kein Fehler, solange nie zwei Zugriffe gleichzeitig stattfinden. Und genau das war lange sichergestellt — allerdings aus einem Grund, den niemand so entworfen hatte.

Main-Task und Render-Task lagen auf **demselben Kern** (die vollständige Geschichte dazu steht in Kapitel 14). Auf einem Kern kann echte Gleichzeitigkeit nicht auftreten: Der FreeRTOS-Scheduler serialisierte die beiden Zugriffe zwangsweise. Der Fehler war vorhanden, aber unerreichbar.

> **Der Konflikt war nicht behoben, sondern verdeckt** — und zwar durch einen zweiten Fehler, der die Firmware an anderer Stelle massiv ausbremste.

7.3 Wie er aufbrach

Mit der Kerntrennung (`boards/LilyGoWatch-S3.json`, `ARDUINO_RUNNING_CORE=0`) laufen `WZ_LoRa_Init()` im `setup()` auf Kern 0 und der Display-Flush auf Kern 1 **wirklich parallel**. Damit war die schützende Serialisierung fort, und der Zugriffskonflikt wurde zum ersten Mal erreichbar.

Das Fehlerbild: Der Renderpfad blieb in `TFT_eSPI::pushSwapBytePixels()` stehen, bis der Task-Watchdog das Gerät neu startete.

Gemessen am Gerät (`WZ_LoRa.cpp:44-45`):

Fassung	Watchdog-Neustarts
`radioSPI(HSPI)` — gemeinsame Peripherie	**3 von ~12 Startvorgängen**
`radioSPI(FSPI)` — getrennte Peripherie	**0 von 12**

Aufschlussreich ist nicht nur die Zahl, sondern der **Zeitpunkt**: Jeder der drei Neustarts erfolgte unmittelbar nach der Logzeile

```
[PWR] ALDO4 (LoRa SX1262): eingeschaltet
```

also genau dann, wenn das Funkmodul erstmals angesprochen wurde. Diese Zuordnung war der Schlüssel zur Diagnose — ein Watchdog-Neustart allein zeigt in keine Richtung, ein Watchdog-Neustart immer an derselben Stelle des Startprotokolls sehr wohl.

7.4 Die Behebung

`WZ_LoRa.cpp:49`:

```
static SPIClass radioSPI(FSPI);
```

`FSPI` ist Arduino-Bus 0 und damit SPI2 — und das ist auf dieser Uhr ohnehin der sachlich richtige Bus: Die vier Funkpins (SCK 3 / MISO 4 / MOSI 1 / CS 5, `variants/t-watch-S3-plus/pins_arduino.h:72-75`) sind genau die FSPI-Standardpins des S3. Die ursprüngliche Zeile hatte das Funkmodul also nicht nur auf die belegte Peripherie gelegt, sondern zugleich von seiner eigenen weggenommen.

⚠️ Der Quelltextkommentar an dieser Stelle (`WZ_LoRa.cpp:48`) verweist noch auf `pins_arduino.h:189-193`; die Definitionen sind seither nach oben gewandert. Das ist die übliche Schwäche von Zeilenverweisen im Fließtext — der Verweis auf den **Namen** (`BOARD_RADIO_SCK`) hält, der auf die Zeile nicht.

Der Bus wird in `WZ_LoRa_Init()` als zweiter Schritt gestartet, direkt nach der Stromschiene und vor dem ersten Chipzugriff (`WZ_LoRa.cpp:103`):

```
radioSPI.begin(BOARD_RADIO_SCK, BOARD_RADIO_MISO, BOARD_RADIO_MOSI, BOARD_RADIO_SS);
dbLOG("[LoRa] SPI: SCK=%d MISO=%d MOSI=%d CS=%d\n", …);
```

Die Logzeile nennt die tatsächlich verwendeten Pins. Das ist Absicht: Eine Pinbelegung, die nur im Header steht, lässt sich beim Fehlersuchen nicht gegen die Wirklichkeit prüfen.

7.5 ⚠️ Was daraus für alles Weitere folgt

Der Befund taugt schlecht als Einzelrezept („Funkmodul auf FSPI legen") und gut als Regel:

> **Ein Fehler, der von einem anderen Fehler verdeckt wird, tritt bei dessen Behebung gemeinsam mit ihr auf.** Wer eine Bremse löst, muss damit rechnen, dass danach Dinge brechen, die vorher nie an die Reihe kamen — und darf den neuen Ausfall nicht der Behebung anlasten.

Praktisch heißt das für dieses Projekt:

1. **Nach jeder Änderung an der Nebenläufigkeit** (Kernzuordnung, Task-Prioritäten, Taktraten) ist mit neu sichtbaren Altlasten zu rechnen. Dasselbe Muster traf zeitgleich die Bedienlogik — siehe Kapitel 16.
2. **Ein Kommentar ist kein Nachweis.** Über der fehlerhaften Zeile stand jahrelang, das Funkmodul habe einen eigenen Bus. Erst der Blick in `esp32-hal-spi.h` und `TFT_eSPI_ESP32_S3.h` zeigte, dass beide Namen auf dieselbe Zahl führen.
3. **Peripherie-Zuordnungen gehören ins Startprotokoll**, nicht nur in den Header.

Die Zeile trägt in der Fassung dieses Zweigs eine ausdrückliche Abweichungsmarkierung gegenüber `wolfgang/main` (`WZ_LoRa.cpp:33`), weil sie beim nächsten Zusammenführen sonst still zurückfiele — mit ihr käme der Watchdog-Neustart zurück.

8 Der Touchcontroller FT6336U

Der FT6336U der T-Watch-S3-Plus kann Wischgesten selbst erkennen — in Hardware, ohne dass die Firmware Koordinaten auswerten muss. Der Weg dorthin führt allerdings über vier Register, von denen keines in der Bedienungsanleitung des Chips steht, und über eine Betriebsbedingung, die nirgends dokumentiert ist.

Dieses Kapitel beschreibt beides. Es ist das Kapitel mit der höchsten Belegdichte im Buch, weil hier fast jede Aussage gegen ein Dokument steht, das etwas anderes behauptet oder schweigt.

8.1 Zwei Gesten-Mechanismen — und der naheliegende ist tot

Der Chip besitzt **zwei voneinander unabhängige** Gesten-Mechanismen:

Weg	Register	Zustand auf diesem Board
`GEST_ID` — der dokumentierte Weg	`0x01`	⚠️ **dauerhaft `0x00`**
„Special Gesture Mode"	`0xD0` / `0xD3`	funktioniert vollständig

`WZ_Touch` bedient den Chip über SensorLib (`TouchDrvFT6X36`) und liefert LVGL die Koordinaten. SensorLibs `getGesture()` liest `GEST_ID` (0x01) — und dieses Register bleibt auf diesem Board **dauerhaft 0x00**, empirisch über mehrere Messreihen belegt (`CR_Touch.h:9–10`). Über den dokumentierten Weg ist die Gestenerkennung des Chips also nicht erreichbar, und eine Bibliothek, die nur diesen Weg kennt, kann den zweiten prinzipiell nicht sehen.

Der zweite Mechanismus steht **weder im Datenblatt noch in der CTPM Application Note**, sondern ausschließlich im Register-Dokument (`documents_original/FT6336U_reg.pdf`):

```
0xD0  ID_G_SPEC_GESTURE_ENABLE   Hauptschalter (1 = ein) — ab Werk 0
0xD1/0xD2, 0xD5–0xD8             Freigabe der Einzelgesten — ab Werk 0xFF
0xD3                             Gesten-Ausgabe  ← hier kommen die Codes heraus
```

Deshalb existiert `CR_Touch` als eigenes Modul neben `WZ_Touch` und greift bewusst an SensorLib vorbei direkt über `Wire1` auf den Chip zu. Die ausgelesenen Codes (`CR_Touch.h:41–46` , alle am Gerät verifiziert):

Code	Geste
`0x20` / `0x21`	Wisch links / rechts
`0x22` / `0x23`	Wisch hoch / runter
`0x24`	Doppeltipp

8.2 ⚠️ Der Hauptbefund: dem Chip fehlt die Panelgröße

Die vier Register `0x98` – `0x9B` halten die Panelauflösung. **Ab Werk stehen sie auf `0x00`** — dem Chip wurde nie gesagt, wie groß das Panel ist.

Die Folge ist keine Ungenauigkeit, sondern ein Totalausfall einer Achse (`CR_Touch.cpp:163–167`):

> Ohne diese vier Bytes ist die **komplette Y-Achse** der Gesten-Engine tot. X funktioniert trotzdem — daher die verwirrende Asymmetrie.

Sauber isoliert gemessen, bei sonst **exakt denselben** Schwellwerten:

Zustand	Ergebnis
ohne `0x98` – `0x9B`	**0 von 6** Hoch/Runter-Wischen erkannt
mit `0x98` – `0x9B`	**16 Gesten in 22 s**

Der Eintrag selbst ist unspektakulär (`CR_Touch.cpp:168–169`):

```
cr_write(0x98, 0x00); cr_write(0x99, 0xF0); // MAX_X = 240
cr_write(0x9A, 0x00); cr_write(0x9B, 0xF0); // MAX_Y = 240
```

⚠️ **Diese Abhängigkeit steht in keinem der vier FocalTech-Dokumente.** Sie war nur durch Messen zu finden. Wer den Chip an einem anderen Panel in Betrieb nimmt, hat hier die erste Stelle zu prüfen — insbesondere dann, wenn *eine* Achse geht und die andere nicht.

Direkt darunter stehen die Schwellwerte, die ab Werk ebenfalls auf `0x00` stehen und in dieser Stellung **unerfüllbar** sind (`CR_Touch.cpp:172–173`): maximale Querabweichung 50 px (`0x92` / `0x93`), minimaler Wischweg 25 px (`0x94` / `0x95`).

8.3 Die Reihenfolge ist zwingend — der Chip überlebt den Reset

Die Konfiguration folgt einem festen Dreischritt: **Engine aus → konfigurieren → Engine an.** Das ist keine Stilfrage (`CR_Touch.cpp:155–159`):

> Der FT6336U behält seinen Zustand über einen ESP32-Reset und ein Neu-Flashen hinweg — nur ein echter Power-Cycle setzt ihn zurück. Läuft die Engine noch von einem früheren Lauf (`0xD0 = 1`), wird eine neue Auflösung bei laufender Engine **nicht übernommen**, und die Y-Achse bleibt tot.

Daraus folgt eine Eigenschaft, die beim Fehlersuchen leicht in die Irre führt: **Der Chip ist nicht Teil des Neustarts.** Eine Firmware, die den Chip falsch konfiguriert hat, kann nach dem Flashen einer korrigierten Fassung weiterhin falsch laufen — bis das Gerät einmal wirklich stromlos war. Ein Fehlerbild, das „nur manchmal" auftritt und sich einem Neustart widersetzt, ist hier zuerst zu vermuten.

Belegt wurde das am 15. Juli 2026 an OE3WAS' eigenem Sketch.

8.4 Die Engine wird vom Host getaktet

Der zweite nicht dokumentierte Befund betrifft nicht ein Register, sondern den Betrieb:

> **Die Gesten-Engine läuft nur weiter, wenn der Host den Touch-Frame (`0x02` – `0x06`) abholt.** Nur dann fällt auch `0xD3` zwischen zwei Wischern wieder auf `0x00` zurück.

Der Abtasttakt ist damit keine Feinheit, sondern die Betriebsbedingung (`CR_Touch.cpp:85–88`). Eine Schleife, die ausschließlich `0xD3` pollt, liefert **45 s lang gar nichts** — A/B belegt.

Am Arduino- `loop()` war das nicht zu halten. Zum Zeitpunkt der Inbetriebnahme lief dieser `loop()` mit **2 Hz** (Kapitel 14 erklärt, warum) — rund sechzigmal zu langsam. Das Fehlerbild war entsprechend diffus: sporadische Gesten (~65 %), verschluckte Wiederholungen, praktisch tote Y-Achse — während dieselbe Logik in einem Standalone-Sketch mit 8-ms-Takt tadellos lief.

> Der Satz **„standalone geht es, in der Firmware nicht"** ist in diesem Projekt zweimal aufgetreten und meinte beide Male dasselbe: nicht die Logik war anders, sondern der Takt. Er gehört als Erstes gemessen, nicht als Letztes.

Die Lösung ist ein eigener FreeRTOS-Task mit festem 8-ms-Takt auf Kern 0 (`CR_Touch.cpp:99` , gestartet in `:199`), damit der ohnehin belastete Render-Kern 1 frei bleibt; die Last ist minimal — ein paar I²C-Bytes alle 8 ms. Deshalb hat dieses Modul bewusst **kein** `CR_Touch_Loop()` .

Dass der Frame zusätzlich zu SensorLib gelesen wird, nimmt LVGL nichts weg: Es sind reine Statusregister, kein FIFO (`CR_Touch.cpp:101–104` , hardwarebelegt — `PRESSED` / `RELEASED` bleiben sauber). SensorLib liest denselben Frame nur alle ~30 ms, und das reicht der Y-Achse nicht.

8.5 ⚠️ Drei Eingriffe, die den Chip beschädigen

Alle drei wurden versucht, alle drei haben geschadet.

`0xD3` niemals beschreiben. Beide Versuche, den Latch aktiv zu löschen — nach jedem Lesen wie auch beim Aufsetzen des Fingers — haben die Zustandsmaschine gestört: Die Vertikale brach auf 0 von 9 ein. Nötig ist es ohnehin nicht, das Register räumt sich beim Frame-Lesen selbst ab (`CR_Touch.cpp:110–113`).

Den Flankenspeicher nicht beim Loslassen zurücksetzen. Naheliegend, um schnelle Wiederholungen derselben Richtung sicher zu erwischen — aber `0xD3` ist gelatcht und hält einen **alten** Wert. Ein Reset entdeckt genau diesen alten Wert neu und meldet eine **falsche** Geste; am 14. Juli belegt: Ein Runter-Wisch meldete Hoch (`CR_Touch.cpp:115–121`).

Die „keep"-Bits in `0xD1` nicht löschen. Bit 7 und Bit 6 sind im Register-Dokument mit *keep* markiert. Werden sie gelöscht (etwa durch `0x1F`), legt das die Gesten-Engine **komplett** lahm — und dieser Zustand überlebt den ESP32-Reset, siehe 8.3 (`CR_Touch.cpp:176–178`). Deshalb steht dort der Werkswert `0xFF` .

Hinzu kommt eine Einschränkung, die bewusst offen blieb: Zwei sehr schnell aufeinander folgende gleiche Wische können zu einem verschmelzen. Eine saubere Entprellung des gelatchten Registers gehört in die Navigationsschicht, nicht in die Erkennung.

8.6 Fehlschläge sind nicht tödlich

`CR_Touch_Init()` liest `0xD0` zurück und setzt `on_CRTOUCH` nur bei bestätigtem Erfolg (`CR_Touch.cpp:187–195`). Bei Misserfolg wird geloggt und zurückgekehrt — **kein Halt**: Die Chip-Gesten sind optionaler Komfort, die LVGL-Gestenerkennung in `WZ_Touch` trägt die Navigation weiterhin.

Die Erfolgsmeldung liest die Konfiguration ebenfalls zurück, statt sie zu behaupten:

```
[CRTOUCH] Gesture Mode aktiv. Auflösung 0x98–0x9B = 00 F0 00 F0 (Soll 00 F0 00 F0)
```

Diese Zeile ist der schnellste Weg, den Zustand aus 8.3 zu erkennen: Stehen dort Nullen, lief die Engine beim Konfigurieren noch — und das Gerät braucht einen echten Power-Cycle, kein weiteres Flashen.

9 Das Display-Panel: BGR und INVON

Das ST7789-Panel der T-Watch-S3-Plus braucht zwei Einstellungen, die beide das Gegenteil dessen sind, was der jeweilige Name nahelegt. Beide wurden zunächst falsch gesetzt, beide lagen übereinander — und beide waren in einem einzigen Flash-Zyklus zu trennen, sobald das richtige Messmittel eingesetzt wurde.

Das Messmittel ist der eigentliche Inhalt dieses Kapitels.

9.1 Das Fehlerbild

Die Erst-Abnahme des Display-Backends scheiterte an falschen Farben. Christians Befund lautete schlicht: **„alles Blaue orange"**.

Das ist ein typisches Fehlerbild dieser Art — es zeigt in keine Richtung. „Orange statt blau" kann von einer vertauschten Kanalreihenfolge kommen, von einer Farbinversion, von einer falschen Bit-Reihenfolge im Puffer oder von einer Kombination daraus. Raten führt hier zu einer langen Reihe von Flash-Zyklen, weil jede Einzelmaßnahme das Bild verändert, ohne es richtig zu machen.

9.2 Der 5-Farben-Boot-Test

Statt zu raten, wurde beim Start eine feste Folge von fünf Vollflächen ausgegeben — **ROT, GRÜN, BLAU, CYAN, WEISS**, je drei Sekunden. Diese fünf genügen, um die vollständige Wahrheitstabelle des Farbpfads abzulesen:

Beobachtung am Panel	Schlussfolgerung
ROT → Blau, BLAU → Rot, GRÜN → Grün	reiner R/B-Tausch (Kanalreihenfolge)
WEISS → Weiß	**keine** Inversion
WEISS → Schwarz	Inversion
CYAN → korrekt	Byte-Swap im Puffer stimmt

Der Test kostet einen Flash-Zyklus und liefert Kanal-Permutation **und** Inversion getrennt voneinander. Genau das war nötig, denn es lagen zwei Defekte übereinander.

> **Farbfragen werden auf der Hardware gemessen, nicht aus Treiber-Parität abgeleitet.**

9.3 Befund 1: Die Inversion war aktiv abgeschaltet worden

Das IPS-Panel braucht `INVON`. Die ST7789-Startsequenz von TFT_eSPI sendet dieses `INVON` bereits von sich aus — der Aufruf `invertDisplay(COLOR_INV)` mit `COLOR_INV = false` (`variants/t-watch-S3-plus/pins_arduino.h:32`) hat es anschließend wieder **abgeschaltet**. Ergebnis: Negativfarben.

Die Ursache liegt in einem Bedeutungsunterschied zwischen den beiden Display-Bibliotheken, den der Name `COLOR_INV` nicht abbildet:

- **Arduino_GFX** wurde mit `ips = true` betrieben (`main.cpp:207`) und sendete effektiv `ips XOR COLOR_INV` — also einen **relativen** Wert.
- **TFT_eSPI** erwartet an dieser Stelle den **Absolutwert**.

Derselbe Wert `COLOR_INV = false` bedeutete in der einen Bibliothek „invertiert" und in der anderen „nicht invertiert". Die Behebung steht in `main.cpp:473`:

```
tft.invertDisplay(!COLOR_INV);
```

9.4 Befund 2: `TFT_RGB_ORDER=1` bedeutet RGB

Der zweite Defekt war die Kanalreihenfolge. Das Panel ist ein **BGR**-Panel; eingestellt war RGB. Die Falle steckt in der Semantik der Einstellung selbst:

> In TFT_eSPI bedeutet `TFT_RGB_ORDER=1` **RGB** und `TFT_RGB_ORDER=0` **BGR** (= `TFT_MAD_BGR`).

Das ist kontraintuitiv genug, dass die vorbereitende Recherche zu dieser Phase sie genau falsch herum wiedergab: `01-RESEARCH.md`, Pitfall 1, riet ausdrücklich, `TFT_RGB_ORDER=1` „nicht wegzulassen" — und lag damit inhaltlich daneben. Der Farbtest auf der Hardware entschied die Frage in einem Durchgang.

Die geltende Einstellung steht in `platformio.ini:104`, samt Warnung im Kommentar:

```
-D TFT_RGB_ORDER=0 ; =TFT_MAD_BGR! TFT_eSPI-Semantik kontraintuitiv (1=RGB, 0=BGR);
                    ; Panel braucht BGR – hardware-verifiziert per Boot-Farbtest 2026-07-23
```

Eine Randnotiz mit Lehrwert: TFT_eSPIs `CGRAM_OFFSET`-Automatik hätte ohne das Define von selbst BGR gewählt. Der Fehler entstand also nicht durch eine fehlende Einstellung, sondern durch eine **überflüssige, falsch verstandene**.

9.5 ⚠️ Der Altzustand war keine gültige Referenz

Der naheliegende Prüfgedanke — „vorher unter Arduino_GFX sahen die Farben doch richtig aus, also stimmte es dort" — ist in diesem Fall falsch.

Auch Arduino_GFX sendete `MADCTL_RGB` (`0x00`) auf ein BGR-Panel. Der R/B-Tausch war also die ganze Zeit vorhanden. Er fiel nur niemandem auf, weil die damalige, aus EEZ Studio generierte Oberfläche durchgehend **blau/grau** gehalten war: In einem Bild ohne kräftige Rot- und Blauflächen nebeneinander ist ein vertauschter Rot- und Blaukanal unsichtbar.

> **Ein „funktionierender" Altzustand belegt nur, dass der Fehler im bisherigen Bildinhalt nicht sichtbar war** — nicht, dass er nicht vorhanden ist. Als Referenz für eine Umstellung taugt er deshalb nicht.

9.6 Der Stand

Einstellung	Wert	Belegstelle
Kanalreihenfolge	`TFT_RGB_ORDER=0` (BGR)	`platformio.ini:104`
Inversion	`invertDisplay(!COLOR_INV)` → INVON	`main.cpp:473`
`COLOR_INV`	`false`	`pins_arduino.h`, beim Block `LCD_WIDTH` / `LCD_HEIGHT` (Zeile 38)
Panelgröße	240 × 240	`pins_arduino.h:30-31`

Abgenommen am 23. Juli 2026 auf der Hardware, gemeinsam mit allen vier Rotationsstufen (`CR_Rotation`), ohne Versatz oder Randstreifen.

10 Die Pins, die etwas anderes sind als sie scheinen

Ein Pin trägt in diesem Projekt bis zu drei Namen: den des Bauteils, an dem er wirklich hängt, den eines Bauteils, das auf einem *anderen* Board dort hängt, und den, den ein `#define` ihm im Quelltext gibt. Nur der erste ist verbindlich — und er steht nicht im Header, sondern auf der Leiterplatte.

Dieses Kapitel sammelt die Stellen, an denen diese drei auseinanderfallen.

10.1 Der Fall Pin 44: GPS empfing Rauschen

Die GPS-Inbetriebnahme scheiterte über längere Zeit an einem Fehlerbild, das nach einem defekten Modul aussah: Die Baudratenerkennung meldete `50 Flanken` und daraus den Fantasiewert **333333 Baud**.

Die Erklärung steht in `variants/t-watch-S3-plus/pins_arduino.h:149-152`:

> Vorher standen hier `RX=44` / `TX=43` neben ungenutzten `SHIELD_GPS_TX` / `RX` (42/41). Pin 44 ist aber zugleich `BOARD_MIC_CLOCK` — die Baudratenerkennung maß daher nur Rauschen.

Pin 44 ist auf dieser Uhr die **Taktleitung des PDM-Mikrofons** (`pins_arduino.h:144`). Der Erkenner hat also nicht etwa nichts gemessen, sondern etwas Falsches: ein Taktsignal, das sich wie eine sehr hohe Baudrate ausnimmt. Ein Ergebnis von „50 Flanken" ist dabei verräterischer als gar kein Ergebnis — es beweist, dass an dem Pin *etwas* liegt, und lenkt den Verdacht damit vom Pin weg auf das Modul.

Richtig sind die Pins, die im Header als vermeintlich ungenutzte `SHIELD_GPS_*` danebenlagen (`pins_arduino.h:153-154`):

```
#define GPS_TX_PIN      42
#define GPS_RX_PIN      41
```

⚠️ **Die entscheidende Beobachtung:** `HAS_MIC` ist in diesem Build **auskommentiert** (`pins_arduino.h:139`). Es lief also gar kein Mikrofontreiber, der den Pin hätte belegen können. Trotzdem war der Pin unbrauchbar — denn:

> **Ein `#define` abzuschalten trennt keine Leiterbahn.** Ein Pin ist mit dem verbunden, womit die Platine ihn verbindet, unabhängig davon, ob die Firmware davon weiß.

Das ist dieselbe Denkfigur wie bei der Stromschienen-Falle in Kapitel 6: Dort brachte ein `false` die Software zum Schweigen, nicht die Hardware; hier verschweigt ein auskommentiertes `#define` eine Verdrahtung, die weiterbesteht.

Der GPS-Fehler hatte im Übrigen **zwei** Ursachen gleichzeitig — die falschen Pins und die nie eingeschaltete Stromschiene BLDO1 (Kapitel 5). Solange beide bestanden, konnte keine Einzelmaßnahme das Modul zum Sprechen bringen, was die Suche erheblich verlängerte.

10.2 Nachtrag: die feste Baudrate

Nach der Pinkorrektur lieferte die Baudratenerkennung in einer Messreihe **9 von 10** Mal exakt 38400 — und scheiterte im zehnten Boot vollständig. Seit dem 31. Juli 2026 steht deshalb ein fester Wert (`pins_arduino.h:163`):

```
#define GPS_FIXED_BAUD  38400
```

Gegenmessung danach: **10 von 10** Boots erfolgreich. Ein Zeitgewinn war das nicht — die Erkennung war schnell, solange das Modul antwortete; sie kostete nur im Fehlschlagfall Zeit, weil sie in ihren Timeout lief. Gewonnen wurde **Determinismus**.

⚠️ Beim Einsatz eines anderen GNSS-Moduls (etwa L76K) ist die Zeile auszukommentieren, dann greift wieder `detectBaudrate()`.

Diese feste Baudrate hatte eine Nebenwirkung, die erst Monate später sichtbar wurde: Der Baudratenerkenner war der **erste** `attachInterrupt()`-Aufrufer des Programms und damit Auslöser eines reproduzierbaren Absturzes. Mit `GPS_FIXED_BAUD` entfiel er, der Absturz galt als verschwunden — und kehrte zurück, sobald das Funkmodul einen neuen `attachInterrupt()` mitbrachte. Die Geschichte steht ausführlich im Kopf von `WZ_LoRa.cpp:64-86`.

10.3 Namen aus anderen Boards, die im Header stehengeblieben sind

Der Variant-Header führt Definitionen mit dem Präfix `WS_` — sie stammen vom Waveshare-Board und beschreiben **nicht** die T-Watch. Zwei davon widersprechen der tatsächlichen Belegung sogar direkt:

Definition	Wert	Tatsächlich auf der T-Watch
`WS_RTC_SCL`	10	10 ist `BOARD_I2C_**SDA**`
`WS_RTC_SDA`	11	11 ist `BOARD_I2C_**SCL**`
`WS_RTC_INT`	39	39 ist `BOARD_TOUCH_SDA`

Die beiden ersten sind gegenüber der echten Belegung **vertauscht**. Wer sie benutzt, verdrahtet den I²C-Bus verkehrt herum.

Benutzt werden sie nicht: `CR_RTC.cpp:31` übernimmt aus dieser Gruppe ausschließlich die Adresse und nimmt die Pins von der richtigen Stelle:

```
bool ok = s_rtc.begin(Wire, WS_RTC_ADDRESS, BOARD_I2C_SDA, BOARD_I2C_SCL);
```

Das ist die richtige Wahl — aber sie ist nirgends erzwungen. Die falschen Definitionen stehen weiterhin im Header und sehen aus wie gültige Angaben.

> **Ein Wert im Variant-Header ist kein Nachweis, dass er für dieses Board gilt.** Vor der Verwendung eines Pins ist zu prüfen, ob er unter einem zweiten Namen bereits vergeben ist — und ob dieser zweite Name überhaupt zu diesem Board gehört.

10.4 Pins, die es nicht gibt

Zwei Display-Pins sind mit `-1` belegt (`pins_arduino.h:20` , `:25`):

Definition	Wert	Bedeutung
`BOARD_TFT_MISO`	–1	Das Panel liest nicht zurück
`BOARD_TFT_RST`	–1	Kein eigener Reset-Pin

`-1` heißt hier „nicht vorhanden”, nicht „noch einzutragen”. Beim Touchcontroller gilt dasselbe: `WZ_Touch.cpp:304` übergibt bewusst `-1` als Reset-Pin, weil die T-Watch keinen hat.

10.5 Die Prüfliste für einen neuen Pin

1. **Im Variant-Header nach dem Zahlenwert suchen**, nicht nach dem Namen — derselbe Pin kann unter mehreren Namen stehen (Pin 44 war `BOARD_MIC_CLOCK` , Pin 39 ist `BOARD_TOUCH_SDA` *und* `WS_RTC_INT`).
2. **Auskommentierte Blöcke mitlesen.** `HAS_MIC` war aus — die Verdrahtung blieb.
3. **Gegen die Hardwaredoku des Herstellers prüfen.** Für die Funkpins ist das ausdrücklich geschehen (`pins_arduino.h:60-61` , gegen LilyGos `docs/hardware/lilygo-t-watch-s3-plus.md`).
4. **Die verwendeten Pins beim Start protokollieren**, nicht nur definieren — siehe die SPI-Logzeile in Kapitel 7.4.
5. **Bei „das Bauteil antwortet nicht” zuerst Pin und Stromschiene prüfen**, erst danach den Treiber. Beide melden sich identisch, nämlich als Schweigen.

11 Zwei Kerne, zwei Aufgaben

Der ESP32-S3 hat zwei Rechenkerne. Diese Uhr nutzt beide — und die Art, wie sie das tut, ist die Voraussetzung dafür, dass sie überhaupt bedienbar ist.

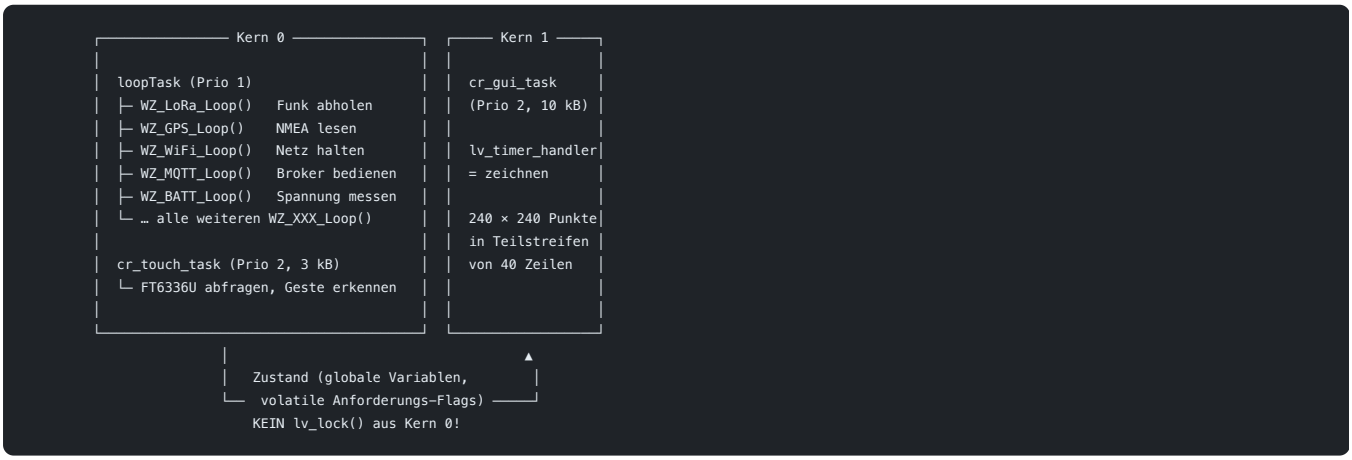
Das Kapitel erzählt zugleich den Fehler, der hier am längsten unentdeckt blieb, weil er als gelöstes Problem galt.

11.1 Die Aufteilung

Kern	Aufgabe	Priorität	Stapel	Wo festgelegt
Kern 0	Arduino-`loopTask` — `setup()`, `loop()`, alle `WZ_XXX_Loop()`	1	8 kB	`boards/LilyGoWatch-S3.json` (`ARDUINO_RUNNING_CORE=0`)
Kern 0	`cr_touch_task` — Finger abtasten, Gesten erkennen	2	3 kB	`CR_Touch.cpp`, `xTaskCreatePinnedToCore(…, 0)`
Kern 1	`cr_gui_task` — LVGL zeichnen	2	10 kB	`CR_GUI.cpp`, `xTaskCreatePinnedToCore(…, 1)`
Kern 1	Arduino-Ereignisse (WLAN-Rückrufe)	—	—	`boards/LilyGoWatch-S3.json` (`ARDUINO_EVENT_RUNNING_CORE=1`)

Dazu kommen die Aufgaben, die das Betriebssystem selbst verteilt und die hier niemand anheftet: der WLAN-Stapel, der Bluetooth-Host und die Zeitgeber von FreeRTOS.

Die Aufteilung als Bild



⚠️ **Warum die Eingabe auf Kern 0 liegt und nicht beim Zeichnen:** Der Finger muss auch dann abgetastet werden, wenn das Zeichnen gerade dauert. Läge beides auf einem Kern, würde ein aufwendiger Bildaufbau die Eingabe verschlucken — und ein verschluckter Wisch ist von einem hängenden Gerät nicht zu unterscheiden.

⚠️ **Die Pfeilrichtung unten im Bild ist eine Regel, keine Beschreibung:** Kern 0 **fordert** Bildschirmwechsel nur an (über einfache Variablen), ausgeführt werden sie im Anzeige-Task. Wer aus dem Hauptablauf heraus `lv_lock()` nimmt oder `lv_scr_load_anim()` aufruft, handelt sich den Watchdog ein — die ausführliche Begründung steht in Kapitel 13.

Der Gedanke dahinter: Das Zeichnen eines Bildschirms von 240×240 Bildpunkten ist Fließarbeit, die zuverlässig Zeit kostet. Der Funkempfang, die Positionsauswertung und die Netzverbindung dagegen sind auf regelmäßige Bedienung angewiesen — ein Paket, das zu spät abgeholt wird, ist verloren. Beides auf demselben Kern bedeutet, dass eines von beiden wartet.

11.2 ⚠️ Der Kernpunkt: die Trennung hängt an zwei Stellen

Dies ist die wichtigste Aussage des Kapitels, und sie hat das Projekt Monate gekostet.

```
xTaskCreatePinnedToCore(cr_gui_task, "LVGL", 10240, nullptr, 2, &_lvgl_task_handle, 1);
//                                                                                  ↑
//                                                  Anzeige-Task auf Kern 1
```

Diese Zeile stand **von Anfang an richtig**. Trotzdem lagen beide Tasks auf demselben Kern.

Der Grund: Wohin der Arduino-`loopTask` gehört, entscheidet diese Zeile nicht. Das entscheidet ein Übersetzungsschalter in der Board-Beschreibung:

```
"-DARDUINO_RUNNING_CORE=1",     ← so stand es bis zum 4. August 2026
```

Damit lag der `loopTask` **ebenfalls auf Kern 1** — genau dort, wo der Anzeige-Task hingeheftet wird. Und da der Anzeige-Task mit Priorität 2 gegen dessen Priorität 1 antritt, gewann er vollständig. Der Hauptablauf kam nur noch in den Pausen zum Zug, die der Anzeige-Task freiwillig ließ.

Gemessene Auswirkung: Der Hauptablauf brach bei eingeschaltetem Bildschirm auf **1 Hz** ein. Nach der Korrektur: **209–236 Hz**. Faktor 200.

> **Wichtig für den, der aus der gemeinsamen Grundlage übernimmt** (Stand 6. August 2026): **Im Zweig von OE3WAS steht weiterhin `-DARDUINO_RUNNING_CORE=1`.** Die Korrektur ist bisher nur in diesem Zweig erfolgt.
>
> Das ist dort **kein Fehler**, solange der Anzeige-Task nicht auf Kern 1 geheftet wird — beide Zeilen ergeben nur *zusammen* einen Sinn. Gefährlich wird es genau beim halben Übernehmen: Wer `xTaskCreatePinnedToCore(…, 1)` übernimmt und die Board-JSON stehen lässt, holt sich **exakt den Zustand, der hier beschrieben ist** — 1 Hz statt 200, und je nach Zeitverhalten den Watchdog aus Kapitel 7.
>
> **Es sind immer beide Stellen:** `CR_GUI.cpp` (Anheften des Anzeige-Tasks) **und** `boards/LilyGoWatch-S3.json` (Kern des `loopTask`).

11.3 Warum der Fehler so lange unbemerkt blieb

Weil die Projektdokumentation an vier Stellen behauptete, die Kerne seien getrennt — und weil das plausibel war. Die `xTaskCreatePinnedToCore()`-Zeile stand sichtbar da, mit der `1` am Ende — bei ihrer Lektüre bestand kein Anlass zu zweifeln.

> **Eine Messung war nie erfolgt.**

Drei Zeilen hätten genügt:

```
dbLOG("[CORE] main=%d gui=%d\n", xPortGetCoreID(), cr_gui_core_id());
```

Diese Zeile steht heute dauerhaft im Startprotokoll. Sie ist **kein Hilfsmittel aus der Fehlersuche, das nach Abschluss entfernt wird**, sondern ein Wachposten: Sie meldet bei jedem Start den tatsächlichen Zustand, nicht den angenommenen.

```
[CORE] main=0 gui=1        ← erwartet
```

Steht dort etwas anderes, ist die Trennung aufgehoben — egal wie die Quelltexte aussehen.

11.4 Die Regel für Nachfolger

> Die Kerntrennung hängt an **zwei** Stellen: an `CR_GUI.cpp:222` **und** an `boards/LilyGoWatch-S3.json`. Eine Änderung an einer der beiden erfordert die Prüfung der anderen.

Besonders heimtückisch ist die Richtung dieser Abhängigkeit. Die Board-Beschreibung ist eine Datei, die beim Einrichten einmal angelegt und danach kaum wieder geöffnet wird. Sie enthält Speicheraufteilung, Taktfrequenz, USB-Betriebsart — lauter Dinge, die mit der Aufgabenverteilung im Betrieb nichts zu tun zu haben scheinen. Dass dort eine Zeile steht, die über die Nutzbarkeit der gesamten Bedienoberfläche entscheidet, ist dort nicht zu vermuten.

11.5 Was aus der Trennung folgt

Sobald zwei Tasks tatsächlich gleichzeitig laufen, gelten Regeln, die zuvor entbehrlich waren:

Der Anzeigebaum gehört dem Anzeige-Task. Jeder Zugriff aus dem Hauptablauf braucht einen Mutex (`lv_lock()` / `lv_unlock()`). Welche Kosten das verursacht und welche Fehler dabei entstehen, steht in Kapitel 13.

Gemeinsam genutzte Hardware wird zum Problem. Solange die Tasks abwechselnd liefen, hat der Zeitplaner alle Zugriffe von selbst hintereinander gelegt. Mit echter Gleichzeitigkeit fällt diese unsichtbare Absicherung weg — und ein Fehler, der jahrelang unentdeckt schlief, wurde in 3 von 12 Starts zum Neustart. Diese Geschichte steht in Kapitel 7 (SPI).

Ein zu schneller Hauptablauf schadet auch. Nach der Korrektur lief er ungebremst mit 1000–4000 Hz und kostete den Anzeige-Task 126 Aussetzer in 29 Sekunden. Warum das so ist und was dagegen hilft, steht in Kapitel 15.

Die letzten beiden Punkte sind die eigentliche Lehre dieses Kapitels: **Eine Änderung an der Aufgabenverteilung ist nie eine lokale Änderung.** Sie deckt auf, was vorher von der Trägheit zusammengehalten wurde.

12 Warum LVGL einen eigenen Task bekam

LVGL lässt sich vollständig aus dem Arduino-`loop()` bedienen — ein `lv_timer_handler()` je Durchlauf genügt. Diese Firmware tut das nicht. Sie gibt der Anzeige einen eigenen FreeRTOS-Task auf dem zweiten Kern. Das kostet etwas, und dieses Kapitel handelt vom Preis.

12.1 Der Task

`CR_GUI.cpp:222`:

```
xTaskCreatePinnedToCore(cr_gui_task, "LVGL", 10240, nullptr, 2, &_lvgl_task_handle, 1);
```

Eigenschaft	Wert
Kern	**1**
Priorität	**2** (der Arduino-`loopTask` hat 1)
Stack	10240 Byte

Der Grund für die Trennung ist das Wesen der beiden Aufgaben: Die Anzeige braucht einen **gleichmäßigen** Takt, sonst ruckelt der Sekundenzeiger sichtbar. Der Main-Loop dagegen bedient Funk, Netz und Sensoren, deren Bearbeitungszeit stark schwankt. In einem gemeinsamen Durchlauf überträgt sich jede Schwankung unmittelbar auf das Bild.

12.2 ⚠️ Kern 1 ist nur die halbe Miete

Die obige Zeile stand **von Anfang an richtig** — und trotzdem lagen beide Tasks auf demselben Kern. Der Kommentar darüber (`CR_GUI.cpp:212–221`) nennt den Grund:

> Der Arduino-`loopTask` lag wegen `-DARDUINO_RUNNING_CORE=1` in `boards/LilyGoWatch-S3.json` **ebenfalls** auf Kern 1 — und dieser Task hat mit Priorität 2 gegen dessen Priorität 1 gewonnen. Ergebnis: Der Main-Loop kam nur noch während des `vTaskDelay()` dran und brach bei eingeschaltetem Schirm auf **1 Hz** ein.

> **Die Kerntrennung hängt an zwei Stellen**, nicht an einer: hier **und** am Board-JSON (dort steht jetzt `ARDUINO_RUNNING_CORE=0`). Wer eine davon ändert, muss die andere mitdenken.

Deshalb gibt es die `[CORE]`-Zeile im Startprotokoll (Kapitel 27.2). Sie meldet den **tatsächlichen** Zustand — erwartet wird `main=0 gui=1`. Die vollständige Fallgeschichte steht in Kapitel 14.

12.3 Der Preis: jeder Zugriff braucht den Mutex

Sobald der Widget-Baum von einem eigenen Task bearbeitet wird, ist er ein geteiltes Betriebsmittel. Jeder Zugriff aus dem Main-Task muss ihn sperren (`CR_GUI.h:13`):

```
lv_lock();
updateStatusLabels();
lv_unlock();
```

Das betrifft **jede** Stelle, die ein Widget anfasst — Statuszeilen, LEDs, Textfelder, Bildschirmwechsel. Es ist der wesentliche Aufwand dieser Architektur, und er ist nicht optional: Ein ungesicherter Zugriff während einer laufenden Renderiteration beschädigt den Baum.

⚠️ **Ein zusätzliches Sperren innerhalb der Renderschleife wäre falsch** — `lv_timer_handler()` nimmt die Sperre bereits selbst (`lv_timer.c`, `lv_lock()` Zeile 81, `lv_unlock()` Zeile 144).

12.4 Wo bewusst *nicht* gesperrt wird

Nicht jeder Zugriff braucht die Sperre. Einige Funktionen des Moduls sind ausdrücklich aus dem Main-Task **ohne** `lv_lock()` aufrufbar (`CR_GUI.h:88–111`) — nach dem Muster „Tearing ist hier hinnehmbar": Sie setzen einfache Werte, deren kurzzeitige Inkonsistenz sich höchstens als ein einzelnes falsch gezeichnetes

Bild äußert, nie als beschädigter Baum.

Das ist eine bewusste Abwägung, keine Nachlässigkeit — und sie ist an der jeweiligen Funktion dokumentiert, nicht dem Aufrufer überlassen.

12.5 ⚠️ Was im Main-Loop nichts zu suchen hat

Die gefährlichste Verwechslung: Der Mutex allein macht einen Zugriff noch nicht unbedenklich. Ein **langer** Vorgang unter gehaltener Sperre blockiert den Render-Task vollständig.

`lv_scr_load_anim()` unter `lv_lock()` im Main-Loop hat drei Watchdog-Neustarts gekostet. Das richtige Muster — ein Anforderungs-Flag, das der GUI-Task selbst abarbeitet — steht in Kapitel 13.

12.6 Der Watchdog

Der Task ist beim Task-Watchdog angemeldet und quittiert **jede** Iteration, auch im Drosselbetrieb (`CR_GUI.cpp:196–204`). Die Quittung steht bewusst **nach** einer vollständig durchlaufenen Iteration:

> Ein echter Hänger — etwa innerhalb `lv_timer_handler()` — darf nicht durch ein verfrühtes Quittungssignal verschleiert werden.

Bleibt die Quittung aus, löst der Watchdog einen Neustart aus; die Ursache landet beim nächsten Start als `ESP_RST_TASK_WDT` in der Absturzauswertung (`CR_Crash.cpp`). Genau so wurden die SPI-Neustarts aus Kapitel 7 sichtbar.

12b Die Bedienoberfläche entsteht im Quelltext

12b.1 Der gewählte Weg

Sämtliche Bildschirme dieser Firmware werden **unmittelbar im C++-Quelltext aufgebaut** — je Bildschirm eine Datei nach dem Muster `CR_<Name>Screen.cpp`, die ihre LVGL-Objekte selbst erzeugt, platziert, gestaltet und mit Ereignisbehandlung versieht.

Der Grund für diese Wahl ist die feine Steuerbarkeit. Ein Entwurfswerkzeug beschreibt, was auf dem Bildschirm steht; der Quelltext beschreibt zusätzlich, **unter welchen Bedingungen** es dort steht und **wie** es dorthin gelangt. Auf einem Gerät mit 240 × 240 Bildpunkten, dessen Anzeige fast durchgehend von Messwerten abhängt, ist der zweite Teil der überwiegende.

Vier Eigenschaften ergeben sich unmittelbar daraus:

Bildpunktgenaue Platzierung mit nachvollziehbarer Begründung. Jede Koordinate steht neben dem Grund, aus dem sie gewählt wurde — Zeilenhöhen, Abstände und Trefferflächen sind im Quelltext kommentiert und dadurch beim Ändern überprüfbar.

Zustandsabhängige Darstellung ohne Umweg. Farbe, Text und Sichtbarkeit ergeben sich direkt aus den Betriebsgrößen. Die Kachel des Funkmoduls etwa unterscheidet drei Zustände — stromlos, empfangsbereit, sendebereit — und liest dafür `on_LORA` und die eingestellte Sendestufe. Solche Verknüpfungen sind der Regelfall, nicht die Ausnahme.

Eigene Zeichenverfahren, wo Widgets nicht reichen. Das Ziffernblatt zeichnet seine Zeiger als gefüllte Polygone auf eine `lv_canvas` (240 × 240, ARGB8888), weil LVGLs Linien-Widget keine veränderliche Strichbreite beherrscht und die verjüngte Zeigerform damit nicht darstellbar wäre (`CR_Watchface.cpp:184–185`). Die Neuzeichnung ist zusätzlich auf die Zeigerachse begrenzt statt auf die gesamte Fläche — eine Optimierung, die außerhalb des Quelltexts nicht formulierbar ist.

Änderungen sind lesbar nachvollziehbar. Eine Verschiebung um zwei Bildpunkte erscheint in der Versionsverwaltung als genau diese eine Zeile.

12b.2 Der Bildschirmverbund: `CR_ScreenMgr`

Die Bildschirme sind nicht fest verdrahtet, sondern melden sich bei einer zentralen Verwaltung an. Jeder Eintrag beschreibt, wohin eine Wischgeste in die jeweilige Richtung führt:

```
void CR_ScreenMgr_Register(CR_ScreenId id, const char *name, CR_ScreenGetRootFn getRoot,
                           CR_ScreenVisibleFn isVisible, CR_ScreenId ringLeft,
                           CR_ScreenId ringRight, CR_ScreenId vertUp, CR_ScreenId vertDown);
```

Damit steht die **gesamte Bedientopologie an einer Stelle** und ist beim Hinzufügen eines Bildschirms in einer Zeile erweiterbar. Das Startprotokoll führt jede Anmeldung mit und schließt mit einer Gesamtzahl ab:

```
[SCRMGR] Screen registriert: Watchface (id=0)
[SCRMGR] Screen registriert: Setup (id=1)
…
[SCRMGR] 19/19 Screens registriert
```

⚠ **Diese Abschlusszeile ist ein Wachposten.** Weicht die Zahl ab, fehlt ein Bildschirm — und ein nicht angemeldeter Bildschirm ist über Wischgesten unerreichbar, ohne dass beim Übersetzen etwas auffiele.

Über `isVisible` kann ein Bildschirm sich zeitweise aus dem Verbund nehmen, etwa wenn das zugehörige Modul nicht übersetzt wurde. Die Wischwege der Nachbarn bleiben dabei stimmig.

12b.3 `CR_EEZShim`: die Anbindung an die gemeinsame Grundlage

🔴 **Dieser Abschnitt gilt ausschließlich für den Zweig von OE3LCR.** `CR_EEZShim` existiert im Zweig von OE3WAS **nicht** — Wolfgang hat es dort bewusst entfernt:

> *„12b.3 wird hinfällig, wenn ich auch EEZ-Screens gleichzeitig handhaben will. Daher bei mir gelöscht, da es sonst Duplikate ergibt."* (OE3WAS, 6. August 2026)

Der Grund liegt auf der Hand, sobald man beide Wege nebeneinander hält: Wer die EEZ-Studio-Oberfläche **weiter benutzt**, hat die `objects`-Struktur bereits — vom Generator erzeugt. Ein zweites, handgeschriebenes `objects` daneben wäre kein Ersatz, sondern ein Doppelgänger, und der Übersetzer sähe jedes Feld zweimal.

Der Shim ist also kein Fortschritt, den OE3WAS noch nachholen müsste, sondern die Folge einer anderen Entscheidung: Dieser Zweig hat die erzeugte Oberfläche abgelöst (12b.1) und braucht deshalb einen Ersatz für das, was der Generator vorher mitlieferte. Wer beide Wege offenhalten will, braucht ihn nicht — er darf ihn sogar nicht haben.

Ein Teil der Firmware stammt aus der gemeinsamen Grundlage mit OE3WAS und greift auf eine `objects`-Struktur zu — etwa `WZ_NTP.cpp`, `WZ_UDP.cpp` und `WZ_Touch.cpp`. Damit diese Module unverändert übersetzt werden können, stellt `CR_EEZShim` genau die **16 tatsächlich erreichbaren Felder** bereit (`CR_EEZShim.h:56`).

Der Nutzen ist die **Zusammenführbarkeit**: Änderungen aus der gemeinsamen Grundlage lassen sich weiterhin übernehmen, ohne die Bildschirme dieses Zweigs anzufassen.

⚠️ **Zwei Eigenschaften sind dabei zwingend zu beachten:**

1. Der Shim stellt Widgets bereit, aber keine Ereignisbehandlung. Ein neu angelegter Bildschirm ist ohne eigenen `LV_EVENT_GESTURE`-Handler **für Wischgesten taub** — er wird angezeigt und lässt sich nicht mehr verlassen. Die Registrierung geschieht im jeweiligen `_Init()`; als Vorlage dient `CR_MsgScreen_Init()`.

2. Auf unbenannte Objekte darf nicht verwiesen werden. Bezeichner der Form `obj2`, `obj3`, … sind Positionsnummern, keine Namen. Sie verschieben sich bei jedem Umbau, und der Verweis zeigt anschließend stillschweigend auf ein anderes Widget — der Übersetzungslauf bleibt dabei fehlerfrei. Zu verwenden sind ausschließlich benannte Felder.

12b.4 Aufbau eines Bildschirms

Jeder Bildschirm folgt demselben Ablauf. Die Reihenfolge ist nicht beliebig:

```
void CR_XxxScreen_Init(void) {
  s_root = lv_obj_create(NULL);                  // 1. eigenständiger Bildschirm
  lv_obj_set_size(s_root, 240, 240);
  lv_obj_remove_flag(s_root, LV_OBJ_FLAG_SCROLLABLE);

  // 2. PFLICHT: Gestenbehandlung – ohne diese Zeile ist der Bildschirm eine Sackgasse
  lv_obj_add_event_cb(s_root, cr_xxxscreen_gesture_cb, LV_EVENT_GESTURE, NULL);

  CR_ScreenHeader_Create(s_root, "TITEL");        // 3. gemeinsame Kopfzeile

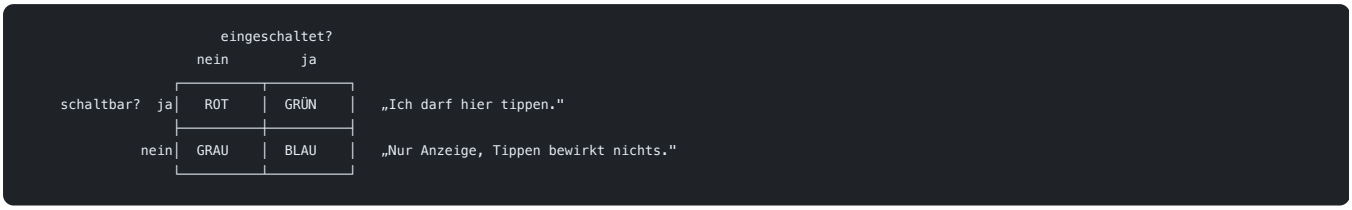
  // 4. Inhalt

  // 5. Anmeldung samt Wischwegen
  CR_ScreenMgr_Register(CR_SCREEN_XXX, "Xxx", CR_XxxScreen_GetRoot, NULL,
                        CR_SCREEN_LINKS, CR_SCREEN_RECHTS,
                        CR_SCREEN_HOCH, CR_SCREEN_RUNTER);
}
```

Ein eigener Gesten-Handler statt der unmittelbaren Registrierung des zentralen ist dann erforderlich, wenn der Bildschirm Sonderfälle kennt — etwa die Sperre der Wischnavigation während eines laufenden Firmware-Updates oder bei geöffneter Abfrage. Der zentrale Handler bleibt dadurch frei von Wissen über einzelne Bildschirme.

12b.4b Die vier Kachelfarben — eine Regel, zwei Aussagen

Die Farbe einer Modulkachel wird an **einer einzigen Stelle** vergeben (`cr_modulescreens_apply_tile_color()` in `CR_ModuleScreens.cpp`), und sie beantwortet zwei Fragen auf einmal:



Farbe	Bedeutung	Reaktion auf Antippen
GRÜN	schaltbar, eingeschaltet	schaltet aus
ROT	schaltbar, ausgeschaltet	schaltet ein
BLAU	nicht schaltbar, aber vorhanden/aktiv	keine — nur Detailseite
GRAU	nicht schaltbar und nicht vorhanden	keine

Der Gedanke dahinter: Das Farbenpaar sagt, ob die Kachel überhaupt ein Bedienelement ist. Rot und Grün heißen *„hier kann ich etwas tun"*, Blau und Grau heißen *„hier gibt es nur etwas zu sehen"*. Ein Modul, das die Hardware gar nicht hat, wird deshalb grau — nicht rot. Rot wäre die Einladung zu einem Tippen, das nichts bewirken kann.

⚠️ **Die zweite Ziffer ist `enabled` , nicht `on_XXX` .** Übergeben wird die Benutzer-Freigabe (`en_XXX`), nicht der Betriebszustand. Eine grüne LoRa-Kachel bedeutet also *„der Benutzer hat es eingeschaltet"* — ob das Funkmodul auch antwortet, sagt erst die Detailseite. Wer beides verwechselt, baut eine Anzeige, die bei stiller Hardware fröhlich grün leuchtet.

⚠️ **GPS und LoRa weichen bewusst ab** und kennen Zwischenstufen (GPS: orange = sucht, blau = letzte Position bekannt, grün = aktueller Fix). Das ist kein Widerspruch zur Regel, sondern ihre Verfeinerung: Beide Module haben einen Zustand zwischen „aus" und „liefert", und den zu verschweigen würde die Kachel zur Lüge machen. Die Bedeutungen stehen im Benutzerhandbuch, Kapitel „Module ein- und ausschalten".

12b.5 ⚠️ Die Trennung der Aufgabenbereiche

> **Bildschirmaufbau und Ereignisbehandlung laufen im Anzeige-Task. Blockierende Arbeit gehört dort nicht hin.**

Ein Tastendruck, der ein Funkmodul einschaltet, eine Netzverbindung aufbaut oder in den Flash-Speicher schreibt, benötigt bis zu mehrere Sekunden. Im Anzeige-Task ausgeführt bliebe die Anzeige für diese Dauer stehen, und die Überwachungsschaltung würde einen Neustart auslösen.

Das durchgängige Muster lautet deshalb:

Ort	Aufgabe
Ereignisbehandlung (Anzeige-Task)	Anforderung vermerken — ein `volatile` -Flag setzen, sonst nichts
`CR_XXX_Loop()` (Hauptablauf)	die Anforderung abarbeiten
Rückmeldung	im nächsten Durchlauf der Anzeigeaktualisierung

Umgesetzt ist das unter anderem beim Ein- und Ausschalten der Module (`CR_ModuleToggle`), beim Funkmodul (`CR_LoRaCtl_RequestPower()`) und beim Firmware-Update (`CR_OTA_RequestUrlUpdate()`).

Umgekehrt gilt ebenso: **Aus dem Hauptablauf darf kein Widget ohne Sperre angefasst werden.** Die Einzelheiten und die dabei möglichen Fehler stehen in Kapitel 13.

12b.6 Was beim Hinzufügen eines Bildschirms zu tun ist

1. `CR_<Name>Screen.cpp/.h` nach dem Muster aus 12b.4 anlegen
2. Kennung in der `CR_ScreenId` -Aufzählung ergänzen
3. `_Init()` aus `main.cpp` aufrufen — **vor** der Abschlussprüfung des Bildschirmverbunds
4. Wischwege der Nachbarn anpassen, damit der Verbund geschlossen bleibt
5. ⚠️ Prüfen: meldet das Startprotokoll die erhöhte Gesamtzahl?

6. ⚠️ Prüfen: lässt sich der Bildschirm in **jede** vorgesehene Richtung wieder verlassen?

Schritt 6 ist der wichtigste. Ein Bildschirm, der sich betreten, aber nicht verlassen lässt, macht das Gerät bis zum Neustart unbedienbar — und beim Übersetzen fällt davon nichts auf.

12b.7 Drei Zifferblätter, eine Screen-ID: die Weiche `CR_FaceSel`

Seit 1.0.4.146 besitzt die Uhr ein zweites Zifferblatt, den „Stromzähler" — ein sechsstelliges HHMMSS-Zählwerk im Stil eines mechanischen Ferraris-Stromzählers. Die Gestaltungsidee ist an „meterClock" von Volos Projects (github.com/VolosR/meterClock) angelehnt; das Vorbild-Repo hat keine Lizenzdatei, übernommen wurde deshalb ausschließlich die Idee, nicht Quelltext, Bilder oder Schrift.

Aufbau des Zählwerks (Stand 1.0.4.171). Vier dunkle Rollen (HH:MM) stehen in einem durchgehenden Gehäuse, zwei rote (SS) in einem eigenen Block 3 px daneben — am echten Zähler sind die Nachkommastellen rot. Alle sechs Rollen sind gleich groß (28 × 50 px) und tragen ein helles Wölbungsband über die Mitte; die Ziffern sind mit `lv_obj_set_style_transform_scale_y` auf 384/256 senkrecht gedehnt, statt die Zellen zu vergrößern. Das Komma zwischen Minuten und Sekunden ist eine runde Ausnehmung, deren **Mitte auf der Unterkante beider Gehäuse** liegt und deren Durchmesser von der letzten dunklen bis zur ersten roten Rolle reicht (`SPALT + 2 × PAD`); sie wird nach beiden Gehäusen angelegt und überdeckt deren abgerundete Ecken. Alle Maße leiten sich aus `CR_WFM_ZW_BOX_*` ab — wer das Zählwerk verschiebt, nimmt Doppelpunkt und Komma mit.

Der Nennwertstreifen am unteren Rand trägt drei Gruppen: links die gemessene Akkuspannung aus `actualVoltage` (WZ_BATT.h, 10-mV-Raster, dieselbe Quelle wie Akku-Schirm und Logbuch), rechts seit 1.0.4.167 die **Restkapazität** (Ladestand × Nennkapazität), in der Mitte die Akkuanzeige. ⚠️ Die Restkapazität ist so gut wie der Ladestand, und der kommt aus der Spannungskennlinie `CR_BattGauge` , nicht aus einer Ladungsbilanz: reproduzierbar, aber keine Messung der entnehmbaren Ladung — unter Last sackt die Spannung, die Anzeige also ebenso. **Mitte heißt hier: mittig zwischen den beiden Kennzahlen**, nicht mittig im Streifen — die beiden Texte sind verschieden breit, sonst sitzt die Pille sichtbar schief. `cr_wfm_nameplate_layout()` rechnet den Versatz aus den echten Labelbreiten (nach `lv_obj_update_layout()`) und läuft erneut, sobald sich die Breite der Spannungsanzeige ändert. Die Nennkapazität unterscheidet 940 mAh (Plus) von 470 mAh (kleine T-Watch S3) über `hw_GPS == CR_HWGPS_KEINS` — dieselbe Unterscheidung, mit der `main.cpp` den Vorgabe-Ladestrom wählt (Entscheidung D-2). Bis 1.0.4.161 stand dort fest „470mAh", auf einer Plus also der falsche Wert.

Die naheliegende Umsetzung — ein zweiter Eintrag in `CR_ScreenId` samt eigener `CR_ScreenMgr_Register()` — wurde bewusst **nicht** gewählt. Sie hätte den Hauptring um ein Glied verlängert, das jeder Wisch-Durchlauf mitschleppt, auch wer nur ein Zifferblatt nutzt (Kapitel 17.3: Ein zusätzlicher Schirm *im* Ring verlängert den Weg für alle, nicht nur für den, der ihn will).

Stattdessen bleibt `CR_SCREEN_WATCHFACE` die einzige Registry-ID. Eine Weiche, `CR_FaceSel` , hält eine feste Funktionszeiger-Tabelle:

```
struct CR_FaceEntry {
  const char *name;
  void (*init)(void);
  lv_obj_t *(*getRoot)(void);
  void (*loop)(void);
  void (*setUnread)(int);
  void (*optionen)(int);   // seit 1.0.5.14: CR_FACEOPT_*-Bits (Doppeltipp, 12b.12)
};
static const CR_FaceEntry kFaces[CR_FACE_COUNT] = {
  { "bahnhofsuhr",  CR_Watchface_Init,      …, CR_Watchface_SetOptionen },
  { "stromzaehler", CR_WatchfaceMeter_Init, …, CR_WatchfaceMeter_SetOptionen },
  { "funkuhr",      CR_WatchfaceRadio_Init, …, CR_WatchfaceRadio_SetOptionen },   // seit 1.0.5.14
};
```

(`CR_FaceSel.cpp`). `CR_FaceSel_Init()` baut **alle** Zifferblätter gleichzeitig auf — sie bestehen dauerhaft im Speicher, nur eines davon ist der aktive `lv_scr` . `CR_FaceSel_GetRoot()` liefert an `CR_EEZShim` / `CR_ScreenMgr` stets den Root des gerade aktiven Eintrags, `CR_FaceSel_Loop()` reicht non-blocking an dessen `_Loop()` durch, und `CR_FaceSel_SetUnreadCount()` reicht an **alle** Zifferblätter zugleich weiter — ein Wechsel zeigt so sofort den richtigen Ungelesen-Stand, statt erst beim nächsten `CR_MsgScreen` -Aufruf. Das dritte Zifferblatt (Funkraumuhr, 1.0.5.14) brauchte genau das: einen Tabelleneintrag und den `CR_FaceId` -Wert `CR_FACE_FUNK = 2` ; an `CR_ScreenMgr` , `CR_EEZShim` oder `CR_MsgScreen` änderte sich nichts. Mitzuziehen sind nur die Texte von `--watchface` , `--info` und der WebUI-Zeile.

Die Wahl selbst liegt in der Preferences-Variable `iWatchface` (NVS-Schlüssel `"wface"` , `prefs.cpp`) — gelesen und begrenzt beim Booten, geschrieben bei jeder bestätigten Auswahl. `prefs.cpp` entscheidet dabei nichts selbst, es liest und schreibt nur; das Laden des gespeicherten Zifferblatts macht `CR_FaceSel_Init()` im Bootpfad.

✅ Zwei gleichzeitig aufgebaute 240×240-Zifferblätter kosten Speicher — am Gerät mit `--memory` gegengeprüft (28.09.2026): **kein nennenswerter Aufschlag**, freier Heap unverändert 39–41 KB, vom LVGL-Vorrat im PSRAM sind 149 von 244 KB belegt. Beide Zifferblätter bestehen ausschließlich aus `lv_obj` / `lv_label` -Rechtecken ohne eigene Zeichenpuffer; teuer wären Leinwände (`lv_canvas`) oder Bilder gewesen (siehe `doku/Entscheidungen.md` E-50).

12b.8 Warum die Scheibenanimation aus `_Loop()` läuft, nicht aus einem `lv_timer`

Das Scheibenfenster des Stromzähler-Zifferblatts zeigt eine rote Marke, die schrittweise weiter rückt. Ein `lv_timer` wäre die naheliegende LVGL-Lösung — und aus demselben Grund wie in Kapitel 15 die falsche: Ein `lv_timer` läuft unabhängig vom Anzeigezustand weiter, auch bei dunklem Schirm, in der Drossel und im Nachtschlaf, und erzeugt dort Zeichenlast ohne jeden Nutzen.

Stattdessen schiebt `CR_WatchfaceMeter_Loop()` die Marke selbst weiter, gedrosselt auf **50 ms** (20 Bilder je Sekunde — bewusst deutlich langsamer als die 3-ms-Taktbremse aus Kapitel 15, weil eine Zeigerbewegung im Stil eines Ferraris-Zählers kein hohes Bildratenziel braucht). Bis 1.0.4.154 waren es 125 ms; die Marke sprang dabei sichtbar. Erhöht wurde die **Bildrate**, nicht die Geschwindigkeit: der Schritt ist von 4 px auf 2 px halbiert, die Marke wandert also unverändert mit rund 40 px/s, nur ohne Sprünge. Ein Durchlauf dauert 140 Schritte, also 7 Sekunden. `CR_GUI.cpp` ruft `CR_FaceSel_Loop()` — und damit die `_Loop()` des jeweils aktiven Zifferblatts — **nur im wachen Zweig** auf (`CR_GUI.cpp:208`); der Zweig für dunklen Schirm, Drossel und Nachtschlaf lässt sie aus (`CR_GUI.cpp:241`). Zusätzlich prüft `CR_WatchfaceMeter_Loop()` selbst, ob der eigene Root überhaupt der aktive Screen ist (`lv_screen_active() == GetRoot()`): Läuft die Bahnhofsuhr, bewegt sich die Marke des Stromzählers nicht mit, obwohl ihr `_Loop()` technisch mitläuft — beide Zifferblätter bestehen ja gleichzeitig. Die Animation kostet damit nur Rechenzeit, wenn der Schirm wach ist **und** das Stromzähler-Zifferblatt tatsächlich zu sehen ist.

12b.8b Das Zählerrad: zwei Ziffern je Rolle, und warum ein halber Pixel ruckelt

Seit 1.0.4.163 rollen die Ziffern, statt umzuspringen. Jede Rolle führt dafür **zwei** Labels übereinander: das sichtbare in der Zellmitte, das nachrückende eine Zellhöhe tiefer und versteckt. Beim Weiterzählen wandern beide gemeinsam nach oben; am Ende tauschen nur die beiden Zeiger ihre Rolle, es wird kein Objekt neu gebaut. Beschnitten wird von der Zelle selbst — ein `lv_obj` clippt seine Kinder, ein zusätzlicher Rahmen ist nicht nötig.

Gerollt wird **genau dann, wenn die Uhr um eine Sekunde weitergegangen ist** — dann drehen alle Stellen, die sich ändern, gemeinsam. Erstbefüllung, Rückkehr aus dem dunklen Schirm (dort läuft `_Loop()` nicht, es fehlen also Sekunden) und Zeitkorrekturen setzen hart. Ohne diese Bedingung liefen beim Aufwachen sechs Walzen gleichzeitig los — und ein Zähler rollt beim Weiterzählen, nicht beim Springen.

> ⚠️ **Die Bedingung gehört an die Uhrzeit, nicht an die Ziffern.** Bis 1.0.4.171 stand in `cr_wfm_rolle_setzen()` „rollen, wenn `neu == (alt + 1) % 10` “. Das stimmt für 9 → 0, aber nicht für die **Übertragsstellen**: bei 59 → 00 geht die Zehnersekunde 5 → 0 und wurde deshalb hart gesetzt, während die Einerstelle noch 400 ms lang rollte. Im Fenster stand dabei einen Moment lang **„09”** (Fund Christian, 29.09.2026). Seit 1.0.4.172 entscheidet der Aufrufer anhand von `now == s_zw_last_time + 1` ; damit dreht auch 23:59:59 → 00:00:00 alle sechs Walzen gemeinsam.

Die Feinteilung der letzten Rolle hängt in derselben Ebene wie die Ziffer und wird mit ihr verschoben; ihr Muster wiederholt sich genau einmal je Zellhöhe, damit die Teilung der nachrückenden Ziffer lückenlos anschließt.

⚠️ Die Regel: ein Bruchteil eines Pixels je Bild ruckelt immer

Am 29.09.2026 lief die Marke im Scheibenfenster plötzlich unruhig, ohne dass an ihrer Bewegung etwas geändert worden war. Ursache: Sie war kurz zuvor von der Linse in die **helle Scheibe** umgehängt worden (damit sie nicht mehr im dunklen Ring erscheint), ihre Bahn war damit von 152 auf 122 px geschrumpft. Die Schrittzahl blieb bei 140 — macht **0,87 px je Schritt**. LVGL kennt nur ganze Pixel: die Marke stand mal still und sprang dann um eins.

> Eine höhere Bildrate hilft dagegen **nicht**. Wer pro Bild weniger als einen ganzen Pixel weiterrückt, verteilt den Stillstand nur auf mehr Bilder.

Richtig ist, die Schrittzahl an die Strecke zu binden und die Dauer über den Takt einzustellen:

```
#define CR_WFM_MARK_SCHRITTE CR_WFM_REFLECT_W   // so viele Schritte wie die Bahn Pixel hat
#define CR_WFM_MARK_MIN_MS 57                   // 122 x 57 ms = 6,95 s Umlauf
```

Damit wandert die Marke bei jedem Schritt um **genau ein Pixel**. Die Regel gilt für jede geradlinige Bewegung im Projekt: Schrittweite ganzzahlig wählen, Geschwindigkeit über den Takt. Beim Zählerrad ist es umgekehrt unkritisch — dort sind 20 Schritte auf 50 px, also 2,5 px je Bild, und die Bewegung folgt zusätzlich einer Cosinuskurve (12b.8), die an den Enden ohnehin langsam ist.

12b.9 Die 3-Sekunden-Druckfläche: warum LVGLs eigener Langdruck nicht reicht

Die unteren 72-px-Ecken des Zifferblatts (Meldungen/Akku-Detail, 12b.4) nutzen LVGLs eingebautes `LV_EVENT_LONG_PRESSED` , das nach 400 ms feuert (Kapitel 16.4). Für die Zifferblatt-Auswahl reicht das nicht: 400 ms wären viel zu kurz für einen Druck, der eine Auswahl absichtlich öffnen soll — und LVGL meldet `LONG_PRESSED` genau einmal, es gibt kein eingebautes Ereignis für „3 Sekunden lang gehalten”.

`CR_FaceSel_AttachLangdruck()` misst die 3 Sekunden deshalb selbst, über die laufenden `LV_EVENT_PRESSING` -Ereignisse:

```
static void cr_facesel_hold_pressing_cb(lv_event_t *e) {
  if (!s_hold_active || s_hold_fired) return;
  if (s_hold_ok && !cr_facesel_ruhig()) s_hold_ok = false;
  if (!s_hold_ok) return;
  if (millis() - s_hold_press_t0 < CR_FACESEL_HOLD_MS) return;  // 3000 ms
  s_hold_fired = true;
  cr_facesel_auswahl_oeffnen();
}
```

(`CR_FaceSel.cpp:289–301`). `LV_EVENT_PRESSED` merkt Zeitstempel und Druckpunkt, jedes folgende `PRESSING` prüft die verstrichene Zeit **und** `cr_facesel_ruhig()` — dieselbe Slop-Rechnung wie `cr_watchface_badge_ruhig()` an den 72-px-Ecken (Quadrat der Distanz zum Druckpunkt gegen `CR_FACESEL_SLOP_PX = 10` , wie `CR_MODSCREENS_TAP_SLOP_PX` in Kapitel 16.4), aber **laufend** statt nur einmal: Ist der Finger irgendwann während der 3 Sekunden aus dem 10-px-Bereich um den Druckpunkt gewandert, bleibt `s_hold_ok` für den Rest dieser Berührung falsch — aus einem langsamen Wisch über das Zifferblatt wird so nie eine Auswahl. Das unterscheidet die Druckfläche von den 72-px-Ecken, die nur beim Loslassen einmal nachsehen: Dort fehlt die volle 3-Sekunden-Spanne, in der sich ein Finger schrittweise verirren könnte.

`LV_EVENT_RELEASED` / `LV_EVENT_PRESS_LOST` setzen den Zustand zurück, **ohne** `lv_indev_wait_release()` . Ein `wait_release` friert auf einer Fläche, auf der auch gewischt wird, die Wegsummierung ein (Kapitel 16.3) und hätte hier jeden Wisch zwischen den beiden Zifferblättern unmöglich gemacht.

Die Druckfläche selbst deckt nur die **obere** Hälfte des Zifferblatts ab (240×168, `TOP_MID` , `CR_WatchfaceMeter.cpp:861–867`) — bewusst kleiner als der volle Schirm, damit die beiden unteren 72-px-Langdruck-Ecken unverändert erreichbar bleiben. `LV_OBJ_FLAG_GESTURE_BUBBLE` lässt Wischgesten weiter zum Root und damit zu `CR_ScreenMgr_GestureCb()` durchsteigen.

12b.10 Die Auswahl übernimmt Wisch-Navigation und Sprung-Wünsche

Während die Auswahl offen ist, darf ein Wisch nicht den normalen Schirm wechseln, sondern muss zwischen den Zifferblättern blättern — dieselbe Aufgabe wie in Kapitel 17.4 bei der Meldungsliste, hier aber am zentralen Gesten-Handler statt an einem eigenen:

```
void CR_ScreenMgr_GestureCb(lv_event_t *e) {
  if (CR_FaceSel_AuswahlOffen()) {
    lv_dir_t dir = lv_indev_get_gesture_dir(lv_indev_active());
    if (dir == LV_DIR_LEFT)       CR_FaceSel_Blaettern(true);
    else if (dir == LV_DIR_RIGHT) CR_FaceSel_Blaettern(false);
    return;  // NICHT weiter zur Ring-Topologie
  }
  …
}
```

(`CR_ScreenMgr.cpp:132–146`). `CR_FaceSel_Blaettern()` lädt dabei den Root des jeweils nächsten/vorigen Zifferblatts direkt per `lv_scr_load_anim()` — die Vorschau beim Blättern ist damit das **echte** Zifferblatt, kein Bild, und der 30-Sekunden-Zeitwächter (unten) startet neu.

Ein zweiter Eingriff sitzt in `CR_ScreenMgr_Loop()` : Ein Sprung-Wunsch — Krone/Heim, die BOOT-Taste, alles, was über `CR_ScreenMgr_RequestScreen()` läuft — wird bei offener Auswahl **verworfen** statt ausgeführt; die Auswahl bricht stattdessen über `CR_FaceSel_Abbrechen()` ab und kehrt selbst zu dem Zifferblatt zurück, das beim Öffnen aktiv war (`CR_ScreenMgr.cpp:259–273`). Ohne diese Prüfung würde ein Kronendruck während offener Auswahl zwei Bildschirmwechsel auf einmal anstoßen — den der Auswahl und den der Heimtaste.

Seit 1.0.5.14 ist der Wisch nur noch Vorschau. `CR_FaceSel_Blaettern()` setzt das Zifferblatt mit `CR_FaceSel_Set(neu, false)` — angezeigt, nicht gespeichert —, und die Auswahl bleibt offen: man blättert beliebig oft im Kreis. Gewählt wird erst mit einem ruhigen Tipp (12b.12). Bis 1.0.5.9 legte bereits der Wisch die Wahl fest (Christian 28.09.: „mit Druck wischen" fühlte sich falsch an), 1.0.5.10–.13 übernahmen zusätzlich nach 3 s ohne Wisch.

Ein dritter, zeitbasierter Abbruch sitzt in `CR_FaceSel_Loop()` selbst: Bleibt die Auswahl 30 Sekunden ohne Eingabe offen, bricht sie ebenso ab wie über Krone/Heim — eine offene Auswahl darf niemals ein Zustand sein, aus dem nur der bewusste Tipp herausführt. Beim Übernehmen wie beim Abbrechen versteckt `CR_FaceSel` konsequent **alle** Auswahlleisten, nicht nur die des gerade aktiven Zifferblatts — sonst zeigte ein später frisch geladener Root beim nächsten Öffnen keine Leiste.

12b.11 Die Funkraumuhr: ein Bild, drei Polygone

Das dritte Zifferblatt (1.0.5.14, Idee OE1MOJ) bildet die Seefunk-Funkraumuhr nach: vier Funkstille-Keile zu je 18° (grün 2182 kHz ab h+00/h+30, rot 500 kHz ab h+15/h+45), eine Minuterie und außen das Alarmzeichen der Telegrafie — 12 Striche zu 4 s mit 1 s Pause. Die Lage der Striche stand in keiner Quelle (Wikipedia nennt nur „alternating red and white bars"); sie ist an einer Vorlage per Winkelprofil vermessen: **jeder Strich beginnt auf der 5-Sekunden-Marke**.

Das Blatt ist ein fertiges Bild. `tools/make_funkuhr.py` zeichnet es am Mac und schreibt es als RGB565- `lv_image_dsc_t` nach `CR_FunkuhrBild.cpp` (je Form 115 KB Flash, kein PSRAM). Die Uhr muss so weder 60 Striche noch 12 Bögen noch 4 Sektoren zur Laufzeit rechnen. Zwei Kniffe machen es auf 240 px scharf:

- Flächen und Linien entstehen **4-fach überabgetastet** und werden **flächentreu** (`BOX`) verkleinert — `LANCZOS` legte helle Säume an die Kanten.
- Die Ziffern werden dagegen **direkt in 240 px und auf ganze Pixel** gesetzt. Verkleinerte Schrift verliert das Hinting und wirkt weich; das war der Befund „wirkt unscharf".

Ein weißer Hof um jede Ziffer schneidet die Keile aus. Er entsteht als Maske: Kontur mit breitem Strich aufziehen, weichzeichnen und bei $\Phi(1) = 0{,}84$ schneiden (schrumpft rund um genau eine Standardabweichung — schließt Buchten wie im Bogen der 2), danach eingeschlossene Löcher von außen fluten (sonst scheint der Keil durch die Innenräume von 0, 6, 8, 9).

Rund und quadratisch. Die quadratische Form ist eine Superellipse $|x|^5 + |y|^5 = 120^5$. Ringe im festen Abstand vom Rand sind dort keine verschobenen Radien, sondern **Parallelkurven** $Q(t) = P(t) - a \cdot n(t)$ — sonst werden sie zu den Ecken hin schmaler. Gesucht wird meist der Punkt einer Parallelkurve unter einem

Zeigerwinkel; dafür hält der Generator je Abstand eine dichte Tabelle t → Winkel. Striche, Balken und Bogenenden stehen senkrecht auf der Kontur. Den Exponenten schreibt der Generator als `cr_funkuhr_quadrat_n` mit ins Abbild, damit die Uhr ihre Zeigerlängen mit derselben Kontur rechnet.

Spitze Zeiger ohne Canvas. `lv_line` kennt nur eine feste Breite. Statt eines 230-KB-Canvas wie bei der Bahnhofsuhr ist jeder Zeiger ein eigenes, durchsichtiges Objekt in der Größe seines Umrisses, das sich in `LV_EVENT_DRAW_MAIN` mit drei `lv_draw_triangle` und einer runden Heckkappe selbst zeichnet. Beim Stellen werden Lage und Größe auf das neue Umrissrechteck gesetzt — LVGL erneuert so nur diesen Bereich (dieselbe Lehre wie beim Sekundenzeiger der Bahnhofsuhr, 260905-e40). Der Sekundenzeiger springt einmal je Sekunde; Stunde und Minute werden nur beim Minutenwechsel neu gestellt.

Plätze vermessen, nicht geschätzt. Batterie und Rufzeichen liegen im freien Feld, das in **beiden** Formen frei ist; die Stelle hat eine Suche im Bild bestimmt (Rechteck mit Rand, nächst der Mitte). Das Rufzeichen beginnt linksbündig mit der „2" der „21" (x = 47). Für Aufnahmen ohne Fenster hat der Emulator `--snap-funk` und `--snap-funk-q` (Kapitel 26).

12b.12 Doppeltipp für alle Zifferblätter — und die Klick-nach-Wisch-Falle

Ein Doppeltipp schaltet seit 1.0.5.14 je nach **Ort des zweiten Tipps**: obere Hälfte die Form der Funkuhr, rechts unten die Akkuanzeige, links unten das Rufzeichen — die beiden letzten für alle Zifferblätter gemeinsam. Die Erkennung sitzt in `CR_FaceSel` (`CR_FaceSel_AttachDoppeltipp()`), jedes Zifferblatt bietet nur `_SetOptionen(bits)` an. Gespeichert werden die Bits `CR_FACEOPT_*` in `iFaceOpt` (NVS-Schlüssel historisch `"wfform"`). Angehängt ist der Handler an die Auswahlfläche, die Langdruck-Ecken und den Root — der Streifen unten in der Mitte gehört keinem Kindobjekt, dort kommt der Tipp beim Root selbst an.

LVGLs eigenes `LV_EVENT_DOUBLE_CLICKED` taugt dafür nicht: es prüft nur den Abstand **zwischen** zwei Tipps, nicht ob **während** eines Tipps gewischt wurde (`indev_proc_short_click()`). Die Weiche merkt sich deshalb den Druckpunkt und wertet nur `LV_EVENT_SHORT_CLICKED` ohne Fingerbewegung (10 px); zwei solche innerhalb 400 ms und 30 px sind ein Doppeltipp. Ein weckender Tipp auf den dunklen Schirm erreicht LVGL gar nicht (Wecksperre in `WZ_Touch.cpp`).

🔴 **Dieselbe Falle traf die Zifferblatt-Auswahl.** LVGL sendet beim Loslassen `LV_EVENT_CLICKED` **bedingungslos**, auch nach einem Wisch (siehe die Tippen-gegen-Ziehen-Abhilfe in `CR_MsgScreen.cpp`). Der Auswahl-Handler wertete dieses `CLICKED` als „Tippen = übernehmen". Solange der Wisch die Wahl ohnehin festlegte, fiel das nicht auf; seit er nur Vorschau ist, schloss das `CLICKED` des ersten Wischs die Auswahl, und der zweite Wisch lief in die normale Schirm-Navigation (Befund am Handgelenk, 1.0.5.13). Seit 1.0.5.14 übernimmt nur noch der ruhige `SHORT_CLICKED`-Tipp in `cr_facesel_tipp_cb()` — überall auf dem Zifferblatt. Die Reihenfolge beim Loslassen ist `RELEASED → SHORT_CLICKED → CLICKED`; ein Tipp bei offener Auswahl wird also gewertet, solange sie noch offen ist, und zählt nicht zugleich als erster Tipp eines Doppeltipps.

12b.13 Der Schreibschirm `CR_Compose`: T9 auf 240 Pixeln

Seit 1.0.5.17 kann man an der Uhr selbst schreiben (`CR_Compose.cpp`, Screen-ID `CR_SCREEN_COMPOSE`, Schleife mit „Senden"). Gewählt wurde eine **Telefontastatur mit Mehrfachtipp**: zwölf Tasten zu 77 × 39 px (rund 9 × 4,5 mm) treffen sich mit dem Finger zuverlässig, eine volle QWERTZ-Reihe hätte Tasten von 2,5 mm. Die Schrift `cr_font_montserrat_16_de` hat 22 px Zeilenhöhe; zwei Zeilen Text, Tastenfeld darunter: 4 · 39 + 3 · 2 = 162 = 240 − 78.

Baustein	Umsetzung
Mehrfachtipp	Das wartende Zeichen steht schon im Text; dieselbe Taste ersetzt es durch das nächste (UTF-8-sicher), ein `lv_timer` (900 ms) schreibt fest
Tipp / Langdruck	nur `LV_EVENT_SHORT_CLICKED` mit 10 px Slop (Klick-nach-Wisch-Falle, 12b.12); `LONG_PRESSED` mit derselben Vorprüfung, nach ihm kommt kein `SHORT_CLICKED`
Ziele	Liste aus aktuellem Ziel, `*`, `iGCB`, fremden DM-Absendern (`CR_MsgScreen_GetView`) und `CR_Stations`; Farbe über `CR_MsgScreen_LayerColor(CR_MsgScreen_LayerOf(…))` — eine Quelle mit dem Meldungsschirm
Senden	`WZ_LoRa_RequestSendTo()` wie `CR_QuickReply`, Farbautomat in `CR_Compose_Loop()` (GUI-Task)
Öffnen von außen	`CR_Compose_Open(dst, returnRoot)`: ✕ führt zu `returnRoot` zurück
Navigation	Registry: senkrecht in **beide** Richtungen „Senden" (Schleife aus zwei Schirmen, auch dort `vertUp = vertDown = CR_SCREEN_COMPOSE`), waagrecht dieselben Nachbarn wie „Senden" (seit 1.0.5.22)
Zwischenablage	`CR_Compose_SetClipboard()` (Doppeltipp in der Detailansicht), Einfügen per Langdruck aufs Textfeld, gekürzt an einer UTF-8-Grenze

T9-Wortvorschlag (seit 1.0.5.23). `CR_T9Dict.cpp` führt drei erzeugte Listen zusammen (`src/t9/`, Generator `tools/t9/gen_t9_dict.py`): Amateurfunk (351, Priorität 220), Deutsch (5606) und Englisch (5000). Jeder Eintrag ist `{code, word, prio}`, nach Code sortiert; `t9_lookup()` sucht binär und liefert erst exakte Treffer, dann Ergänzungen. `CR_T9_Lookup()` holt je Liste die besten acht, ordnet gemeinsam (exakt vor Ergänzung, dann Priorität, dann der kürzere Code) und entfernt Doppelte ohne Groß/Klein („BAKE"/„bake"). Im Schreibschirm steht das Wort wie beim Mehrfachtipp schon im Text (`s_t9Len` Byte am Ende) und wird beim Weiterschalten ausgetauscht; `cr_compose_commit()` schreibt es fest. Speicher: die Tabellen liegen als `static const` im Flash (DROM, ~290 KB), **kein**

RAM zur Laufzeit; eine Suche braucht rund 250 Byte Stapel. Ein gepacktes Format (Codes beim Suchen ausrechnen) spräche ~190 KB, lohnt bei 60 % freiem Flash nicht. Die DE/EN-Listen stammen aus wordfreq und stehen unter CC BY-SA 4.0 (`src/t9/LIZENZ.md`). Im Emulator: `EMU_TIPPEN="**784"` (zweimal Modus = T9, dann Q-T-H).

Langdruck im Meldungsschirm. LVGL schickt nach einem Langdruck beim Loslassen trotzdem `CLICKED` — an das **alte** Objekt, obwohl der Schreibschirm schon steht. In der Detailansicht hieße das: die Ansicht schließt sich und lädt den Schreibschirm sofort wieder weg. Die Flag `s_composeLongHandled` (gelöscht bei jedem neuen `PRESSED`) verbraucht diesen Klick; bewusst kein `lv_indev_wait_release()` , das die Wischgesten einfriert.

Doppeltipp in der Detailansicht. Der einfache Tipp schließt die Ansicht, also muss er auf den zweiten warten: ein einmaliger Timer (350 ms) schließt, ein zweiter Tipp im Fenster löscht ihn und kopiert. Der Timer prüft vor dem Schließen, ob die Detailansicht noch aktiv ist — sonst holte ein verspäteter Tipp nach einem Wisch zurück.

Im Emulator: `--snap-compose <datei.ppm> [Ziel] [Text]` und `--snap-compose-ziele …` mit aufgeklappter Liste (Gruppenplätze wie auf der Uhr vorbelegt).

12b.14 Ein Ziel für „Senden“: Ebenenliste, Kopfzeile, Schleife (seit 1.0.5.26)

Befund Christian 02.10.: In einer **leeren** Gruppe kam man nie an die Textbausteine. Das Ziel der Gruppe bekam „Senden“ nur aus der Detailansicht einer Meldung, und in der Ebenenliste führten links wie rechts zur Übersicht.

Woher „Senden“ sein Ziel nimmt (`cr_quickreply_screen_load_start_cb` , beide LVGL-Ladepfade geprüft: `prev` oder `act` ist der alte Schirm):

Gekommen aus	Ziel	Quelle
Detailansicht	Absender bzw. Gruppe der Meldung	`CR_MsgScreen_GetReplyTarget()`
Liste einer Ebene (Wisch rechts)	Gruppe der Ebene, ALL/DM → `*`	`CR_MsgScreen_GetListTarget()`
Schreibschirm (Schleife)	dessen Ziel	`CR_Compose_GetDst()`
sonst (Übersicht, Module, Krone …)	`*`	–

Dort lässt es sich per Tipp auf die Kopfzeile umstellen, bis der Schirm neu betreten wird.

Rückweg in dieselbe Ebene. Der Wisch rechts setzt `s_list_to_senden` . Kommt der Meldungsschirm danach direkt aus „Senden“ zurück, gilt derselbe Weg wie aus der Detailansicht (`s_back_from_detail`): Ebene und Blatt bleiben. Jedes andere Betreten löscht den Merker und beginnt wie bisher auf der Übersicht.

Eine Zielliste für beide Schirme: `CR_DstPicker` . Die Liste stand bis 1.0.5.25 in `CR_Compose` . Für „Senden“ wurde sie in ein eigenes Modul gezogen statt kopiert: Reihenfolge (aktuelles Ziel, `*` , `iGCB` , DM-Partner aus dem Verlauf, gehörte Stationen), Kennfarben (`CR_DstPicker_Color()` über `CR_MsgScreen_LayerColor`) und Tipp-Prüfung (10 px Slop, `SHORT_CLICKED`) gibt es damit nur einmal. Es ist höchstens eine Liste offen. Jeder Schirm schließt sie beim Wischen und beim Betreten (`CR_DstPicker_Close()`), damit sie nie auf einem unsichtbaren Root liegen bleibt.

Leere Ebene. `s_empty_label` zeigt jetzt drei Zeilen in Schwarz (Kontrastlehre 31.07.) mit dem Ziel der Ebene. Er wird in `cr_msgscreen_render_list()` gesetzt, wo auch die Sichtbarkeit entschieden wird.

Im Emulator: `--snap-leer <datei.ppm> [Ebene]` , `--snap-senden …` und `--snap-senden-ziele …` (Ebene 0 = ALL, 1 = DM, ab 2 die Gruppenplätze; Vorgabe 7 = Gruppe 2324).

12b.15 Wecken zeigt die Meldung (seit 1.0.5.27)

Vorgabe Christian 02.10.: Display finster → Meldung → Display an **mit der Meldung** → nach der Blank-Zeit finster → Touch bringt die Meldung zurück → nach der Merkfrist bringt Touch die Uhr.

Drei Tasks, drei Merker (`CR_Wecken.cpp` , Lehre lv-lock-im-main-loop: der Empfang fasst LVGL nie an):

Stelle	Task	Wirkung
`CR_Wecken_BeiMeldung()`	Empfang (Main)	nur bei dunklem Schirm und erlaubtem Weg: `handlePressPWR()` , `s_zeigeWunsch`
`CR_Wecken_GuiLoop()`	GUI, **nach** `CR_MsgBridge_Loop()`	`CR_MsgScreen_ZeigeNeueste()` → Detail der neuesten Meldung in **ihrer** Ebene, `s_halten` ; verlässt man die Detailansicht, endet das Halten
`CR_Wecken_HaeltMeldung()`	Main (Rückkehr zur Uhr), IMU-Wecken	hell: immer halten; dunkel: bis `nMerkMin` ab `CR_Wecken_Dunkel()` abgelaufen ist

Die 10-s-Rückkehr zur Uhr (`ScreenSwitchTimer`) ist ausgesetzt, solange gehalten wird. Ist die Merkfrist um, greift sie wieder, auch bei dunklem Schirm. Der nächste Touch zeigt dann die Uhr. Der Touch-Weg selbst bleibt unverändert: Er weckt nur und lässt den aktiven Schirm stehen. Das ist während des Haltens die Meldung.

Die Hebe-Geste (`cr_imu_wecken()`) fragt `CR_Wecken_HaeltMeldung()` **vor** `handlePressPWR()` (die Frist zählt gegen `blanked`) und lässt dann `CR_ScreenMgr_RequestHome()` aus. Die Krone geht immer zur Uhr, und `s_halten` erlischt im nächsten GUI-Durchlauf, weil das Detail nicht mehr aktiv ist.

Merkfrist `nMerkMin` (NVS `merkMin` , `isKey()` -Muster wie `nTimeout` , weil 0 gültig ist), Stufen 0/1/2/5/10/30, Regler im Setup. Dort sitzen jetzt drei Regler, die Rufzeichenzeile ist entfallen, das Raster steht als `CR_SETUPSCREEN_*_Y` . Konsole: `--setMERK` . Emulator: `--snap-setup <datei.ppm>` .

12b.16 Der Wecker: `CR_Alarm` und `CR_AlarmScreen` (seit 1.0.5.28)

Christian 02.10.: „eine Wecker (Alarm) Seite, wo eine x? Anzahl an Alarmen eingestellt werden kann (täglich, wöchentlich oder mit fix Datum)“. Entscheidungen: 8 Wecker, Wochentage einzeln und „einmal am Datum“, Schlummerzeit und Signal je Wecker.

Daten. `CR_AlarmEintrag` , 12 Byte: `belegt, an, hh, mm, tage` (Bit 0 = Mo … Bit 6 = So), `signal` (Vibration/Ton), `schlummer` , `monat, tag, jahr` . Acht Plätze, also 96 Byte, liegen als ein Blob `wecker` im NVS. `putBytes()` wird geprüft (Memory nvs-20kb-knapp). Steht ein Datum, gilt nur dieses. Weder Tage noch Datum heißt „einmal“. Der GUI-Task schreibt (`CR_Alarm_Set/Delete`), der Main-Task liest; ein `portMUX` schützt die Kopie. `CR_Alarm_Stand()` zählt jede Änderung mit, damit die Liste neu zeichnet, wenn der Main-Task einen einmaligen Wecker abschaltet.

Fälligkeit (`CR_Alarm_Loop` , Main-Task, 250-ms-Raster): Geprüft wird das **Fenster** `(letzte Prüfung, jetzt]` gegen die Weckzeit von heute und gestern (Mitternacht). Damit geht keine Minute verloren, auch wenn ein Schlafzyklus über sie hinweg gelaufen wäre. Nach dem Start, nach dem Stellen der Uhr oder nach einem Sprung von mehr als 10 min wird das Fenster auf eine Sekunde gesetzt, so werden keine alten Wecker nachgeholt. Ohne gestellte Uhr (vor 2023) ist nichts fällig.

Klingeln. `CR_Power_KeepAwake(true)` (symmetrisch freigegeben in `cr_alarm_ende`), bei dunklem Schirm `handlePressPWR()` , dann `CR_ScreenMgr_RequestScreen(CR_SCREEN_ALARM_RING)` . Alle 2 s kommt das Signal: `CR_Haptic_Play(CR_HAPTIC_FX_EIN)` und/oder `CR_Audio_Frei(988 Hz, 4 × 140 ms)` . Ohne spielbereites Audio vibriert ein Ton-Wecker zusätzlich. Nach 10 min ist Schluss. **Schlummern** setzt `s_schlummerBis` , **Aus** schaltet Einmal- und Datumswecker ab. Die Knöpfe im GUI-Task setzen nur Merker (`CR_Alarm_Request…`). Die Krone ruft im Main-Task `CR_Alarm_Schlummern()` statt `CR_ScreenMgr_RequestHome()` .

Nachtschlaf. `CR_Sleep` kürzt seinen Zyklus (sonst bis 60 s) auf `CR_Alarm_SekundenBisNaechster()` , das auch ein offenes Schlummern berücksichtigt. Die Uhr wacht damit zur Weckminute auf, und die Fensterprüfung fängt einen Rest von Sekunden ab.

Schirme. Die Liste hat fünf Karten zu 40 px je Seite; waagrecht wird geblättert, senkrecht gilt die Säule Zifferblatt ↔ Wecker ↔ Kalender. Der Editor hat zwei Walzen (20-px-Schrift, Zeilenabstand 6, drei volle Zeilen = 88 px; 1.0.5.28 klemmte 24 px in feste 78 px, oben und unten war je eine halbe Zeile weg, Befund Christian am Handgelenk), eine Tagesreihe, drei Zeilen, die beim Antippen weiterschalten, und ein Datums-Overlay. Er hat keine Wisch-Nachbarn; die Walzen scrollen senkrecht, hinaus geht es über das X und die Krone. Die Klingel-Ansicht hat bewusst keinen Wisch, damit niemand einen Wecker versehentlich wegwischt. Schriften nur in Größen, die es schon gab (24/36): Mit 22/46 waren es 128 KB Flash mehr, so sind es 12 KB.

Belegt am Gerät: Anlegen, Liste mit nächstem Termin, `test` , `aus` . Ein echter einmaliger Wecker klang um 06:35:00 von selbst und war nach „aus“ abgeschaltet. **Offen:** Schlummern bis zum erneuten Klingeln und Wecken aus dem Nachtschlaf (der läuft nur ohne USB).

Emulator: `--snap-wecker <datei.ppm> [0|1|2|3]` (Liste, Bearbeiten, Datumswalzen, Klingeln; vier Beispiel-Wecker).

Nachtrag 1.0.5.33: Liste, Schlummern aus, Tagesspannen

- **Liste:** je Seite 4 Wecker (Karten 0..3, 36 px im Abstand 38 ab y = 32), darunter **fest** die Zeile „+ neuer Wecker“ (Karte 4, y = 184) bzw. „alle 8 Plätze belegt“ (`s_cardSlot` −3, Tipp ohne Wirkung), darunter Seitenpunkte (y = 226). Vorher standen 5 Karten je Seite und „+“ als letzter Eintrag – ab dem 5. Wecker rutschte es auf Seite 2, die man nicht sah (Befund Christian 03.10.).
- **Schlummern aus:** `schlummer == 0` heißt jetzt „aus“ (vorher als 5 min gelesen, gespeichert war 0 nie). `CR_Alarm_Schlummern()` schaltet dann aus – Krone und Konsole verhalten sich wie „Aus“; die Klingel-Ansicht zeigt nur den Knopf „Aus“ über die ganze Breite. `--wecker add … aus` .
- **Tagesspannen:** `CR_Alarm_WiederholungText()` fasst ab drei aufeinanderfolgenden Tagen zusammen („Mo-Sa“, „Di-Do Sa“). Die beiden grauen Zeilen der Karte haben eine **feste** Höhe von einer Zeile: mit automatischer Höhe bricht `LV_LABEL_LONG_MODE_DOTS` nicht ab, sondern um – „Mo Di Mi Do Fr Sa“ lief in die Signalzeile.
- Emulator: `--snap-wecker 4|5` (6 Wecker, Seite 1/2), `6|9` (8 Wecker, Seite 2/1), `7|8` (Klingeln und Bearbeiten ohne Schlummern).

12c Die Schriften

Auf dieser Uhr sind drei verschiedene Schriftsysteme im Einsatz, und sie beantworten drei verschiedene Fragen. Wer eine Schrift ändern will, muss zuerst wissen, mit welchem der drei er es zu tun hat — sie werden auf völlig verschiedene Weise erzeugt und eingebunden.

12c.1 Der Überblick

System	Wo	Umfang	Wofür
LVGL-Standard	`src/lv_conf.h:620–640`	20 Größen, Montserrat 8–48	allgemeine Beschriftungen
Eigene Schriften	`SMashCom42/fonts/`	4 Dateien, rund 447 KB	Umlaute und das Ziffernblatt
Emoji-Bildschrift	`src/CR_EmojiData.cpp`	130 Bilder (98 Emojis, 32 Flaggen)	farbige Emojis im Funkverkehr

Die Standardschrift ist `lv_font_montserrat_14` — gesetzt über `LV_FONT_DEFAULT` in `lv_conf.h` (Zeile 690).

12c.2 ⚠️ Warum es eigene Schriften braucht: die eingebauten können kein „ä”

Der wichtigste Punkt dieses Kapitels, und er kostete eine eigene Fehlersuche.

LVGLs eingebaute `lv_font_montserrat_*` sind mit dem Zeichenbereich `-r 0x20-0x7F,0xB0,0x2022` erzeugt — **reines ASCII**. Ein „ä” in einer MeshCom-Meldung erschien deshalb als leeres Kästchen.

> **Der Zerleger ist unschuldig.** Er bricht nur bei Steuerzeichen ab (`b < 0x20`), UTF-8-Bytes laufen unverändert durch. Der Text war die ganze Zeit richtig — er ließ sich nur nicht darstellen.

Das ist die typische Verwechslung bei Zeichensatzfehlern: Ein fehlendes Zeichen sieht nach einem Fehler in der Verarbeitung aus, liegt aber in der Darstellung.

Die Abhilfe sind zwei eigene Schriften mit erweitertem Bereich (`CR_Fonts.h:28–48`):

Datei	Größe
`fonts/cr_font_montserrat_14_de.c`	191 KB
`fonts/cr_font_montserrat_16_de.c`	222 KB

Das `_de` steht für den erweiterten Zeichenbereich, nicht für eine deutsche Schriftart. Der Unterschied zur eingebauten Fassung sind zwei Bereiche:

Bereich	deckt ab	seit
`0xA0–0xFF` Latin-1-Supplement	ä ö ü Ä Ö Ü ß, Französisch, Spanisch, Nordisch	1.0.3.x (260804-c8w)
`0x100–0x17F` Latin Extended-A	Polnisch, Tschechisch, Slowakisch, Ungarisch, Kroatisch, Baltisch	**1.0.4.173**

Der zweite Block kam am 29.09.2026 dazu, weil **ę ż ć ś ń** aus polnischen Meldungen als leere Kästchen erschienen. Statt der fünf Zeichen wurde der ganze Block genommen — **gemessene Kosten 18,8 KB Flash für beide Schriften zusammen** (2.774.169 → 2.793.465 Byte, 33,3 → 33,6 % der App-Partition). Einzelne Zeichen nachzuziehen wäre billiger, kostete aber jedes Mal einen Build und ein Release.

⚠️ Im europäischen Amateurfunk ist das keine theoretische Frage: Die Uhr empfängt, was das Netz trägt, nicht was wir vorgesehen haben.

12c.3 Wie eine eigene Schrift erzeugt wird

Der vollständige Aufruf steht **im Kopf jeder erzeugten Datei** — das ist die verlässlichste Stelle, weil er dort nicht veralten kann:

```
npx lv_font_conv --no-compress --no-prefilter --bpp 4 --size <14|16> \
  --font Montserrat-Medium.ttf \
  -r 0x20-0x7F,0xA0-0xFF,0x100-0x17F,0x2013-0x2014,0x2018-0x201E,0x2026,0x20AC,0x2022 \
  --font FontAwesome5-Solid+Brands+Regular.woff -r <dieselbe Symbolliste wie LVGL> \
  --format lvgl -o cr_font_montserrat_<groesse>_de.c --force-fast-kern-format
```

Die Quelldateien liegen in der LVGL-Quelle unter `scripts/built_in_font/`.

Zwei Entwurfsentscheidungen stecken darin:

Dieselbe Rezeptur wie LVGL, nur erweitert. Dadurch bleiben **alle** `LV_SYMBOL_*`-Glyphen erhalten (der zweite `--font`-Aufruf mit den rund 58 FontAwesome-Codepoints), und die Schrift ist ein unmittelbarer Ersatz für die eingebaute. Ohne diesen zweiten Teil hätte der Umstieg sämtliche Symbolzeichen verloren — Batteriesymbol, WLAN-Zeichen, Pfeile.

4 Bit je Bildpunkt (`--bpp 4`): 16 Graustufen für die Kantenglättung. Höher lohnt bei dieser Schriftgröße nicht, niedriger sieht auf einem 240 × 240-Schirm sichtbar rau aus.

> ⚠️ **`-o` muss auf den endgültigen Pfad zeigen.** `lv_font_conv` leitet Schutzmakro *und* Variablennamen aus dem Ausgabedateinamen ab: Eine Erzeugung nach `neu14.c` ergibt `#ifndef NEU14` und `const lv_font_t neu14` — die Datei lässt sich dann nicht einfach an ihren Platz kopieren. Am 29.09.2026 einmal hineingelaufen.

12c.4 Die Ziffernblatt-Schriften

Zwei weitere Dateien liegen in `SMashCom42/fonts/` und stammen aus dem früheren Entwurfswerkzeug:

Datei	Größe	Verwendung
`ui_font_mplus_rounded1c_bold_24.c`	93 KB	Ziffernblatt
`ui_font_mplus_rounded1c_medium_18.c`	67 KB	Ziffernblatt, Meldungsschirm

Sie sind mit `D3-20` aus `ui/smc42/fonts.h` in ein **eigenes** Verzeichnis umgezogen (`CR_Fonts.h:6-10`) — inhaltlich unverändert bis auf ein entfallenes, ungenutztes Deskriptor-Array. Der Grund ist derselbe wie bei der handgeschriebenen Bedienoberfläche (Kapitel 12b): Was zum Quelltext gehört, soll nicht in einem erzeugten Verzeichnis liegen, wo es beim nächsten Erzeugen verschwinden könnte.

Angesprochen werden sie direkt über `&ui_font_mplus_rounded1c_*`, nicht über ein Array.

12c.5 Emojis sind keine Schrift, sondern Bilder

Der dritte Weg funktioniert grundsätzlich anders. Farbige Emojis lassen sich nicht als Schrift darstellen — eine Schrift kennt nur Deckkraft je Bildpunkt, keine Farbe. Deshalb liegt hier eine **Bildschrift** (`lv_imgfont`, `CR_Emoji.cpp:54`): LVGL fragt für ein Zeichen kein Glyph ab, sondern ein Bild.

Zeichenvorrat	**130** Bilder: 98 Emojis und 32 Flaggen (`CR_EmojiData.cpp`, Stand 1.0.5.9)
Umfang	der größte Einzelposten unter allen Schriften
Bildgröße	fest, `CR_EmojiPixelSize`

⚠️ **Drei Einschränkungen**, die aus dem Verfahren folgen und im Kapitel zur Bildschrift ausführlicher stehen:

1. **Die Bilder skalieren nicht.** Sie sind immer gleich groß, unabhängig von der Schriftgröße der Umgebung (`CR_Emoji.h:37`).
2. **Ein Emoji verbraucht genau ein Zeichen.** Der Variantenselektor `FE0F`, den viele Telefone mitsenden, muss vorher herausgefiltert werden — sonst rückt der Textzeiger falsch weiter (`CR_Emoji.cpp:24`).
3. **Flaggen brauchen einen Umweg.** Eine Flagge besteht aus zwei Regionalzeichen (U+1F1E6…U+1F1FF), die Bildschrift kennt aber nur ein Bild je Zeichen. Seit 1.0.4.1 fasst `CR_Emoji.cpp` das Paar vor der Darstellung zu **einem** Zeichen im privaten Bereich zusammen (`0xF0000 + a·26 + b`), für das ein Flaggenbild hinterlegt ist.

⚠️ **Die Absenkung ist begrenzt (seit 1.0.5.3).** Ein Emoji-Bild sitzt etwas unter der Grundlinie, damit es optisch mit dem Text abschließt: um ein Viertel seiner Bildhöhe, höchstens aber um `font->base_line`, also nie tiefer als die Unterlänge der Umgebungsschrift. So bleibt jedes Bild innerhalb seiner Zeile.

🔴 **Eine Fehldiagnose, die hier festgehalten wird.** Anlass für diese Begrenzung waren „Punkte von Emojis vor der eigentlichen Meldung“ in der Meldungsliste. Die Begrenzung ist richtig, die Ursache war aber eine andere: Die Listenkarten hatten seit Juli das Polster des Default-Themes (13 px), und `lv_obj_set_pos()` der Kinder zählt ab dem Inhaltsbereich. Die Textzeile lag dadurch **unter dem Kartenrand**. Sichtbar waren nur die höchsten Teile der ersten Zeile, also Emoji-Oberkanten, die rote Nadel 📍 und Umlautpunkte. Behoben in 1.0.5.8 mit `pad_all = 0` an den Karten (`CR_MsgScreen.cpp`). Seitdem zeigt jede Karte Rufzeichen, Zeit und die erste Textzeile.

Gefunden erst, als der Schirm gerendert statt die Glyphe betrachtet wurde: Der Emulator hat dafür `--snap-list <datei.ppm> [n]`. Er legt n Beispielmeldungen an, öffnet die Ebene ALL und speichert das Bild. Der Gegenbau mit LVGL 9.5 und alter `lv_conf.h` zeigte dasselbe Bild, die neue Grundlage war also nicht schuld.

Schlägt die Erzeugung fehl, bleibt es bei Platzhaltern — die Uhr läuft weiter (`CR_Emoji.cpp:58`), nach demselben Muster wie bei den Chip-Gesten (Kapitel 8.6).

12c.6 ⚠️ Ein Sparhebel, der bisher nicht gezogen wurde

In `lv_conf.h` sind **alle 20** Montserrat-Größen von 8 bis 48 eingeschaltet. Tatsächlich verwendet werden davon nur wenige — die Oberfläche arbeitet überwiegend mit den beiden eigenen `_de`-Schriften und den beiden Ziffernblatt-Schriften.

Jede aktivierte Größe kostet Flash. Bei 26 % Flash-Belegung (Stand 1.0.3.0) drückt das nicht, weshalb bisher nichts abgeschaltet wurde. **Es ist damit kein Problem, sondern ein bekannter, ungenutzter Spielraum** — festgehalten, damit er bei Platznot nicht erst gesucht werden muss.

⚠️ Vor dem Abschalten ist zu prüfen, welche Größen LVGL selbst voraussetzt: `LV_FONT_DEFAULT` zeigt auf `lv_font_montserrat_14`, und einzelne Widgets greifen auf die Standardschrift zurück, wenn keine gesetzt ist.

12c.7 Wo man nachsieht

Frage	Antwort
Welche LVGL-Größen sind an?	`SMashCom42/src/lv_conf.h:620–640`
Welche eigenen Schriften gibt es?	`SMashCom42/fonts/` — Deklaration in `CR_Fonts.h`
Wie wurde eine Schrift erzeugt?	Kopf der jeweiligen `.c`-Datei, erste Zeilen
Welche Emojis kennt die Uhr?	`SMashCom42/src/CR_EmojiData.cpp`
Warum fehlt ein Zeichen?	Zuerst den Zeichenbereich prüfen, nicht den Zerleger — siehe 12c.2

13 Was im Hauptablauf verboten ist

Die Uhr hat zwei Kerne und zwei Abläufe (Kapitel 11): den Arduino-Hauptablauf auf Kern 0 und den Zeichenablauf auf Kern 1. Wer die Grenze zwischen beiden verletzt, bekommt keinen Absturz mit Fehlermeldung, sondern **sporadische Aussetzer, die wie Hardwarefehler aussehen**.

Dieses Kapitel sammelt die Regeln — jede einzelne aus einem Fall, der Zeit gekostet hat.

13.1 Kein `lv_*` außerhalb des Zeichenablaufs

LVGL ist nicht threadsicher. Es gehört dem Zeichenablauf auf Kern 1. Jeder Aufruf aus dem Hauptablauf greift in Datenstrukturen, die gerade gezeichnet werden.

Betroffen ist vor allem der **Empfangspfad**: Ein Funkpaket trifft als Interrupt ein, wird im Hauptablauf zerlegt — und der naheliegende nächste Schritt wäre, es direkt in die Meldungsliste zu schreiben. Genau das ist verboten.

```
Hauptablauf (Kern 0)          CR_MsgBridge          Zeichenablauf (Kern 1)
Paket zerlegen  ─── Push ──→  Warteschlange  ─── Pop ──→  Liste zeichnen
```

`CR_MsgBridge` existiert allein für diese Grenze. Der Hauptablauf schreibt hinein, der Zeichenablauf holt ab.

> ⚠️ Der Fehler äußert sich nicht als Absturz, sondern als **gelegentlich zerstörter Bildinhalt** — und man sucht tagelang am Bildschirmtreiber.

13.2 Kein `lv_lock()` im Hauptablauf

Die scheinbar saubere Lösung — „dann sperre ich LVGL eben kurz" — ist die schlechtere. Ein `lv_scr_load_anim()` unter `lv_lock()` aus dem Hauptablauf heraus kostete **dreimal einen Watchdog-Neustart**: Der Zeichenablauf wartet auf die Sperre, der Hauptablauf wartet auf die Bildschirmumschaltung, und die Überwachung schlägt zu.

Wie es stattdessen gelöst wird

Richtig ist ein **Wunsch-Flag**: Wer etwas will, das im anderen Ablauf passieren muss, **schreibt es auf** statt es selbst zu tun. Der andere Ablauf sieht bei nächster Gelegenheit nach und erledigt es dort, wo es hingehört.

Das Muster besteht aus drei Teilen, und alle drei sind nötig:

Teil	Wer	Was
1. Die Variable	—	`static volatile`, klein (`bool` oder `int8_t`)
2. Der Wunsch	der Ablauf, der *nicht* darf	setzt die Variable — **und tut sonst nichts**
3. Die Ausführung	der Ablauf, der *darf*	prüft die Variable in seiner Schleife, setzt sie zurück, handelt

Warum `volatile`: Beide Abläufe laufen auf **verschiedenen Kernen** (Kapitel 11). Ohne dieses Wort darf der Übersetzer annehmen, niemand sonst ändere die Variable, und den Zugriff wegkürzen.

Warum **kein** Mutex: Genau der wäre das `lv_lock()`, das den Watchdog verursacht hat. Bei einem einzelnen `bool` oder `int8_t` ist keiner nötig — die Schreiboperation ist unteilbar, und schlimmstenfalls wird ein Wunsch einen Durchlauf später bemerkt. Das ist ein Zehntelsekunde, kein Fehler.

Beide Richtungen im Projekt — mit Beleg

Das Muster wird in **beide** Richtungen benutzt, und wer es nachschlägt, sollte das passende Vorbild erwischen:

Zeichenablauf → Hauptablauf (der häufigere Fall — der Finger tippt eine Kachel, aber ein- und ausgeschaltet wird im Hauptablauf):

```
// CR_ModuleToggle.cpp
static volatile int8_t s_req_wifi = -1;   // -1 = kein Wunsch, 0 = aus, 1 = an

void CR_ModuleToggle_Request(CR_ModuleToggleId id, bool wantOn) {   // aus dem Zeichenablauf
  s_req_wifi = wantOn ? 1 : 0;                                      // mehr passiert hier nicht
}
```

Ausgeführt wird es in `CR_ModuleToggle_Loop()`, aufgerufen aus `loop()` in `main.cpp`.

⚠️ `int8_t` statt `bool`, weil hier **drei** Zustände nötig sind: „kein Wunsch" muss sich von „Wunsch: aus" unterscheiden lassen. Ein `bool` kann das nicht — mit ihm wäre jeder Durchlauf ein Ausschaltbefehl.

Hauptablauf → Zeichenablauf (der Fall aus diesem Abschnitt):

```
// CR_GUI.cpp
static volatile bool s_force_suspend = false;

void CR_GUI_Suspend() { s_force_suspend = true; }    // aus dem Hauptablauf aufrufbar
…
bool lowPower = s_force_suspend || blanked;          // gelesen NUR in cr_gui_task()
```

Die Regel dahinter, in einem Satz: Wer die Sperre nehmen müsste, um etwas zu tun, darf es nicht selbst tun — er darf es nur *wollen*.

13.3 Kein `delay()`, kein Blockieren

`WZ_XXX_Loop()` läuft in jedem Durchgang des Hauptablaufs. Jede blockierende Zeile bremst **alles** — Tastenabfrage, MQTT, Funkempfang.

Stattdessen die `Timeout`-Klasse (`lib/timeout/`), die auf `millis()` beruht:

```
if (timer.time_over()) { … ; timer.start(1000); }
```

Die eine erlaubte Ausnahme ist `radio.transmit()`: Es blockiert bei SF11 mehrere hundert Millisekunden und lässt sich nicht aufteilen. Deshalb darf **ausschließlich der Hauptablauf** senden — im Zeichenablauf würde das Bild für eine halbe Sekunde einfrieren.

13.4 Warum der Takt trotzdem zählt

Lange lief der Hauptablauf mit **1–2 Hz** statt der heutigen 240 Hz (Kapitel 14 und 15). Das hatte eine Nebenwirkung, die niemand geplant hatte: Die Gestenerkennung von LVGL **funktionierte zufällig gerade deswegen**.

> ⚠️ **Nach jeder Änderung am Takt muss die gesamte Bedienung neu vermessen werden.** Nach der Beschleunigung waren drei Dinge kaputt, die vorher gingen — Wischgesten verpufften, Schieberegler gewannen gegen Wischer, Tipps kamen nicht an. Alle drei hatten dieselbe Wurzel und keiner war im Bedienteil selbst zu finden. Kapitel 16 erzählt es im Einzelnen.

13.5 Die Prüfliste

Vor jedem Einbau in `WZ_XXX_Loop()` oder in den Empfangspfad:

Frage	Bei „ja"
Ruft der Code `lv_*` auf?	in den Zeichenablauf verlegen, Flag oder Warteschlange benutzen
Blockiert er länger als ein paar Millisekunden?	mit `Timeout` zerlegen
Sperrt er etwas, worauf der Zeichenablauf wartet?	umbauen — das ist der Watchdog-Fall
Sendet er?	muss im Hauptablauf bleiben, nie im Zeichenablauf

14 Der Fall REND-03: drei Fehler, von denen der erste die anderen verdeckte

Dieses Kapitel erzählt eine Fehlersuche vollständig — mit den Irrwegen. Das ist Absicht. Der Befund allein („Kerne waren nicht getrennt") ließe sich in zwei Sätzen abhandeln. Was ihn lehrreich macht, ist die Frage, warum er über **Monate** unbemerkt blieb, obwohl das Symptom täglich sichtbar war.

14.1 Das Symptom

Bei eingeschaltetem Bildschirm lief der Hauptablauf mit **1 Hz** — ein Durchlauf pro Sekunde. Schaltete sich der Bildschirm ab, sprang der Wert auf mehrere tausend.

Praktische Folgen: Der Sekundenzeiger ruckelte. Funkpakete wurden verzögert abgeholt. Und — hinterher betrachtet das eigentliche Ärgernis — **jede Zeitmessung im Hauptablauf war unbrauchbar**, weil zwischen zwei Messpunkten eine ganze Sekunde lag.

14.2 Die Erklärung, die alle glaubten

Es gab eine plausible Erklärung, und sie stand seit Monaten in der Projektdokumentation:

> *Die Renderlast bremst den Hauptablauf. Der Bildschirm zu zeichnen kostet Zeit, und diese Zeit fehlt anderswo.*

Dafür sprach ein starkes Indiz: Bildschirm an → langsam, Bildschirm aus → schnell. Der Zusammenhang war reproduzierbar, sofort einleuchtend und passte zu jeder Messung. Es wurde sogar nachgemessen, wie lange ein Einzelbild braucht — die Zahlen bestätigten das Bild.

Die Erklärung war falsch. Nicht ungenau, nicht unvollständig: falsch. Sie beschrieb eine Wirkung als Ursache.

Das ist die erste Lehre dieses Kapitels: Eine Erklärung, die zu allen beobachteten Daten passt, ist deshalb noch nicht richtig. Ein *verdrängter* Task und ein *ausgebremster* Task sehen von außen identisch aus.

14.3 Was tatsächlich los war

Es waren **drei** Fehler. Der erste hat die beiden anderen unsichtbar gemacht.

Fehler 1 — beide Tasks auf demselben Kern

Die Aufgaben lagen nicht auf getrennten Kernen, sondern beide auf Kern 1. Der Anzeige-Task hat Priorität 2, der Hauptablauf Priorität 1 — also verdrängte der Anzeige-Task ihn vollständig. Der Hauptablauf kam nur noch dann zum Zug, wenn der Anzeige-Task freiwillig pausierte.

Damit erklärt sich auch das Bildschirm-Indiz, das alle in die Irre führte: Bei abgeschaltetem Bildschirm stellt der Anzeige-Task seine Arbeit ein — und gibt den Kern frei. Es war nie die *Last*, es war die *Verdrängung*.

Die Ursache steht in Kapitel 11 im Detail: `xTaskCreatePinnedToCore()` war richtig, aber die Board-Beschreibung schickte den Hauptablauf auf denselben Kern.

Fehler 2 — 85 % der Zeit am Türsteher

Erst nachdem Fehler 1 behoben war, wurde der zweite sichtbar. Der Hauptablauf nahm bei **jedem** Durchlauf den Anzeige-Mutex, um eine Leuchtanzeige auszuschalten, **die sich gar nicht geändert hatte**.

Der Denkfehler steckt in einer Zeile, die harmlos aussieht:

```
if (CETtimer.time_over()) { /* LED aus */ }
```

`time_over()` liest sich wie ein Ereignis — „der Zeitgeber ist gerade abgelaufen". Es ist aber ein **Zustand**: Einmal abgelaufen, liefert die Methode dauerhaft `true` (`Timeout.cpp:55–59`), und der Konstruktor setzt sie sogar von vornherein auf `true` (`Timeout.cpp:6`). Da dieser Zeitgeber nur alle fünf Minuten neu gestartet wird, war er praktisch immer abgelaufen.

Ergebnis: bei jedem Durchlauf ein Mutex-Zugriff auf einen Baum, der einem anderen Kern gehört.

Gemessen: 812–883 Millisekunden reine Wartezeit pro Sekunde. 82–88 % der verfügbaren Zeit.

Fehler 3 — zu schnell ist auch falsch

Mit den ersten beiden Fehlern behoben lief der Hauptablauf mit **1000–4000 Durchläufen pro Sekunde** — und kostete den Anzeige-Task auf dem anderen Kern **126 Aussetzer in 29 Sekunden**.

Fehler 2 hatte bis dahin unfreiwillig als Bremse gewirkt. Ihn zu beheben legte offen, dass eine absichtliche Bremse fehlte. Näheres in Kapitel 15.

14.4 Der Nebenbefund, den erst die Gleichzeitigkeit hervorbrachte

Nach der Korrektur häuften sich Neustarts durch die Überwachungsschaltung: **3 in 12 Startvorgängen**, jedes Mal unmittelbar nach dem Einschalten des Funkmoduls.

Die Ursache lag seit jeher im Quelltext:

Bauteil	Schnittstelle	Ergibt auf dem ESP32-S3
Display	`USE_HSPI_PORT`	SPI3
Funkmodul	`SPIClass radioSPI(HSPI)`	SPI3

Beide auf derselben Schnittstelle. Das war immer schon so — und immer schon falsch. Nur hatte es nie geschadet: Solange beide Tasks sich einen Kern teilten, legte der Zeitplaner alle Zugriffe zwangsläufig hintereinander. Diese unsichtbare Absicherung fiel mit der Kerntrennung weg.

> **Fehler 1 hatte einen zweiten Fehler jahrelang zugedeckt.** Ihn zu beheben, hat den anderen nicht verursacht — es hat ihn nur endlich sichtbar gemacht.

Das ist der Grund, warum die Behebung eines alten Fehlers eine Firmware zunächst *instabiler* erscheinen lassen kann. Der Fehlschluss, die Änderung sei ursächlich, führt zu ihrer Rücknahme — und begräbt beide Fehler erneut.

14.5 Ergebnis

	vorher	nachher
Hauptablauf, Bildschirm an	**1 Hz**	**209–236 Hz**
Aussetzer der Anzeige	—	**0**
Neustarts in 10 Startvorgängen	—	**0**

Christians Abnahme am Gerät: *„Zeiger läuft rund und sieht gut aus, kein Stopp im Moment mehr … Bedienung geht jetzt PERFEKT flüssig!“*

14.6 Was daraus zu lernen ist

Erstens: Eine Annahme, die nie gemessen wurde, ist keine Erkenntnis — auch wenn sie dokumentiert ist. Die Kerntrennung stand an vier Stellen in der Projektdoku. Drei Zeilen `xPortGetCoreID()` hätten die Behauptung jederzeit widerlegt. Seitdem gilt in diesem Projekt: Was das Startprotokoll nicht meldet, ist nicht bekannt.

Zweitens: Fehler stapeln sich. Nach dem ersten Fund weiterzusuchen ist nicht Übereifer, sondern Pflicht — ein grober Fehler macht feinere unsichtbar. Ein Abbruch nach dem ersten Erfolg liefert eine halb reparierte Firmware, die für vollständig repariert gehalten wird.

Drittens: Die überzeugendste Erklärung ist die gefährlichste. „Renderlast bremst den Hauptablauf” war einleuchtend, passte zu allen Daten und hat die Suche monatelang in die falsche Richtung gelenkt. Ein Indiz, das zu einer Erklärung passt, ist kein Beweis für sie — zu prüfen ist stets, ob es nicht ebenso gut zu einer anderen passt.

Viertens — und das ist die praktische Regel: Wachposten bleiben im Quelltext. Die Zeile `[CORE] main=0 gui=1` ist kein Rückstand aus der Fehlersuche, der aufzuräumen wäre. Sie ist der Grund, warum dieser Fehler nicht ein zweites Mal jahrelang unbemerkt bleiben kann.

15 Warum ein zu *schneller* Loop ebenfalls schadet

Kapitel 14 endet mit einem Erfolg: Der Main-Loop lief nicht mehr mit 1 Hz, sondern mit über tausend Durchläufen je Sekunde. Dieses Kapitel handelt davon, warum das der falsche Zielwert war — und warum die Firmware seither eine ausdrückliche **Bremse** enthält.

15.1 Der Zustand nach der Entfesselung

Nach der Behebung von REND-03 hatte das `loop()` keinen einzigen blockierenden Aufruf mehr. Es raste mit **1000–4000 Durchläufen je Sekunde** durch und pollte MQTT, UDP, GPS und LoRa tausendfach ins Leere (`main.cpp:149–150`).

Das ist zunächst nur Verschwendung — jeder dieser Durchläufe kostet Strom, und die Akkulaufzeit ist ein erklärtes Projektziel. Der eigentliche Schaden lag jedoch woanders.

15.2 Der gemessene Schaden: der andere Kern leidet mit

Main-Task und Render-Task laufen auf **getrennten** Kernen (Kapitel 11). Die naheliegende Erwartung ist deshalb, dass ein schneller Main-Loop den Render-Task nicht berührt.

Die Messung sagt etwas anderes (`main.cpp:151–153`):

Zustand	`GUI_LAG`-Ausreißer in 29 s
ohne Bremse (1000–4000 Hz)	**126**
mit Bremse	**0**

Bei sonst **gleichem Programmstand**. Getrennte Kerne sind eben nicht vollständig entkoppelt: Sie teilen sich den Flash-Cache und den Speicherbus. Ein Kern, der ununterbrochen Code und Daten nachlädt, entzieht dem anderen Bandbreite — ohne dass irgendein Synchronisationsmittel im Spiel wäre, an dem sich das ablesen ließe.

> **Zwei Kerne heißt nicht zwei unabhängige Rechner.** Wer auf dem einen Kern Leerlauf erzeugt, kann auf dem anderen Aussetzer verursachen.

Sichtbar wurde das übrigens nicht als Zahl, sondern als **stehender Sekundenzeiger** — Christians Befund lautete schlicht „Uhr steht still".

15.3 Die Bremse

`main.cpp:159`:

```
#define CR_LOOP_PACING_MS 3
```

Angewendet am Ende jedes Durchlaufs (`main.cpp:1109–1113`). Die Wahl von 3 ms ist begründet, nicht geraten (`main.cpp:153–156`):

- **Kein Verbraucher im `loop()` braucht mehr als ein paar hundert Hertz.** Die schnellste Größe ist der Tastendruck auf die PWR-Taste, und 3 ms liegen weit unter jeder Wahrnehmungsschwelle.
- Gemessen bleiben rund **200–230 Hz** — das Zwanzigfache des Abnahmekriteriums D-06 (≥ 10 Hz).

Der Zielwert ist also nicht „so schnell wie möglich", sondern „schnell genug, mit Abstand".

15.4 ⚠️ Die Ausnahme — und der Fehler darin

Während eines laufenden Firmware-Updates wird bewusst **nicht** gebremst: Dort zählt jeder Durchlauf von `CR_OTA_Loop()` für den Durchsatz.

Diese Ausnahme war zunächst falsch formuliert, und der Fehler ist lehrreich, weil er ausschließlich aus einem **Namen** entstand (`main.cpp:1101–1108`):

```
// falsch:
if (!CR_OTA_IsRunning()) { delay(CR_LOOP_PACING_MS); }
// richtig:
if (CR_OTA_GetState() != CR_OTA_RUNNING) { delay(CR_LOOP_PACING_MS); }
```

`CR_OTA_IsRunning()` liefert `on_OTA` und beantwortet damit die Frage **„läuft der Web-Server?"** — so auch in `CR_OTA.h:92` dokumentiert. Es beantwortet **nicht** die Frage „läuft ein Update?".

Der Server läuft aber, solange der Update-Schirm geöffnet ist (Kapitel 23). Die Bremse fiel damit bereits beim **bloßen Betreten** des Schirms weg. Gemessen am Gerät am 5. August 2026:

```
235 Hz  →  980 Hz,  sobald der OTA-Schirm offen war
```

Und damit war exakt jener ungebremste Zustand wiederhergestellt, der dem Render-Task die 126 Aussetzer gekostet hatte.

> **Ein Bezeichner, der eine andere Frage beantwortet als die, die an der Aufrufstelle gestellt wird, ist ein Fehler — auch wenn er sich richtig liest.** `IsRunning` klang nach „das Update läuft"; gemeint war „der Server ist bereit". Beides ist plausibel, und der Übersetzer prüft es nicht.

Richtig ist der Update-**Zustand**, nicht die Server-Bereitschaft.

15.5 Was daraus folgt

1. **Ein Taktwert ist zu begründen, nicht zu maximieren.** Die Frage lautet nicht „wie schnell geht es?", sondern „was ist die schnellste Größe, die dieser Loop bedienen muss?"
2. **Nach jeder Änderung am Takt ist die Bedienung neu zu vermessen** — Kapitel 16 zeigt an drei Belegen, was sonst geschieht.
3. **Diagnosezeilen sind Wachposten, keine Reste.** `[LOOPHZ]` steht bewusst ohne Debug-Level-Schranke im normalen Betriebsprotokoll (`main.cpp:938–941`); genau dadurch fiel der Sprung auf 980 Hz überhaupt auf. Die Messpunkte sind in Kapitel 27 beschrieben.

16 ⚠️ Die Bedienauswertung hängt an der Abtastrate

Dieses Kapitel behandelt die Klasse von Fehlern, die in diesem Projekt am häufigsten und am teuersten aufgetreten ist. Sie ist deshalb so schwer zu erkennen, weil sie **keinen** Fehler im eigenen Quelltext darstellt: Die betroffenen Stellen sind sämtlich korrekt geschrieben. Falsch ist eine unausgesprochene Annahme über die Geschwindigkeit, mit der sie aufgerufen werden.

16.1 Der Mechanismus

LVGL wertet Bedienvorgänge **je Abtastung** aus, nicht je Zeiteinheit. Der Eingabetreiber wird aus dem Anzeige-Task heraus abgefragt; wie oft das geschieht, ergibt sich aus dem Takt des Systems. Sämtliche Schwellwerte — Wischweg, Tippgenauigkeit, Reglerwert — beziehen sich auf **einzelne Abtastungen**, nicht auf Millisekunden.

Daraus folgt:

> **Ein und derselbe Fingerbewegungsablauf führt bei unterschiedlicher Abtastrate zu unterschiedlichen Ergebnissen.** Der Quelltext bleibt dabei unverändert.

Zwei gegenläufige Wirkungsrichtungen sind zu unterscheiden:

Abtastrate	Wirkung auf Wischgesten	Wirkung auf Widgets
niedrig	Weg wird zu grob erfasst, Geste geht verloren	Widget bekommt wenige Werte, wirkt träge
hoch	Geste wird zuverlässig erkannt	Widget übernimmt jede Zwischenposition, reagiert überempfindlich

Der derzeitige Systemtakt beträgt **235–330 Hz** (`[LOOPHZ]` im Betriebsprotokoll). Alle nachfolgend beschriebenen Schwellwerte sind auf diesen Bereich abgestimmt.

16.2 Wischgesten: `CR_Gesture`

LVGL summiert den Fingerweg in `gesture_sum` auf und meldet eine Geste, sobald die Summe `gesture_min_distance` überschreitet. Vor dem Aufsummieren steht jedoch ein Geschwindigkeitstor:

```
if ((LV_ABS(vect.x) < gesture_min_velocity) &&
    (LV_ABS(vect.y) < gesture_min_velocity)) {
    gesture_sum = 0;        // ← Summe wird VERWORFEN, nicht bloß nicht erhöht
}
```

⚠️ **Entscheidend ist, dass der Zähler bei Unterschreitung nicht stehen bleibt, sondern zurückgesetzt wird.** Die Prüfgröße `vect` ist die Bewegung **seit der letzten Abtastung** — bei hoher Abtastrate also ein sehr kleiner Wert. Mit LVGLs Voreinstellung `gesture_min_velocity = 3` müsste der Finger in **jeder einzelnen** Abtastung mindestens 3 Pixel zurücklegen, sonst beginnt die Wegsummierung von vorn. Bei 300 Abtastungen je Sekunde entspräche das einer Fingergeschwindigkeit, die im Betrieb nicht vorkommt.

Eingestellt ist deshalb (`CR_Gesture.cpp:28-29`):

Parameter	Wert	Bedeutung
`gesture_min_velocity`	**0**	Tor abgeschaltet — `LV_ABS(vect) < 0` trifft nie zu, der Zähler wird nie zurückgesetzt
`gesture_min_distance`	**50 px**	die eigentliche, abtastratenunabhängige Bedingung

Damit lautet die Regel schlicht: **„Der Finger hat sich um mehr als 50 Pixel in eine Richtung bewegt"** — unabhängig davon, in wie vielen Schritten das geschah.

Das Startprotokoll führt die Einstellung mit:

```
[GEST] Wisch-Arbitrierung gesetzt: min_velocity=0 (0 = Gate aus), min_distance=50 px
```

> ⚠️ **`gesture_min_velocity` darf nicht zur Lösung anderer Probleme heraufgesetzt werden.** Der Wert 0 ist selbst die Abhilfe gegen den Verlust von Wischgesten, und er wirkt **global auf alle Bildschirme**. Konflikte zwischen Widget und Geste sind örtlich zu lösen (Abschnitt 16.3), nicht über diesen Parameter.

16.3 Schieberegler: nur der Knopf nimmt den Finger an

Ein Schieberegler leitet seinen Wert in `update_knob_pos()` aus der **absoluten** Fingerposition her, und zwar bei **jeder** Abtastung mit Kontakt. Berührt der Finger die Schiene neben dem Knopf, springt der Wert sofort dorthin — noch bevor überhaupt eine Bewegung stattgefunden hat.

⚠️ **Eine Auswertung der Bewegungsrichtung greift hier zwangsläufig zu spät**: Im Augenblick des Aufsetzens ist die Bewegung null.

Eingestellt ist deshalb (`CR_SliderGuard.cpp` , angewandt auf alle drei Regler des Geräts):

```
lv_obj_add_flag(slider, LV_OBJ_FLAG_ADV_HITTEST);
lv_obj_set_ext_click_area(slider, 12);
```

`LV_OBJ_FLAG_ADV_HITTEST` ist LVGLs eigene Vorkehrung dafür (`lv_slider.c:268` , Kommentar dort: *„react only on dragging the knob(s)“*). Der Regler beantwortet die Trefferprüfung nur noch positiv, wenn der Druckpunkt in seiner **Knopffläche** liegt. Ein Kontakt auf der übrigen Schiene erreicht ihn nicht mehr — er springt nicht, und die Wischgeste läuft unbehelligt zum Bildschirm-Wurzelobjekt durch.

Der Knopf ist nur so breit wie der Regler hoch (20 px) und mit dem Finger allein kaum zu treffen. `ext_click_area` vergrößert deshalb **die Trefferfläche des Knopfes**, ohne die Schiene wieder scharf zu schalten (`lv_slider.c:271` rechnet den Wert im Trefferprüfungspfad auf die Knopffläche auf). 12 px Rand ergeben ein Ziel von rund 44 × 44 px — der gängige Richtwert für Fingerbedienung.

Für den verbleibenden Fall — der Finger greift den Knopf und wandert dann senkrecht — hält `CR_SliderGuard` den Wert fest. Dabei sind zwei Eigenschaften von LVGL zu beachten:

Beobachtung	Folge für die Umsetzung
`lv_slider_set_value()` löst **kein** `LV_EVENT_VALUE_CHANGED` aus (kein Vorkommen in `lv_bar.c`)	Der Nachtrag muss von Hand erfolgen, sonst verharren gekoppelte Größen wie die Bildschirmhelligkeit auf einem Zwischenwert
`update_knob_pos(obj, false)` leitet den Wert **beim Loslassen erneut** aus der Fingerposition her	Eine Korrektur allein während des Ziehens genügt nicht

⚠️ **Der Nachtrag von `LV_EVENT_VALUE_CHANGED` erfolgt genau einmal, beim Loslassen.** Bei jeder Abtastung nachgetragen wären es bis zu 330 Ereignisse je Sekunde, von denen jedes die volle Wirkungskette auslöst — bei der Helligkeit etwa das Neusetzen des PWM-Werts samt Beschriftung. Das Gerät wirkt dann spürbar hängend.

> ⚠️ **`lv_indev_wait_release()` ist an dieser Stelle keine Lösung.** Der Aufruf lässt `indev_proc_press()` ab der folgenden Abtastung sofort zurückkehren (`lv_indev.c:1408` , vor dem Aufruf von `indev_gesture()`) und friert damit die Wegsummierung ein, bevor sie die 50 px für den Bildschirmwechsel erreicht. Auf Bildschirmen, deren einziger Ausgang eine senkrechte Wischgeste ist, entstünde daraus eine Sackgasse.

16.4 Kacheln: Tippen gegen Wischen gegen Langdruck

Die Kacheln des Modulgitters unterscheiden drei Kontaktarten:

Kontakt	Wirkung	Schwelle
Tippen, Finger unter der Schwelle	Modul ein-/ausschalten	`CR_MODSCREENS_TAP_SLOP_PX` = **10 px**
Finger wandert weiter	Klick wird verworfen, Geste gilt	dieselbe Schwelle
Kontakt länger gehalten	Detailbildschirm öffnet sich	`LV_INDEV_DEF_LONG_PRESS_TIME` = **400 ms**

Die 10 px sind aus LVGLs eigenem `scroll_limit` übernommen — Projektkonvention: Schwellwerte werden aus dem Rahmenwerk übernommen, nicht geschätzt. Dieselbe Konstante findet sich in `CR_MsgScreen.cpp` und `CR_QuickReply.cpp` , seit 1.0.5.3 auch im Zeitzonen-Schirm.

⚠️ **Fall Zeitzonen-Schirm (1.0.5.3).** Dort stellte jeder Tipp auf eine Stadtzeile die Zone um. Ein Wisch, der auf einer Zeile begann, löste beim Loslassen trotzdem einen `CLICKED` aus. LVGL prüft beim Klick nicht, wie weit der Finger gewandert ist, sondern nur, ob ein Rollvorgang lief, und der Schirm rollt nicht. Wer von unten nach oben wegwischte, hatte damit eine andere Zeitzone eingestellt. Die Abhilfe folgt demselben Muster wie bei den Kacheln: `LV_EVENT_PRESSED` merkt sich

den Startpunkt (`cr_tz_pressed_cb`), und Zeilen- wie Blätterknopf wirken nur, wenn der Finger höchstens `CR_TZ_TIPP_SLOP_PX` = 10 px gewandert ist (`cr_tz_tipp_ruhig`).

🔴 Offener Befund: rund ein Drittel der Tippvorgänge bleibt wirkungslos

Auszählung eines Betriebsmitschnitts von 45 s (Firmware 1.0.0.7, Loop-Takt 291–329 Hz):

	Anzahl
Berührungen gesamt	**13**
davon Langdruck → Detailbildschirm	3
davon Modul geschaltet	3
davon Wischgeste	3
ohne erkennbare Wirkung	**4 (31 %)**

Beide Schwellwerte stammen aus einer Zeit, in der der Systemtakt bei 1 Hz lag. Bei 330 Hz wird jede Zitterbewegung des Fingers erfasst, die zwischen zwei Abtastungen zuvor unsichtbar blieb — die 10-px-Grenze wird damit deutlich früher überschritten. Die 400-ms-Grenze wirkt in dieselbe Richtung: Ein Bedienvorgang, der scheinbar wirkungslos bleibt, verleitet zum Nachdrücken, und Nachdrücken erzeugt einen Langdruck.

⚠️ **Nicht geraten wird an dieser Stelle.** Die Betriebsprotokollzeilen führen bereits die Koordinaten mit (`[TOUCH] X=… Y=… PRESSED`). Eine zusätzliche Zeile beim Verwerfen — mit der tatsächlich zurückgelegten Strecke — macht die Verteilung messbar. Erst danach ist zu entscheiden, ob die Schwelle anzuheben ist oder ob eine andere Auswertung nötig wird.

16.5 Regeln für Eingriffe in diesen Bereich

> **1. Eine Änderung des Systemtakts ist eine Änderung der gesamten Bedienung.** Nach jedem Eingriff in die Aufgabenverteilung, die Taktbremse oder die Anzeigeschleife ist die Bedienung vollständig neu zu prüfen — nicht nur an der Stelle, an der etwas auffällt.
>
> **2. Schwellwerte werden aus dem Rahmenwerk übernommen, nicht geschätzt.** 10 px stammen aus LVGLs `scroll_limit` , 50 px aus der Wischweg-Bedingung.
>
> **3. Konflikte zwischen Widget und Geste sind örtlich zu lösen.** Globale Eingabegeräte-Parameter wirken auf alle 19 Bildschirme.
>
> **4. Vor der Änderung eines Schwellwerts ist die tatsächliche Verteilung zu messen.** Ein heraufgesetzter Wert, der ein Symptom beseitigt, verschiebt es in aller Regel nur.

17 Die Schirm-Topologie

Die Uhr hat rund zwanzig Schirme und kein einziges Menü. Wie man trotzdem überall hinkommt — und warum niemand sich verläuft — entscheidet eine einzige Datenstruktur.

17.1 Jeder Schirm trägt seine Nachbarn selbst ein

Es gibt **keine zentrale Verdrahtungsstelle**. Jedes Modul meldet sich am Ende seiner `_Init()` selbst an und gibt dabei seine vier Nachbarn mit:

```
void CR_ScreenMgr_Register(CR_ScreenId id, const char *name, CR_ScreenGetRootFn getRoot,
                           CR_ScreenVisibleFn isVisible, CR_ScreenId ringLeft, CR_ScreenId ringRight,
                           CR_ScreenId vertUp, CR_ScreenId vertDown);
```

`CR_ScreenMgr.h:85`. Ein `CR_SCREEN_NONE` in einer Richtung heißt: Dorthin führt kein Weg.

Der Vorteil zeigt sich beim Ändern. Wer einen Schirm einhängt, fasst genau eine Stelle an — die seines eigenen Moduls. Der Nachteil ist die Kehrseite derselben Medaille: **Die Nachbarschaften sind gegenseitig, aber nichts erzwingt das.** Trägt A seinen rechten Nachbarn B ein und B nicht seinen linken Nachbarn A, entsteht eine Einbahnstraße — der Übersetzer merkt davon nichts.

> ⚠️ Deshalb wird die Nachbarschaftstabelle im Bedienhandbuch **aus dem Quelltext gelesen** und nicht abgeschrieben (`tools/handbuch.py` , `topologie_lesen()`). Eine abgeschriebene Tabelle wäre nach der ersten Änderung falsch, und niemand würde es merken.

17.2 Zwei Ringe, ein Kreuzungspunkt

Ring	Richtung	Was dort liegt
Hauptring	waagrecht	was man im Betrieb braucht: Meldungen, Senden, Module, Stationen
Säule	senkrecht	was man selten braucht: Info, Setup, Update

Beide sind **geschlossen** und kreuzen sich beim Ziffernblatt. Wer immer in dieselbe Richtung wischt, kommt dort wieder an. Das ist die tragende Entscheidung des ganzen Bedienkonzepts: Es gibt keinen Zustand, aus dem man nicht durch stures Weiterwischen herausfindet.

Der waagrechte Ring in der Reihenfolge, in der er durchlaufen wird:

```
Ziffernblatt → Meldungen → Meldung → Senden → Modul 1 → Stationen → Ziffernblatt
```

Die Säule ebenso:

```
Ziffernblatt ↓ Info ↓ Setup ↓ Update ↓ Ziffernblatt
```

Dass die Säule oben wieder beim Ziffernblatt herauskommt, steht ausdrücklich im Quelltext (`CR_Watchface.cpp:787`): `vertUp` zeigt auf das **letzte** Glied der Kette statt auf `CR_SCREEN_NONE` . Ohne OTA endet die Kette bei Setup, und die Schleife schließt sich dort.

17.3 Sackgassen sind erlaubt — aber nur nach unten

Nicht alles liegt im Hauptring. **Detailschirme** hängen als Sackgasse unter ihrer Kachel: Von der GPS-Kachel geht es runter ins GPS-Detail, von dort nur wieder hoch. Dasselbe gilt für das **Radar** unter den Stationen.

Das ist Absicht. Eine Sackgasse kostet niemanden einen Weg — der Hauptring bleibt unverändert, egal wie viele Details darunter hängen. Ein zusätzlicher Schirm *im* Ring dagegen verlängert für jeden den Weg, auch für den, der ihn nie braucht.

Die Kachelseiten sind der Grenzfall: Sie liegen **waagrecht** außerhalb des Rings, bilden **senkrecht** aber selbst einen geschlossenen Ring.

Schirm	←	→	↑	↓
Modul 1	Senden	Stationen	**Modul 2**	**Modul 2**
Modul 2	Senden	Stationen	**Modul 1**	**Modul 1**

Registriert wird beides in `CR_ModuleScreens.cpp`, bei den `CR_ScreenMgr_Register()`-Aufrufen für „Modul 1" und „Modul 2". Die Kopfzeilen lauten seit 1.0.5.43 **UHR** und **MESHCOM** (Anordnung nach Häufigkeit, Christian 06.10.). Die Detailseiten führen auf die Seite zurück, auf der ihre Kachel steht: LoRa (`CR_LoRaScreen.cpp`) und Gateway auf Seite 2, alle übrigen auf Seite 1. ⚠️ Wer eine Kachel auf die andere Seite legt, nimmt auch ihre Farbnachführung mit (Prüfung gegen `s_grid_root` bzw. `s_grid2_root` in der jeweiligen `*_tile_loop()`), sonst bleibt die Farbe ohne Fehlermeldung stehen.

Seite 2 trägt seit dem 6. August 2026 dieselben waagrechten Nachbarn wie Seite 1. Sie wird dadurch **kein** Ringglied — der Hauptring bleibt gleich lang, weil beide Seiten auf dieselben Ziele zeigen. Vorher stand dort zweimal `CR_SCREEN_NONE`, mit der Absicht, den Ring nicht zu verlängern. Befund Christians: *„da bleibe ich immer hängen und sehe sonst erst, dass ich auf Modulkachel 2 bin"*.

> ⚠️ **Die Lehre daraus** — sie gilt über diesen Fall hinaus: Ein wirkungsloser Wisch ist von einem hängenden Gerät nicht zu unterscheiden. Wer eine Seite aus dem Ring heraushalten will, spiegelt besser die Nachbarn der Hauptseite, statt gar keinen Weg anzubieten.

> ⚠️ **Bis zum 6. August 2026 war Seite 2 auch senkrecht eine Sackgasse**: `vertUp` von Seite 1 stand auf `CR_SCREEN_NONE`, `vertDown` von Seite 2 ebenso. Befund Christians beim Ausprobieren: *„es geht nur 1× runter und wieder 1× rauf wischen"*. Das war formal korrekt und in der Hand falsch — wer zweimal in dieselbe Richtung wischt, erwartet weiterzukommen, nicht stehenzubleiben.
>
> Bei **zwei** Seiten sieht der Ring wie bloßes Hin-und-Her aus, und das ist er auch. Sein Wert zeigt sich beim Erweitern: Eine dritte Kachelseite hängt sich ohne Änderung an dieser Stelle in die Kette (Seite 2 `vertDown` → Seite 3, Seite 3 `vertDown` → Seite 1).

17.4 Der Ringschluss beim Blättern

Die Meldungsliste ist der einzige Schirm, auf dem **senkrechtes Wischen nicht den Schirm wechselt**, sondern innerhalb des Schirms blättert. Vier Meldungen je Blatt (`CR_MSGSCREEN_LIST_ROWS`, `CR_MsgScreen.h:64`), hoch zu den älteren, runter zu den neueren.

Damit das geht, fängt `cr_msgscreen_gesture_cb()` die senkrechten Gesten ab und reicht nur die waagrechten an den Schirmverwalter weiter. Liste und Detail sind konsequent mit `vertUp = vertDown = CR_SCREEN_NONE` registriert — die senkrechte Richtung ist auf diesen beiden Schirmen schlicht für etwas anderes vergeben.

Bis zum 5. August 2026 endete das Blättern hart. War das letzte Blatt erreicht, war ein weiterer Wisch ein folgenloser No-op — „Ende ist Ende". Das klang beim Entwurf vernünftig und war in der Bedienung falsch:

> Ein Blatt, das stumm stehen bleibt, ist von einem hängenden Schirm nicht zu unterscheiden. Der Träger weiß nicht, ob seine Geste nicht erkannt wurde oder ob es nichts mehr zu sehen gibt — und wischt noch dreimal, um es herauszufinden.

Vor allem widersprach es dem Rest des Geräts: Jede andere Schleife der Uhr ist geschlossen. Seit `CR_MsgScreen.cpp:318` gilt das auch hier — hinter dem letzten Blatt kommt das erste, vor dem ersten das letzte:

```
const int maxOff = cr_msgscreen_max_offset();
if (maxOff == 0) return;                // alles passt auf EINE Seite
int next = s_scroll_offset + direction * CR_MSGSCREEN_LIST_ROWS;
if (next < 0)          next = maxOff;   // vor der ersten Seite -> ans Ende
else if (next > maxOff) next = 0;       // hinter der letzten Seite -> an den Anfang
```

Der Sonderfall oben ist der wichtige: Passt der gesamte Ringpuffer auf ein einziges Blatt, gibt es nichts zu blättern. Ohne diese Zeile liefe die Liste bei jedem Wisch auf sich selbst um und würde sich neu zeichnen — sichtbares Flackern ohne jede Wirkung.

`cr_msgscreen_max_offset()` liefert den Anfang des letzten Blattes, **abgerundet** auf ein Vielfaches der Zeilenzahl. Bei 11 Meldungen und vier Zeilen sind das drei Blätter mit den Offsets 0, 4 und 8; das letzte zeigt nur drei Einträge. Genau dieser abgerundete Wert ist zugleich das Sprungziel für den Umlauf nach unten.

17.5 Eine Namensfalle, die ins Handbuch durchschlug

Der Name in `CR_ScreenMgr_Register()` ist kein reiner Bezeichner — er erscheint in der Kopfzeile des Schirms, in jeder `[SCRMGR]`-Protokollzeile und in der generierten Nachbarschaftstabelle des Bedienhandbuchs.

Das Ziffernblatt hieß dort bis zum 5. August 2026 `"Watchface"`. Im Handbuch stand dadurch ein Wort, das auf keinem Schirm der Uhr zu sehen ist, und die Tabelle verwies auf einen Schirm, den das umgebende Kapitel anders nannte. Der Name heißt jetzt `"Ziffernblatt"` (`CR_Watchface.cpp:753`).

> ⚠️ **Lehre:** Was in die Registry geschrieben wird, ist Anzeigetext, kein Programmierername. Wer hier einen englischen Bezeichner einträgt, veröffentlicht ihn — im Zweifel in einem Handbuch, das jemand ausdruckt.

Derselbe Fehler steckte doppelt in der Tabelle selbst: Die Spalte „Schirm" nahm den registrierten Namen, die vier Nachbarspalten leiteten sich einen aus der Enum-Kennung ab (`Module Grid2` statt `Modul 2`). Ursache war die Reihenfolge — ein Schirm nennt seine Nachbarn über die Kennung, deren Klartextname aber steht in einer anderen Datei und ist erst bekannt, wenn alle Registrierungen gelesen sind. Der Generator läuft deshalb seit demselben Tag in zwei Durchgängen.

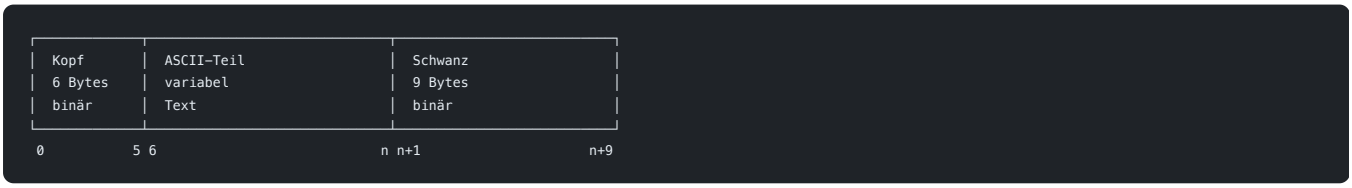
18 Der MeshCom-Paketaufbau

Dieses Kapitel beschreibt ein MeshCom-Paket Byte für Byte. Es ist der Teil der Firmware, bei dem Raten am teuersten war: Ein falsch belegtes Byte führt nicht zu einer Fehlermeldung, sondern zu **Stille** — das Netz nimmt das Paket entgegen, quittiert es sogar, und wirft es weg.

> Die Beschreibung stammt aus zwei Quellen: aus mitgeschnittenen Paketen (2026-08-02 bis 2026-08-05) und aus der MeshCom-Firmware selbst (`icssw-org/MeshCom-Firmware` , Zweig `dev` , `src/aprs_functions.cpp`). Wo beide vorliegen, gilt die Quelle — die Messung sagt, was *ist*, die Quelle sagt, was *geprüft wird*.

18.1 Der Aufbau im Überblick

Ein Paket besteht aus drei Teilen. Nur der mittlere ist lesbar.

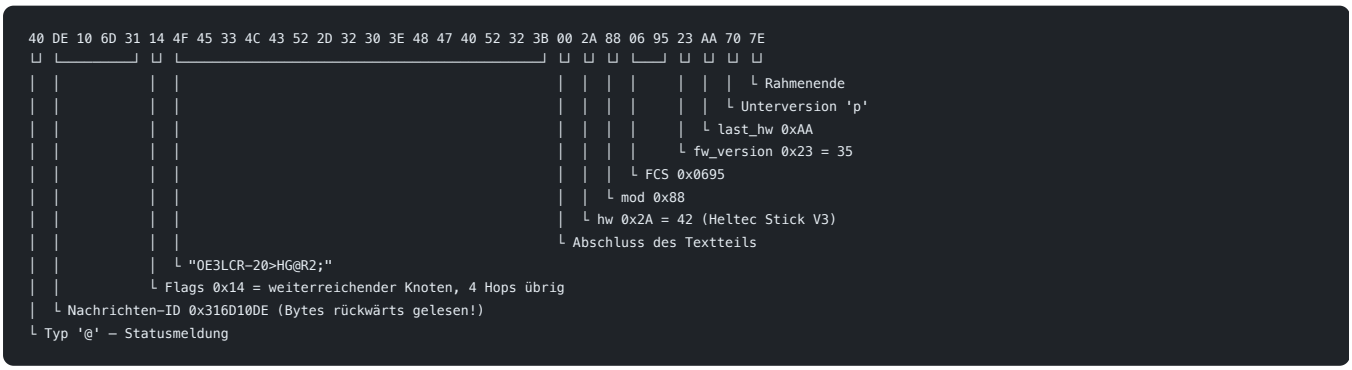


Teil	Bytes	Inhalt
Kopf	0	Typzeichen
	1–4	Nachrichten-ID, **little endian**
	5	Flags (Hops, Betriebsart, Herkunft)
ASCII	6 … n	`ABSENDER[,HOP…]>ZIEL<Typ>TEXT`
Schwanz	n+1	`0x00` — Abschluss des Textteils
	n+2	Hardware-Kennung des Absenders
	n+3	Modulation + Landeskennung
	n+4, n+5	Prüfsumme (FCS), **high byte zuerst**
	n+6	Firmware-/Protokollfassung
	n+7	Kennung der zuletzt sendenden Station **mit Bit 0x80**
	n+8	Unterversion
	n+9	`0x7E` — Rahmenende

Aufgebaut wird der Schwanz in `cr_mesh_append_tail()` (`CR_MeshCom.cpp:353`), der Kopf und der ASCII-Teil in `cr_mesh_build()` (`:363`).

18.2 Ein echtes Paket, vollständig zerlegt

Mitgeschnitten am 5. August 2026, 31 Bytes. Es ist ein **Original** — der Absender hat es selbst erzeugt, kein Digipeater hat es angefasst:



> ⚠️ **Zur Entstehung dieser Zeichnung** (Befund OE3WAS, 6. August 2026): In der ersten Fassung war die Klammer über dem Textteil **sechs Spalten zu kurz**, wodurch jede Beschriftung rechts davon auf das falsche Byte zeigte — „hw" stand über `3B`, dem Semikolon des Textteils. Solche Zeichnungen gehören **berechnet**, nicht von Hand gesetzt: Jedes Byte belegt drei Spalten (`XX` + Leerzeichen), eine Gruppe von *n* Bytes also `3n − 1`. Wer nachträglich ein Byte einfügt, verschiebt alles dahinter.

Dasselbe Paket, wenn unsere Uhr es sendet

Der Aufbau ist identisch — nur zwei Bytes unterscheiden sich, und beide sagen aus, *welches Gerät* gesendet hat:

```
… 3B 00 3D 88 06 95 23 BD 70 7E
        ⌊          |  ⌊
        |          |  └ last_hw 0xBD  (0x3D | 0x80 — dieselbe Uhr, zuletzt gesendet)
        |          └ fw_version 0x23 = 35 (unverändert — siehe unten!)
        └ hw 0x3D = 61  (T-Watch-S3, unsere Kennung)
```

61 ist die Kennung, die Kurt (OE1KBC) unserer Uhr am 5. August 2026 zugeteilt hat; sie steht in der MeshCom-Firmware als `#define T_WATCH_S3 61`. Im Quelltext dieses Zweigs: `CR_MESH_SOURCE_HW` in `CR_MeshCom.h`.

⚠️ `last_hw` **ist nicht willkürlich:** Es ist dieselbe Kennung mit gesetztem obersten Bit — aus `0x2A` wird `0xAA`, aus unserem `0x3D` wird `0xBD`. Beim Original stimmen beide Felder überein; erst eine Weiterleitung lässt sie auseinanderlaufen (siehe das zweite Beispiel unten).

⚠️ `fw_version` **bleibt 35, obwohl unsere Firmware anders zählt.** Das ist kein Versehen, sondern Vorschrift: Das Netz verwirft Pakete, deren `fw_version` zwischen 0 und 35 liegt. Wer hier seine eigene Versionsnummer einträgt, sendet unsichtbar.

Das `mod` **-Byte ist seit 1.0.3.58 zusammengesetzt, nicht fest:** unteres Nibble die Modulation (fest `8` — SF11/CR6/BW250, diese Uhr stellt ihre Funkparameter nicht um), oberes Nibble die **Landeskennung** `iCTRY` aus dem NVS (`CR_MeshCom_SourceMod()`). Beim Default EU8 (`iCTRY = 8`) ergibt das bitidentisch das `0x88` aller früheren Mitschnitte. Umstellbar per `--setCTRY` — angenommen werden nur EU8 und US, die auf dieser Antenne funkgleich sind; 868/915-MHz-Länder lehnt der Parser mit Begründung ab (Stufe 2, die echte SX1262-Umkonfiguration, ist bewusst nicht gebaut).

Dasselbe Paket, sechs Sekunden später wieder gehört — diesmal **weitergereicht**:

```
… 40 DE 10 6D 31 93 4F 45 33 4C 43 52 2D 32 30 2C 4F 45 33 4C 43 52 2D 30 32 3E … 00 2A 88 0B 0F 23 8A 70 7E
                  ⌊                              └──────────────────────┘              ⌊       ⌊
                   |                                zweites Rufzeichen im Pfad           |       |
                   └ Flags 0x93 = 0x80 (über MQTT) + 0x10 + 3 Hops                      |       └ last_hw 0x8A
                                                                                        └ hw 0x2A unverändert
```

Drei Dinge sind an diesem Vergleich zu lernen:

1. Die **Nachrichten-ID bleibt gleich** (`DE 10 6D 31`). Nur an ihr lässt sich erkennen, dass es dieselbe Meldung ist — der Inhalt allein genügt nicht.
2. Der **Hop-Zähler sinkt** (4 → 3), und Bit 0x80 kam hinzu: Diese Fassung lief über den MQTT-Server.
3. `hw` bleibt **0x2A**, aber `last_hw` wechselt von **0xAA** auf **0x8A**. Die beiden Felder benennen verschiedene Stationen: `hw` ist das Gerät, das die Meldung *verfasst* hat, `last_hw` das, welches sie *zuletzt gesendet* hat. Beim Original sind beide identisch, bei jeder Weiterleitung nicht mehr.

18.3 Byte 0 — die vier Typen

Zeichen	Hex	Bedeutung	Wohin es führt
`:`	0x3A	Textnachricht	Meldungsliste
`!`	0x21	Position mit Klartextkoordinaten	Stationsliste und Radar
`@`	0x40	Status-/Positionsmeldung in Kurzform	wird gelesen, nicht angezeigt
`A`	0x41	Empfangsbestätigung	färbt die eigene Sendekarte grün

Alles andere wird verworfen (`CR_MeshCom.cpp:192`) — lieber ein Paket weniger als eine erfundene Meldung.

> ⚠️ **Das Typzeichen steht ZWEIMAL im Paket**: einmal als Byte 0 und ein zweites Mal im ASCII-Teil hinter `QUELLE>ZIEL`. Das ist kein Fehler, sondern der Aufbau der Quelle (`snprintf("%s>%s%c%s", …)`). Wer nur eines der beiden setzt, baut ein Paket, das der Zerleger der Gegenstelle nicht auseinandernimmt.

18.4 Bytes 1–4 — die Nachrichten-ID

Little endian: niederwertigstes Byte zuerst. Aus `DE 10 6D 31` wird `0x316D10DE`.

Der Aufbau ist nicht zufällig. Die oberen drei Bytes sind je Station konstant, das unterste zählt hoch — abgelesen an echten Aussendungen und in `CR_MeshCom_NewMsgId()` (`:326`) nachgebaut:

```
uint32_t h = 2166136261u;              // FNV-1a
for (const char *p = call; *p; p++) { h ^= (uint8_t)*p; h *= 16777619u; }
s_station = h & 0x00FFFFFFu;           // 24 Bit aus dem Rufzeichen
…
uint32_t id = (s_station << 8) | s_counter;
```

Die Stationskennung wird **aus dem Rufzeichen abgeleitet** und ist deshalb über Neustarts hinweg stabil. Der Zähler startet dagegen bei `millis() & 0xFF` — sonst vergäbe die Uhr nach jedem Neustart wieder dieselben IDs, und das Netz hielte neue Meldungen für Wiederholungen.

18.5 Byte 5 — die Flags

Bit	Maske	Bedeutung
0–3	`0x0F`	verbleibende Weiterleitungen (max_hop)
4	`0x10`	`bMESH` — **ich reiche fremde Pakete weiter**
5	`0x20`	`app_offline`
6	`0x40`	`track`
7	`0x80`	lief über den MQTT-Server

Die Uhr sendet seit 1.0.5.33 wie die Vorlage **4 Hops bei Text** (und Quittungen) und **2 bei Position** (`CR_MESH_MAX_HOP_TEXT` / `_POS`, Vorlage `configuration_global.h:363-364`); bis 1.0.5.32 waren es 3 für beides. Bit 0x10 folgt seit 1.0.5.33 dem Digipeater-Schalter (`en_MESHRELAY`) — eine Selbstauskunft muss stimmen (siehe Kapitel 19.5). Am Gerät belegt: Bake mit Byte 5 = `0x12`.

Bit 0x80 ist beim Empfang die einzige Möglichkeit zu unterscheiden, ob ein Nachbar per Funk oder ein Gateway über MQTT geantwortet hat.

Hop-Zähler über 7 (seit 1.0.5.25). Ältere Knoten ohne App oderten `0x20` in `max_hop`; stand das Nibble auf 0, wurde daraus beim Herunterzählen `0x1F`, also 15 Hops auf der Luft (Abgleich v4.35v, R1). Die Vorlage behebt das nur beim Erzeuger. Mehr als `MAX_HOP_LIMIT` 7 kann regulär nicht entstehen; `CR_MeshCom_BuildRelay()` reicht so ein Paket deshalb nicht weiter (`[MESH] Weiterleitung abgelehnt: Hop-Zaehler …`). Angezeigt wird es trotzdem.

18.6 Der ASCII-Teil

```
OE3LCR-55>*:Hier ist ein Text
└────┬────┘│└┬└──────┬──────┘
     │     ||  │      └ Nutztext, UTF-8 (Umlaute und Emojis gehen durch)
     │     ||  └ Typzeichen, zum zweiten Mal
     │     |└ Ziel: "*" = an alle, sonst eine Gruppennummer
     │     └ Trennzeichen
     └ Absender, volles Rufzeichen mit SSID-Suffix
```

Ist die Meldung über Digipeater gelaufen, stehen deren Rufzeichen **kommagetrennt hinter dem Absender**: `OE3LCR-20,OE3LCR-02>…`. Der Absender ist immer der Teil vor dem ersten Komma.

> Genau daran lassen sich Originale von Weiterleitungen unterscheiden — **ein Original hat kein Komma im Pfad**. Diese Unterscheidung war der Schlüssel zur Lösung des Sendeproblems (Kapitel 19.1).

18.7 Der Positionsteil

Bei Typ `!` ist der Nutztext kein Freitext, sondern ein festes Format:

```
4748.77N/01614.19E[ Kommentar/A=000873/B=100
└──┬──┘│└──┬──┘└┘
   │   ||   │    └ Symbolzeichen – '[' = Person zu Fuß
   │   ||   │      (Symboltabelle steht VOR der Länge)
   │   ||   └ Länge: DDDMM.mm, drei Stellen Grad
   │   │└ Symboltabelle '/'
   │   └ Halbkugel N/S
   └ Breite: DDMM.mm, zwei Stellen Grad
```

Zusatz	Bedeutung
`/A=000873`	Höhe in **Fuß**, sechsstellig mit führenden Nullen
`/B=100`	Akkustand in Prozent

> ⚠️ **Grad und Minuten, nicht Dezimalgrad.** `4748.77N` sind 47° 48,77′ = **47,8128°**. Wer die Ziffern als Kommazahl liest, erhält 47,4877 — rund **40 km daneben**. Der Fehler fällt bei Stationen in der Nachbarschaft kaum auf, weil die Position ungefähr stimmt. Die Umrechnung steht deshalb als eigene Funktion mit dieser Warnung (`cr_mesh_parse_position()` , `:141`).

> ⚠️ **Die Höhe geht in Fuß hinaus.** 873 Fuß sind 266 m. Wer Meter einsetzt, meldet sich rund dreimal zu tief.

Das Symbolzeichen ist `[` (Person), **nicht** das Voreingestellte `#` der Quelle — `#` ist das APRS-Zeichen für einen **Digipeater**, und als solcher darf sich die Uhr nicht ausgeben.

18.8 Die Quittung — ein Paket, das die Regeln bricht

Eine Empfangsbestätigung ist **kein gewöhnliches Paket**. Sie hat genau 12 Bytes, keinen ASCII-Teil, kein Rufzeichen — und weder Prüfsumme noch Rahmenende:

Byte	Inhalt
0	`'A'`
1–4	**eigene, neue** Nachrichten-ID der Quittung
5	Flags (Hops; **ohne** 0x80 — das setzt nur ein Gateway)
6–9	die **bestätigte** Nachrichten-ID
10	`0x00` = Knoten, `0x01` = Gateway
11	`0x00` — Abschluss

`CR_MeshCom_BuildAck()` (`:441`). Die beiden IDs sind der Kern: Byte 1–4 identifiziert *diese Quittung*, Byte 6–9 sagt, *worauf* sie sich bezieht.

> ⚠️ Wer hier `cr_mesh_append_tail()` anhängt — weil es bei allen anderen Paketen so ist —, baut ein Paket, das die Gegenstelle nicht kennt.

18.9 Die Prüfsumme

Zwei Bytes, high byte zuerst, gebildet als **16-Bit-Summe aller vorangehenden Bytes** — einschließlich der beiden unmittelbar davorstehenden (`hw` und `mod`):

```
unsigned int fcs = 0;
for (size_t i = 0; i < pos; i++) fcs += (unsigned int)out[i];
out[pos++] = (uint8_t)((fcs >> 8) & 0xFF);
out[pos++] = (uint8_t)(fcs & 0xFF);
```

Keine CRC, keine Polynomdivision — eine schlichte Addition mit Überlauf. Am Beispielpaket aus 18.2 nachgerechnet ergibt sie exakt `0x0695` .

18.10 Mindestlängen und Puffergrößen

Konstante	Wert	Bedeutung
`CR_MESH_MIN_LEN`	10	kürzer wird gar nicht erst zerlegt
`CR_MESH_CALL_LEN`	16	Rufzeichen samt SSID
`CR_MESH_PATH_LEN`	64	vollständiger Pfad mit allen Hops
`CR_MESH_TEXT_LEN`	180	Nutztext
`CR_MESH_SEEN_SLOTS`	16	Gedächtnis für die Entdopplung

Die Textlänge ist an `CR_MsgScreen` angelehnt, damit beim Weiterreichen nichts abgeschnitten wird, was dort noch hineinpasst.

19 Senden: die Fallstricke

Empfangen war an einem Nachmittag fertig. Senden hat vier Tage gedauert — nicht weil es schwer wäre, sondern weil **jeder Fehler gleich aussieht**: Das Paket geht hinaus, wird quittiert, und erscheint auf keinem Knoten.

Dieses Kapitel sammelt jeden Fallstrick, in den wir tatsächlich getreten sind, samt der Frage, die ihn am Ende gelöst hat.

19.1 Der systematische Denkfehler

Die drei Felder `hw`, `fw_version` und `sub_version` standen monatelang auf `0x00`. Sie waren **aus mitgeschnittenen Paketen abgeleitet** — und dort *sind* sie null.

Der Haken: Die Mitschnitte enthielten überwiegend **weitergereichte** Pakete. Ein Digipeater setzt diese Felder beim Weitersenden auf null; nur ein **Original** trägt echte Werte.

Sichtbar wurde es erst, als der Mitschnitt nach einem einfachen Merkmal getrennt wurde: **Hat der ASCII-Teil ein Komma im Pfad?** Ohne Komma ist es ein Original.

```
Original (fremd):  00 2A 88 06 53 23 AA 70 7E     ← hw 0x2A, fw 0x23, last_hw 0xAA
Unsere Aussendung: 00 00 88 xx xx 00 00 23 7E     ← alles null, und 0x23 an falscher Stelle
```

> **Lehre:** Ein Protokoll lässt sich aus Messdaten ableiten — aber nur, wenn man weiß, **welche Rolle** der Sender des Musters hatte. Wer Originale und Weiterleitungen in einen Topf wirft, leitet den Aufbau eines weitergereichten Pakets ab und hält ihn für den allgemeinen Fall.

Danach wurde die Quelle gelesen statt weiter gemessen. Das ist die zweite Lehre, und sie war schon einmal fällig: Beim Decoder hatte Raten 80 % gebracht — die fehlenden 20 % kosteten mehr Zeit als das Lesen der Quelle gekostet hätte.

19.2 `fw_version = 0` — der Fehler, der alles erklärte

Das war die Ursache. Der Zerleger der Gegenstelle verwirft Pakete mit falscher Firmwarefassung:

```
// aprs_functions.cpp:467
if (fw_version > 0 && fw_version < 35) { /* "Packet discarded, wrong FW-version" */ }
```

Unsere `0` rutschte **nur durch die Lücke** in dieser Bedingung: `> 0` nimmt die Null aus. Das Paket wurde also nicht wegen, sondern **trotz** seiner Angabe angenommen — von einer Nachlässigkeit im fremden Code, auf die sich nichts verlassen darf.

Richtig ist **35**. Die Zahl entsteht in der Quelle aus `shortVERSION()`, das aus `"4.35"` die Stellen 2–3 herausschneidet. Bestätigung aus zweiter Richtung: `WZ_UDP.cpp:14` definiert für Wolfgangs KEEP-Telegramme längst `SOURCE_VERSION "4.35"`.

19.3 Das fehlende Pflicht-Bit im `last_hw`

Bit **0x80** ist in `last_hw` Pflicht — die Quelle setzt es ausnahmslos („mit lastHeard Bit", `aprs_functions.cpp:115`). Unseren Aussendungen fehlte es ganz.

Gegenprobe an echten Originalen:

gesehene `hw`	zugehöriges `last_hw`
0x0A (10)	0x8A
0x2A (42)	0xAA
0x3D (61)	**0xBD** ← unsere seit dem 5. August

Deshalb leitet sich der Wert im Quelltext ab, statt zweimal gepflegt zu werden:

```
#define CR_MESH_LAST_HW  (0x80 | CR_MESH_SOURCE_HW)
```

19.4 Die Unterversion an der falschen Stelle

Wir schrieben `0x23` (`'#'`) an die Position der Unterversion. Die Quelle setzt `'#'` dort **nur, wenn gar keine Unterversion vorliegt**. Wir sprechen 4.35**p**, also gehört `0x70` hin — und die `0x23` gehört zwei Bytes weiter nach vorn, als `fw_version`.

Beides zusammen ergab ein Paket, in dem **dieselbe Zahl an der falschen Stelle stand**. Solche Fehler sind besonders zäh, weil das Byte im Mitschnitt vorkommt und dadurch plausibel wirkt.

Die richtige Reihenfolge (`encodeAPRS()`, Zeile 1082–1090):

```
… FCS_hi | FCS_lo | fw_version | last_hw | sub_version | 0x7E
```

19.5 Ein Bit, das eine Unwahrheit behauptete

Bit `0x10` in den Flags stand fest gesetzt. In der Quelle hängt es an `bMESH` — und **dieselbe Variable** entscheidet, ob ein Knoten fremde Pakete weiterreicht (`lora_functions.cpp:297`); ihr Standardwert ist `false`.

Das Bit ist also die Selbstauskunft *„ich bin ein weiterreichender Knoten"*. Bis 1.0.5.32 stand es fest auf 0 — auch noch, als die Uhr seit 260806 weiterleiten konnte. Seit 1.0.5.33 folgt es dem Digipeater-Schalter:

```
const uint8_t hops = (type == CR_MESH_TYPE_TEXT) ? CR_MESH_MAX_HOP_TEXT : CR_MESH_MAX_HOP_POS;
out[pos++] = (uint8_t)((CR_MeshCom_IstDigipeater() ? 0x10 : 0x00) | (hops & 0x0F));
```

> Das Bit im *empfangenen* Paket steuert nichts — es landet nur im mheard-Verzeichnis. Weglassen kostet uns also nichts und behauptet nichts Falsches. **Wer es setzt, muss vorher die Weiterleitung gebaut haben.**

19.6 Die Prüfsumme aus einem fremden Paket

Der allererste Sendeversuch (2026-08-02) trug einen Abschluss, der aus einem mitgeschnittenen fremden Paket kopiert war — **samt dessen Prüfsumme**. Das Netz verwarf die Aussendung stillschweigend; kein Digipeater reichte sie weiter.

Die FCS **muss** über den eigenen Inhalt gerechnet werden. Sie ist die einzige Stelle des Pakets, die sich nicht abschreiben lässt.

19.7 Grün beweist weniger, als es aussieht

Der teuerste Irrtum war kein Byte, sondern eine Fehlannahme über die Rückmeldung:

> **Eine Quittung bedeutet: irgendein Knoten in Reichweite hat den Rahmen angenommen.** Sie bedeutet **nicht**, dass die Meldung auswertbar war oder irgendwo angezeigt wurde.

Digipeater prüfen den Rahmen — Länge, Prüfsumme, Struktur. Den Inhalt prüft erst die Station, die ihn anzeigen soll. Deshalb kamen tagelang grüne Karten für Pakete, die nirgends ankamen.

Seit dem 5. August nennt das Protokoll deshalb **den Absender der Quittung** und ob sie über ein MQTT-Gateway kam (Bit 0x80). Dazu meldet die Uhr `>>> WEITERGEREICHT`, wenn sie ihre eigene Aussendung über einen Digipeater zurückhört — das ist der einzige harte Beleg dafür, dass das Netz sie tatsächlich verteilt. Vorher fiel dieser Fall stumm unter die Entdopplung.

19.8 Position: schweigen ist besser als raten

```
if (lat == 0.0 && lon == 0.0) return 0;
```

Ohne echten Fix wird **nicht** gesendet. Die Quelle hält es genauso („position 0.0/0.0 no message sent").

Die Sendekarte (Baustein 0 der Schnellantwort) kennt seit 260821-v8l keinen festen Heimat-Rückfall mehr, sondern dieselbe mehrstufige Kette: aktueller Fix → zuletzt gemessener Fix dieser Laufzeit (ohne Altersgrenze) → konfigurierte Position (`--setLAT` / `--setLON`, gleiche Plausibilitätsprüfung wie die Bake) → keine Angabe. Fehlt jede Quelle, entfällt der Standort-Teil der Meldung komplett — auch hier gilt: eine erfundene Position wäre schlimmer als gar keine.

Der Zerleger verwirft 0/0 auch beim **Empfang**: Der Nullpunkt liegt im Golf von Guinea und ist als echte Stationsposition praktisch ausgeschlossen — er ist das Kennzeichen eines Geräts, das ohne Fix trotzdem sendet.

19.8.1 Die Quellenwahl der Bake (seit 1.0.4.50, E-35)

Bis 1.0.4.49 baute `wz_lora_build_pos_packet()` das Bakenpaket **ausschließlich** aus der Knotenposition `fLATpos` / `fLONpos` / `iALTpos`. Die wird nur von einem echten Fix (`CR_GpsFix_Loop()`, E-17) oder über `--setLAT` / `--setLON` gesetzt — **nie** aus der Telefonposition. Auf einer T-Watch S3 ohne „Plus" (kein GNSS, E-32) sendete die Bake damit nie das, was die Uhr wusste.

Seit 1.0.4.50 gibt es **genau eine Stelle**, die entscheidet, welche Position hinausgeht:

```
static wz_posq_t wz_lora_pos_quelle(wz_pos_t *out);   // WZ_LoRa.cpp
```

Ablauf, in dieser Reihenfolge:

1. `CR_GpsFix_GetTxPosition()` fragen. Liefert sie `true`, ist die Position sendefähig: entweder aktueller Fix (`CR_GpsFix_HasFix()` → `WZ_POSQ_FIX`) oder eine **in dieser Laufzeit gemessene** Position, jünger als `CR_GPSFIX_TX_MAX_AGE_S` (3600 s). Ob die vom Telefon stammt, erkennt die Quellenwahl daran, dass `CR_GpsFix_PhoneAgeS()` und das Alter der gewählten Position übereinstimmen (±1 s für die Sekundenteilung) — `CR_GpsFix_SetFromPhone()` setzt beide Zeitstempel auf denselben Wert. Ergebnis `WZ_POSQ_APP`, sonst `WZ_POSQ_LIVE`.
2. Liefert sie `false`, greift der Rückfall auf die gespeicherte Knotenposition — **aber nur bei** `hw_GPS != CR_HWGPS_KEINS`. Auf einer Uhr ohne GNSS schweigt die Bake stattdessen (`WZ_POSQ_KEINE`) und nennt den Grund im Log.
3. Erst danach laufen die drei Plausibilitätsprüfungen (Platzhalter `CR_POS_DEFAULT_LAT/LON` mit `fabs(...) < 1e-5`, Wertebereich, 0/0) — jetzt auf dem **gewählten** Kandidaten statt fest auf `fLATpos`.

⚠️ **`CR_GpsFix_GetTxPosition()` war bis 1.0.4.50 toter Code** — die Funktion existierte samt Doxygen-Kopf und begründeter Frist, hatte aber **keinen einzigen Aufrufer**, nur Erwähnungen in Kommentaren. Das erklärt, warum ihr Kopf länger existierte als ihr Aufrufer, und ist der Grund, warum dieser Umbau **keine neue Funktion und keine neue Frist** erfunden hat.

Der Track-Automat bleibt fix-gebunden. `wz_lora_track_loop()` steigt weiterhin bei `if (!en_TRACK || !CR_GpsFix_HasFix()) return;` aus; die Quellenwahl liefert dort also immer `WZ_POSQ_FIX`. Er benutzt sie nur, um dieselben Werte aus **einer** Quelle statt aus zweien zu lesen — kein neues Verhalten.

Die Frist ist bewusst nicht die der ECOGPS-Sparpause (600/900 s): ECOGPS entscheidet über die Stromschiene, die Bake über das, was ins Netz geht. Eine zweite Zahl neben `CR_GPSFIX_TX_MAX_AGE_S` wäre die Doppelquelle, die wir sonst überall vermeiden.

`WZ_LoRa_BeaconPosSource()` gibt den Quellennamen für Anzeigen zurück (`--info`: `Bakenposition: …`) — sie fragt nur die Quellenwahl, baut kein Paket und sendet nichts.

19.9 Die Sperren beim Senden

Sperre	Wert	Warum
Mindestabstand	**10 s**	verhindert Fluten und den Doppeltipp am Handgelenk
Frist für die Quittung	**30 s** (bis August 15 s)	danach gilt die Aussendung als nicht angekommen; eine spätere Quittung macht die Karte nachträglich grün
Bake-Vorrang	—	die Positionsbake weicht **jeder** Benutzermeldung

> ⚠️ Wird vor Ablauf der 10 Sekunden erneut getippt, färbt sich die Karte **sofort rot** — obwohl nichts fehlgeschlagen ist. *Rot unmittelbar nach dem Tipp* heißt „zu früh", *rot nach 30 Sekunden* heißt „keine Quittung". Die Unterscheidung liegt in der Zeit, nicht in der Farbe.

19.10 Wo gesendet werden darf

`radio.transmit()` **blockiert** — bei SF11 mehrere hundert Millisekunden. Zwei Regeln folgen daraus:

1. **Nur aus dem Haupt-Task.** Im Render-Task würde die Aussendung das Bild einfrieren.
2. **Keine `lv_*`-Aufrufe im Empfangspfad.** LVGL gehört dem Render-Task; der Empfang läuft im Haupt-Task, angestoßen von einem Interrupt-Flag. Die Brücke dazwischen ist `CR_MsgBridge`.

Beide Regeln stehen ausführlich in Kapitel 13.

19.11 Was NICHT das Problem war

Zwei plausible Thesen, die geprüft und **widerlegt** wurden — sie stehen hier, damit sie niemand erneut verfolgt:

These	Befund
„Die Hardware-Kennung wird gefiltert"	❌ Alle vier Fundstellen geprüft: gesetzt, gelesen, angezeigt — **nie gefiltert**. Wirkung hat allein das 0x80-Bit im Nachbarbyte.
„Die Nodes halten `OE3LCR-55` für sich selbst"	❌ `is_equ()` vergleicht Länge **und** Inhalt; `-55` und `-20` sind verschiedene Stationen.

Ebenfalls gegengeprüft und in Ordnung: Prüfsummenformel (an einem vollständigen Fremdpaket nachgerechnet, `0x0653` exakt getroffen), Feldreihenfolge, Typbyte, ID-Bytefolge, Flags, das Format `CALL>*:TEXT` und `MOD 0x88`.

19b Kollisionserkennung: CSMA/CA mit Hardware-CAD

Bis zum 6. August 2026 sendete diese Uhr **blind**. Wollte sie funken, funkte sie — ohne vorher zu horchen, ob gerade jemand anderer auf dem Kanal ist.

Das ist kein Schönheitsfehler. Es ist der Fehler, der ein Funknetz von innen beschädigt.

19b.1 Warum eine Kollision doppelt weh tut

LoRa hat keinen Schiedsrichter. Kein Knoten teilt Sendezeiten zu, es gibt keine Reihenfolge und keine Rückmeldung „warte noch". Zwei Aussendungen, die sich zeitlich überlappen, überlagern sich im Empfänger zu Unsinn — die Prüfsumme schlägt an, und **beide** Pakete sind weg.

Der Schaden ist deshalb nie nur der eigene:

verloren geht	Folge
die eigene Aussendung	man merkt es und wiederholt — ärgerlich, aber harmlos
die fremde Aussendung	der andere merkt nichts. Er hat gesendet, es kam nur nichts an
alles, was daraus gefolgt wäre	jeder Knoten, der dieses Paket weitergereicht hätte, reicht nichts weiter. Der Verlust wandert durch das ganze Netz

Die dritte Zeile ist die teure. In einem Flood-Netz wie MeshCom lebt eine Meldung davon, dass Knoten sie wiederholen. Wer eine Aussendung zerstört, löscht nicht ein Paket, sondern einen **Ast** des Verteilbaums.

⚠️ Und der Zerstörer erfährt davon nichts. Eine Kollision sieht am eigenen Gerät genauso aus wie ein erfolgreicher Sendevorgang: Paket hinaus, kein Fehler, fertig.

19b.2 Woher das Verfahren kommt

Das Verfahren ist nicht unsere Erfindung und soll es nicht sein. Es stammt von **Martin, DK5EN** aus der MeshCom-Firmware (`icssw-org/MeshCom-Firmware`, 45 Commits). Wie schwer das Problem im echten Netz wog, sagt einer seiner Commit-Titel deutlicher als jede Erklärung:

> `Fix LoRa RX blind windows causing ~32% message loss`

Ein Drittel aller Meldungen. Weitere Bausteine derselben Arbeit: *„Priority-Queue, Trickle-HEY RFC 6206, CSMA-Prioritaeten, Statistik"*, *„CAD-Spinloop auf nRF52 und ESP32 behoben"*, *„RACE-01..05 — atomare Ring-Buffer-Indizes, CAD Critical Sections"*.

Seine Beschreibung des Netzverhaltens liegt im offiziellen Zweig unter `docs/prio-talk-flood-networking.md` („Autor: Martin DK5EN"). Der Quelltext dazu ist bei uns nachlesbar: `configuration_global.h:139` (die Parameter), `esp32_main.cpp:2370` (der Sendepfad mit dem CAD-Aufruf), `lora_functions.cpp:2022` (die Backoff-Formel).

Warum wir es übernehmen statt selbst zu entwerfen: Die Zahlen sind im laufenden Netz erprobt. Ein Knoten, der andere Wartezeiten würfelt als alle übrigen, ist kein besserer Teilnehmer, sondern ein unberechenbarer. Kurts Randbedingung gilt: **die Verfahrenslogik ist unantastbar.**

19b.3 CAD — die Hardware horcht selbst

Der SX1262 kann etwas, was Software nicht kann: Er erkennt eine **laufende LoRa-Aussendung an ihren Präambelsymbolen**, ohne das Paket empfangen zu müssen. Das heißt **Channel Activity Detection**, kurz CAD.

Das ist wesentlich feiner als eine Pegelmessung. LoRa-Signale liegen oft **unter** dem Rauschpegel — ein einfacher „ist da Energie?"-Test würde sie übersehen und den Kanal für frei erklären, während mitten in der Übertragung gesendet wird.

Ein Scan über 4 Symbole dauert bei SF11/BW250 rund **33 ms**. Das ist der Preis: 33 ms Verzug vor jeder Aussendung, und in dieser Zeit ist der Empfang unterbrochen.

Zwei Scans, nicht einer

Ein einzelner CAD-Treffer kann Rauschen sein. Deshalb prüft die Vorlage nach — und wir auch: meldet der erste Scan „belegt", folgt ein zweiter. Erst wenn **beide** belegt melden, wird gewartet. Meldet der zweite frei, war der erste ein Fehlalarm; die Uhr zählt diese Fälle mit, damit sich später beurteilen lässt, wie treffsicher die Einstellung ist.

19b.4 Wenn der Kanal belegt ist: gestaffelt warten

Nach einem belegten Kanal einfach sofort wieder zu fragen wäre die schlechteste aller Möglichkeiten: Alle Knoten, die gewartet haben, stürzen im selben Augenblick los, sobald die fremde Aussendung endet — und kollidieren nun miteinander. Deshalb wartet jeder Knoten, und zwar unterschiedlich lang.

Die Wartezeit setzt sich aus zwei Teilen zusammen:

```
Wartezeit  =  Basiszeit (nach Wichtigkeit)  +  Zufall (0…10 Slots × 35 ms)
```

Die Basiszeit richtet sich nach der Wichtigkeit. Bei Gedränge sollen die wichtigen Pakete zuerst durch, nicht die, die zufällig zuerst wollten:

Klasse	Was	Basiszeit
1	**Quittung**	3000 ms
2	Meldung, die ein Mensch abgeschickt hat	3000 ms
3	(Weiterleitung — bei uns noch nicht)	4500 ms
4	**Positionsbake**	5500 ms

Dass die Quittung vorne steht, hat einen handfesten Grund: Bleibt sie aus, hält der Absender seine Meldung für verloren und **wiederholt sie**. Eine verspätete Quittung erzeugt also zusätzlichen Funkverkehr. Die Bake dagegen hat es niemals eilig — kommt sie eine halbe Minute später, merkt es kein Mensch.

Bei Wiederholungen sinkt die Basiszeit (ein Sechstel beim zweiten, ein Drittel beim dritten Versuch). Wer schon zweimal nicht durchkam, wartet kürzer — sonst verhungert er hinter Knoten, die gerade erst anfangen. Ab dem dritten Fehlversuch wird nur noch kurz getaktet nachgefragt (100 ms), damit ein Auftrag nicht endlos liegen bleibt.

⚠️ Der Zufall muss je Gerät verschieden ausfallen

Ein Slot ist 35 ms lang: 28 ms für den CAD-Scan, 2 ms zum Umschalten auf Senden, 5 ms Reserve. Zehn Slots ergeben höchstens 350 ms Streuung — genug, damit CAD die Belegung eines Nachbarn sicher erkennt, bevor man selbst dran ist.

Das funktioniert aber **nur, wenn nicht alle Geräte dieselbe Zahlenfolge würfeln**. Täten sie es, wählten sie nach jedem belegten Kanal wieder denselben Slot und kollidierten erneut — der ganze Backoff wäre wirkungslos, und zwar auf eine Weise, die im Einzeltest nie auffällt. Darum kommt der Zufall aus `esp_random()`, dem Hardware-Rauschen des ESP32, und nicht aus einem Zahlengenerator mit festem Startwert.

19b.5 Ruhe nach einem fremden Paket

CAD allein deckt einen Fall nicht ab, und es ist ein häufiger: **unmittelbar nach einem fremden Paket reichen andere Knoten genau dieses Paket weiter.** Wer in dem Moment sendet, in dem der Kanal gerade frei geworden ist, trifft punktgenau in die Weiterleitung der Nachbarn.

Deshalb gilt ein zweiter Zeitzaun: Nach jedem Empfang läuft die Wartezeit neu an (`CR_Csma_NoteRx()` im Empfangspfad). In der Vorlage ist das `iReceiveTimeOutTime = millis()`.

⚠️ **Folge für die Bedienung:** Eine Meldung kann jetzt einige Sekunden später hinausgehen als früher — dann nämlich, wenn kurz davor ein Paket hereinkam. Das ist gewollt. Zum Vergleich: Die erste Quittung aus dem Netz brauchte in der Messreihe vom 3. August zwischen 3,4 und 10 Sekunden. Gegen diese Zeiten fällt der Zaun kaum auf.

19b.6 Bei uns: Zustandsautomat, keine Warteschleife

RadioLib bietet einen bequemen Einzeiler an: `radio.scanChannel()`. **Er ist für uns unbrauchbar.** Die Bibliothek wartet darin in einer Schleife auf den Fertig-Pin des Funkchips — ohne Zeitgrenze. Ein hängender CAD-Vorgang hielte die Uhr für immer fest, und der Interrupt-Watchdog würde sie neu starten. Das ist genau das Verbot aus Kapitel 13: keine Warteschleifen im Hauptlauf.

Stattdessen benutzen wir die zerlegte Fassung derselben Bibliothek. Sie passt sogar besser zu uns als der Einzeiler, weil unser Empfang den Fertig-Pin ohnehin schon abfragt:

Schritt	Aufruf
Scan anstoßen	`startChannelScan()` — Standby, Pin-Zuordnung, CAD starten
Ergebnis abholen	DIO1 abfragen (wie beim Empfang), dann `getChannelScanResult()`
zurück in den Empfang	`startReceive()` — löscht dabei die Meldeflaggen und damit DIO1

Daraus wird ein Automat mit drei Zuständen (`CR_Csma.cpp`):

```
RUHE  —(Sendewunsch, Wartezeit abgelaufen)—▶  Scan starten  —▶  SCAN 1
SCAN 1 —(fertig)—▶  frei?   —▶ JA: Torwächter sagt „senden"
                    belegt? —▶ Gegenprobe                —▶  SCAN 2
SCAN 2 —(fertig)—▶  frei?   —▶ JA (Fehlalarm des ersten Scans)
                    belegt? —▶ Wartezeit setzen, zurück in den Empfang, RUHE
```

Jeder Aufruf kehrt sofort zurück. Ein Sendeauftrag wird bei belegtem Kanal **nicht verworfen** — er bleibt stehen und fragt beim nächsten Takt wieder. Bei 240 Hz Taktrate kostet das nichts.

⚠️ **Die Reihenfolge im Sendepfad ist kein Zufall.** Der Torwächter steht **vor** dem Paketbau, nicht dahinter. Denn `CR_MeshCom_NewMsgId()` verbraucht eine Nachrichten-ID und trägt sie in die Merkliste des Weiterleitungs-Nachweises ein (Kapitel 19). Stünde die Kanalprüfung dahinter, würde jeder Anlauf bei belegtem Kanal eine ID verbrennen und diese Liste mit vier Plätzen zumüllen.

19b.7 Zwei Dinge, die der Einbau bei uns erzwungen hat

Die Quittung wurde zu einem Wunsch

Bisher wurde eine Quittung mitten im Empfangspfad abgestrahlt, direkt nachdem das Paket zerlegt war. Das geht nicht mehr: Der Torwächter braucht mehrere Takte, und im Empfangspfad darf nicht gewartet werden. Also derselbe Weg wie beim Tipp auf einen Textbaustein (Kapitel 13.2) — der Empfang **merkt sich den Wunsch**, der Hauptlauf erledigt ihn, sobald der Kanal frei ist.

Vier Plätze. ⚠️ Ist die Liste voll, fällt eine Quittung aus — und das ist die richtige Entscheidung: Eine Quittung, die erst nach vielen Sekunden hinausgeht, nützt dem Absender nichts mehr, weil er da längst wiederholt hat.

Der verwaiste Scan

Ein Fehler, den die Bauweise selbst erzeugt und der ohne Vorkehrung still bleibt: Ein Scan wird angestoßen, und bevor sein Ergebnis abgeholt ist, **verschwindet der Auftraggeber** — die Bake wird über die Kachel abgeschaltet, der Sendeauftrag verworfen. Dann fragt niemand mehr nach. Der Automat bliebe im Scan stehen, DIO1 bliebe hoch, und der Empfangspfad würde das **CAD-Ergebnis für ein Paket halten** und `readData()` darauf ansetzen.

Deshalb läuft im Hauptlauf ein Wächter (`CR_Csma_Loop()`), und zwar an genau der richtigen Stelle: **hinter allen Sendewegen, vor dem Empfang.** Bis dorthin kommt der Lauf nur, wenn niemand senden wollte — läuft dann noch ein Scan, ist er verwaist. Der Wächter räumt zwei Fälle ab: Scans, die hängen (Zeitgrenze 300 ms, das Neunfache der normalen Dauer), und Scans, nach denen 200 ms lang niemand gefragt hat. Beide enden im Empfang.

19b.8 Warum eine eingereihte Aussendung nie storniert wird

Eine naheliegende Idee, die Martin in seiner Analyse ausdrücklich verwirft — und die Begründung gehört hierher, weil die Frage bei jeder Erweiterung wiederkommt:

Wenn wir während der Wartezeit hören, dass ein anderer Knoten dasselbe Paket schon weitergereicht hat — warum brechen wir dann unsere eigene Weiterleitung nicht ab?

Weil es das Netz an seiner empfindlichsten Stelle zerreißt. Der Fall heißt **Brückenknoten**:

```
A1–A2–A3 ———— [BR] ———— B1–B2–B3
```

`BR` ist der einzige Knoten, der beide Gruppen hört. A1 sendet; A2 und BR empfangen. A2 ist schneller und reicht weiter. BR hört das — und würde, mit Stornierung, seine eigene Weiterleitung streichen. **Ergebnis: B1, B2 und B3 erfahren nie von der Meldung.**

Der Grund ist grundsätzlich: **Ein Knoten kann nicht wissen, ob seine Aussendung einzigartige Information trägt.** BR weiß nicht, dass es B1–B3 gibt und dass sie nur über ihn erreichbar sind. Er weiß nur „dieses Paket kenne ich schon". Darauf zu stornieren, bricht das Weiterreichen.

Die Entdopplung verhindert deshalb nur das **erneute Einreihen**, niemals das Absenden eines bereits eingereihten Pakets. Was verschwenderisch aussieht, ist der Preis dafür, dass ein Netz ohne Kenntnis seiner eigenen Form funktioniert.

19b.9 Was noch offen ist

Stufe 2: die Vorprüfung auf Präambel und Kopf. CAD erkennt Präambelsymbole. Ist die Präambel schon vorbei und läuft gerade die Nutzlast — bei SF11 und einem langen Paket können das mehrere Sekunden sein —, kann CAD den Kanal für frei erklären, obwohl mitten in einem Empfang gesendet würde. Die Vorlage fängt das ab, indem sie **vor** dem Scan die Meldeflaggen des Funkchips liest (`esp32_main.cpp:2340`): steht dort „Präambel erkannt" oder „Kopf gültig", ist ein Paket im Anflug und die Aussendung wird verschoben.

Bei uns fehlt das noch, und zwar mit Absicht: In RadioLib 7.7.1 und 7.8.1 nimmt `startReceive()` keine Flaggenparameter mehr, und unser Empfang hängt an der Abfrage von DIO1. Die Flaggen zu erweitern, ohne den Empfang zu beschädigen, ist ein eigener Schritt — und der Empfang ist das Letzte, was in diesem Projekt kaputtgehen darf.

Noch nicht gemessen. Die Uhr zählt bereits mit, wie oft der Kanal belegt war, wie oft der zweite Scan einen Fehlalarm entlarvt hat, wie viele Versuche höchstens nötig waren und wie oft ein Scan aufgeräumt werden musste. Was diese Zahlen im Alltag sagen, ist offen — sie brauchen einen Konsolenbefehl und einen Tag im Netz.

⚠️ **Ebenfalls zu messen: der Takt.** Jeder Sendevorgang kostet jetzt mindestens 33 ms CAD. Die Erfahrung aus Kapitel 16 gilt: Nach jeder Änderung am Zeitverhalten ist die Bedienung neu zu vermessen, nicht zu vermuten.

19c Wartelisten statt Sendering — und die Seite /lora

Am 12. September 2026 bekam die Weboberfläche eine Seite **LoRa** nach dem Vorbild des „LoRa Queue"-Felds in der MeshCom-Firmware von DK5EN (`web_functions.cpp`, Abschnitt WQ-01). Beim Nachbau fiel die erste Frage sofort: Der Node zeigt einen Sendering mit **20** Plätzen, die Uhr zeigt **13**. Warum?

Der Node hat einen Ring, die Uhr hat Wartelisten

DK5ENs Firmware führt **einen** Sendering (`MAX_RING 20`, `configuration_global.h`) für alles, was auf die Luft soll — eigene Texte, Quittungen, Weiterleitungen, Baken, Server-Pakete — mit fünf Vorrangklassen (crit, high, normal, low, bg). Ist der Ring voll, weist der Node eigene Meldungen ab (**QRT**), und schon vorher warnt er (**QRS**). Für einen Digipeater mit Gateway, der auch Server-Verkehr auf die Luft bringt, ist das die richtige Bauform.

Die Uhr hat **keinen** gemeinsamen Ring. Jede Paketart hat ihre eigene, feste Warteliste in `WZ_LoRa.cpp`, und `WZ_LoRa_QueueStats()` zählt sie zusammen:

Warteliste	Plätze	Variable	CSMA-Vorrang
Quittungswünsche (ACK)	4	`s_ackWish[]`	1 (höchster)
Pong-Wünsche	2	`s_pongWish[]`	1
DM-Quittungen	2	`s_dmAckWish[]`	1
eigener Text	1	`s_txState == WZ_TX_PENDING`	2
Weiterleitung + Gateway-Rückweg	3 (gemeinsam)	`s_relayPkt[]`, `s_relayIsGateway[]`	3
Positionsbake	1	`s_beaconRequested`	4

Summe: **13**. Die Zahl ist also keine Wahl, sondern die Kapazität, die es gibt. Die Seite zeichnet die Kästchen in der Reihenfolge des CSMA-Vorrangs (Kapitel 19b), so wie sie auf den Kanal kämen.

Status | Meldungen | Stationen | LoRa | Akku | Einstellu
Update | aus

LoRa

Messung

SENDERING

1 / 13 Plätze belegt

Quittung | Text | Bake | Relais | Gateway

QUITTUNG	TEXT	BAKE	RELAIS	GATEWAY
0	0	0	1	0

SENDEDRUCK

● RUHIG

Nichts staut sich, der K

Kanal belegt (CSMA, 5

Relais-Plätze frei

DICHT ab 50 % belegt
wenn kein Relais-Platz

ENTDOPPLUNG

≈107 min Reichweite

0 | 10 | 40

Relais-Ring **64 IDs** ÷ neue IDs je 5 min
sicher ab **10 min**

Text-Ring 16 IDs ≈ **27** min

NEUE KENNUNGEN

3 neu · 21 Dubl

Anteil neu

im letzten 5-min-Fenst

KANALAUSLASTUNG

9,3 % gesamt

rx **7,3 %** | tx **2,0 %** Fenster vor 217 s

Luftzeit je Paket, 5-min-Fenster

Die Seite /lora am 12.09.2026, 17:28: ein Relais-Paket im Ring, Kanal ruhig, Entdopplung reicht 107 min zurück, 9,3 % Kanalauslastung

Warum der Weiterleitungs-Ring nur 3 Plätze hat

Von den 13 Plätzen kann nur einer wirklich überlaufen: der Weiterleitungs-Ring. Die übrigen sind Zustände (ein Text, eine Bake) oder Wünsche, die spätestens mit der nächsten Aussendung erledigt sind.

Der Ring ist mit Absicht klein, und er verdrängt bei Vollstand **nicht den ältesten**, sondern lässt das **frische** Paket fallen (`wz_lora_queue_pkt()`):

> Ein Paket, das erst nach vielen Sekunden hinausgeht, hat seinen Zweck verloren — die Nachbarn haben es längst von einem anderen Knoten.

Dazu kommt die Alterung aus E-45: Was 60 s im Ring liegt, wird verworfen. Zusammen heißt das: mehr Plätze würden nur mehr veraltete Luftzeit erzeugen. Der Fall wird mit einer Logzeile belegt (`[MESH RELAY] Warteschlange voll – ein Paket faellt aus`); häuft sie sich, ist der Ring zu klein. In den Mitschnitten vom 12.09. kam sie nicht vor.

Sendedruck: RUHIG, DICHT, VOLL

Weil die Uhr eigene Meldungen nie abweist, gibt es kein QRS/QRT. Die Firmware rechnet stattdessen in `cr_webui_funk_druck()` — eine Quelle für Seite und Konsole:

Zustand	Bedingung
VOLL	kein Platz im Weiterleitungs-Ring frei (frische Relais-Pakete fallen weg)
DICHT	Kanal im letzten 5-min-Fenster bei ≥ 50 % der CSMA-Anläufe belegt, **oder** ≥ 5 Einträge in den Wartelisten
RUHIG	sonst

Die beiden Schwellen sind Vorschläge vom 12.09. und noch nicht am Verkehr geeicht.

Das 5-Minuten-Fenster (`CR_Funklog`)

`CR_Funklog.cpp` führt wie die Vorlage (`PRIO_STAT_INTERVAL_S 300`) ein Fenster von 5 Minuten. Beim Abschluss wird eine Kopie festgehalten, die laufenden Zähler beginnen von vorn; bis zum ersten Abschluss liefert die Seite die laufenden Werte mit Hinweis.

Wert	Quelle
Luftzeit rx	`radio.getTimeOnAir(len)` je empfangenem Rahmen, in `WZ_LoRa_HandleRx()` **vor** dem Decoder (auch Unlesbares zählt)
Luftzeit tx	derselbe Wert in `wz_lora_tx_start()`, aus `s_txFlightMaxMs`
neu / wiederholt	eigene Merkliste mit 64 Kennungen (Bytes 1–4), unabhängig vom Relais-Schalter
Kanal belegt	Delta von `CR_Csma_GetBusyCount()` gegen `CR_Csma_GetFreeCount()` im Fenster

Entdopplungs-Reichweite = 64 ÷ neue Kennungen je Fenster × 5 min. Das ist die Zeit, die die Relais-Merkliste (`CR_MESH_RELAY_SLOTS 64`) beim aktuellen Verkehr zurückreicht; die Seite markiert unter 10 min orange. Die Anzeige-Merkliste für Texte hat nur 16 Plätze und steht als zweite Zahl daneben.

Der Paketstrom

Der Mitschnitt hält die letzten 20 Rahmen (`CR_FUNKLOG_ZEILEN`, 150 Byte je Zeile, statisch, 3,3 KB) im Format von `charBuffer_aprs()` der Vorlage, damit Christian dieselbe Zeile wie am Node liest:

```
16:53:09 :134EF3E5 3 101 8/8 LH:AA DK8VW-99,OE3LCR-20>10 :... und die haben Antennen
```

Feld	Herkunft im Rahmen
`16:53:09`	Lokalzeit der Uhr; `--:--:--` solange die Uhr nicht gestellt ist
`:134EF3E5`	Kennung, Bytes 1–4 little-endian
`3`	verbleibende Hops, Byte 5 Bits 0–3
`101`	Byte 5: Bit 7 Server, Bit 6 Track, Bit 4 Mesh (`aprs_functions.cpp:160ff` ; Bit 5 App-offline wird nicht gezeigt)
`8/8`	Modulation / Hardware des Absenders aus dem Schwanz (`tail[2]` , oberes / unteres Nibble)
`LH:AA`	letzter Digipeater, `tail[6]` (Bit 7 = „letzte sendende Station")
`DK8VW-99,OE3LCR-20>10`	ASCII-Teil ab Byte 6 bis zum ersten Steuerzeichen: Absender, Pfad, Ziel
`:...`	Typzeichen und Nutzlast, gekürzt auf 60 Zeichen; zwischen Ziel und Typzeichen ein Leerzeichen wie in der Vorlage

Der Schwanz ist 9 Byte: `0x00 | hw | mod | FCS_hi | FCS_lo | fw | last_hw | sub | 0x7E` (Kapitel 18). Eigene Aussendungen stehen mit derselben Zeile und der Marke **TX** im Strom.



Paketstrom mit Kennungsfarben: 8E6E0134 trifft als „neu" ein (TTL 2), kommt zweimal als „dup" über OE3LCR-11 zurück (TTL 1 und 0) und geht als „TX" mit TTL 1 von der Uhr weiter; LH nennt den jeweils letzten Digipeater

Schnittstellen

- `/lora` — die Seite; `/api/lora.json?ab=<seq>` — Werte plus Zeilen mit `seq > ab`, neueste zuerst. Die Seite fragt alle 5 s und fügt nur Neues ein; `seq` läuft ab 1 durch.
- `--lora` — dieselben Zahlen und Zeilen an der Konsole (`CR_Funklog_Print()`).
- Der Modulname `CR_Funklog` blieb, als der Reiter von „Funk" in „LoRa" umbenannt wurde: die Seite zeigt nur die Luftschnittstelle, kein UDP — so wie die Meldungsliste die Herkunft „Funk" seit 1.0.4.16 als „LoRa" beschriftet.

20 Empfang und Zerlegung

Der Zerleger ist **defensiv gebaut**: Was nicht passt, wird verworfen statt geraten. Diese Haltung hat einen Grund — die Beschreibung des Protokolls stammt teilweise aus Beobachtung, und eine Meldung weniger ist besser als eine erfundene.

20.1 Die Prüfkette

`CR_MeshCom_Parse()` (`CR_MeshCom.cpp:210`) steigt an jeder Stelle aus, an der etwas nicht stimmt:

Prüfung	Bei Verstoß
Länge ≥ 10 Bytes	verwerfen
Typ ist einer der vier bekannten	verwerfen
ASCII-Teil ≥ 3 Zeichen	verwerfen
`>` im ASCII-Teil vorhanden	verwerfen
Absender sieht aus wie ein Rufzeichen	verwerfen

Erst danach werden Ziel, Text, Zeit und Position gelesen.

20.2 Wo der Text aufhört

Der ASCII-Teil beginnt bei Byte 6 und läuft **bis zum ersten Steuerzeichen**:

```
while (asciiLen < maxAscii) {
  uint8_t b = (uint8_t)ascii[asciiLen];
  if (b < 0x20) break;
  asciiLen++;
}
```

Das `0x00`, mit dem der Schwanz beginnt, ist damit zugleich das Ende des Textes — eine Längenangabe braucht es nicht.

> ⚠️ **Bytes ≥ 0x80 werden durchgelassen.** Dort steht UTF-8: Umlaute und Emojis. Eine Prüfung auf „druckbares ASCII" (0x20–0x7E) hätte jede Meldung mit einem Umlaut mitten im Wort abgeschnitten.

20.3 Entdopplung — die wichtigste Funktion überhaupt

Jedes Paket kommt zwei- bis dreimal an: einmal direkt, dann über einen oder mehrere Digipeater. Ohne Entdopplung stünde jede Meldung mehrfach in der Liste.

```
bool CR_MeshCom_IsDuplicate(uint32_t msgId) {
  if (msgId == 0) return false;             // 0 gilt als "keine ID"
  for (uint8_t i = 0; i < CR_MESH_SEEN_SLOTS; i++)
    if (s_seen[i] == msgId) return true;
  s_seen[s_seenNext] = msgId;
  s_seenNext = (uint8_t)((s_seenNext + 1) % CR_MESH_SEEN_SLOTS);
  return false;
}
```

Ein Ringpuffer über **16 Kennungen** (`CR_MESH_SEEN_SLOTS`). Er merkt sich nicht den Inhalt, sondern die ID — nur sie bleibt beim Weiterreichen unverändert.

> ⚠️ **Die Reihenfolge ist entscheidend.** Die Quittung auf eine empfangene Meldung wird **hinter** der Entdopplung ausgelöst. Dasselbe Paket dreimal zu quittieren würde das Netz belasten statt ihm zu helfen.

Wiederholte Direktmeldungen (seit 1.0.5.16)

Ab MeshCom v4.35u wiederholt ein Knoten eine **Direktmeldung** bis zu dreimal im Abstand von 40 s (aufgegeben nach 160 s, also 4 × 40 s; Vorlage `lora_functions.cpp:3003`, `MAX_RETRANSMIT 3`), auch wenn er sein Echo schon gehört hat. Jede Kopie trägt eine **eigene** Kennung: die Bits 10–11 der

`msg_id` werden per XOR gekippt (Vorlage `pn_retry.h`). So reichen Digipeater die Kopie weiter, obwohl sie das Original schon hatten. Der Ring oben sieht aber vier verschiedene Meldungen.

Deshalb gibt es eine zweite Prüfung, nur für Meldungen in **PN-Form**: Text an ein persönliches Rufzeichen (kein `*`, keine Gruppe) mit Quittungswunsch `{nnn` am Ende. Verglichen wird der **Kern** `msg_id & 0xFFFFF3FF` und, über die Vorlage hinaus, das **Absender-Rufzeichen**. Die gekippten Bits sind die unteren Bits der Gerätekennung, zwei Knoten könnten sich sonst im Kern gleichen. Ein Eintrag gilt 5 Minuten.

Ring	Wer fragt	Schlüssel	Kopie wird …
Anzeige (`CR_MeshCom_IsPnRepeat`)	Funkweg **und** Drahtweg gemeinsam	Kern + Absender	**quittiert**, aber nicht angezeigt, keine Vibration, kein App-Eintrag
Gateway (`CR_MeshCom_IsPnRepeatGw`)	Hinweg zum Server	Kern + Absender	nicht noch einmal hochgeladen
Weiterleitung (`CR_MeshCom_IsRelayDuplicate`)	Digipeater	**volle** ID	weitergereicht – sonst erreicht die Wiederholung ihr Ziel nie

> ⚠️ **Quittiert wird jede Kopie.** Kam unsere Quittung auf das Original nicht an, holt erst die Quittung auf die Kopie sie nach. Die Prüfung steht deshalb **vor** dem Quittungsblock (der schneidet `{nnn` ab, danach fehlt die PN-Form), abgebrochen wird erst danach.

Beim Start rechnet `CR_MeshCom_PnSelfTest()` 13 Fälle durch: Kopien müssen erkannt, eine andere Meldung, ein anderer Absender und eine abgelaufene Frist **durchgelassen** werden.

```
[MESH] PN-Selbstpruefung: ok (13 Faelle, Kopien erkannt, Fremdes durchgelassen)
[MESH] PN-Wiederholung 12345E78 von OE0AAA-1 (Kern 12345278) - quittiert, nicht nochmal anzeigen
```

Briefkasten-Telegramme aus MeshCom v4.40a (seit 1.0.5.24)

Ab v4.40a kann ein Knoten Direktmeldungen für abwesende Empfänger verwahren (`--store own/list/heard` , ab Werk **aus**). Die Uhr versteht beide Telegramme dieser Rolle (seit 1.0.5.24, damals noch als 4.35A); seit 4.40A (1.0.5.31) meldet sie die Verwahrung als Status `0x04` an die App.

Verwahrmeldung an den Absender. Der Briefkasten schickt uns eine gewöhnliche Textmeldung `"%-9.9s:sto%03u %s"` , etwa `OE0AAA-1 :sto017 OE0BBB-2` (Vorlage `sto_notice.cpp`). Sie hat kein `{` und kein `:ack` ; ohne Erkennung stand sie als Meldung in der Liste. `CR_MeshCom_StoParse()` prüft so scharf wie die Vorlage: Marke fest an Byte 9, drei Ziffern, kein `{` , kein `:ack` / `:rej` . Ausgewertet wird sie in `WZ_LoRa_NoteTextAck()` , also auf Funk- und Drahtweg gleich. Die Ampel der Sendekarte bleibt unberührt: Die echte Quittung schickt der Empfänger nach der Zustellung, nach abgelaufener Frist heilt sie wie jede späte Quittung. Den App-Status 0x04 „verwahrt“ gibt es erst mit 4.40A: Der Status selbst ist neu in v4.40a, eine 4.35-App kennt ihn nicht. (Den Rufzeichen-Anhang des 0x41-Rahmens gibt es schon seit v4.35t, bei uns seit 1.0.5.25.)

Zustellung an den Empfänger. Der Briefkasten stellt mit Absender, Text und `{nnn` des Originals zu, aber mit einer **neuen** `msg_id` (`millis()` des Briefkastens), Pfad `SRC,BRIEFKASTEN` und `max_hop` 0 (`msgstore_glue.cpp:92-118`). Hatte die Uhr die Meldung schon, greifen weder der ID-Ring noch der PN-Ring. `CR_MeshCom_IsDmResent()` führt dafür einen Inhaltsring: 16 Plätze, 24 h, Schlüssel Absender (ohne Groß/Klein) + Text samt `{nnn` .

Neuer Eingang mit gleichem Absender und Text …	Herkunft `msg_id & 0xFFFFF000`	Ergebnis
Briefkasten-Zustellung	**anders**	nachgereicht: quittiert, nicht angezeigt
PN-Kopie (Bits 10–11 gekippt)	gleich	hier durchgelassen, der PN-Ring entscheidet
Absender nach Neustart, gleicher Zählerstand	gleich	angezeigt (echte neue Meldung)

Quittiert wird auch hier jede Kopie, denn der Briefkasten löscht erst auf die Quittung. Beim Start rechnet `CR_MeshCom_StoSelfTest()` 15 Fälle durch.

```
[MESH] Briefkasten-Selbstpruefung: ok (15 Faelle, :sto und Zustellung erkannt, Fremdes durchgelassen)
[MESH] Verwahrmeldung von OE0CCC-3: unsere Direktmeldung 017 (12345011) liegt dort fuer OE0BBB-2 bereit - Quittung folgt nach der Zustellung
[MESH] Direktmeldung von OE0AAA-1 ueber OE0AAA-1,OE0CCC-3 nachgereicht (Briefkasten, 0004D2F1) - schon gezeigt, quittiert, nicht nochmal anzeigen
```

20.4 Die eigene Aussendung, zurückgehört

Eine Sonderrolle spielen Pakete, die man selbst verfasst hat und über einen Digipeater zurückbekommt. Ohne Behandlung landeten sie als fremde Meldung in der eigenen Liste.

`WZ_LoRa.cpp` merkt sich deshalb die letzten **vier** selbst vergebenen Kennungen (`WZ_LORA_OWN_IDS`). Trifft eine davon wieder ein, meldet das Protokoll:

```
[MESH] >>> WEITERGEREICHT: eigene Aussendung 316D10DE von OE3LCR-20 zurueckgehoert
       (3 Hops uebrig, ueber MQTT) - das Netz verteilt sie
```

Das ist der **einzige harte Beleg**, dass das Netz eine Meldung tatsächlich verteilt — härter als jede Quittung (siehe Kapitel 19.7). Bis zum 5. August fiel dieser Fall stumm unter die Entdopplung.

Die Gateway-Regel (seit 1.0.5.6)

Das Zurückhören zählt seit 1.0.5.6 auch als **Bestätigung**, aber nur in einem engen Fall:

Bedingung	geprüft in
Textmeldung (`:`) an alle (`*`)	`wz_lora_eigenes_echo()`
die Uhr hat sie **beim Senden selbst** zum Server getragen, und der Upload gelang	`s_txServerId` , gesetzt in `WZ_LoRa_SendTextTo()`
sie kommt über Funk zurück (erstes Echo oder spätere Wiederholung)	Uplink-Zweig und Duplikat-Zweig des Empfangs

Anlass (30.09.2026): Eine Meldung an `*` blieb rot, obwohl sie beim Paperwhite ankam und OE3LCR-10/-14 sie weiterreichten. Keine einzige Quittung kam. Die MeshCom-Firmware erklärt, warum das vorkommen kann: Ein Gateway quittiert nicht, was schon über ein anderes Gateway beim Server liegt (`lora_functions.cpp:884–886`). Für genau diesen Fall hat sie eine eigene Regel (`:917–927`): Hört ein Gateway seine eigene Meldung an `*` zurück, meldet es der App die Wolke mit Häkchen. Die Uhr ist dabei strenger als die Vorlage, der dort schon der Gateway-Schalter genügt: Gewertet wird nur, wenn der eigene Upload nachweislich gelang.

Nachweis ohne Kabel: `--info` zeigt „Gateway-Regel …: n in der Frist, m nachträglich, zuletzt am TT.MM. hh:mm“. Die Zähler liegen im RTC-Speicher (`RTC_NOINIT_ATTR`) und überstehen Neustart, Panic und Watchdog, nicht aber einen leeren Akku. Bei der Einführung hatten zwei Testmeldungen an `*` regulär Quittungen bekommen; die Regel hatte noch nicht gegriffen.

⚠️ **Die Kennung einer Quittung ist keine Stationskennung.** Die Vorlage vergibt sie als `millis()` des Quittierenden (`lora_functions.cpp:942ff`). Wer aus den oberen 24 Bit eine Station ablesen will, liest bei fremden Gateways eine Laufzeit. `--lora` zeigt Quittungen seit 1.0.5.6 als „Quittung fuer “ mit dem Zusatz „(Gateway)“, wenn Byte 10 das so ausweist.

20.5 Zeittelegramme

Das Netz sendet alle fünf Minuten ein Zeittelegramm — 288-mal am Tag. Sie erscheinen **nicht** in der Meldungsliste; sie würden sie fluten und die Uhr am Schlafen hindern.

Erkannt werden sie am Präfix `{CET}` :

```
{CET}2026-08-05 21:11:55
```

> ⚠️ **`{CET}<` heißt: Das Netz hält diesen Zeitstempel selbst für falsch.** Die MeshCom-Firmware reicht solche Telegramme nicht weiter. Sie dürfen die Uhr also **nicht stellen**. Der Fall wird ausdrücklich abgefangen — vorher fiel er nur zufällig richtig aus, weil `sscanf` am `<` scheiterte.

Danach folgt eine Plausibilitätsprüfung (Jahr 2024–2099, Monat 1–12, Stunde ≤ 23 …). Ein verstümmeltes Telegramm darf die Systemuhr nicht verstellen.

20.6 Positionen

Positionsmeldungen erscheinen ebenfalls nicht in der Meldungsliste — ihre Koordinaten wandern in die Stationsliste und auf das Radar. Das Format und die 40-km-Falle stehen in Kapitel 18.7.

Die Prüfungen beim Lesen:

Prüfung	Grund
Halbkugel ist `N` / `S` bzw. `E` / `W`	sonst ist es kein Koordinatenfeld
Grad ≤ 90 bzw. ≤ 180, Minuten < 60	Zahlendreher abfangen
nicht 0/0	Kennzeichen eines Geräts ohne Fix

Die Symboltabelle wird bewusst **nicht** geprüft: Im Mitschnitt kamen `/` , `-` und `&` vor, und die APRS-Norm erlaubt an dieser Stelle ohnehin Overlays.

20.7 Der Weg vom Interrupt bis auf den Schirm

```
Interrupt (SX1262)  →  Flag setzen
        ↓
Haupt-Task, WZ_LoRa_Loop()  →  Paket abholen, CR_MeshCom_Parse()
        ↓
Entdopplung  →  eigene Aussendung?  →  Zeittelegramm?  →  Position?
        ↓
CR_MsgBridge_Push()   ← Warteschlange
        ↓
Render-Task, CR_MsgScreen  →  Liste zeichnen
```

> ⚠️ **Im Empfangspfad steht kein einziger `lv_*`-Aufruf.** LVGL ist nicht threadsicher und gehört dem Render-Task; der Empfang läuft im Haupt-Task. `CR_MsgBridge` ist genau dafür da: Der Haupt-Task schreibt in die Warteschlange, der Render-Task holt ab. Wer diese Trennung verletzt, bekommt keinen Absturz, sondern sporadisch zerstörte Bildinhalte — und sucht tagelang an der falschen Stelle.

20.8 Was im Protokoll steht

Seit dem 5. August gibt die Uhr empfangene **und** selbst gesendete Pakete im **gleichen Format** aus. Das war die Voraussetzung, um beide überhaupt vergleichen zu können:

```
[LoRa RX]  #2  49 Bytes  RSSI -86.0 dBm  SNR -0.5 dB
[LoRa HDR] 40 DE 10 6D 31 93 4F 45 33 4C 43 52 2D 32 30 2C (49 Bytes gesamt)
[LoRa END] 31 33 3B 00 2A 88 0B 0F 23 8A 70 7E (ab Byte 37)
[MESH] @ von OE3LCR-20 an * (ID 316D10DE, 3 Hops uebrig, weitergereicht, ohne POS)
```

Kopf und Schwanz in Hex, dazwischen die Auswertung im Klartext. Ohne diese Ausgabe wäre der Vergleich zwischen Originalen und Weiterleitungen (Kapitel 19.1) nicht möglich gewesen — sie ist kein Beiwerk, sondern das Messgerät.

21 Funkparameter: Sync-Wort und Präambel

Alle Bytes des vorigen Kapitels nützen nichts, wenn die Funkschicht nicht passt. Sie ist die unerbittlichste Schicht des ganzen Systems: **Stimmt ein Wert nicht, hört man nichts.** Keine Fehlermeldung, keine gestörten Pakete, keine halben Meldungen — Stille.

21.1 Die Parameter

Parameter	Wert	Wo festgelegt
Frequenz	**433,175 MHz**	`RF_FREQUENCY`
Bandbreite	**250 kHz**	`LORA_BANDWIDTH`
Spreizfaktor	**SF11**	`LORA_SF`
Coderate	**4/6**	`LORA_CR`
Präambel	**8 Symbole**	`LORA_PREAMBLE_LENGTH`
Sync-Wort	**0x2B**	`LORA_SYNC_WORD_MESHCOM`

Alle sechs stehen in `variants/t-watch-S3-plus/pins_arduino.h:88–112` — also in der Board-Ebene, nicht im Modul. Das ist Absicht: Ein anderes Board kann in einer anderen Region auf anderen Werten funken, ohne dass eine Zeile MeshCom-Code angefasst wird.

Gesetzt werden sie in `WZ_LoRa_Init()` (`WZ_LoRa.cpp:109–129`).

21.2 Das Sync-Wort ist ein Türsteher

Das Sync-Wort steht in jeder LoRa-Präambel und wird **von der Hardware** geprüft. Ein Paket mit falschem Sync-Wort erreicht die Firmware nie — der Chip verwirft es, bevor ein Interrupt entsteht.

Das macht es zum härtesten aller Fehler: Man sieht nicht ein einziges Symptom. Kein Zähler steigt, kein Protokoll meldet etwas, das Funkmodul ist nachweislich in Ordnung.

> ⚠️ **0x2B ist nicht der Standardwert.** RadioLib und die meisten Beispiele verwenden `0x12` („private network") oder `0x34` („public LoRaWAN"). Wer eine Uhr mit dem Vorgabewert baut, bekommt ein technisch einwandfreies Gerät, das nie etwas hört.

21.3 Die Präambel

Acht Symbole, für Senden und Empfang identisch. Sie dient dem Empfänger dazu, sich auf das Signal einzuschwingen.

> ⚠️ **Merkposten gegen eine falsche Erinnerung:** In frühen Projektnotizen stand „Präambel 32". Der Wert war nie 32 — er steht seit dem allerersten Commit dieser Datei (13. Juli 2026) auf 8, und das Boot-Protokoll bestätigt es bei jedem Start:
>
> ```
> [LoRa] Init SX1262: 433.175 MHz, BW 250 kHz, SF 11, CR 6, Praeambel 8
> ```
>
> Die Zeile gibt es genau deshalb: Funkparameter gehören ins Protokoll, damit man sie **ablesen** statt aus dem Gedächtnis behaupten kann.

21.4 Was SF11 für alles andere bedeutet

Der Spreizfaktor bestimmt die Luftzeit — und die wirkt bis in die Bedienung hinein:

Folge	Größenordnung
Dauer einer Aussendung	mehrere hundert Millisekunden
Zeit bis „abgestrahlt" (blau)	rund **1 Sekunde**
Zeit bis zur Quittung (grün)	typisch **4–11 Sekunden**

Die Quittungszeiten sind gemessen, nicht geschätzt: Über sieben Aussendungen lagen sie bei 3,5 / 3,5 / 3,4 / 10,0 / 9,1 / 6,9 / 6,3 Sekunden. Daraus folgt die Frist von **15 Sekunden**, nach der eine Aussendung als nicht angekommen gilt — sie muss den langsamsten gemessenen Fall mit Reserve überdecken.

Und daraus folgt die wichtigste Regel des Sendepfads: `radio.transmit()` **blockiert für die Dauer der Aussendung.** Im Render-Task aufgerufen, würde es das Bild für eine halbe Sekunde einfrieren. Deshalb sendet ausschließlich der Haupt-Task (Kapitel 13 und 19.10).

21.5 Ein eigener SPI-Bus

Der SX1262 hängt **nicht** am Display-Bus, sondern an einem eigenen:

```
[LoRa] SPI: SCK=3 MISO=4 MOSI=1 CS=5
```

Zwei Geräte an einem Bus wären möglich, aber der Bildschirm überträgt in Schüben und der Funkchip antwortet auf Interrupts — beides gleichzeitig führt zu Wartezeiten in genau dem Task, der keine haben darf. Details in Kapitel 7.

21.6 Die Stromschiene

Das Funkmodul hängt an **ALDO4**, nicht an ALDO3. Diese Verwechslung kostete bei der Inbetriebnahme einen halben Tag: Bei falsch geschalteter Schiene meldet der Chip weder Fehler noch Anwesenheit — `radio.begin()` läuft in einen Zeitablauf, was auch bei einem Verdrahtungsfehler passieren würde.

Die vollständige Zuordnung aller Schienen steht in Kapitel 5 und in Anhang A.

21.7 Prüfreihenfolge, wenn nichts empfangen wird

Von der Hardware nach oben — jede Stufe schließt die darunterliegende aus:

1. **Schiene an?** `[PWR] ALDO4 (LoRa SX1262): eingeschaltet`
2. **Chip gefunden?** `Found SX126x` samt Versionszeichenkette im RadioLib-Protokoll
3. **Parameter richtig?** die `Init SX1262`-Zeile mit allen sechs Werten ablesen
4. **Empfang läuft?** `[LoRa] bereit – Empfang laeuft (Sync 0x2B)`
5. **Kommt überhaupt etwas an?** einige Minuten horchen — bei ruhigem Verkehr vergehen leicht mehrere Minuten zwischen zwei Paketen

> ⚠️ **Punkt 5 ist der, an dem man sich am leichtesten selbst täuscht.** Am 5. August fiel der Empfangszähler eines Tragetests zweieinhalb Stunden lang auf null — die Uhr schien defekt. Sie war es nicht: Der Träger saß im Kino, und ein Kinosaal ist ein Betonbunker. **Zu jedem Empfangsbefund gehört die Frage, wo das Gerät in dieser Zeit war.**

21b Bluetooth: die zweite Funkschnittstelle

Die Uhr hat einen zweiten Funkweg — nicht ins Amateurfunknetz, sondern zum Telefon in der Tasche. Das Fernziel ist die MeshCom-Handy-App: Meldungen dort tippen statt mit den fünf Bausteinen des Senden-Schirms, und empfangene Meldungen am größeren Bildschirm mitlesen.

Dieses Kapitel beschreibt, was davon gebaut ist — und ausführlicher, was **bewusst nicht** gebaut ist. Zwei Wege sind offen geblieben, und beide aus Gründen, die schwerer wiegen als der Nutzen.

21b.1 Warum in zwei Stufen

Der Weg zur App hatte zwei voneinander unabhängige Unbekannte. Sie wurden getrennt beantwortet, weil sie verschiedene Antworten brauchen:

Stufe	Frage	Wie zu beantworten
A	Passt BLE überhaupt in den Speicher?	messen — ohne Protokollkenntnis beantwortbar
B	Welchen Dienst erwartet die App?	**lesen** — nicht raten

Die Trennung war keine Förmlichkeit. Wäre Stufe A negativ ausgefallen, wäre die gesamte Protokollarbeit umsonst gewesen.

21b.2 Stufe A: der Speicher

Der interne RAM ist auf dieser Uhr das Nadelöhr, nicht der Flash — mbedTLS ist genau daran gescheitert (`-32512`), weshalb HTTPS über eine schlanke Eigenlösung läuft (Kapitel 25). BLE stand unter demselben Verdacht.

Deshalb fiel die Wahl auf **NimBLE** statt des Arduino-eigenen Bluedroid: grob 30–50 kB gegen 80–100 kB.

Gemessen am Bau, nicht geschätzt:

	ohne BLE	mit BLE	Zuwachs
RAM (statisch)	194.768 B (59,4 %)	201.876 B (61,6 %)	**+7,1 kB**
Flash	1.985.077 B (23,9 %)	2.192.909 B (26,3 %)	+208 kB

Statisch passt es also mit großem Abstand — rund 126 kB interner RAM bleiben frei.

Der Laufzeitbedarf entsteht erst beim Start des Stacks und lässt sich nur am Gerät messen. Dafür protokolliert `WZ_BLE_Init()` den internen Heap **vor und nach** dem Start und gibt die Differenz aus — nach dem Vorbild der `[PWR]` -Wachposten (Kapitel 5.3). ✅ Gemessen am 6. August: Der Stack kostet real **66,7 kB** — mehr, als die statischen Zahlen ahnen ließen, und tragbar erst, seit `LV_MEM_POOL_ALLOC` den LVGL-Vorrat ins PSRAM verlegt und damit 128 kB internen RAM freigegeben hat (14 → 145 kB frei). Die Lehre daraus: **Speicher immer am Gerät messen** — Linker-Statistik und `ESP.getFreeHeap()` täuschen beide.

21b.3 ✅ Der Befund, der Stufe B klein machte

Die Erwartung war ein eigenes Rahmenformat für Meldungen zwischen Uhr und App — mit Kopf, Längenfeld und Typkennung, das zu entziffern gewesen wäre.

Ein Blick in die Quelle beendete das (`reference/meshcom-firmware/src/phone_commands.cpp:104`):

```
if(toPhoneBuff[0] == ':' || toPhoneBuff[0] == '!' || toPhoneBuff[0] == '@')
```

Das sind **genau die Pakettypen aus Kapitel 18** — `:` Text, `!` Position, `@` Wetter.

> **Der BLE-Dienst transportiert dieselben MeshCom-Rohpakete wie der Funk.** Es gibt kein eigenes Nachrichtenformat. Der vorhandene Zerleger bleibt unverändert brauchbar; die Brücke muss nur Bytes durchreichen.

Damit schrumpfte Stufe B von „ein Protokoll nachbauen" auf „Bytes weiterleiten".

⚠️ **Korrektur 17.08. — „weiterleiten" heißt nicht „nackt durchreichen".** Die App will das Rohpaket zwar unverändert, aber **eingepackt**, so wie `sendToPhone()` es tut (`phone_commands.cpp:50–95`): ein Kennbyte `0x40` davor, die Unix-Zeit (4 Byte, big-endian) und ein Nullbyte dahinter (`addBLEOutBuffer()` , `loop_functions.cpp:525`). Bis 1.0.3.69 ging der Rahmen ohne beides hinaus — die App zeigte in keiner Gruppe eine Meldung (Befund Christian). Und die Vorlage reicht nicht *jedes* Paket weiter: nur Textmeldungen an den Knoten, an `*` oder an eine eigene Gruppe, Positionen nur bei angemeldeter App, Systemtelegramme (`{CET}` , `{MCP}` , `{SET}`) nie.

⚠️ Dieser Befund war **nicht** zu erraten. Ein selbst entworfenes Rahmenformat hätte plausibel ausgesehen und nie funktioniert — dieselbe Lehre wie beim Sendepfad (Kapitel 19), nur diesmal vor dem ersten Fehlversuch gezogen.

21b.4 Der Dienstaufbau

Maßgeblich ist die **ESP32-Fassung** der MeshCom-Quelle, also unsere Chipfamilie (`src/esp32/esp32_main.cpp:1587–1644`):

Kennung	Richtung	Eigenschaft
`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`	—	Dienst (Nordic UART Service)
`6E400003-…`	Uhr → App	`NOTIFY`
`6E400002-…`	App → Uhr	`WRITE`

✅ **Ein Nebenbefund mit Gewicht:** Die MeshCom-ESP32-Firmware benutzt **selbst NimBLE** (`NimBLECharacteristic` , `:1600` / `:1611`). Die Stackwahl aus Stufe A ist damit nicht nur vertretbar, sondern deckungsgleich mit dem Original — samt seiner Speichereigenschaften.

21b.5 Der Empfangsweg und die Task-Grenze

Seit 1.0.3.70 (17.08.) entscheidet `WZ_LoRa_HandleRx()` **nach** der Entdopplung, was in den Meldungs-Ring der App wandert (`CR_BlePhone_QueueFrame()`): Textmeldungen an uns, an `*` oder an eine eigene Gruppe (`CR_BlePhone_ZielFuerApp()` , Gruppen aus `--setgrc`); Positionen nur bei angemeldeter App; der Draht-Weg (`WZ_UDP.cpp`) reicht mit demselben Filter das vollständige GATE-Paket ab Byte 4 durch. Eine Meldung erreicht die App damit **genau einmal**, egal ob sie zuerst über Funk oder Draht kam. Der Ring hat 20 Plätze (Vorlage `MAX_RING`) und liegt im PSRAM; ohne angemeldete App sammeln sich die Meldungen dort mit dem Bit `0x20` („App war offline") im Flag-Byte und gehen beim nächsten Anmelden hinaus — nach den Konfigurationstelegrammen und **vor** dem `CONFFIN` , wie in `esp32_main.cpp:2817` . Direktmeldungen mit Quittungswunsch (`Text{nnn`) werden vor der Übergabe gekürzt (`CR_MeshCom_TruncateText()` , Prüfsumme neu), Quittungen `:acknnn` werden nicht angezeigt, sondern als Häkchen-Telegramm an die App gereicht.

Die Häkchen-Stufen (seit 1.0.5.25, mit 4.40A fünf)

Der Rahmen folgt `ack_attribution.h` der Vorlage (seit v4.35t): `0x41 | ID (LE) | Status | n | Rufzeichen (n Byte)` , danach wie jeder Rahmen die Zeit. `n = 0` ist das alte Format, byteidentisch mit früher.

Status	Bedeutung	Auslöser an der Uhr	Rufzeichen im Anhang
`0x00`	gehört	eigene Aussendung über einen Digipeater zurück	letztes Glied des Pfades
`0x01`	Gateway/Server	12-Byte-Quittung (`:423`); eigene Meldung an `*` als Gateway zurückgehört (`:1351`)	leer bzw. eigenes Rufzeichen
`0x02`	Empfänger	Text-Quittung `:ackNNN` (`:1144`), Funk und Draht	Absender der Quittung
`0x03`	aufgegeben (seit 1.0.5.31)	Frist einer eigenen **Direktmeldung** abgelaufen (Karte rot)	Zielrufzeichen
`0x04`	verwahrt (seit 1.0.5.31)	Verwahrmeldung `:stoNNN` eines Briefkastens auf unsere Direktmeldung	Rufzeichen des Briefkastens

Bis 1.0.5.24 kannte `wz_lora_app_status()` nur zwei Stufen und meldete die 12-Byte-Quittung als `0x02` . Die App zeigte damit für jede Gateway-Quittung „vom Empfänger bestätigt".

Seit 4.40A (1.0.5.31, Vorlage `lora_functions.cpp:1694` und `:3090`) ist die Reihenfolge keine Zahlenreihe mehr. `wz_lora_app_stufe_erlaubt()` entscheidet anhand des zuletzt gemeldeten Stands (`s_ownAppFlags` = Stufe + 1):

Neue Stufe	erlaubt, wenn bisher …
`0x00` , `0x01`	nichts oder eine niedrigere dieser beiden — nie nach `0x02` / `0x03` / `0x04`
`0x04`	weder `0x02` noch `0x03` noch schon `0x04`
`0x03`	weder `0x02` noch `0x03` noch `0x04` — ein Briefkasten hält die Meldung, das ist kein Aufgeben
`0x02`	alles außer `0x02` — eine späte Quittung heilt auch „aufgegeben" und „verwahrt"

Der Befund vom 05.09. („Häkchen verschwand“) bleibt damit ausgeschlossen. Die Vorlage meldet `0x03`, wenn ihre Wiederholungsleiter aufgibt (3 Wiederholungen im 40-s-Takt, aufgegeben nach 160 s). Die Uhr wiederholt nicht, wartet seit 1.0.5.33 aber ebenso lange: Offene Direktmeldungen stehen in `s_dmOffenId[4]` (Ziel, Startzeit), bis das `:ack` des Empfängers kommt oder `WZ_LORA_DM_AUFGABE_MS` (160 s) um ist – dann 0x03 mit dem Zielrufzeichen. Die Sendekarte geht nach 30 s ohne Quittung in `WZ_TX_WAITING` (blau, sperrt **nicht**: Bake und neue Aufträge laufen wieder, ein neuer Tipp hat Vorrang) und wird erst nach 160 s rot. In 1.0.5.31/.32 kam 0x03 schon nach 30 s.

`--ackinfo on|off` (Vorlage `bAckInfo`, `command_functions.cpp:2391`): Die App schaltet es selbst ein, um zu zeigen, wer alles gehört und quittiert hat. Dann geht **jede** Meldung hinaus, auch eine schon gemeldete oder niedrigere Quittungsstufe (`0x00` – `0x02`, nie ein überholtes `0x03` / `0x04`). Der Schalter ist flüchtig, steht nie im NVS und gilt nur für die Verbindung, in der er gesetzt wurde (`s_ackInfoGen == WZ_BLE_ConnGen()`). Das Echo `--ackinfo on|off` geht wie `addBLECommandBack()` als Textmeldung von `response` an `*` zurück, mit Bit `0x20`.

Seit 1.0.4.48 (260903-d5i) wertet dieselbe Funktion die Text-Quittung auch *aus*, statt sie nur zu verschlucken: `WZ_LoRa_NoteTextAck(src, dst, text, ueberDraht)` korreliert `nnn` gegen `s_txAwaitingAck` / `s_txTimedOut` / `s_ownIds[]` und setzt über `wz_lora_tx_note_acked()` bzw. `wz_lora_tx_note_late_acked()` denselben Zustand wie der 12-Byte-ACK-Pfad — die Sendekarte wird grün. Aufgerufen wird sie aus **beiden** Empfangswegen (`WZ_LoRa_HandleRx()` und `WZ_UDP.cpp::getUDP()`); bis 1.0.4.47 kannte nur der Funkweg die Textform, über den Draht stand die Quittung als gewöhnliche Meldung in der Liste. Verlangt werden jetzt drei Ziffern hinter dem Marker (`:ack089`), damit ein Fließtext wie „:ackermann” nicht verschluckt wird.

Bis 1.0.3.69 ging **jedes** Funkpaket **vor** der Entdopplung nackt hinaus — siehe die Korrektur in 21b.3.

In der Gegenrichtung gilt die Regel aus Kapitel 13.1 unverändert:

> **Im Empfangsrückruf wird nichts verarbeitet.** Er läuft im NimBLE-Task, also außerhalb des Hauptablaufs. Er legt das Telegramm in eine Warteschlange und kehrt zurück; abgeholt wird im Hauptablauf.

Bemerkenswert daran: Die MeshCom-Quelle löst es genauso (`xQueueSend` im Rückruf, `esp32_main.cpp:356–364`). Zwei unabhängig entstandene Firmwares kommen bei derselben Task-Grenze zum selben Muster — ein Hinweis darauf, dass es keine Geschmacksfrage ist.

Die Warteschlange ist bewusst kurz (8 Einträge) und verwirft bei Überlauf. Ein Rückstau bedeutet, dass der Hauptablauf nicht nachkommt — dann ist Verwerfen ehrlicher als eine wachsende Warteschlange, die den Speicher aufbraucht.

21b.6 Die Anmeldung und der Konfigurations-Rundlauf

Nach dem Verbinden schickt die App ein Anmelde-Telegramm (`[Länge][0x10][0x20][0x30]`) und **wartet auf Antwort** — bleibt sie aus, steht die App mit einer Verbindung da, über die nichts kommt. Die Uhr antwortet mit dem vollständigen Konfigurationssatz als JSON-Telegramme (`0x44`-Kennung): `I` Grunddaten, `IS1` Bauzeit, `SE` Sensoren, `SW` WLAN, `SN` Knoten, `SN1` Via/Autosymbol, `W` Wetter, `G` Position, `SA` Symbol, `IO`, `TM`, `AN`, je Station ein `MH`, zuletzt `CONFFIN` (`CR_BlePhone.cpp`, Reihenfolge wie `esp32_main.cpp:297`).

Die MH-Rahmen (seit 1.0.5.31 nach dem 4.40-Vertrag)

`MH` meldet, wen die Uhr **direkt** gehört hat: das letzte Glied des Pfades (`msg_source_last` der Vorlage), nicht den Urheber. Eine Meldung von OE0AAA-1 über OE0BBB-2 ist für die App ein Empfang von OE0BBB-2. Die Tabelle dafür führt `CR_MHeard` (24 Plätze, 12 h), gefüllt im Funkempfang neben `CR_Stations`, das weiter die Urheber für den Stationsschirm sammelt. Bis 1.0.5.30 kamen die Rahmen aus `CR_Stations`, mit leerem Datum und `0` für HW, MOD und Entfernung.

Aufbau nach `mh_phone.h` (v4.40a): die 13 alten Felder in alter Reihenfolge, dann die neuen.

Feld	Quelle an der Uhr
`DATE` / `TIME`	Ortszeit des Empfangs; ohne gestellte Uhr **kein** Rahmen (wie die Vorlage)
`PLT`	Typzeichen `:` `!` `@` als Zahl
`HW`	`last_hw & 0x7F` aus dem Paketschwanz
`MOD`	`mod` aus dem Schwanz; ohne Bit 7 in `last_hw` stammt er nicht vom letzten Sender → `| 0xF0`
`RSSI` / `SNR`	dieser Empfang
`DIST`	km auf 0,1 gerundet, nur aus einer **direkt** gehörten Position; `-1` = unbekannt
`PL`	Pfadglieder (Urheber = 1)
`MESH`	Byte 5, Bit `0x10`
`NCNT`	`/Nnn` aus einer direkt gehörten Position
`AGE`	Minuten seit dem letzten Empfang
`EX`, `NB`, `GW`, `VIA`	`0` — die Uhr führt keine Nachbarmatrix

`HM` und `ROLE` fehlen; die Vorlage lässt sie bei „unbekannt“ ebenso weg. Passt der Rahmen nicht in 243 Zeichen, fallen die neuen Felder weg. Beim Anmelden geht ein Schnappschuss der Liste hinaus (neueste zuerst), danach ein Live-Rahmen je Station höchstens einmal pro Minute, hinter wartenden Meldungen. `CR_MHeard_SelfTest()` prüft beim Start Schwanz, Pfadglieder, `/N` (auch den Fall, der **nicht** treffen darf) und den JSON-Text Zeichen für Zeichen.

`IS1` und `SN1` kamen mit MeshCom v4.35v (seit 1.0.5.25 auch bei uns): `I` und `SN` lagen an der BLE-Längengrenze, die Vorlage lagerte aus. `IS1` trägt `BDATE` (`YYYYMMDD-HHMMSS` aus `__DATE__` / `__TIME__`), `SN1` die Felder `VIA`, `VIACALL`, `WSPWD`, `ASYM`. Die Uhr hat weder Via-Rufzeichen noch Autosymbol (`false`/leer). `WSPWD` bleibt **leer**, das Webkennwort geht nie über BLE. Unser `SN` behält sein leeres `WSPWD` für ältere Apps. Wird `SN` nach einem Befehl nachgereicht, folgt `SN1` einen Abstand später.

Zwei Lehren stecken darin:

> **Der Satz muss vollständig sein.** Der erste Anlauf schickte nur die vier Telegramme mit echten Daten und ließ die übrigen weg — die App baut ihre Ansicht aber aus dem VOLLSTÄNDIGEN Satz auf und blieb weiß. Ein Telegramm mit `false` / `0` ist keine Erfindung, sondern die Auskunft „dieses Gerät hat diese Hardware nicht”.

> **Zwischen den Telegrammen wird gewartet** (300–400 ms, wie die Vorlage) — deshalb ist das ein Zustandsautomat mit Zeitgeber, keine Schleife.

Setzt der Betreiber einen PIN (`--setBTcode`), verlangt die Uhr ihn bei der Anmeldung: Die App hasht den auf sechs Stellen nullgefüllten PIN-**String** mit SHA-256 und schickt die 32 Byte mit (`phone_commands.cpp:283-291`); die Uhr bildet denselben Hash und vergleicht. Ohne gültige Anmeldung wird kein Befehl ausgeführt. Ohne gesetzten PIN ist der Zugang offen — wie in der Vorlage, Festlegung Christian.

Eine Anmeldung gilt nur für die Verbindung, in der sie erfolgt ist (seit 1.0.5.4). `WZ_BLE` zählt bei jedem Verbinden eine Nummer weiter (`WZ_BLE_ConnGen()`), `CR_BlePhone` merkt sich bei der Anmeldung diese Nummer und prüft sie vor jeder Änderung (`cr_blephone_angemeldet()`). Bis 1.0.5.3 blieb der Anmeldemerker nach dem Trennen stehen, wenn gerade kein Handshake lief. Ein anderes Telefon, das sich danach verband, galt dann ohne PIN als angemeldet. Die Bindung an die Nummer greift auch dann, wenn Trennen und Neuverbinden zwischen zwei Durchläufen des Hauptlaufs liegen.

⚠️ Gehasht wird die ZEICHENKETTE `"012345"`, nicht die Zahl `12345` — wer die Zahl hasht, bekommt einen Hash, der nie passt, und der Fehler sieht aus wie ein falsch getippter PIN.

21b.7 🔬 Der Textkanal ist der Steuerkanal

Der wichtigste Messbefund der App-Anbindung (Mitschnitte 07.08. und 13./14.08.): **Die App benutzt die dafür vorgesehenen Binärtelegramme nicht.** In drei Minuten Mitschnitt mit stehender Verbindung kam kein einziges `0x50` (Rufzeichen) — stattdessen `0x20` (Zeitstempel) und `0xA0`, der Textkanal. Durch ihn schickt die App **Konsolenbefehle**:

App-Handlung	Was wirklich ankommt
Rufzeichen ändern	`--setcall OE3LCR-66` (klein geschrieben)
„Position senden"	`--setlat … --setlon … --setalt …` in EINEM Telegramm
Country-Auswahl	`--setctry EU8` — sofort beim Umstellen des Dropdowns
Symbol-Auswahl	`--symid /` + `--symcd >` als zwei Befehle (gemessen 14.08.: NICHT `0x95`)
APRS-Kommentar	`--atxt <text>` (nicht der Alias `--aprscomment`)

Die Unterscheidung trifft die Vorlage selbst (`phone_commands.cpp:536`): beginnt der Inhalt mit `--`, ist es ein Befehl, sonst eine Meldung.

> **Jeder neue Konsolenbefehl ist damit automatisch auch ein App-Befehl.** Und umgekehrt: Was die Uhr der App meldet, kann als Befehl zurückkommen (siehe Echo-Filter, 21b.8).

21b.8 Zeit und Position vom Telefon

Zeit (`0x20`, 4 Byte Unix-UTC): niedrigster Rang aller Zeitquellen (Entscheidung Christian 13.08.) — sie stellt nur eine **ungestellte** Uhr (Plausibilitätsgrenze 2025-01-01, dieselbe wie bei der Funknetz-Zeit). Ist die Systemzeit plausibel, wird nur die Abweichung geloggt; gemessen am Gerät: „Abweichung +0 s", die RTC geht exakt. In die RTC wird höchstens einmal je Laufzeit geschrieben.

Position: Der Binärweg (`0x70` / `0x80` / `0x90`: je 4-Byte-Wert plus Save-Flag, `0x0A` = feste Position → sofort speichern, `0x0B` = periodisch) ist vollständig implementiert — die App benutzt ihn aber nicht, sie schickt das `--setlat/--setlon/--setalt`-Trio durch den Textkanal. Beide Wege führen auf dieselbe Kette:

1. Das Trio gilt nur **komplett und frisch** (10-s-Fenster, `WZ_PARSER_POS_TRIO_MS`) — eine Höhe ohne frisches Breite/Länge-Paar wird verworfen und **geloggt** (Lehre vom 07.08.: stilles Verwerfen ist im Mitschnitt von „App schickt nichts" nicht zu unterscheiden).
2. `CR_GpsFix_SetFromPhone()` prüft Plausibilität und Rangfolge: **echter GPS-Fix > Telefon-Position > aufgehobene Altposition** (Begründung in E-12, `doku/Entscheidungen.md`).
3. Die erste Telefon-Position jeder Laufzeit wird sofort in den NVS gesichert — sonst stand nach jedem Reset wieder „H 0 m" am Schirm.
4. Ein gültiger GPS-Fix überschreibt zusätzlich sofort die Knotenposition (`fLATpos` / `fLONpos` / `iALTpos`) - dieselben Werte, die `--setLAT` / `--setLON` / `--setALT` und die App setzen. Bis 1.0.4.49 sendete die Bake (`WZ_LoRa_SendPosition()`) **immer genau diese** Knotenposition, ohne Ruhe- oder Altersbedingung - wie die Standard-MeshCom-Firmware (E-17, `doku/Entscheidungen.md`, löst E-16 ab). **Seit 1.0.4.50 wählt `wz_lora_pos_quelle()`** zwischen Fix, frischer Laufzeit-Position (App oder eigener Fix, Frist `CR_GPSFIX_TX_MAX_AGE_S`) und der Knotenposition; letztere nur auf Uhren **mit** GNSS (E-35, Technikbuch 19.8.1). Eine Abweichung zur Vorlage bleibt bewusst bestehen: der Fix wird zusätzlich alle 15 Minuten in den NVS gesichert (`CR_GpsFix_Loop()`) und überlebt damit einen Neustart - die Vorlage hält ihn nur im Arbeitsspeicher.

Der App-Schalter **TRACK** (SN-Telegramm) meldet seit 1.0.3.93 einen eigenständigen Zustand (`en_TRACK`, eigene Kachel „Track") - er ist NICHT mehr an die Positionsbake gekoppelt (bis 1.0.3.92 spiegelte TRACK `en_POSBEACON`). Bei GPS-Fix sendet Track Positionen im SmartBeaconing-Takt (10-20 s) ausschließlich über das eigene Gateway; über Funk höchstens alle 5 Minuten (`WZ_LORA_TRACK_LORA_SECS`), dann über denselben Weg wie die normale Bake.

Der Echo-Filter (`CR_BlePhone_IstPosEcho`): Die App spiegelt die im `G`-Telegramm gemeldete Node-Position periodisch als set-Trio zurück. Ohne Filter überschrieb dieses Echo jede frischere Position — der „Höhe 0"-Befund. Verglichen wird gegen das zuletzt Gemeldete, gerundet auf die Genauigkeit, mit der es die App erreicht hat (`%.5f`), mit einer Toleranz von einer halben letzten Stelle ($5 \cdot 10^{-6}$ Grad ≈ 0,5 m); die Höhe muss exakt stimmen.

Seit 1.0.4.50 deckt der Filter BEIDE Wege. Bis dahin stand er nur im Textkanal (`WZ_Parser.cpp`, `--setALT`-Zweig); der Binärweg `0x90` rief `CR_GpsFix_SetFromPhone()` ungefiltert auf. Solange die Bake `fLATpos` las, war das folgenlos. Seit die Telefonposition die Bake speist (E-35), hielte sich eine gespiegelte Altposition **selbst am Leben**: Uhr meldet Position → App spiegelt sie → Uhr übernimmt sie als „frisch" → Bake sendet sie wieder. Der `0x90`-Zweig fragt deshalb jetzt vor der Übernahme `CR_BlePhone_IstPosEcho()`, loggt eine Ablehnung (`[BLEPHONE] Telefon-Position ist das Spiegelbild der eigenen Knotenposition – ignoriert`) und verbraucht das Trio trotzdem, damit der nächste Satz sauber mit `0x70` beginnt.

⚠ Der Filter liefert bewusst `false`, solange noch nie ein `G`-Telegramm gemeldet wurde (`s_gGemeldet`) — sonst gälte die **erste** echte Telefonposition nach dem Verbinden fälschlich als Echo. Der Vergleichswert wird beim **Melden** gesetzt (`cr_blephone_telegramm_pos()`), nicht von der Bake; die Quellenwahl aus 1.0.4.50 berührt ihn nicht.

> ⚠ Der Filter ist die konkrete Form eines allgemeinen Satzes: **Alles, was die Uhr der App meldet, kann als Befehl zurückkommen.** Bei jedem neuen Feld in einem Antwort-Telegramm gehört diese Frage mitgedacht.

Symbol und Kommentar (seit 1.0.3.60 — die frühere 0x95-Verwerfung „das Symbol steht fest" ist revidiert, Entscheidung Christian 14.08.): Das APRS-Symbol kommt wahlweise binär (`0x95`: `[Tabelle][Zeichen]`, Wache `/` oder `\` wie die Vorlage) oder als `--symid` / `--symcd`-Textbefehl — die App nutzt gemessen den **Textweg**, `0x95` bleibt trotzdem implementiert (dasselbe Doppelweg-Muster wie bei der Position, deren Binärtrio die App ebenfalls links liegen lässt); der Kommentar über `--atxt` / `--aprscomment` (max. 40 Zeichen, `none` löscht, `"` und `\` sind wegen des JSON- und APRS-Wegs abgelehnt). Alle drei Werte liegen

im NVS (`symid` / `symcd` / `atxt`) — und **die Positionsbake und das SA-Telegramm lesen dieselben Werte**. Das ist die Lehre aus dem Zustand davor: Das SA-Telegramm meldete der App fest `#` , während der Funk `[` (Person zu Fuß) hinaustrug — das fünfte Beispiel für „Anzeige liest andere Quelle als die Wirkung".

21b.9 Der Weg Telefon → Funk: seit 1.0.3.64 offen (E-13)

Bis 1.0.3.63 wurden in der App getippte Meldungen angenommen und verworfen — „eine Aussendung löst ausschließlich der Lizenzinhaber aus". Beim Durchtesten der App fiel auf, dass diese Sperre **symbolisch geworden war**: Über denselben `0xA0` -Kanal konnte die App längst `--sendPOS` und `--sendHEY` ausführen, also sehr wohl Aussendungen auslösen.

Entscheidung Christian (14.08., E-13 in `doku/Entscheidungen.md`): Chat-Meldungen (führendes `:` , das Format der Vorlage) gehen über `WZ_LoRa_RequestSend()` hinaus — derselbe Weg wie die Sendekarten, samt Rufzeichen-Wache, Statusmaschine und Sendesperren (Reglerstufe 0 und Akku-Schutz stoppen auch den App-Chat). Bewusst **ohne PIN-Pflicht**; ein gesetzter `--btcode` schützt aber beide Wege — die Anmelde-Wache steht seit diesem Umbau VOR Chat und Befehlen.

Seit 17.08. (1.0.3.70) mit Ziel: Die App schreibt `{ZIEL}Text` — ZIEL ist ein Rufzeichen (Direktmeldung) oder eine Gruppennummer; ohne Klammer geht es an `*` (Vorlage `sendMessage()` , `loop_functions.cpp:3100`). Direktmeldungen bekommen wie in der Vorlage den Quittungswunsch `{nnn` angehängt (nnn = untere acht Bit der Nachrichten-ID). Umlaute und Emojis kommen von der App als `%XX` -Folgen und werden wie in der Vorlage nur für die UTF-8-Einleitungen zurückgewandelt. Jede eigene Textmeldung geht als **Echo** an die App (0x40-Rahmen) — daran kennt sie die ID und setzt später die Häkchen: „gehört" (unsere Meldung kam über einen Digipeater zurück) und „quittiert".

⏳ **Noch offen:** die Antwort auf eine *empfangene* Direktmeldung mit Quittungswunsch (`SendAckMessage` der Vorlage, Text-DM `:acknnn` zurück an den Absender) — braucht einen eigenen Sendeweg und berührt die Regel „Aussendung nur durch den Lizenzinhaber". ⚠️ Nicht verwechseln mit der Gegenrichtung: Eine *eingehende* Text-Quittung wird seit 1.0.4.48 ausgewertet (s.o.) — offen ist ausschließlich das **Senden** einer solchen Quittung.

Das Echo an die App ist unverändert: Es entsteht in `WZ_LoRa_SendTextTo()` über `CR_BlePhone_QueueFrame()` . Der seit 1.0.4.48 zusätzlich abgelegte Eintrag in der **Meldungsliste** (`CR_MsgBridge_Push(..., CR_MSG_ORIGIN_EIGENE, ...)` im Sende-Automaten) ist davon getrennt und geht nicht an die App.

Was weiterhin bewusst nicht gebaut ist

Der Einstellungs-Dienst `0xF0A0/0xF0A1`

Die nRF52-Fassung der MeshCom-Firmware bietet einen zweiten Dienst an, über den die App die Geräteeinstellungen liest und schreibt. Übertragen wird `s_meshcom_settings` **als roher Speicherblock**.

Ein Blick in die Struktur beendet die Überlegung (`reference/meshcom-firmware/src/esp32/esp32_flash.h:28,105`):

```
char node_opwd[40]    = {0};   // das WLAN-Kennwort
char node_passwd[15]  = {0};   // das Geräte-Kennwort
```

> **Ein unverschlüsselter Funkdienst, der Zugangsdaten herausgibt, kommt nicht in Betracht.** Kapitel 25 beschreibt den Aufwand, mit dem in diesem Projekt sichergestellt wird, dass Zugangsdaten das Gerät nicht verlassen — ein BLE-Dienst, der sie auf Anfrage sendet, hebt diesen Aufwand vollständig auf.

Die Uhr wird über die Konsole und den Setup-Schirm eingestellt. Das ist umständlicher und bleibt so.

21b.10 ⚠️ Strom

BLE-Werbung ist ein **Dauerverbraucher**. Deshalb steht `en_BLE` per Vorgabe auf *aus* (`prefs.cpp` , NVS-Schlüssel `enBLE`); `HAS_BLE` schaltet lediglich den Code frei, nicht den Funk.

Der Grund ist nicht Vorsicht, sondern Messbarkeit:

> Die Laufzeit-Grundlast der Uhr ist erst seit den Logbuch-Läufen vom August belastbar (Kapitel 24) — und ein dauernder Verbraucher, der vorher zugeschaltet wird, macht beide Größen hinterher untrennbar.

Das ist exakt die Falle aus Kapitel 6, wo ein „abgeschaltetes" GNSS-Modul jede Sparmessung vor dem 3. August 2026 entwertet hat. Sie wird hier nicht wiederholt.

Die Sendeleistung ist auf die niedrigste Stufe gesetzt: Die Gegenstelle ist ein Telefon in Armlänge, nicht eine Station am Berg. Der Laufzeitbedarf des Stacks ist am Gerät gemessen: **66,7 kB** interner RAM — möglich wurde das erst, als `LV_MEM_POOL_ALLOC` den LVGL-Vorrat ins PSRAM verlegte und damit 128 kB internen

RAM freigab.

21b.11 Der Stand

✅ Uhr wirbt im MeshCom-Muster	`MC-<4 Hexstellen>-<Rufzeichen>` — nur so findet die App sie
✅ Meldungen an die App	seit 1.0.3.70: `0x40`-Rahmen + Zeit, nach Entdopplung, Filter wie Vorlage, 20er-Ring für die Zeit ohne App, auch Draht-Weg
✅ Anmeldung `0x10` mit Antwort-Rundlauf	vollständiger Konfigurationssatz bis `CONFFIN`
✅ PIN-Schutz	SHA-256, aktiv sobald `--setBTcode` gesetzt ist
✅ Befehle der App (`0xA0`)	laufen durch denselben Parser wie die Konsole
✅ Zeit-Übernahme (`0x20`)	stellt nur eine ungestellte Uhr
✅ Positions-Übernahme	Textkanal-Trio + Binärweg, Echo-Filter, Rangfolge E-12
✅ Netzwerkwerte an die App	SW mit Live-IP/GW/DNS/SUB + S2, wie die Vorlage
✅ APRS-Symbol + Kommentar (`0x95` /Textweg)	seit 1.0.3.60 — Bake und SA lesen dieselben NVS-Werte
✅ Chat-Fenster → Funk	seit 1.0.3.64 (E-13), Weg der Sendekarten samt allen Sperren; seit 1.0.3.70 mit Ziel `{ZIEL}`, %-Dekodierung, Echo + Häkchen an die App
✅ Eingehende Text-Quittung auswerten	seit 1.0.4.48 — `WZ_LoRa_NoteTextAck()`, Funk **und** Draht, färbt die Sendekarte grün
⏳ Quittung auf empfangene Direktmeldung	**senden** (`:acknnn` zurück an den Absender) fehlt noch (eigener Sendeweg)
🔴 Einstellungs-Dienst `0xF0A0`	bewusst offen gelassen — enthält Zugangsdaten

22 Der nichtflüchtige Speicher (NVS)

Alles, was einen Neustart überleben soll, liegt im NVS des ESP32 — Rufzeichen, WLAN-Zugänge, Modulschalter, die Entladekurve. Verwaltet wird das ausschließlich in `prefs.cpp`, über die Arduino-Klasse `Preferences`.

Die Fallen dieses Kapitels haben eines gemeinsam: **Sie äußern sich nicht als Fehler, sondern als Wert, der stillschweigend nicht gespeichert wurde.**

22.1 Zwei Funktionen, dieselbe Reihenfolge

`Read_Prefs()` liest beim Start, `Save_Prefs()` schreibt. Beide sind **spiegelbildlich aufgebaut** — dieselben Schlüssel in derselben Reihenfolge. Das ist keine Kosmetik: Ein Schlüssel, der nur in einer der beiden Funktionen vorkommt, ist entweder nicht speicherbar oder nicht lesbar, und beides fällt beim Ausprobieren nicht auf.

Jedes `preferences.begin()` braucht sein `preferences.end()` vor dem Verlassen der Funktion.

22.2 Schlüssel sind höchstens 15 Zeichen lang

Eine harte Grenze des NVS. Deshalb die abgekürzten Namen:

Schlüssel	Bedeutung
`enWIFI`, `enGPS`, `enLORA`, `enMQTT`	Modulschalter
`enPOSB`, `enACK`	Positionsbake, Quittungen
`wifi1s` … `wifi3s`	Netznamen der drei Plätze
`wifi1p` … `wifi3p`	Kennwörter, verschlüsselt
`brightness`, `BTcnt`	Helligkeit, Startzähler

> ⚠ Ein zu langer Schlüssel wird **stillschweigend abgeschnitten**. Zwei Schlüssel, die sich erst ab dem 16. Zeichen unterscheiden, sind derselbe Eintrag.

22.3 Die Default-Falle

Die teuerste Falle dieses Kapitels. `Save_Prefs()` schreibt nur, wenn sich der Wert geändert hat:

```
if (preferences.getString("sCALL", "") != sCALL) { preferences.putString("sCALL", sCALL); }
```

Das spart Schreibzyklen — der Flash hält nicht unbegrenzt viele aus. Aber es hat eine Bedingung:

> ⚠ **Der Vergleichs-Default in `Save_Prefs()` darf nicht derselbe Ausdruck sein wie der in `Read_Prefs()`.** Sonst gilt: Wert kommt aus dem Compile-Makro → ist gleich dem Default → wird nie geschrieben → beim nächsten Start kommt er wieder aus dem Makro. Der Eintrag entsteht **nie**.

Genau so kam das Rufzeichen monatelang aus dem Übersetzungsmakro statt aus dem Speicher. Am Gerät sah alles richtig aus — bis jemand ein anderes Rufzeichen setzen wollte.

22.4 Die Schemaversion — Aufräumen, nicht Wegwerfen

`PREFVERSION` (derzeit **4**) steht in `globals.h`. `Inc_counter()` vergleicht sie beim Start mit dem gespeicherten Wert. Bei Abweichung läuft ein Ablauf in **drei Schritten** (`prefs.cpp:46`):

```
if (VP != VersionPreferences) {
    preferences.end();
    Read_Prefs();        // 1. alle definierten Parameter LESEN
    nvs_flash_erase();   // 2. Bereich löschen
    nvs_flash_init();
    Save_Prefs();        // 3. alle zuvor gelesenen NEU SCHREIBEN
```

> ⚠️ **Es geht dabei nichts verloren, was noch gebraucht wird.** Zuerst werden alle Parameter gelesen, die die Firmware kennt; dann wird der Bereich gelöscht; dann werden genau diese Werte zurückgeschrieben. Rufzeichen, WLAN-Zugänge und alle übrigen Einstellungen überleben den Versionswechsel.

Was verschwindet, sind ausschließlich veraltete Parameter — und das ist der Zweck der Übung. Ein Schlüssel geht genau dann weg, wenn `Read_Prefs()` ihn **nicht mehr liest**: Was nicht gelesen wird, wird auch nicht zurückgeschrieben. Der Bereich enthält danach genau die Schlüssel, die die laufende Firmware kennt, und keinen einzigen mehr.

Daraus folgt die praktische Regel: **`PREFVERSION` zu erhöhen ist gefahrlos.** Wer ein Feld umbenennt oder seine Bedeutung ändert, soll die Nummer erhöhen — der alte Schlüssel wird dabei sauber entsorgt, statt als Altlast liegen zu bleiben.

> ⚠️ Die Kehrseite steht in 22.5: Ein alter Schlüssel, der noch als Rückfall **gelesen** wird, überlebt den Versionswechsel deshalb ebenfalls. Er ist kein Versehen, sondern eine Entscheidung — man muss sie nur kennen.

22.5 Einen einzelnen Wert löschen

Das Aufräumen veralteter Schlüssel erledigt der Versionswechsel aus 22.4 von selbst. Etwas anderes ist es, **im laufenden Betrieb einen einzelnen Wert loszuwerden** — etwa wenn ein Nutzer einen WLAN-Zugang wieder entfernt. Dafür gibt es keinen Automatismus: `preferences.remove()` muss ausdrücklich aufgerufen werden, und zwar für **jeden** Schlüssel, der zu einer Sache gehört:

```
void Clear_Wifi_SSID(uint8_t slot) {
  case 1:
    preferences.remove("wifi1s");
    sWifi1SSID = "";
    preferences.remove("sSSID");   // ⚠ alter Schlüssel als Rückfall
    sSSID = "none";
```

> ⚠️ **Platz 1 hat zwei Schlüssel.** Aus der Zeit vor den drei WLAN-Plätzen liegt dort noch der alte Einzelschlüssel (`sSSID`, `sPWD`), der beim Lesen als Rückfall dient. Wer nur den neuen löscht, bekommt beim nächsten Start den alten Wert zurück — und hält den Löschbefehl für kaputt.
>
> Und weil `Read_Prefs()` diesen alten Schlüssel weiterhin liest, **übersteht er auch einen Versionswechsel** (22.4). Er ist damit der einzige Weg, auf dem ein Altwert dauerhaft im Bereich bleibt — genau deshalb muss er hier ausdrücklich mit entfernt werden.

Dass Netzname und Kennwort **getrennt** gelöscht werden (`Clear_Wifi_SSID` und `Clear_Wifi_Override`), ist ebenfalls Absicht: Ein Netz mit geändertem Kennwort behält seinen Namen. Der Konsolenbefehl `--setSSID <platz> off` ruft beide auf.

22.6 Das Akku-Logbuch im NVS

Ein Sonderfall: 180 Messpunkte, alle fünf Minuten einer, also 15 Stunden. Sie liegen als Ringpuffer im NVS und überstehen damit auch einen leergelaufenen Akku — das ist der ganze Sinn der Sache, denn genau dann will man wissen, was passiert ist.

Ein Eintrag hat ein Feld `flags` mit acht Zustandsbits, von denen seit dem 6. August 2026 alle belegt sind. **Wer ein neuntes braucht, verbreitert das Feld — und damit die Größe jedes Eintrags im NVS.**

Unterhalb von 15 % Ladung wird jeder Messpunkt sofort gesichert statt gesammelt: Das Ende der Kurve ist der interessanteste Teil, und es ist genau der Teil, den ein plötzliches Abschalten verschluckt.

23 Firmware-Update über Funk

Die Uhr lässt sich ohne Kabel aktualisieren. Der Entwurf folgt der Tasmota-Vorgabe und besteht aus wenigen, ausdrücklich getroffenen Entscheidungen — jede davon schränkt die Möglichkeiten ein, und jede tut das aus einem benannten Grund.

23.1 Die fünf Grundentscheidungen

Aus dem Kopf von `CR_OTA.h:9–18`:

Kennung	Entscheidung	Grund
D-01	Der Web-Server läuft **nur**, solange der Update-Schirm geöffnet ist	Strom (Akkulaufzeit ist Projektziel) und keine dauerhafte Angriffsfläche
D-03	Daraus folgt die Freigabe: **ohne Uhr in der Hand kein Server**, ohne Server kein Update	zusätzlich verlangt jeder Endpunkt das Passwort
D-05	Das Passwort stammt aus der nicht versionierten Konfiguration (`OTA_PASSWORD`)	nie aus dem Code, nie aus dem Protokoll — siehe Kapitel 25
D-10	Die Uhr sucht **niemals von selbst** nach Updates	der URL-Weg läuft ausschließlich auf ausdrückliche Anforderung
E	Die Versionsprüfung löst **niemals** ein Update aus	reine Anzeige, ausgelöst beim Betreten des Schirms

Es gibt folglich **keinen** Start in `setup()` — nur `CR_OTA_Start()` aus dem Schirm heraus.

> **Die Zugangsbeschränkung ist keine zusätzliche Schicht, sondern eine Folge des Entwurfs.** Wer die Uhr nicht in der Hand hat, kann den Server nicht starten; wer ihn nicht starten kann, findet keinen Endpunkt vor.

⚠️ Genau diese Kopplung an den geöffneten Schirm war zugleich die Quelle eines Fehlers an ganz anderer Stelle: Die Taktbremse des Main-Loops fragte versehentlich die **Server-Bereitschaft** statt den **Update-Zustand** ab und fiel damit schon beim Betreten des Schirms weg (Kapitel 15.4).

23.2 Zwei Wege

Weg	Ablauf	Wofür
Datei-Upload	Browser lädt die `.bin` direkt auf die Uhr	ein einzelnes Gerät, beliebiger Stand
URL	Die Uhr holt sich die Datei selbst von einer hinterlegten Adresse	der Normalfall bei gepflegtem Server

Beide teilen sich denselben Ablaufzustand (`CR_OTA.h:74`). Der URL-Weg ist bequemer, der Upload-Weg unabhängig von einem erreichbaren Server — deshalb gibt es beide.

23.3 Das Server-Schema

Neben der Firmware-Datei liegt eine `version.json` **im selben Ordner** (`CR_OTA.h:29–40`), mit genau zwei Schlüsseln:

```
{ "version": "1.0.3.0", "notes": "Kurzbeschreibung, optional" }
```

- `version` ist **Pflicht** und muss exakt vier durch Punkt getrennte Zahlenfelder haben — dasselbe Format wie `CR_VERSION` (Kapitel 4.3). Verglichen wird numerisch Feld für Feld.
- `notes` ist optional und wird dem Benutzer angezeigt.

⚠️ Die `notes` schreibt der Betreiber selbst; `ota_release.sh` lässt sie bewusst unangetastet und zieht nur die Nummer nach. Eine stehengebliebene Beschreibung aus einer früheren Fassung wird also weiterhin angezeigt — sie ist beim Hochzählen mitzupflegen (`ota_release.sh --notes "…"`).

23.4 ⚠️ Zwei Dateien, ein Stand

Auf dem Server liegen **zwei** Abbilder, und sie werden von verschiedenen Seiten geholt:

Datei	Wer holt sie	Inhalt
`firmware.bin`	**die Uhr** über den OTA-Weg	nur die Anwendung
`firmware.factory.bin`	**der Browser** über den Web-Flasher	Vollabbild samt Bootloader

> **Wer nur eine der beiden Dateien erneuert, hat danach zwei verschiedene Stände im Umlauf.** `ota_release.sh` kopiert deshalb immer beide.

⚠️ Umgekehrt gilt: In den OTA-Ordner gehört ausschließlich das Anwendungsabbild. Die Factory-Datei dort abgelegt führt beim Update zu „Flash Read Failed".

23.5 Was der Ablauf voraussetzt

Die Update-Kette ist nur so verlässlich wie ihr schwächstes Glied, und das ist nicht die Technik, sondern die Gewohnheit: Der Server muss nach **jedem** Flashen nachgezogen werden. Warum, steht in Kapitel 4.5 — es hat zweimal je einen Arbeitstag gekostet.

Der Blank-Timer ist während eines laufenden Updates ausgesetzt (`main.cpp:960–966`): Der Schirm steht auf 60 Sekunden, ein Update dauert länger, und ohne diese Klammer ginge die Uhr mitten im Schreiben des Abbilds dunkel. Der Countdown läuft dabei bewusst weiter — nur sein Zuschlagen wird unterdrückt, sodass nach dem Aufheben sofort wieder das normale Verhalten gilt.

24 Der Akku: was gemessen ist — und was nicht

Die Akkulaufzeit ist ein erklärtes Projektziel. Dieses Kapitel sagt, was darüber wirklich bekannt ist, und trennt das scharf von dem, was nur plausibel klingt. Diese Trennung ist hier besonders nötig: Zwei Zahlen, die lange als gesichert galten, haben sich als falsch erwiesen — die eine, weil das Messverfahren defekt war, die andere, weil sie aus einer Zeit stammte, in der es die Hälfte der heutigen Verbraucher noch gar nicht gab.

24.1 Drei Größen, die auseinanderzuhalten sind

Größe	Woher	Belastbar?
Spannung in mV	`actualVoltage`, gelesen vom AXP2101	✅ ja — die einzige Größe, die Vergleiche trägt
Prozent	`CR_BattGauge`, Kennlinie über der Spannung (bis 5.8.2026: Schätzung des Leistungsreglers)	⚠️ reproduzierbar, aber lastabhängig — siehe unten
Restlaufzeit	wird nirgends berechnet	— es gibt sie nicht, und das ist Absicht

> ⚠️ **Die Prozentanzeige ist eine Schätzung des Leistungsreglers, keine Messung.** Sie fällt deutlich schneller als die Spannung. Im Lauf vom 4. August sank die Spannung gleichmäßig um rund 40 mV pro Stunde, während die Prozentzahl von 70 auf 18 stürzte. Für jeden Vergleich zwischen zwei Läufen gilt deshalb: **über die Spannung vergleichen, nie über das Prozent.**

Die Anzeige auf dem Ziffernblatt zeigt trotzdem Prozent — weil das für den Träger die verständlichere Größe ist. Für die Entwicklung ist sie es nicht.

Der Beleg: dieselbe Spannung, zwei Anzeigen

Am 5. August 2026 lieferte ein Tageslauf mit 75 Messpunkten den harten Nachweis. Die Uhr lud vormittags am Kabel und entlud sich abends am Handgelenk — dieselben Spannungen kamen also zweimal vor, einmal steigend, einmal fallend:

Spannung	beim Laden	beim Entladen
3744 mV	20 %	**50 %**
3751 mV	24 %	62 % / 52 %
3757 mV	25 %	53 %
3806 mV	31 %	58 %

30 Prozentpunkte Unterschied bei identischer Spannung. Das ist keine Ungenauigkeit mehr, sondern eine andere Größe: Die Anzeige folgt dem *Verlauf*, nicht dem Ladezustand. Der Regler schätzt aus der Richtung mit, in die sich die Spannung zuletzt bewegt hat.

Damit ist jede Laufzeitrechnung aus Prozenten wertlos — auch die scheinbar harmlose Form „von 63 % auf 20 % in 2 h 50 min, macht 15 %/h".

✅ Die Behebung: eine Spannungskennlinie (ab 6. August 2026)

`batt_percent` kommt seither nicht mehr aus dem Leistungsregler, sondern aus einer festen Kennlinie über der gemessenen Spannung (`CR_BattGauge`). Die Umschaltung geschieht an einer einzigen Stelle (`WZ_BATT.cpp`, als Abweichung markiert) — dadurch profitieren Ziffernblatt, Info-Schirm **und** die Prozentspalte des Logbuchs gemeinsam.

> **Der Hauptgewinn ist nicht Genauigkeit, sondern Reproduzierbarkeit.** Dieselbe Spannung ergibt jetzt immer denselben Prozentwert. Erst dadurch werden zwei Messläufe überhaupt vergleichbar — vorher verglich man zwei Zahlen, die aus verschiedenen Vorgeschichten stammten.

Der Bezugsbereich ist bewusst **3400–4100 mV** und nicht der theoretische Zellenbereich 3300–4200 mV: Die Uhr lädt nur bis 4100 mV (Ladeschluss, bewusst niedriger für die Lebensdauer) und schaltet bei 3480 mV geordnet ab (bis 1.0.3.118: 3400 mV). „100 %" heißt damit *voll geladen, wie diese Uhr lädt*, und „0 %" heißt *gleich ist Schluss*. Eine Anzeige, die bei Ladeschluss 95 % zeigt und bei Abschaltung 8 %, wäre für den Träger nutzlos.

Gegenprobe an den Messpunkten von oben:

Spannung	Kennlinie	PMU beim Entladen	PMU beim Laden
3744 mV	**49 %**	50 % ✓	20 % ✗
3751 mV	**50 %**	52 % ✓	24 % ✗
3806 mV	**61 %**	58 % ✓	31 % ✗

Aufschlussreich ist das Muster: Der Leistungsregler lag beim **Entladen** durchweg ungefähr richtig und beim **Laden** weit daneben. Die Kennlinie liefert nun in beiden Richtungen den entlade-korrekten Wert.

⚠️ **Was daran gemessen ist und was nicht:** Die Stützstellen stammen aus der üblichen LiPo-Ruhespannungskurve und sind an zwei eigenen Messpunkten gegengeprüft — sie sind **nicht** aus einem vollständigen eigenen Entladelauf gewonnen. Sobald ein Lauf von voll bis zur Abschaltung im Logbuch steht, gehören sie daraus neu gesetzt. Bis dahin ist dies eine begründete Annäherung, keine Messung.

⚠️ **Was eine Spannungskennlinie prinzipiell nicht kann:** Unter Last bricht die Spannung ein (Sendespitze, Vibrationsmotor), beim Laden liegt sie über der Ruhespannung. Die Glättung im Modul trägt Lastspitzen weg, beseitigt den Effekt aber nicht. Beides ist der Preis für eine reproduzierbare Zahl — der Reglerwert hatte diese Fehler ebenfalls, nur zusätzlich zu seiner Verlaufsabhängigkeit.

⚠️ **Für die Auswertung ändert sich nichts:** `tools/battlog.py` rechnet weiterhin ausschließlich mit Millivolt. Aufzeichnungen von vor diesem Stand tragen in der Prozentspalte noch den alten Wert; das Ablageformat bleibt unverändert, damit die bisherige Aufzeichnung nicht verworfen wird.

> ⚠️ **Fallstrick beim Messen:** Ein Lauf mit wenig Funkverkehr sieht sparsam aus. Derselbe 5.-August-Lauf ergab 52,6 mV/h — die Hälfte des Vergleichslaufs vom Vortag. Die Erklärung stand nicht in den Zahlen, sondern im Kalender: Der Träger war im Kino, und der Empfangszähler stand zweieinhalb Stunden lang auf null. Ein Funkmodul, das nichts hört, hat auch nichts zu verarbeiten. **Zu jeder Kurve gehört die Frage, was die Uhr in dieser Zeit tatsächlich zu tun hatte** — der RX-Zähler jedes Messpunkts beantwortet sie.

24.2 Das Logbuch (`CR_BattLog`)

Eine Entladekurve lässt sich nicht am Schreibtisch messen: Die Uhr hängt am Handgelenk, und genau dort entsteht der Verbrauch, der interessiert. Deshalb schreibt die Uhr ihre eigene Kurve mit.

Größe	Wert	Konstante
Messabstand	5 min	`CR_BATTLOG_INTERVAL_MS`
Plätze	180 → **15 Stunden**	`CR_BATTLOG_SLOTS`
Schreibabstand in den Flash-Speicher	15 min	`CR_BATTLOG_FLUSH_MS`
Sofortsicherung unterhalb	15 %	`CR_BATTLOG_LOW_PERCENT`

Jeder Messpunkt trägt neben Spannung und Prozent auch **welche Verbraucher liefen** — Laden, USB, WLAN, GPS, LoRa, Bildschirm. Ohne diese Angaben wäre die Kurve wertlos: Zwei Läufe mit unterschiedlicher Modulbelegung sind nicht vergleichbar, und die Belegung hinterher zu rekonstruieren gelingt nicht.

Zwei Entwurfsentscheidungen verdienen Beachtung:

Der Schreibabstand ist bewusst gröber als der Messabstand. 15 Minuten statt 5 bedeutet 96 statt 288 Schreibvorgänge am Tag. Der Flash-Speicher der Uhr verteilt Schreibzugriffe selbst (Wear-Leveling), aber es gibt keinen Grund, ihn ohne Not zu belasten.

Unterhalb von 15 % wird jeder Messpunkt sofort gesichert. Das Ende der Kurve ist genau der Teil, wegen dem das Logbuch überhaupt gebaut wurde — und wenn der Leistungsregler abschaltet, gibt es keine zweite Gelegenheit.

Ausgegeben wird die Kurve beim Start, aber nur am Kabel (`main.cpp:463`). Im Akkubetrieb wären 180 Logzeilen bei jedem Start reiner Ballast — und würden Strom kosten, worum es hier ja gerade geht.

24.3 Die Schutzabschaltung

Bis zum 4. August 2026 hatte die Firmware **keinerlei** Schutzabschaltung. Ein leerlaufender Akku wurde einfach so weit entladen, bis der Leistungsregler von sich aus abschaltete.

Seither gilt (`CR_Power.cpp:48–61`):

Schwelle	Wert	Wirkung
Warnung	**3600 mV** (bis 1.0.4.93: 3550)	Ladebalken wird rot, rotes „!" am Prozenttext, kurzer Klick; fällt mit Stufe 1 zusammen
Abschaltung	**3480 mV**	letzter Messpunkt + Meldungen gesichert, 3 s Abschied-Schirm „Akku leer – bitte laden" (seit 1.0.4.94), langes Brummen, `pmu.shutdown()`

> ⚠️ **Warum 3480 und nicht 3400 (seit 1.0.3.119).** Am 29.08.2026 lief die Uhr bis 3397 mV und war dann ohne geordnete Abschaltung weg (Zähler unverändert, alle Meldungen des Tages verloren). Die PMU-eigene Abschaltung (VOFF) liegt bei 2600 mV — der Killer ist früher: DC1 hält unter ~3,4 V Akkuspannung die 3,3-V-Schiene nicht mehr, der Brownout-Detektor setzt den Chip zurück. Eine Software-Stufe bei 3400 mV kommt dagegen immer zu spät. Dazu: 40 mV Hysterese am Abschaltzähler (eine Messung knapp über der Schwelle nullt ihn nicht mehr) und Meldungssicherung ins NVS bei Stufe 1, Stufe 2 und alle 15 Minuten. | Rückweg | Warnschwelle + 100 mV | Warnung setzt sich von selbst zurück |

Beide Schwellen sind entprellt: Erst **5 aufeinanderfolgende Messungen** im 2-Sekunden-Raster lösen aus — also 10 Sekunden durchgehend unterhalb der Schwelle.

Das ist keine Vorsicht um der Vorsicht willen, sondern zwingend: Eine LoRa-Aussendung oder der Vibrationsmotor ziehen kurzzeitig so viel Strom, dass die Spannung einbricht. Ohne Entprellung würde die Uhr sich mitten im Senden abschalten — bei halb vollem Akku.

Am Ladekabel wird nie gewarnt und nie abgeschaltet. Der Kabelzustand wird als Erstes geprüft, und alle Entprellzähler werden dabei zurückgesetzt.

Für den Schreibtischtest gibt es das Flag `CR_POWER_SHUTDOWN_TEST`, das die Schwellen auf 3950/3900 mV hebt — damit lässt sich die Abschaltung an einem normal geladenen Akku auslösen, statt ihn erst leerlaufen zu lassen. Im Auslieferungsbild ist davon kein Byte enthalten.

24.4 Was gemessen ist

Lauf 2 — vollständig, mit Abschaltung (5. August 2026)

Firmware 1.0.1.0, Gerät am Handgelenk getragen. Betriebsbedingungen vollständig:

Module	WLAN, GNSS, LoRa — alle aktiv
Bildschirmhelligkeit	**70 %**
Abschaltzeit des Bildschirms	**30 s**, jeweils voll ausgeschöpft
Funkbetrieb	mehrere **Aussendeversuche**, dazu empfangene Meldungen

```
06:51 UTC   3987 mV   96 %      ← Kabel ab
08:41       3826 mV   65 %
10:21       3746 mV   43 %
12:06       3636 mV   20 %
12:56:44    3427 mV   11 %      ← letzter Rasterpunkt
12:58:01    3389 mV   10 %      ← Abschalt-Messpunkt, KEIN Rasterpunkt
```

Laufzeit 6 h 07 min. Der letzte Eintrag fällt bewusst aus dem 5-Minuten-Raster: Ihn schreibt `CR_Power` unmittelbar vor `pmu.shutdown()`, zusammen mit einer erzwungenen Sicherung in den Flash-Speicher. **Damit ist die Schutzabschaltung erstmals am Gerät nachgewiesen** — Auslösung bei 3389 mV gegen eine Schwelle von 3400 mV, mit vollständiger Kette aus entprellter Erkennung, Datensicherung und geordnetem Abschalten.

Lauf 1 — nachts, unvollständig (4./5. August 2026)

Firmware 1.0.0.7, gleiche Modulbelegung, Gerät abgelegt:

```
20:33 UTC   3827 mV   70 %      ← Kabel ab
22:03       3774 mV   55 %
23:03       3711 mV   42 %
01:14       3635 mV   18 %      ← letzte Aufzeichnung, Ursache offen (24.6)
```

⚠️ Nicht mit vollem Akku gestartet; die Zahl ist keine volle Laufzeit.

Vergleich beider Läufe

Vergleichbar sind die Läufe nur über eine **gemeinsame Spannungsstrecke**, hier 3826 → 3636 mV:

	Dauer für 190 mV	Bildschirm an	Aussendungen
Lauf 1 (nachts, abgelegt)	**4 h 41 min**	**0** von 47 Messpunkten	keine
Lauf 2 (tagsüber, getragen)	**3 h 25 min**	**8** von 75 Messpunkten	mehrere

Lauf 2 verbrauchte auf derselben Strecke rund ein Viertel mehr. **Zwei** Betriebsunterschiede kommen dafür in Betracht, und beide traten gemeinsam auf:

1. **Der Bildschirm** war tagsüber wiederholt aktiv (70 % Helligkeit, 30 s Nachleuchtdauer), nachts durchgehend abgeschaltet.
2. **Aussendungen** fanden nur tagsüber statt. Eine LoRa-Aussendung zieht kurzzeitig ein Vielfaches des Ruhestroms — der Empfangsbetrieb allein ist damit nicht vergleichbar.

> ⚠️ **Beide Ursachen sind hier nicht trennbar.** Da sie gemeinsam auftraten, lässt sich der Mehrverbrauch keinem von beiden zuordnen. Eine Zuordnung erforderte zwei weitere Läufe, in denen jeweils nur eine der beiden Größen verändert wird.

⚠️ Methodischer Hinweis: das 5-Minuten-Raster unterschätzt kurze Ereignisse

Die Angabe „8 von 75 Messpunkten" ist **keine Einschaltdauer**, sondern eine Stichprobe. Bei einer Nachleuchtdauer von 30 s und einem Messabstand von 300 s beträgt die Trefferwahrscheinlichkeit je Einschaltvorgang rechnerisch nur etwa **10 %** — die tatsächliche Zahl der Einschaltvorgänge liegt also deutlich höher als acht.

Dasselbe gilt für Aussendungen: Sie dauern Bruchteile einer Sekunde und werden vom Raster praktisch nie erfasst.

> **Das Logbuch eignet sich zur Aufzeichnung von Zuständen, nicht von Ereignissen.** Wer den Einfluss kurzer Vorgänge quantifizieren will, benötigt eine Ereigniszählung — etwa einen Zähler für Einschaltvorgänge und Aussendungen, der im jeweiligen Messpunkt mitgeschrieben wird. Ein solcher existiert bislang nicht.

24.5 ⚠️ Was NICHT gemessen ist

Diese Liste ist der wichtigere Teil des Kapitels.

Die volle Laufzeit ist unbekannt. Der einzige vollständige Lauf startete bei 70 %. Eine ältere Notiz nennt „rund 14 Stunden" — die stammt aus der Zeit **vor** GPS und LoRa und ist damit gegenstandslos. Eine spätere Messung ergab „etwa 6 Stunden", betraf aber einen Lauf mit defektem Abschaltverhalten (siehe unten).

Die Stromwirkung des Loop-Takts ist hergeleitet, nie gemessen. Seit dem Fall REND-03 (Kapitel 14) läuft der Hauptablauf mit rund 235 statt 1 Hz — das Zweihundertfache. Dass die 3-ms-Taktbremse den Mehrverbrauch auffängt, ist eine **Annahme**. Belegt ist sie nicht.

Der größte Einzelverbraucher ist nicht sicher bestimmt. Lange galt GPS mit rund 30 mA als Hauptposten. Eine Vergleichsmessung hat das auf etwa 20 % des Gesamtverbrauchs korrigiert — die Zahl gilt aber nur für den einen gemessenen Lauf.

⚠️ **Und alle Sparmessungen vor dem 3. August 2026 sind wertlos.** Bis dahin setzte das Abschalten von GPS nur ein Flag und kappte die Stromschiene nicht — ein Lauf mit „GPS aus" zog exakt denselben Strom wie einer mit GPS an. Die vollständige Geschichte steht in Kapitel 6.

Die Temperaturangabe — ein Lehrstück über „gültig gerechneten Unsinn"

Das Betriebsprotokoll meldete über Monate `Temp: 385.70 °C`. Der Wert wurde als offensichtlicher Anzeigefehler abgetan und nicht untersucht. Beides war falsch: Er war **kein** Anzeigefehler, und er war präzise erklärbar.

Nachgerechnet. XPowersLib rechnet den Rohwert so um:

```
#define XPOWERS_AXP2101_CONVERSION(raw)   (22.0 + (7274 - raw) / 20.0)
```

Mit `raw = 0` ergibt das `22 + 7274/20 = 385,7` — **exakt der gemeldete Wert**.

Belegt am Datenblatt. Der Grund für den Rohwert null steht in der Registerbeschreibung des AXP2101, Register `0x30` (`adc_ch_en0`):

```
tdie_ch_en   Bit 4   RW   POR = 0b
             die temperature measure ADC channel enable
             0: disable   1: enable
```

Der Kanal ist nach dem Einschalten **abgeschaltet**, und `initBATT()` aktivierte drei andere Kanäle — Akku-Erkennung, Akkuspannung, USB-Spannung — nur diesen nicht. Der ADC maß nie, das Ergebnisregister blieb null, die Umrechnung machte daraus eine Zahl.

> ⚠️ **Genau das macht diese Fehlerart gefährlich: Es gab nichts abzufangen.** Der Wert war kein Fehlercode, kein `NaN` und keine Bereichsüberschreitung — die Bibliothek prüft `raw > 16383` und liefert dann `NAN`, aber null liegt im gültigen Bereich. Eine Plausibilitätsprüfung hätte den Wert verworfen, aber die Ursache nie gefunden.

Nach dem Einschalten des Kanals (`pmu.enableTemperatureMeasure()`), gemessen während des Ladens:

```
[BATT] Chg:1 USB:1  54%  Bat:3.809 V  USB:4.530 V  Chip:44.7 °C
[BATT] Chg:1 USB:1  54%  Bat:3.815 V  USB:4.700 V  Chip:43.5 °C
```

⚠️ Und es ist nicht die Akkutemperatur

Der zweite Befund wiegt schwerer als der erste. `getTemperature()` liest die Ergebnisregister `0x3C/0x3D`. Die ADC-Kanalliste des Datenblatts ordnet diese eindeutig zu:

Kanal	Funktion	Ergebnisregister
0	BAT voltage	0x34/0x35
1	Vbus voltage	0x36/0x37
2	Vsys voltage	0x38/0x39
3	**TS voltage** — Akku-Temperaturfühler	0x3A/0x3B
4	**die temperature** — Chip	**0x3C/0x3D**

Gemessen wird also die **Sperrschichttemperatur des Leistungsreglers**, nicht die des Akkus. Das passt zu den Messwerten: 43–45 °C während des Ladens sind Verlustwärme des Reglers, nicht die Temperatur einer Zelle.

Die Akkutemperatur läge auf **Kanal 3** und setzte einen Heißleiter am `TS`-Pin voraus (Datenblatt 7.7.4: 10 kΩ bei 25 °C, zwischen `TS` und Masse). Ob die T-Watch-S3-Plus einen solchen bestückt hat, ist **nicht geprüft** — dafür wäre der Schaltplan des Herstellers heranzuziehen (Bezugsquellen in Kapitel 2b.5).

Die Beschriftung im Betriebsprotokoll lautet deshalb seit dem 5. August `Chip:` statt `Temp:` — die alte legte die Verwechslung nahe.

> **Die allgemeine Lehre:** Ein unplausibler Messwert ist kein Grund, ihn auszublenden. Er ist ein Hinweis darauf, dass eine Messkette an einer bestimmten Stelle nicht das tut, was angenommen wird. Hier waren es zwei Annahmen auf einmal: dass der Sensor misst, und dass er den Akku misst.

24.6 🔴 Ein offener Befund: das Logbuch bricht bei 180 Einträgen ab

Im Lauf vom 4. August endete die Aufzeichnung um 01:14 UTC nach **genau 180 Einträgen** — der vollen Puffergröße. **Die Uhr lief zu diesem Zeitpunkt weiter**, vom Träger am Morgen bestätigt.

Ausgeschlossen ist (Absturzzähler vorher/nachher verglichen): kein Watchdog-Neustart, kein Absturz, kein Neustart überhaupt. Ebenfalls geprüft und in Ordnung: die Ringpufferlogik (`CR_BattLog.cpp:122–123`) und die Aufrufkette im Sekundenraster (`main.cpp:905`).

Es bleibt eine **ungeprüfte** Hypothese: der Hauptablauf auf Kern 0 könnte stehengeblieben sein, während der Anzeige-Task auf Kern 1 weiterlief — die Uhr fühlt sich dann lebendig an, obwohl Funk, GPS und Logbuch stillstehen.

Was Lauf 2 dazu beiträgt

Der Abbruch ist nicht reproduzierbar. Lauf 2 schrieb ohne Unterbrechung bis zum Abschaltpunkt. Damit scheidet ein grundsätzlicher Fehler in der Ringpufferlogik aus — sie bewältigt den Überlauf.

Ein Unterschied fällt auf und ist als Spur festzuhalten: Lauf 2 unterschritt die Marke von 15 % (`CR_BATTLOG_LOW_PERCENT`), ab der **jeder** Messpunkt sofort in den Flash-Speicher geschrieben wird. Lauf 1 endete bei 18 % — dort galt noch der gebündelte Schreibabstand von 15 Minuten, und alle Messpunkte seit dem

letzten Schreibvorgang lagen ausschließlich im Arbeitsspeicher.

⚠️ Das erklärt jedoch **höchstens 15 Minuten** fehlender Aufzeichnung, nicht die beobachteten knapp sechs Stunden. Der Befund bleibt offen.

> **Merksatz für die Fehlersuche daran:** Die Antwort steht im Arbeitsspeicher, nicht im Flash-Speicher. `CR_BattLog_Dump()` gibt das RAM-Feld aus. Ein Rücksetzen vor dem Mitlesen löscht genau den gesuchten Beweis. **Erst mitlesen, dann zurücksetzen** — beim ersten Versuch ist genau dieser Fehler passiert und hat die Klärung um einen Tag verzögert.

24.8 Der Lade-Wächter (1.0.3.113)

Anlass. Christian, 28.08.: „Der Ladestrom passt oft nicht, meist nach dem Flashen — USB ab- und anstecken hilft." Gemessen im Bootlog nach dem Flashen von .112: 18–23 s nach dem Start zweimal `[PWR] USB-Quelle zu schwach (VINDPM)`, davor `USB:4.714 V` am Eingang; 25 Minuten später 5,125 V und Ladephase „fertig". Der Gesundheitszähler stand bei 145 VINDPM-Minuten seit dem 26.08. — das ist kein Einzelfall, sondern der Normalfall an diesem Mac-Port unter Bootlast (WLAN-Scan, GPS-Schiene, LoRa, Display).

Was der AXP2101 kann — und was nicht. Er hat keinen VBUS-Schalter: die Leitung lässt sich per Software nicht öffnen. Er kann den Eingangsstrom begrenzen (Register 0x16), das Laden sperren (0x18 Bit 1), die VINDPM-Schwelle setzen (0x15, 3,88–5,08 V in 80-mV-Schritten; Werkswert wird beim Start geloggt) und sich selbst neu starten (0x10 Bit 1, POWOFF/POWON aller Schienen). Die Schwelle zu senken bringt wenig: er ist ein Linearlader, unter ~4,3 V Eingang lädt er eine 4-V-Zelle nicht mehr.

Drei Stufen, alle im Main-Task (PMU-I2C-Regel), 2-s-Raster in `CR_Power_Loop()`:

1. Nach 60 s VINDPM: Soft-Neuanstecken — Laden sperren, Limit 100 mA, 200 ms, Limit und Ladestrom neu, Laden frei (`cr_power_chargefix_apply()`, auch per `--ladefix`).
2. Nach 3 min: Ladestrom flüchtig auf `CR_POWER_CHGCUR_STEPS[0]` (125 mA). Der Sollwert `cr_power_chgCurrentMA` bleibt unangetastet — kein NVS, bewusst wie E-11 bei GPS. Rückweg, sobald VINDPM 30 s lang weg ist (Anti-Flattern: beim Boot wechselte VINDPM binnen 5 s zweimal).
3. Nach 8 min, nur mit `--setPMURESET on` (NVS `pmuReset`, Vorgabe aus), Akku < 4000 mV, nicht während OTA: `pmu.reset()`. Davor Zähler + `CR_BattHealth_Flush()` und der NVS-Merker `pmuRstPend`; beim nächsten Start wird er eingelöst und sperrt Stufe 3, bis das Kabel einmal weg war — **eine Reset-Schleife an einem dauerhaft schwachen Port ist damit ausgeschlossen.**

Konten. `CR_BattHealth` zählt Soft-Neuanstecken (`bhLadefix`) und PMU-Neustarts (`bhPmuRst`) dauerhaft; `--akku` zeigt dazu Ladephase (gecacht, kein I2C aus fremden Tasks), VINDPM-Schwelle, Eingangsspannung und den Stand des Wächters.

Was nicht bewiesen ist. Ob Stufe 1 den Mac-Port tatsächlich „umstimmt", entscheidet die Messung beim nächsten Auftreten (`--ladefix`, dann `[BATT]`-Zeilen vergleichen). Stufe 2 wirkt nach Datenblatt sicher, Stufe 3 ist das Rezept vom 18.08. — beides ohne Gerätetest ausgeliefert (Uhr am Handgelenk, OTA), daher Stufe 3 ab Werk aus.

24.7 Merksätze für Messungen an diesem Gerät

> 1. **Über die Spannung vergleichen, nie über das Prozent.**
> 2. **Nur gemeinsame Spannungsstrecken vergleichen** — Läufe mit unterschiedlichem Startstand sind über die Dauer nicht vergleichbar.
> 3. **Die Modulbelegung jedes Laufs mitschreiben.** Sie steht in den Spalten des Logbuchs und lässt sich hinterher nicht rekonstruieren.
> 4. **Nur eine Größe je Lauf verändern.** Bildschirm und Aussendungen traten in Lauf 2 gemeinsam auf und sind deshalb nicht trennbar.
> 5. **Kurze Ereignisse zählen, nicht abtasten.** Das 5-Minuten-Raster erfasst einen 30-Sekunden-Vorgang rechnerisch nur in etwa 10 % der Fälle; Aussendungen praktisch nie.
> 6. **Erst mitlesen, dann zurücksetzen.**
> 7. **Spannung VOR einer Aussendung messen, nie danach.** Senden zieht kurzzeitig den größten Strom des ganzen Geräts; die Spannung bricht ein und braucht danach Zeit zur Erholung. Eine Messung im Nachlauf liefert einen zu niedrigen Wert, der aussieht wie ein gültiger.

24.7.1 Warum Merksatz 7 auch den Quelltext betrifft

Merksatz 7 ist keine reine Messanleitung. `readBATT()` läuft im **1-Sekunden-Raster** des Hauptlaufs (`main.cpp:927`) — also blind gegenüber dem Sendegeschehen. Während der Aussendung selbst wird nicht gemessen (der Sendeaufruf hält den Hauptlauf an), **wohl aber unmittelbar danach**, wenn der Akku sich noch nicht erholt hat.

⚠️ Und an einer Stelle wandert dieser Wert nach außen: **Die Positionsbake schickt den Akkustand mit ins Netz** (`WZ_LoRa.cpp`, `CR_MeshCom_BuildPos()`). Stammt er aus dem Nachlauf der vorherigen Aussendung, meldet die Uhr dem Netz dauerhaft einen zu schlechten Akku als sie hat.

Deshalb misst der Sendepfad **vor** der Bake einmal ausdrücklich nach, statt den zuletzt zufällig entstandenen Wert zu verwenden.

24b Die Stromsparstufen: Ziel, Wirkung, Mechanik

Die Uhr zieht im Realbetrieb rund 150 mA im Mittel — bei 940 mAh Akku sind das etwa sechs Stunden. Ein A/B-Messtag (20. August 2026) hat die Verdächtigen sortiert: **Nicht die Weckrate** des Prozessors dominiert (diese Hypothese wurde widerlegt), sondern zwei Dauerläufer — die **GPS-Suche ohne Fix** und die **WLAN-Netzsuche unterwegs**. Der LoRa-Empfang, der Kernauftrag der Uhr, ist mit ~5 mA der kleinste Funkposten und wird bewusst nie angetastet.

Gegen diese Verbraucher stehen sechs Maßnahmen. Jede folgt derselben Bauart: ein `en_ECOxxx`-Schalter nach dem Sechs-Ebenen-Modell (Kapitel 3), im NVS persistiert, per Konsole schaltbar, in Text-Sicherung und WebUI sichtbar. Dieses Kapitel beschreibt für jede Maßnahme **Ziel** (welcher Verbrauch soll weg), **Wirkung** (was passiert am Gerät) und **Mechanik** (wo im Quelltext, mit welchen Konstanten).

24b.1 Der GPS-ECO-Automat in CR_Power

Vier der sechs Maßnahmen betreffen die GPS-Stromschiene (BLDO1). Damit sie sich nicht gegenseitig ein- und ausschalten, laufen sie durch **einen** Automaten in `CR_Power_Loop()` (`CR_Power.cpp`), der im 30-Sekunden-Raster prüft und sich den wirksamen Abschaltgrund in `s_ecoReason` merkt:

Grund	Name	Schalter	Bedeutung
0	keiner	—	Schiene läuft (oder `en_GPS` ist aus)
1	Telefon	`en_ECOGPS`	App liefert Positionen, GPS wäre doppelt
2	Ruhe	`en_ECOIMU`	Uhr liegt still, Position ändert sich nicht
3	Fix-Suche	`en_ECOFIX`	10 min erfolglos gesucht → Sparpause
4	Bake-Vorlauf	`en_ECOBAKE`	GPS nur auf Bestellung vor der Bake

Es gilt: **nur ein Grund zugleich**, und `en_GPS` (Nutzer-Hauptschalter, Ebene 4) bleibt Master — steht er auf aus, hat der Automat nichts zu entscheiden. Grund 4 hat Top-Priorität vor der Ruhe-Erkennung: Steht eine Bake an, wird die Schiene auch dann geweckt, wenn die Uhr still liegt und Grund 2 sie sonst hielte. Jeder Wechsel schreibt eine `[PWR]`-Logzeile mit Grund im Klartext; `--info` nennt den aktiven Grund.

24b.2 ECOCPU — Prozessortakt (Vorgabe: an)

- **Ziel:** Grundlast des Rechenkerns senken. Der Hauptlauf dreht mit 240–300 Hz auch bei dunklem Schirm — der Kern ist nie untätig genug, um von allein zu sparen.
- **Wirkung:** Bei dunklem Schirm `setCpuFrequencyMhz(80)`, beim Wecken zurück auf 240 (`CR_Power.cpp`, im selben Automaten-Umfeld). Für die Bedienung folgenlos, weil der Takt vor dem ersten sichtbaren Bild wieder oben ist.
- **Ehrlichkeit der Messung:** Der Stufe-1-Messlauf (Prag, 14.08.2026) war **nicht beweiskräftig** — 70,1 mV/h mit ECOCPU gegen 59,6 mV/h ohne, aber bei völlig anderem Nutzungskontext (WLAN-Anteil, Weckrate, Fix-Anteil). Lehre für alle weiteren Messungen: mV/h-Vergleiche nur im flachen Teil der Entladekurve und nur bei protokolliertem Kontext (Kapitel 24.7).

24b.3 ECOGPS — Telefon liefert (Vorgabe: an)

- **Ziel:** Den Empfänger (~25 mA dauernd) abschalten, wenn die verbundene App dieselbe Information frei Haus liefert.
- **Wirkung:** Grund 1 greift, wenn zehn Minuten kein eigener Fix kam **und** die App-Positionen frisch sind; reißt die App-Versorgung ab, geht die Schiene wieder an.
- **Mechanik:** `CR_Power.cpp`, Grund-1-Zweig; die App-Position kommt über den 0xA0-Kanal (Kapitel 21b) und wird gegen ihr Alter geprüft.
- **Messwert:** Stufe-2-Lauf: **43,8 mV/h, –27 %** gegen den Stand davor — die bislang am saubersten belegte Einzelmaßnahme.

24b.4 ECOIMU — Ruhe-Erkennung (Vorgabe: an, seit 1.0.3.70)

- **Ziel:** Nicht getragene Uhr erkennen: dann sind GPS **und** Positionsbake sinnlos.
- **Wirkung:** Nach 10 min Stillstand (BMA423) Grund 2 — Schiene aus, Bake pausiert; die erste Bewegung hebt beides auf. Nach 2 min Ruhe wird zusätzlich die Schirm-Blankzeit auf 15 s gekappt. Die Hebe-Geste (Kippen, nicht Heben — dokumentierte Grenze, siehe Quick 260901) weckt den Schirm.
- **Mechanik:** `CR_Imu.cpp` liefert Ruhe-Sekunden (`CR_Imu_IsStillFor()`); der Automat konsumiert sie. Wichtig für Messungen: Am benutzten Schreibtisch erreicht der Ruhe-Zähler die 10 min praktisch nie — die Maßnahme wirkt auf dem Nachttisch, nicht am Arbeitsplatz.

24b.5 ECOFIX — Sparpause bei erfolgloser Suche (Vorgabe: an, seit 1.0.3.80er)

- **Ziel:** Das teuerste Muster beenden, das der A/B-Messtag belegt hat: GPS-**Suche ohne Fix** zieht mehr als GPS mit Fix (volle Korrelator-Leistung) — und drinnen sucht sie endlos vergebens.

- **Wirkung:** 10 min ohne Fix und ohne Aussicht → Grund 3, Schiene aus für `CR_POWER_FIXPAUSE_MS` = 30 min, dann neuer Versuch. Duty-Cycle drinnen: ~25 % Schienen-Laufzeit statt 100 %.
- **Mechanik:** `CR_Power.cpp` Grund-3-Zweig mit eigener Pausenuhr (`s_ecoFixOffAtMs`). Eine Schutzvorkehrung aus der Praxis: Nach dem **Einschalten** der Schiene zählt die Fix-Uhr neu — sonst kappte ECOFIX die Schiene 38 s nach dem Anlauf, weil die alten „10 min ohne Fix" noch standen.

24b.6 ECOWLAN — Modem-Schlaf + Unterwegs-Pause (Vorgabe: an, seit 1.0.4.31)

- **Ziel:** Die zwei WLAN-Posten aus der A/B-Messung: das Dauerlauschen im verbundenen Zustand und die **Endlossuche unterwegs**, die per Definition erfolglos bleibt, solange kein bekanntes Netz in Reichweite ist.
- **Wirkung, verbunden:** Nach jedem erfolgreichen Connect `WiFi.setSleep(WIFI_PS_MAX_MODEM)` — der Funkchip wacht nur zum DTIM-Sammeltakt des Routers auf statt zu jedem Beacon. Preis: einzelne Draht-Meldungen (UDP) kommen um Beacon-Intervalle verspätet. Bei `en_ECOWLAN` aus wird explizit `WIFI_PS_MIN_MODEM` gesetzt (Core-Default, keine stille Änderung).
- **Wirkung, unterwegs:** Nach `WZ_WIFI_TRAVEL_PAUSE_STREAK` = 3 **komplett** erfolglosen Suchrunden (Rundum-Scan **plus** gezielte Slot-Scans, kein konfigurierter Platz gesehen) wechselt der Verbindungsautomat in den neuen Zustand `WIFI_TRAVEL_PAUSE`: `WiFi.mode(WIFI_OFF)` für `WZ_WIFI_TRAVEL_PAUSE_MS` = 10 min. Der Treiber ist dabei wirklich aus — das spart auch die Scan-Bursts (~100 mA-Spitzen).
- **Drei Auswege aus der Pause** (`WZ_WiFi.cpp`, Zustand `WIFI_TRAVEL_PAUSE`):
  1. Pausenzeit abgelaufen → neue Suchrunde;
  2. **Bewegung nach Ruhe**: Lag die Uhr während der Pause mindestens `WZ_WIFI_TRAVEL_PAUSE_IMU_REST_S` = 30 s still (nur bei `en_ECOIMU` an messbar) und bewegt sich dann, endet die Pause sofort — das Heimkommen-Szenario. Die Ruhe-Vorbedingung verhindert, dass normales Handgelenk-Wandern beim Gehen die Pause dauernd sprengt;
  3. `--wlan scan` (Sofort-Scan von Hand).
- **Abgrenzung:** Der Fehlversuchs-Zähler `s_scanRoundsFailStreak` ist **getrennt** vom Eskalations-Zähler `s_timeoutStreak` der Trennungs-Diagnose (1.0.3.119, Treiber-Neustart ab 3, Uhr-Neustart ab 6 Dauer-Timeouts). Eine Unterwegs-Pause ist kein „WLAN-Leck" und darf die Eskalation weder füttern noch verhindern. Ein erfolgreicher Connect nullt die Fehlversuchs-Kette. `--wlan` (Status) nennt Pause und Restzeit.

24b.7 ECOBAKE — GPS auf Bestellung (Vorgabe: aus, seit 1.0.4.32)

- **Ziel:** Die Konsequenz aus ECOFIX zu Ende gedacht: Wer GPS nur für die 30-Minuten-Bake braucht, braucht keinen Dauerbetrieb mit Sparpausen — sondern gar keinen Dauerbetrieb. Aus „an, mit Pausen" wird „aus, mit Terminen".
- **Wirkung:** Grund 4 parkt die Schiene, solange die nächste Bake weiter als der Vorlauf entfernt ist. `WZ_LoRa_BeaconDueWithin(CR_POWER_ECOBAKE_LEAD_MS)` mit 3 min Vorlauf meldet den Termin; die Schiene läuft an, `CR_GpsAid` (Kapitel 28) drückt den Warm-Fix auf ~13 s, kalt am Grenzsignal bleiben 2–3 min Physik — der Vorlauf deckt beides. Nach der Bake fällt `BeaconDueWithin` wieder auf false, die Schiene parkt. Rechnerisch: wenige Minuten Empfängerlaufzeit je halbe Stunde statt 100 %.
- **Mechanik:** `WZ_LoRa_BeaconDueWithin()` (`WZ_LoRa.cpp`) ist eine **reine Zeitplan-Abfrage** aus LoRa-eigenen Statics — kein Eingriff in den Sende-/Empfangspfad (die Lehre „kein frühes return im RX-Pfad" aus Stufe 3 gilt weiter). Sonderfälle: `--sendPOS` setzt die Sofort-Anforderung, der Automat weckt die Schiene ohne Wartezeit neu (`[PWR] ECOBAKE: Sofort-Bake angefordert`); die allererste Bake nach dem Start (Kurztakt `WZ_LORA_POSBEACON_FIRST_SECS` = 120 s) gilt als „steht bevor".
- **Warum Vorgabe aus:** Die Maßnahme ändert spürbar das Verhalten — Radar und Entfernungen bekommen zwischen den Baken keine frischen Eigenpositionen, und bei ausgeschalteter Positionsbake (`en_POSBEACON` aus) liefert `BeaconDueWithin` dauerhaft false: GPS bleibt dann ganz aus, bis `--sendPOS` kommt. Das ist gewollt (kein Termin = kein Verbrauch), muss aber eine bewusste Entscheidung des Nutzers sein. Kein Fix bis zum Termin → die Bake fällt aus (MeshCom-Regel: nie eine erfundene Position senden), die Schiene geht **trotzdem** aus — Strom geht vor Vollständigkeit, der nächste Takt kommt bestimmt.

24b.8 ECOSLEEP — Nachtschlaf des Prozessors (Vorgabe: aus, seit 1.0.4.105)

- **Ziel:** Der Prozessor bleibt sonst die ganze Nacht wach, obwohl er nichts zu tun hat. Der ESP32-S3 kann im **Light Sleep** den Takt anhalten und behält dabei RAM und Zustand — aufgeweckt ist er in Millisekunden, ohne Neustart.
- **Wirkung:** Der Automat `cr_sleep_automat()` (`CR_Sleep.cpp`) öffnet eine **Phase**, wenn gleichzeitig gilt: Nachtruhe (`CR_Nachtruhe_IstNacht()`, 22:00–06:59), Uhr seit **120 s still** (`CR_Imu_IsStillFor(120)`), Schirm dunkel, kein USB, keine App. In der Phase ruht das WLAN (`WZ_WiFi_SleepPause(true)`), und der Prozessor schläft in Zyklen von höchstens 60 s. Geweckt wird über GPIO: **DIO1** (Pin 9, der SX1262 meldet ein fertiges Paket), **Touch** (16) und **PMU** (21). Nach jeder Weckung bleibt der Prozessor **300 ms** wach, wenn das Paket keine Reaktion brauchte, sonst **2000 ms** (Fix C, 1.0.4.142). Die Phase endet, sobald eine Bedingung wegfällt; der Grund steht im Nachtschlaf-Journal (NVS, `--info`).
- **Mechanik — vier Fallen, die eine Phase fälschlich beendeten** (Messnächte 15.–19.09., alle behoben in 1.0.4.142/.143):
  - **A — Vergleich über die Schlaflücke:** `CR_Imu_Loop()` läuft im Light Sleep nicht; die erste Abtastung nach dem Aufwachen verglich gegen eine bis zu 60 s alte Probe (gemessen 325 LSB gegen 38/39 ohne Schlaf). Jetzt meldet `CR_Imu_NoteSleepGap()` die Lücke, und die erste Probe danach erneuert nur die Bezugsprobe.
  - **B — Schwelle im Rauschen:** `CR_IMU_MOTION_LSB` lag mit 40 nur 1 LSB über dem gemessenen Tischrauschen; jetzt 80.
  - **C — feste Wachzeit:** 2 s nach jeder Weckung, auch für Pakete ohne Belang (83 × 2 s = 166 von 169 s Wachzeit einer Phase); jetzt 300 ms / 2000 ms.
  - **D — die Uhr schüttelte sich selbst wach:** Jede angezeigte Meldung vibriert, der BMA423 sah das als Bewegung (232–824 LSB). `CR_Haptic_Play*()` meldet jetzt über `CR_Imu_NoteShake()` ein Fenster, in dem Abtastungen die Ruhe-Uhr nicht nullen.

- **Warum Vorgabe aus:** In einer Phase fehlen Draht-Meldungen und die App — das ist ein sichtbarer Unterschied, den der Nutzer bewusst wählen muss.
- **Ergebnis:** Siehe 24b.10. Kapitel 29.6 beschreibt die ältere Stufe „LoRa-Warten statt Phasenende" (1.0.4.132, Entscheidung E-48).

24b.9 ECODIGI — Digipeater-Pause in der Nacht (Vorgabe: aus, seit 1.0.4.145)

- **Ziel:** Die Aussendungen sparen, die die Uhr nachts **für andere** macht. Als Digipeater (`en_MESHRELAY` , Kapitel „Node, Digipeater, Gateway" in `CR_Gateway.h`) strahlt sie fremde Pakete erneut aus, damit sie weiter kommen. Senden kostet am meisten Strom — und jede anstehende Aussendung hält den Prozessor zusätzlich wach.
- **Anlass (Messung 20./21.09.2026, 1.0.4.144, ECOSLEEP an):** Die Nachtschlaf-Fixes wirkten — die längste Phase lief 6 h 19 min, 91 % Schlafanteil (Vornächte 74–76 %, Phasen von 5–13 min). Der Verbrauch über die feste Spanne 3900 → 3780 mV blieb aber **gleich**: 33,5 mV/h gegen 32,5 / 31,2 in den Vornächten. Der Schlafanteil ist also nicht der Hebel. Was die Zähler derselben Nacht zeigen:

Größe	Nacht 20./21.09.
LoRa-Pakete empfangen	1635
davon ohne jede Reaktion	1332 (81 %)
als Digipeater weitergereicht	**303**
eigene Aussendungen gesamt	363
„LoRa-Warten" (Phase blieb wegen Sendearbeit wach)	249

  Eine Weiterleitung ist bei SF 11 / BW 250 / CR 4/6 rund **0,95 s Sendezeit** (~85 Byte).
- **Wirkung:** In der Nachtruhe reicht die Uhr **fremde** Pakete nicht weiter. Unberührt bleiben der **Empfang** (DIO1 weckt weiter, jedes Paket wird dekodiert, gezählt und — wenn es eine Meldung ist — angezeigt), die eigene Bake, Quittungen, `{ping}` -Antworten und der Gateway-Weg ins Internet (`en_GATEWAY`).
- **Mechanik:** Eine einzige Prüfung in `WZ_LoRa.cpp` , im Weiterleitungsblock **unmittelbar vor** `wz_lora_queue_relay()` :

  ```
  if (en_ECODIGI && CR_Nachtruhe_IstNacht()) {
    s_rxNachtAus++;
    dbLOG("[MESH RELAY] ... – Nachtruhe (ECODIGI), nicht weitergereicht\n", ...);
    break;
  }
  ```

  Die Stelle ist mit Absicht **die letzte** im Block:

  1. **Ehrliche Zählung.** Davor fallen alle Pakete heraus, die ohnehin nie weitergereicht würden: an uns gerichtet, `{ping}` / `{pong}` , `{CET}<` , Wiederholungen (`CR_MeshCom_IsRelayDuplicate`), vom Gateway zugestellt (`CR_Gateway_SkipMesh`), Hops aufgebraucht oder eigenes Rufzeichen im Pfad (`CR_MeshCom_BuildRelay` liefert 0). `nachtAus` zählt deshalb nur **echte** unterlassene Aussendungen. Es gilt: `relay + nachtAus` = was ohne Pause gesendet worden wäre; beide schließen sich über das `break` gegenseitig aus.
  2. **Entdopplung bleibt gepflegt.** Die ID ist bereits vermerkt. Fällt die Sperre um 07:00, strahlt die Uhr keine alten Wiederholungen nachträglich aus.
  3. **Zweiter, stiller Gewinn:** `wz_lora_note_reaktion()` wird **nicht** gerufen — es gab keine Reaktion. Damit gilt die Weckung als „ohne Reaktionsbedarf", und Fix C gibt das schnelle Wachfenster (300 ms statt 2000 ms). Außerdem belegt das Paket keinen Platz in der Sendeschlange, also meldet `WZ_LoRa_IsIdle()` früher „frei", und die Phase muss nicht auf einen freien Kanal warten.

  Der Preis von (1) und (2): Das Weiterleitungspaket wird gebaut und verworfen — einige Mikrosekunden Rechenzeit gegen eine Zahl, auf die man sich verlassen kann.
- **Sichtbarkeit:** `--setECODIGI` ohne Zusatz, `--info` (Zeile „Digipeater-Pause"), WebUI (Statuszeile „Schalter"), Sicherung (`CR_Backup`). Der Zähler steckt in `WZ_LoRaRxStats::nachtAus` . NVS-Schlüssel `ecodigi` .
- **Warum Vorgabe aus:** Es ist die einzige Sparstufe, die **dem Netz** etwas nimmt. Wo die Uhr die einzige Brücke zwischen zwei Stationen ist, erreichen deren Meldungen einander nachts nicht mehr. Das darf nie ohne Wissen des Nutzers geschehen.
- **Messplan:** Siehe 24b.10.

24b.10 Zusammenspiel und offene Messungen

Die Reihenfolge im Automaten entscheidet: Grund 4 (Termin) schlägt Grund 2 (Ruhe) — eine still liegende Uhr mit aktiver Bake bekommt ihren Fix. Grund 3 (Fix-Suche) bleibt als Obergrenze auch im ECOBAKE-Suchfenster sinnvoll: Findet die Schiene im Vorlauf nichts, beendet spätestens der Bake-Termin selbst den Versuch.

Was belegt ist: ECOGPS –27 % (Stufe-2-Lauf). **Was aussteht:** je ein sauberer mV/h-Lauf im flachen Spannungsfenster für ECOWLAN (unterwegs, ohne bekanntes Netz) und ECOBAKE (getragen, Bake an) — beide nach den Merksätzen aus Kapitel 24.7, mit protokolliertem Kontext. Erst diese Läufe machen aus „sollte sparen" ein „spart".

Nachtverbrauch, feste Spanne 3900 → 3780 mV, ohne Kabel:

Nacht	Stand	mV/h	Schlafanteil	längste Phase
11./12.09.	ohne Nachtschlaf	43,8	—	—
15./16.09.	ECOSLEEP, vor Fix A–D	32,5	74–76 %	5–13 min
16./17.09.	ECOSLEEP, vor Fix A–D	31,2	74–76 %	5–13 min
20./21.09.	ECOSLEEP + Fix A–D (1.0.4.144)	33,5	91 %	6 h 19 min
21./22.09.	+ ECODIGI (1.0.4.145)	*offen*		

⚠️ Die 18,9 mV/h der Nacht 13./14.09. sind **kein** Vergleichswert — sie stammen aus einem anderen Abschnitt der Entladekurve. ⚠️ Nie über einzelne Stunden rechnen: der Schirm bricht die Spannung kurzzeitig ein (–5 bis +70 mV/h Streuung).

Befund 20./21.09.: Light Sleep ist umgesetzt, und er funktioniert — 91 % Schlafanteil. Dass der Verbrauch trotzdem nicht sank, heißt: Was **während** des Schlafs weiterläuft, dominiert. Die Kandidaten sind LoRa (Empfänger dauernd an, 1635 Weckungen, 363 Aussendungen) und BLE (wirbt die ganze Nacht).

Messplan ECODIGI: Eine Nacht mit `--setECODIGI on`, sonst alles wie am 20./21.09. (ECOSLEEP an, Digipeater an, Gateway an, gleicher Liegeplatz). Belegt ist die Stufe, wenn **beide** Bedingungen gelten: 1. `nachtAus` liegt in der Größenordnung der 303 Weiterleitungen und `relay` bleibt nachts nahe 0 — sonst greift die Sperre nicht; 2. der Verbrauch über dieselbe Spanne sinkt **deutlich unter 31 mV/h** (die untere Grenze der Vornächte). Ein Wert zwischen 31 und 34 wäre Streuung, kein Beleg.

Bleibt (1) erfüllt, (2) aber nicht, ist die Aussendung nicht der Hebel — dann ist als Nächstes die DIO1-Weckung selbst an der Reihe (81 % der Pakete wecken umsonst).

25 Zugangsdaten und das öffentliche Abbild

Die Firmware, die auf Christians Uhr läuft, enthält sechs Geheimnisse im Klartext. Die Firmware, die auf dem Web-Flasher liegt, enthält null. Beide werden aus demselben Quelltext gebaut.

Dieses Kapitel beschreibt, wie diese Trennung hergestellt **und wie sie bei jedem Bau nachgewiesen** wird.

25.1 Was im Entwicklungsabbild steckt

Geheimnis	Inhalt
`CALL`	Rufzeichen
`SSID_NAME1`	WLAN-Name
`SSID_PWD1`	WLAN-Kennwort
`MQTT_SERVER`	private Broker-Adresse
`OTA_PASSWORD`	Update-Kennwort
`WIFI2_SSID`	zweiter WLAN-Name

Sie stammen aus der persönlichen Konfigurationsdatei und landen als Zeichenketten im Abbild. Wer eine solche `.bin` weitergibt, gibt sie mit — sie sind mit `strings` in Sekunden sichtbar, ohne jedes Werkzeug.

> ⚠️ **Das betrifft ausschließlich das Entwicklungsabbild auf dem Rechner des Entwicklers.** Diese Datei entsteht beim gewöhnlichen `pio run`, liegt unter `.pio/build/` und wird **nirgendwo hin weitergegeben** — nicht ins Versionsverzeichnis, nicht auf den OTA-Server, nicht in den Web-Flasher. Es ist kein Leck und war nie eines. Beschrieben wird hier eine **Gefahr für den Fall der Weitergabe**, nicht ein bestehender Zustand.
>
> Nachprüfbar ist beides: Der Befund oben stammt aus einer Messung am 6. August 2026 am eigenen Entwicklungsabbild (6 von 9 Werten gefunden). Das öffentliche Abbild entsteht auf einem anderen Weg (25.2) und wird vor jedem Ausspielen geprüft (25.3).

25.2 Die Auswahl beim Übersetzen

`globals.h:56–65` entscheidet, welche Konfiguration hereinkommt:

```
#ifdef CR_RELEASE_BUILD
  #include "CR_config_release.h"     // ← nur Platzhalter
#elif __has_include("WZ_config.h")
  #include "WZ_config.h"             // OE3WAS, nicht versioniert
#else
  #include "RC_config.h"             // OE3LCR
#endif
```

`CR_RELEASE_BUILD` wird **ausschließlich** in der Umgebung `t-watch-s3-plus-release` gesetzt und geht beiden anderen Pfaden vor. Das öffentliche Abbild bekommt damit ausschließlich Platzhalter aus `CR_config_release.h`.

⚠️ Weder `WZ_config.h` noch `RC_config.h` gehören ins Versionsverzeichnis. Zum Anlegen einer eigenen dient `WZ_config_example.h` als Vorlage.

⚠️ Diese wenigen Zeilen sind zugleich die Stelle, die bei Zusammenführungen mit `wolfgang/main` bereits **zweimal** verlorenging (Kapitel 2.3).

25.3 Die zweite Verteidigungslinie

Die richtige Umgebung zu wählen ist eine Absicht — und Absichten scheitern. Deshalb prüft `tools/ota_release.sh` das fertige Abbild, bevor irgendetwas ausgespielt wird (`:106–125`).

Das Verfahren ist bewusst schlicht: Aus `RC_config.h` werden **alle** Zeichenkettenwerte gezogen und im Abbild gesucht.

```
# vereinfacht
sed -n 's/^[[:space:]]*#define[[:space:]]\+[A-Z_0-9]\+[[:space:]]\+"\([^"]*\)".*/\1/p' RC_config.h
  → für jeden Wert: grep -qF "wert" firmware.bin firmware.factory.bin
```

Regel	Grund
Werte unter 6 Zeichen werden übersprungen	sonst Zufallstreffer
Geprüft werden **beide** Abbilder	OTA-Datei *und* Browser-Datei
Ein Fund = **Abbruch**, nichts wird ausgespielt	„vermutlich wurde das Dev-Abbild gebaut"

Bei Erfolg meldet das Skript:

```
✓ Abbild ist frei von Zugangsdaten
```

> **Die Prüfung sucht nicht nach dem, was das Abbild enthalten *sollte*, sondern nach dem, was es nicht enthalten *darf*** — und sie prüft es am fertigen Erzeugnis, nicht an der Absicht, die zu ihm geführt hat.

Das ist der entscheidende Unterschied zu einer Konfigurationsprüfung: Ein falsch gewähltes Bauziel, ein vergessenes Flag oder ein Zwischenstand aus einem früheren Lauf werden sämtlich erfasst, weil am Ergebnis gemessen wird.

25.4 Wo Geheimnisse sonst noch austreten

Zwei Wege außerhalb des Abbilds sind bekannt und zu beachten:

Weg	Fund
Startprotokoll	Eine Protokollzeile in `WZ_WiFi` gab das WLAN-Kennwort im Klartext aus (`Connecting to <SSID> <PWD>`)
Bildschirmfotos	Der MQTT-Detailschirm zeigt die private Broker-Adresse aus der Konfiguration — eine Falle für Handbuch-Abbildungen

⚠️ Vor einer Veröffentlichung von Handbuch oder Technikbuch sind deshalb Bildschirmfotos auf echte Zugangsdaten zu prüfen und WLAN-Namen aus Startprotokollen zu schwärzen.

⚠️ Unabhängig davon liegen in der Versionsgeschichte beider Fernablagen noch Zugangsdaten aus einem früheren Stand. Sie sind dort nicht mehr zu entfernen, ohne die Geschichte umzuschreiben — ein Wechsel dieser Zugangsdaten ist die einzige wirksame Maßnahme.

25.5 Die Regel

> **Ein Abbild mit Zugangsdaten verlässt niemals das Gerät, auf dem es gebaut wurde.**

Praktisch heißt das:

1. Ausgespielt wird ausschließlich über `tools/ota_release.sh` — nie eine von Hand kopierte `.bin`.
2. Die Erfolgsmeldung `✓ Abbild ist frei von Zugangsdaten` ist Teil des Ablaufs, nicht Zierrat. Bleibt sie aus, wird nichts hochgeladen.
3. Neue Geheimnisse gehören in die Konfigurationsdatei — dann greift die Prüfung von selbst, ohne dass jemand die Liste in 25.1 pflegen muss.

26 Der Emulator — und wo er *nicht* als Nachweis taugt

Ein Flash-Zyklus dauert rund eine Minute, eine Sichtprüfung am 240 × 240 großen Schirm kostet Aufmerksamkeit, und für jede Layout-Frage die Uhr in die Hand zu nehmen, bremst. Deshalb gibt es einen nativen Emulator, der die eigenen Schirme am Rechner darstellt.

Der zweite Teil der Kapitelüberschrift ist der wichtigere.

26.1 Was er ist

Ein SDL2-Programm für macOS, das **die echte LVGL-Quelle der Firmware** zusammen mit **den echten `CR_`-Modulen** übersetzt — `CR_EEZShim`, `CR_Watchface`, `CR_MsgScreen`, `CR_ScreenMgr`, `CR_ScreenHeader`, `CR_Stations`, `CR_StationScreen`, `CR_RadarScreen`, `CR_QuickReply`.

Die Darstellung ist damit **1 : 1** die der Uhr (240 × 240, im Fenster mit doppelter Vergrößerung). Es handelt sich nicht um einen Nachbau der Oberfläche, sondern um dieselbe Oberfläche in anderer Umgebung.

Befehl	Wirkung
`make`	baut `build/emulator`
`make run`	öffnet das Fenster
`make app`	baut ein doppelklickbares `Emulator.app`
`make smoke`	kopfloser Test: 100 Bilder rendern
`make clean`	räumt `build/` weg

Voraussetzung ist SDL2 (`brew install sdl2`) und ein einmal gelaufener Firmware-Bau — er erzeugt die LVGL-Quelle, gegen die der Emulator übersetzt.

⚠️ **Wer `clean` sagt, muss auch `app` sagen:** Die Verknüpfung auf dem Schreibtisch zeigt auf `build/Emulator.app`, und `clean` löscht das Bundle mit. `make run` allein hilft nicht — das baut die nackte Binärdatei.

⚠️ Der Emulator liegt in einem **eigenen Verzeichnis**; `SMashCom42/` bleibt vollständig unberührt. Er ist ausschließlich örtliches Arbeitsmittel und niemals Bestandteil eines Beitrags an Wolfgangs Zweig (Kapitel 2.1).

26.2 ⚠️ Wo er als Nachweis versagt

Dies ist der Grund, warum das Kapitel im Buch steht.

> **Eine grüne Zeile im Emulator heißt nicht, dass eine Meldung angekommen ist.** Sie heißt nur, dass die Anzeigelogik den Zustandswechsel richtig verarbeitet. Ob wirklich gesendet wurde, sagt ausschließlich die echte Uhr.

Der Emulator kennt keine Funkstrecke, keinen Akku, keinen I²C-Bus und keinen zweiten Kern. Er kann daher folgende Fragen **nicht** beantworten:

Frage	Warum nicht
Kommt eine Aussendung im Netz an?	keine Funkschicht — Kapitel 19
Wie viel Strom kostet eine Funktion?	kein Verbrauchsmodell — Kapitel 24
Reagiert die Bedienung flüssig?	anderer Takt, andere Abtastrate — Kapitel 16
Antwortet ein Bauteil?	keine Hardware — Kapitel 5, 8, 10
Ruckelt der Sekundenzeiger?	anderes Rendermodell — siehe 26.3

Positiv gewendet, und darin liegt sein Wert: Er beantwortet **Layoutfragen** verlässlich — Anordnung, Textlängen, Umbrüche, Sichtbarkeit, Bildschirmwechsel. Genau dafür ist er gebaut.

26.3 ⚠️ Ein Unterschied, der schweigend zuschlägt

Der Emulator rendert **einfädig** (`emulator/lv_conf.h:124–129`):

```
#define LV_USE_OS   LV_OS_NONE     // NICHT LV_OS_PTHREAD
```

Der Grund ist kein Vereinfachungswunsch, sondern eine reproduzierte Verklemmung: Der pthread-Render-Thread verkeilt sich mit dem synchronen Canvas-Layer-Aufruf in `CR_Watchface_Init()` — der Haupt-Thread hängt in `lv_draw_add_task`, der Render-Thread wartet in `lv_thread_sync_wait`. Per Stapelprobe belegt.

Die Firmware dagegen rendert mit `LV_OS_FREERTOS` und eigener Sperrdisziplin (Kapitel 12). **Die beiden Umgebungen unterscheiden sich also genau in dem Punkt, der auf der Uhr die meisten Schwierigkeiten gemacht hat.**

⚠️ Besonders tückisch: Der kopflose Test (`make smoke`) blieb bei der Verklemmung **grün**. Ein bestandener Selbsttest belegt hier also nicht, dass die Anzeige läuft.

> **Ein Emulator, der an einer Stelle bewusst anders gebaut ist als das Zielsystem, kann genau dort nichts beweisen** — und diese Stelle ist bei ihm ausgerechnet die Nebenläufigkeit.

26.4 Auch die Bedienung ist nachgebildet, nicht gleich

Die Wischerkennung läuft auf **SDL-Ebene** (Mausbewegung zwischen Drücken und Loslassen), nicht über LVGL-Gesten. Der Grund ist derselbe wie auf der Uhr: Scrollbare Flächen verschlucken Gesten, bevor sie ankommen.

Das ist sinngemäß dasselbe Vorgehen wie bei den Chip-Gesten des FT6336U (Kapitel 8) — aber eben nur sinngemäß. Wie sich eine Geste **auf der Uhr** anfühlt, ist im Emulator nicht zu messen; diese Frage hängt an der Abtastrate und gehört ans Gerät (Kapitel 16).

Mock-Meldungen entstehen auf Tastendruck (`M`), nicht durch einen Zeitgeber — es gibt keine simulierte Funkaktivität, die versehentlich für echte gehalten werden könnte.

26.4b Aufnahmen ohne Fenster

Für Layoutfragen genügt oft ein Bild. `build/emulator --snap-list <datei.ppm> [n]` legt n Mock-Meldungen an und nimmt die Meldungsliste auf, `--snap-funk <datei.ppm>` die Funkraumuhr mit der aktuellen Ortszeit, `--snap-funk-q` dieselbe in der quadratischen Form, `--snap-compose <datei.ppm> [Ziel] [Text]` den Schreibschirm mit vorbelegtem Entwurf und `--snap-compose-ziele` denselben mit aufgeklappter Zielliste (Gruppenplätze wie auf Christians Uhr vorbelegt). Diese Aufrufe setzen selbst `SDL_VIDEODRIVER=dummy`: SDL rendert in Software, **es öffnet sich kein Fenster** — wer am Mac arbeitet, muss nichts wegklicken. Eine andere Uhrzeit liefert `TZ`, etwa `TZ=UTC-5:30` (eine halbe Stunde Versatz schiebt den Minutenzeiger aus einem Bereich, den man vermessen will). Umwandeln: `sips -s format png datei.ppm --out datei.png`.

26.5 Die Regel

> **Der Emulator beantwortet Layoutfragen und spart Flash-Zyklen. Jede Aussage über Verhalten, Zeit, Strom oder Funk gehört ans Gerät.**

Im Zweifel gilt die Reihenfolge aus Anhang C.5: Was nicht am Gerät gemessen wurde, ist eine Annahme — und wird in diesem Projekt auch so bezeichnet.

27 Die eingebauten Messpunkte

Dieses Buch behauptet an vielen Stellen Zahlen. Dieses Kapitel sagt, womit sie gemessen wurden — damit ein Nachfolger sie nachprüfen kann, statt sie zu glauben.

Die Messpunkte sind ausdrücklich **kein Debug-Rest**, der beim Aufräumen entfernt werden dürfte. Jeder einzelne ist entstanden, weil eine Annahme sich als falsch erwiesen hat, und jeder ist als **Wachposten** gegen die Rückkehr genau dieses Fehlers gedacht.

27.1 `[LOOPHZ]` — der Takt des Main-Loops

```
[LOOPHZ] 240 Hz
```

Zählt die `loop()`-Durchläufe im letzten Sekundenfenster — die Ausgabe steht in `main.cpp` bei `s_loopIterCount` (zum Redaktionsschluss Zeile 964).

⚠️ Diese Zeile ist bewusst **nicht** an einen Debug-Level gebunden, sondern läuft unconditional im normalen Betriebsprotokoll mit. Der Grund: Sie ist der zentrale Nachweis für das Abnahmekriterium D-06 (≥ 10 Hz), und sie soll ohne Umschalten sichtbar sein.

Genau diese Entscheidung hat sich bezahlt gemacht — der Sprung von 235 auf 980 Hz beim Betreten des Update-Schirms (Kapitel 15.4) fiel nur deshalb auf.

Beobachtung	Bedeutung
~200–240 Hz	Normalzustand mit Taktbremse
~1000 Hz und mehr	Bremse greift nicht — Render-Aussetzer zu erwarten
einstellig	schwerer Fehler, siehe Kapitel 14

27.2 `[CORE]` — die tatsächliche Kernzuordnung

```
[CORE] main=0 gui=1
```

Einmalig in der ersten Sekunde jedes Starts (`main.cpp:943–953`). Der Kommentar dort nennt den Zweck ungeschminkt:

> Die Zeile ist kein Debug-Rest, sondern eine Wache: Der 1-Hz-Einbruch entstand genau dadurch, dass Main-Task und Render-Task unbemerkt auf **demselben** Kern lagen, während die gesamte Projektdokumentation von getrennten Kernen ausging.

Erwartet wird `main=0 gui=1`. Stehen dort zwei gleiche Zahlen, ist Kapitel 11 zu lesen — und mit hoher Wahrscheinlichkeit auch Kapitel 14.

Es sind drei Zeilen Code. Hätte es sie früher gegeben, wäre eine monatelang falsche Dokumentation nie entstanden.

27.3 `GUI_LAG` — Aussetzer des Render-Tasks

```
[GUI_LAG] #7 gap=180ms prevRender=41ms max=210ms
```

Misst den Start-zu-Start-Abstand zwischen zwei Iterationen des Render-Tasks und zählt Ausreißer über einer Schwelle (`CR_GUI.h:30–48`, gemeldet in `main.cpp:926–935`).

Zwei Einstellungen, beide messwertbelegt:

Einstellung	Wert	Begründung
`CR_GUI_LAG_STATS`	1	abschaltbar für Gegenproben und den Produktivbetrieb
`CR_GUI_LAG_THRESHOLD_MS`	**120**	rund das 1,5-fache des gemessenen typischen Abstands (Median 81 ms bei n=40, unabhängige Gegenmessung 90 ms bei n=27)

⚠️ Die Schwelle stand vorher auf 60 ms und lag damit **unter** der Normalkadenz — sie zählte praktisch jede zweite Iteration fälschlich als Ausreißer.

> **Ein Schwellwert, der im Normalbetrieb dauernd auslöst, ist als Anomalie-Detektor wertlos.** Er ist an der gemessenen Normalkadenz zu kalibrieren, nicht an einer Wunschvorstellung von ihr.

⚠️ Der Messpunkt ist ehrlich in dem, was er **nicht** leistet: Die Kalibrierung hat die Renderleistung nicht verbessert. Die Sweep-Tick-Rate liegt weiterhin bei ~17–18/s statt der angestrebten 30/s — das ist eine offene Frage, kein gelöstes Problem.

27.4 `[SLGUARD]` — der Schiebereglerschutz

```
[SLGUARD] senkrechter Wisch: 14 Abtastungen festgehalten, Wert zurueck auf 40
```

Ausgegeben in `CR_SliderGuard.cpp:128`. Der Kommentar darüber trifft den Kern des ganzen Kapitels (`:112`):

> Die `[SLGUARD]`-Zeile ist **Messung, keine Krücke**.

Sie beantwortet die bis dahin ungemessene Frage, wie viele Abtastungen ein Schieberegler während eines Wischens tatsächlich verschluckt — eine Größe, die unmittelbar am Loop-Takt hängt (Kapitel 16).

27.5 Das Akku-Logbuch

Der einzige Messpunkt, der **über einen Neustart hinweg** erhalten bleibt: Die Uhr schreibt alle fünf Minuten selbst mit, ins NVS (`CR_BattLog.h`).

Eigenschaft	Wert
Messabstand	5 min
Plätze	180 → **15 Stunden** Aufzeichnung
Schreibabstand ins NVS	15 min
unter 15 % Ladung	jeder Messwert sofort gesichert
Format-Kennung	`CR_BATTLOG_FORMAT 2`

Je Eintrag stehen Spannung, Ladestand und ein Flag-Byte: Laden, USB, WLAN, Schirm an, Zeitstempel belastbar, GPS, LoRa und **GPS-Fix**.

⚠️ Das Fix-Bit (`0x80`) ist **das letzte freie Bit**. Wer ein weiteres Merkmal aufnehmen will, vergrößert damit den Eintrag — und der liegt im NVS, dessen Platz begrenzt ist.

Warum das Fix-Bit überhaupt nötig war, ist ein Lehrstück für sich: Ohne es ist ein Sendezähler von 0 **nicht deutbar** — „Bake defekt" und „nie einen Fix gehabt" sehen identisch aus. Dieselbe Lücke bestand zuvor beim Empfangszähler und führte zu einer Verbrauchszahl, die für den Alltag nicht galt (Kapitel 24).

Ausgelesen und ausgewertet wird mit:

```
~/.platformio/penv/bin/python tools/battlog.py --lesen
```

Das Werkzeug trennt Kabel- von Akkubetrieb, stellt neben jede Verbrauchszahl die Empfänge je Stunde und den Fix-Anteil — und spricht die Warnungen selbst aus („kein einziger Empfang — die Zahl gilt für Funkstille"). ⚠️ Prozentwerte werden bewusst **nie** hochgerechnet; warum, steht in Kapitel 24.

⚠️ Das serielle Auslesen startet die Uhr neu. Das ist normal und für das Logbuch sogar nötig — es wird nur beim Start ausgegeben.

27.6 Der LoRa-Selbsttest

`CR_LoRaCtl.cpp:117` fährt nach dem Hochlaufen einen vollständigen Aus-/Einschaltzyklus des Funkmoduls und protokolliert ihn:

```
[LORATEST] --- Runde 1/1: ABSCHALTEN (on_LORA=1) ---
[LORATEST] --- Runde 1: EINSCHALTEN (on_LORA=0) ---
[LORATEST] === Runde 1: on_LORA=1 -> OK ===
```

Er prüft damit genau die Eigenschaft aus Kapitel 6.4: dass Wiedereinschalten **nicht** das Gegenteil von Ausschalten ist, sondern die vollständige Startroutine erfordert — samt Sync-Wort und Präambel, ohne die die Uhr im selben Netz taub wäre.

27.7 `[LoRa TXHDR]` — der Paketkopf im Klartext

```
[LoRa TXHDR] 3A BD ... (42 Bytes gesamt, Text)
```

Gibt den Kopf jeder eigenen Aussendung hexadezimal aus (`WZ_LoRa.cpp:356` , `:470`). Diese Ausgabe war die **Voraussetzung** dafür, den MeshCom-Paketaufbau überhaupt aus Messdaten abzuleiten (Kapitel 18).

Sie ist zugleich das Mittel, mit dem sich die Hardware-Kennung am Netz gegenprüfen ließe: `3D` an Position 2 und `BD` im Schwanz. ⚠️ Diese Gegenprobe **steht noch aus** — sie erfordert eine Aussendung, und Aussendungen löst ausschließlich der Lizenzinhaber aus.

27.8 Die Startprotokoll-Wachposten

Kein eigenes Werkzeug, sondern ein durchgehendes Muster: **Jede Zustandsänderung an der Hardware wird zurückgelesen und protokolliert**, statt als erfolgt angenommen zu werden.

```
[PWR] ALDO4 (LoRa SX1262): eingeschaltet        ← oder FEHLER
[MODTOGGLE] GPS aus, BLDO1 abgeschaltet         ← oder NOCH AN (Fehler)
[CRTOUCH] Gesture Mode aktiv. Auflösung 0x98–0x9B = 00 F0 00 F0 (Soll 00 F0 00 F0)
[LoRa] SPI: SCK=3 MISO=4 MOSI=1 CS=5
[SCRMGR] 20/20 Screens registriert
```

> **Ein Befehl, der nicht ankommt, meldet sich damit selbst** — statt sich als unerklärliches Folgeproblem zu tarnen: als Treiberfehler (Kapitel 5), als zu kurze Akkulaufzeit (Kapitel 6) oder als tote Achse (Kapitel 8).

27.9 Die Regel hinter allen Messpunkten

> **Wo etwas nicht gemessen wurde, steht das in diesem Projekt ausdrücklich dabei.**

Der Grund ist Anhang C.5: Die Sammlung der Sätze, die über Monate in der Dokumentation standen, plausibel klangen und falsch waren. Keiner von ihnen war unplausibel — keiner war gemessen.

Deshalb gilt für jeden neuen Messpunkt: Er wird eingebaut, **bevor** die Frage beantwortet wird, nicht danach. Und er bleibt drin.

28 Warum der GPS-Fix Minuten dauerte — und was die Uhr dem Empfänger seit 1.0.3.104 mitgibt

Das GNSS-Modul der T-Watch-S3-Plus ist ein **u-blox MIA-M10Q**. Sein Datenblatt verspricht einen Kaltstart in 28 s; am Handgelenk dauerte der erste Fix regelmäßig mehrere Minuten, obwohl das Satelliten-Balkendiagramm (Kap. 12b) längst Satelliten zeigte. Dieses Kapitel erklärt, warum beides zugleich wahr ist, und was `CR_GpsAid` dagegen tut.

28.1 Die 28 Sekunden sind ein Laborwert

Data Sheet UBX-22015849 R08, Tabelle 2: Kaltstart GPS+Galileo 28 s, GPS+Galileo+BeiDou 27 s — gemessen im **Simulator, alle Satelliten bei –130 dBm, Raumtemperatur, per Befehl ausgelöst**. Zwei Zeilen weiter stehen die Empfindlichkeiten: Kaltstart **–148 dBm**, Verfolgen **–167 dBm**. Zwischen beiden liegen 19 dB. Ein Satellit, den das Modul verfolgt, ist noch lange nicht stark genug, um seine **Ephemeriden** (die Bahndaten, 18–36 s Sendezeit je Satellit) fehlerfrei zu liefern. Genau das ist das Bild „Satelliten sichtbar, kein Fix": Das Modul hört sie, kann aber nicht lesen, wo sie sind. Integration Manual UBX-21028173 R05, 3.12: unter schlechten Signalbedingungen „may take several minutes or even completely fail".

Ein Empfänger braucht für den Fix vier Dinge: grobe **Zeit**, grobe **Position**, den **Almanach** (grobe Bahnen aller Satelliten, gültig Wochen) und die **Ephemeriden** (genaue Bahnen der gerade sichtbaren, gültig ~4 h). Was er davon noch hat, bestimmt die Startart:

Start	Zeit	Position/Almanach	Ephemeriden	TTFF (Datenblatt)
Kalt	—	—	—	28 s (Labor), Minuten (Handgelenk)
Warm	grob	ja	müssen neu gelesen werden	~30 s, am Handgelenk Minuten
Heiß	genau	ja	noch gültig (< 4 h)	1 s
AssistNow Autonomous	grob	ja	vorausberechnet (3–6 Tage)	3–4 s

28.2 Die Platine: Backup-Zelle ja, Quarz nein

Der Schaltplan der GPS-Zusatzplatine (`doku/hardware/T-Watch-S3-Plus-GPS V1.0`) zeigt zwei Dinge:

- `V_BCKP` (Pin J5) hängt am Netz `VRTC` , gespeist von einer **MS412FE** — einer wiederaufladbaren Seiko-Zelle (3 V, 1 mAh), geladen über 1N4148 + 1 kΩ aus `VDD3V3` , also aus der GPS-Schiene **BLDO1** (Kap. 5). Solange die Zelle Spannung hat, überlebt der batteriegepufferte Speicher (BBR: Almanach, Ephemeriden, Konfiguration) das Abschalten der Schiene. Die Zelle lädt **nur, solange BLDO1 an ist**.
- `RTC_I` ist offen, `RTC_O` liegt auf GND. Das ist wörtlich die Beschaltung aus Integration Manual 4.2.3 **„RTC not used"**: Das Modul hat keine eigene Uhr. Fällt die Schiene, ist die Zeit weg — und ohne Zeit kann das Modul seine gespeicherten Ephemeriden nicht zuordnen. Jeder Neustart ist damit bestenfalls ein **Warmstart**, nie ein Heißstart.

Und wir schalten die Schiene oft ab: die Kachel (Kap. 5), die Akku-Schutzstufe 2 (Kap. 24) und `CR_Power` nach 10 min ohne Fix (`CR_POWER_FIXSEARCH_MS`) mit 30 min Pause (`CR_POWER_FIXPAUSE_MS`). Bis 1.0.3.103 folgte auf jedes Wiedereinschalten das Neu-Lesen der Ephemeriden aus dem Himmel.

Das Integration Manual nennt für genau diese Beschaltung das Rezept (4.2.3): *„Time information can be sent to the receiver at every startup. Coarse time information (accuracy of the order of seconds) is sufficient for a warm start and to use AssistNow data."*

28.3 Was `CR_GpsAid` tut

`CR_GpsAid.cpp/.h` ist ein additives CR_-Modul (Kap. 2): `WZ_GPS.cpp` bleibt Besitzer der UART und reicht nur einen Schreib-Zeiger (`CR_GpsAid_Bind`) und jedes empfangene Byte (`CR_GpsAid_FeedByte`) herein. Am Ende von `SetupUBLOX()` ruft es `CR_GpsAid_OnModuleReady()` , das in dieser Reihenfolge sendet:

1. **`UBX-CFG-VALSET`** (Ebenen RAM+BBR): `CFG-ANA-USE_ANA = 1` — AssistNow Autonomous, ab Werk **aus**. Das Modul rechnet aus einmal empfangenen Ephemeriden eine Bahn-Vorhersage für ~3 Tage und legt sie im BBR ab; die Zelle hält sie. Dazu `CFG-NAVSPG-ACKAIDING = 1` , sonst quittiert das Modul Hilfsdaten nicht (`UBX-MGA-ACK-DATA0`).
2. **`UBX-MGA-INI-TIME_UTC`** aus der Systemzeit der Uhr (PCF8563-RTC beim Boot, NTP, Telefon; Kap. 22), nur wenn das Jahr ≥ 2025 ist. Die Genauigkeit wird **ehrlich** angegeben — 3 s mit NTP, 10 s ohne —, denn das Manual warnt: zu optimistisch angegebene Zeit verschlechtert den Start. Hat das Modul bereits Zeit, ignoriert es die Hilfe; senden schadet also nie.
3. **`UBX-MGA-INI-POS_LLH`** aus `CR_GpsFix_GetLastPosition()` (Kap. 19.8), mit 10 km Unsicherheit bei ≤ 1 h Alter, sonst 100 km. Auch hier gilt: eine Position, die um mehr als die angegebene Unsicherheit falsch ist, schadet — deshalb großzügig.

Bytelayouts und Schlüssel stammen aus der u-blox M10 SPG 5.10 Interface Description (UBX-21035062): MGA-INI-TIME_UTC 3.13.9.3, MGA-INI-POS_LLH 3.13.9.2, CFG-VALSET 3.10.5, Prüfsumme 3.4 (8-Bit-Fletcher über Klasse, ID, Länge, Nutzlast). Als Referenzcode diente die SparkFun-Bibliothek u-blox GNSS v3 (`setUTCTimeAssistance()` , `setAopCfg()`) — sie selbst ist für den knappen internen RAM zu schwer, daher handgebaute Rahmen und ein 32-Byte-Parser.

28.4 Messen statt glauben: die TTFF-Zeile

Bis zum Fix fragt `CR_GpsAid_Loop()` alle 2 s `UBX-NAV-STATUS` ab (höchstens 20 min). Meldet das Modul einen Fix, steht im Log:

```
[GPSAID] Fix (3D) nach 4 s seit Start - Modul-TTFF 3810 ms, Modul laeuft seit 6 s
[GPSAID] AssistNow Autonomous: AN, Orbitrechner ruht (Abschalten unbedenklich)
[GPSAID] erster gueltiger NMEA-Fix nach 5 s seit Start
```

`Modul-TTFF` ist das Feld `ttff` aus NAV-STATUS (Interface Description 3.15.17), die zweite Zahl unsere eigene Uhr bis zum ersten gültigen RMC-Satz. `--gps` zeigt beide Werte des letzten Fixes. Quittungen werden ebenfalls geloggt — `ACK` / `NAK` für CFG-VALSET, die MGA-Quittungen mit Klartext („Empfänger kennt die Zeit nicht" hieße: Zeit-Aiding kam nicht an). Als Nebenertrag wurde am 27.08.2026 erstmals die Antwort auf die alten Legacy-Befehle aus `SetupUBLOX()` sichtbar: `UBX-CFG-RATE` bekommt noch ein ACK, **`UBX-CFG-CFG` ein NAK** — das „Speichern in Flash und BBR" hatte also nie stattgefunden. Der Aufruf wurde entfernt; die Persistenz übernimmt seither die BBR-Ebene von `CFG-VALSET`, gehalten von der Backup-Zelle. Erster Bootlog von 1.0.3.104:

```
[GPSAID] CFG-VALSET gesendet: AssistNow Autonomous an, MGA-Quittungen an (RAM+BBR)
[GPSAID] Zeit-Aiding gesendet: 2026-08-27 14:48:28 UTC (+-3 s)
[GPSAID] Positions-Aiding gesendet: 47.93942 / 16.13960, 314 m, +-100 km (Alter > 1 h)
[GPSAID] ACK fuer CFG-RATE (Legacy)
[GPSAID] ACK fuer CFG-VALSET
[GPSAID] MGA-Quittung fuer Zeit-Aiding: angenommen
[GPSAID] MGA-Quittung fuer Positions-Aiding: angenommen
```

Messrezept: Fix holen, Schiene abschalten (Kachel), eine Minute warten, wieder einschalten. Sekunden = Backup-Zelle und Zeit-Aiding arbeiten. Minuten = Zelle leer (lädt nur bei laufender Schiene) oder BBR verloren. Nach 1–2 Tragetagen mit Fix sollte AssistNow Autonomous greifen — dann auch nach Stunden ohne Schiene ein Fix in wenigen Sekunden.

Fix-Logbuch (1.0.4.90): Weil der USB-Ring die `[GPSAID]`-Zeilen nach Minuten ohne Kabel verliert (Spaziergang 06.09.: 21 598 Zeilen verworfen, keine Fixzeit mehr lesbar), merkt sich `CR_GpsAid` die letzten zehn Modulstarts in `RTC_NOINIT_ATTR` (überlebt Neustart und Absturz, nicht stromlos): Uhrzeit, Sekunden bis zum ersten NMEA-Fix, Modul-TTFF, ob Bahndaten eingespeist waren, oder „kein Fix, Modul n s an". Ausgabe mit `--gps` ohne Argument, neueste zuerst.

28.5 Die Messung vom 27.08.2026 (Uhr am Kabel vor dem Haus, JN87ct)

Schritt	Ergebnis
Signal am Messplatz (GSV, 90 s)	24–26 Satelliten gehört, Spitzen 27–32 dBHz, nur 4 über 28 dBHz — Grenzbereich zum Dekodieren
Neustart 18:22:23 mit Zeit-/Positions-Aiding, **ohne** Bahndaten im BBR	erster Fix 18:25:11 (5 Satelliten, HDOP 3,5): **~2–3 min**
Schiene stromlos (Kachel), 20 s später wieder an	`[GPSAID] Fix (3D) nach 16 s - Modul-TTFF 13174 ms` → **13 s**
danach	`AssistNow Autonomous: AN, Orbitrechner ruht`

Der erste Fix am Grenzsignal bleibt Minutenarbeit — das ist Physik (Kap. 28.1). Aber jeder weitere Start mit gefüllter Backup-Zelle und Zeit-Aiding kommt in Sekunden, wo die Uhr vor 1.0.3.104 wieder von vorn anfing. Nebenbefund derselben Messung: ein manuelles `--gps off` / `--gps on` innerhalb von 1,6 s fiel zwischen zwei 30-s-Raster der Eco-Logik, die Fix-Uhr behielt ihren Stand und ECOFIX kappte die Schiene 38 s nach dem Einschalten — seit 1.0.3.105 beginnt die Fix-Uhr bei jeder Flanke `on_GPS` aus→an neu (`CR_Power.cpp`).

Stört sich die Uhr selbst? (A/B-Test 18:43–18:46, `tools/gsv_snr.py`): 75 s mit WLAN, dann `--wlan off` (Funkchip wirklich aus, seit 1.0.3.106 als Konsolenbefehl nach dem Kachel-Muster) und 90 s ohne. Neun Satelliten in beiden Fenstern: mittlere Differenz **–0,2 dB**, Median +0,9 dB, Spitzen 29–30 dBHz in beiden Fällen. Das WLAN der Uhr hat keinen messbaren Einfluss auf den GNSS-Empfang; das Grenzsignal ist Standort und Antennenlage, nicht Eigenstörung. Messrezept für andere Verdächtige (Display, LoRa): `--setGPSDEBUG 2`, zwei Mitschnitte, `gsv_snr.py`.

28.6 Was bewusst NICHT gemacht wurde

- **Kein `UBX-CFG-RST`** — jeder Reset, der das BBR löscht, macht aus dem nächsten Start einen Kaltstart (IM Tab. 23).
- **Keine Power-Save-Modi** (PSMOO/PSMCT) während der Suche: „The receiver is not able to download satellite data (e.g. the ephemeris) while it is working in on/off or cyclic tracking operation" (IM 3.6.2.7).
- **Kein Dynamikmodell „WRIST"** — laut Interface Description „not available in all products"; ein unbekannter Schlüssel ließe die ganze VALSET-Nachricht scheitern.
- **Kein Live/Predictive Orbits** (die Nachfolger von AssistNow Online/Offline): bräuchten Internet unterwegs bzw. einen Dienst-Schlüssel; Autonomous leistet das Wesentliche ohne beides.

Die Zeit-Aiding-Genauigkeit ließe sich mit einem Puls auf `EXTINT` in den Mikrosekundenbereich bringen (IM 3.10.2) — der Pin ist auf der Platine nicht herausgeführt.

28.7 Bahnvorhersagen per WLAN: AssistNow Predictive Orbits (1.0.4.83–.88)

Der Aiding-Weg aus 28.3 gibt dem Empfänger Zeit und Position, aber keine Bahnen. Die kommen seit 1.0.4.88 vom u-blox-Dienst **AssistNow Predictive Orbits** (Nachfolger von AssistNow Offline, das seit 31.05.2026 außer Support ist und keine Tokens mehr ausgibt).

Ablauf (`CR_GpsAno.cpp` , Main-Task, ein Schritt je Durchlauf):

1. Ohne Profil-Token (`--setANOTOKEN` , NVS `anoTok` , XOR-verschleiert wie das OTA-Kennwort) ist das Modul stumm. GPS läuft wie ohne das Modul — Grundsatz Christian 06.09.
2. Ohne Chipcode: sobald das Modul läuft, `CR_GpsAid_PollIdent()` (UBX-SEC-UNIQID + UBX-MON-VER, beide als Hex-Rahmen inkl. Sync und Prüfsumme gemerkt), dann `POST api.thingstream.io/ztp/assistnow/credentials` mit `{"token","messages":{"UBX-SEC-UNIQID","UBX-MON-VER"}}` → `chipcode` (28 Zeichen, NVS `anoChip`). Im Portal entsteht dabei ein Thing mit dem Namen der Chip-ID. Der Nutzer tippt die Chip-ID nirgends ein — die Uhr liest sie per UBX-SEC-UNIQID (6 Byte, v2 beim M10) und schickt den rohen Rahmen; das Portal braucht nur den Profil-Token (Anleitung Klick für Klick im Handbuch, Kapitel Module → „Schneller GPS-Fix mit Bahndaten”).
3. Mit Chipcode: fehlen Sätze für den heutigen UTC-Tag oder sind sie älter als 24 h → `GET assistnow.services.u-blox.com/GetAssistNowData.ashx?chipcode=…&data=uporb_7&gnss=gps,glo` (7 Tage GPS+GLONASS, 29 988 Byte, 357 UBX-MGA-ANO-Rahmen à 84 Byte, 51 je Tag) ins PSRAM (`CR_GpsAid_AnoSetRaw`). Läuft das Modul, wird sofort eingespeist.
4. Einspeisen (`CR_GpsAid`): nach Zeit- und Positions-Aiding die Sätze des UTC-Tages, einer je Loop-Durchlauf, mit Warten auf die MGA-Quittung (CFG-NAVSPG-ACKAIDING ist an; Frist 250 ms). Gemessen: 51 Sätze in 1,8 s, alle angenommen.

HTTPS: derselbe tls_mini/BearSSL-Weg wie die OTA-Versionsprüfung, verschlüsselt ohne Ankerprüfung (die stürzt in tls_mini sporadisch ab, CR_OTA.cpp). Zwei Fallstricke, beide am 06.09. gemessen: (a) der TLS-Eingangspuffer muss **16 709 Byte** fassen — mit dem OTA-Wert 8 192 brach der 30-KB-Abruf mit HTTP –11 und 0 Byte ab, weil der Dienst volle 16-KB-Sätze schickt; (b) `setReuse(false)` plus Auswerten von Content-Length, sonst wartet das Lesen bis zur 15-s-Frist (ZTP dauerte 16,5 s statt ~2 s). Der Abruf blockiert den Hauptlauf ~2 s (Abruf) bzw. ~2 s (ZTP); Fristen: 90 s nach Boot, 24 h nach Erfolg, 60 min nach Fehler, nach Neustart frühestens 60 min nach dem letzten Abruf (NVS `anoTs` , Kontingent Evaluation 300/Monat). Speicherwache: größter freier interner Block ≥ 20 KB, sonst 5 min vertagt.

Messung 06.09.2026, draußen, 3–4 GPS-Satelliten mit 21–35 dBHz, echter Kaltstart per `--gps kalt` (CFG-RST mit BBR-Löschung — ein Abschalten der Schiene reicht wegen der Pufferzelle NICHT): mit Bahndaten erster Fix nach **54 s**, ohne nach **147 s** (Modul-TTFF 123,6 s). Am Fensterbrett vorher 0–1 Satellit ohne Pegel: kein Fix in 5 min, mit oder ohne — Bahndaten helfen erst, wenn Signale da sind.

Flash-Ablage (1.0.4.89): Partition `coredump` (64 KB) als Zwischenspeicher — Kopf 16 Byte (Magic „ANO1”, Länge, Abrufzeit, CRC32), dahinter der rohe Strom. Für Meldungen war diese Partition der K.o. (ein Absturz-Abzug überschreibt sie, E-23), für einen Zwischenspeicher ist das egal: Prüfung schlägt fehl, Abruf folgt. Nichts im Projekt liest Abzüge (CR_Crash nutzt RTC_NOINIT + NVS); der Core prüft beim Boot nur lesend und meldet unsere Daten als „Incorrect size of core dump image” (eine Zeile, harmlos). Gemessen: 29 988 Byte in 576 ms geschrieben, nach Neustart übernommen und beim Modulstart eingespeist, kein neuer Abruf. Alter = Abruf-Zeitstempel im Kopf/NVS, nicht die Ladezeit. **Offen:** Ankerprüfung, sobald tls_mini stabil ist. Werkzeuge: `tools/assistnow_ztp.py` (Registrierung, Abruf und Übertragung vom Mac aus, `--gpsano b64`), `--gps ident` , `--gps kalt` , `--gpsano holen|ztp|feed` .

28.8 Der interne Vorverstärker: drei Modi, ein Zähler zum Messen (1.0.4.112)

Frage Christian (12.09.): Draußen beim Farbtest der Satellitenbalken „nur ganz selten gelb, grün überhaupt nicht — kann der Chip intern einen Vorverstärker setzen?”

Befund im Integration Manual (UBX-21028173 R05, 2.1.2 und 4.4.1): Der MIA-M10Q hat vor dem SAW-Filter einen internen LNA mit drei Betriebsarten: **normal gain (0)**, **low gain (1, Werk)** und **bypass (2)** für Designs mit aktiver Antenne. Die Betriebsart steht im Konfigurationsschlüssel `CFG-HW-RF_LNA_MODE` (0x20A30057) und lässt sich zur Laufzeit in RAM und BBR schreiben; sie wirkt erst nach einem Reset des Empfängers. u-blox schreibt dazu nur einen Satz: „The normal gain mode is not recommended for MIA-M10Q” — ohne Begründung. Für eine Uhr mit Chipantenne am Handgelenk ist die naheliegende Vermutung Sättigung bei starken Signalen oder Stromverbrauch; beides ist messbar, also wird gemessen statt geglaubt.

Was die Uhr seit 1.0.4.112 kann (alles in `CR_GpsAid`):

Befehl	Wirkung
`--gps lna`	Merker der Uhr anzeigen und den Empfänger nach seinem Modus fragen (CFG-VALGET, RAM-Ebene)
`--gps lna normal|low|bypass`	Modus schreiben (CFG-VALSET RAM+BBR), Hotstart nur des GNSS-Teils, erneutes Aiding, Rückfrage; Merker ins NVS (`gpsLna`)
`--gps rf`	UBX-MON-RF einmal abfragen: AGC-Zähler, Rauschen je ms, CW-Störanzeige, Jamming-Zustand, Antennenstatus, I/Q
`--gps rf <s>` / `--gps rf 0`	dieselbe Zeile alle s Sekunden (Messreihe draußen) / aus
`--gps itfm on|off`	Jamming-Monitor des Empfängers (CFG-ITFM-ENABLE, Antenne passiv); ab Werk aus

Der Merker wird bei jedem Modulstart im Start-VALSET mitgesendet (derselbe Rahmen, der AssistNow Autonomous und die MGA-Quittungen einschaltet). Weil die Backup-Zelle das BBR hält, ist der Wert beim Start ohnehin schon wirksam; das Mitsenden deckt nur eine leere Zelle oder ein anders bespieltes NVS ab und wirkt dann ab dem nächsten Modulstart.

Warum der Reset nur den GNSS-Teil trifft: UBX-CFG-RST mit `navBbrMask 0x0000` (Hotstart: Bahndaten, Almanach und Zeit bleiben, die Fixzeit je Modus bleibt vergleichbar) und `resetMode 0x02` (Controlled Software reset, GNSS only). Der vollständige Software-Reset 0x01 lädt die Konfiguration aus dem BBR neu und würde dabei die 38 400 Baud verlieren, die `WZ_GPS.cpp` per `$PUBX,41` nur im RAM setzt — die Uhr verstünde ihren Empfänger nicht mehr. Der Empfänger quittiert CFG-RST nicht; nach 1,5 s läuft `CR_GpsAid_OnModuleReady()` erneut (Aiding, Bahndaten, TTFF-Uhr), danach die VALGET-Rückfrage.

Die Messgröße: UBX-MON-RF (0x0A 0x38) liefert je RF-Block 24 Byte, darunter `agcCnt` (0..8191), `noisePerMS`, `jamInd` (CW-Unterdrückung, relativ), `jammingState` (nur mit ITFM), `antStatus` / `antPower` sowie I/Q-Offset und -Amplitude. Der **AGC-Zähler** ist der eigentliche Hebelmesser: er sagt, wie weit der Chip nachverstärken muss, damit der Wandler ausgesteuert ist. Sinkt er nach dem Umschalten bei gleichem Rauschen deutlich, kommt vorne mehr Signal an.

Erste Messung am Kabel, Fensterbrett, 12.09. 20:13 Uhr (kein Fix, nur Tracking):

Modus	AGC	Rauschen/ms	CW	Antenne
low (Werk)	2046 (24 %), zweite Abfrage 1488 (18 %)	90	4–7	ok, Speisung an
normal	558 (6 %), dreimal identisch	93–94	7–10	ok, Speisung an
low (zurück)	2046 (24 %)	92	5	ok, Speisung an

Umschalten, Rückfrage und Rückweg funktionieren (ACK auf VALSET, VALGET meldet den neuen Wert, Aiding läuft danach erneut durch). Der AGC-Zähler fällt in `normal` auf gut ein Viertel, das Rauschen je ms bleibt gleich — genau das Bild, das ein Nutzer im u-blox-Portal für den M10 beschrieb (rund 20 dB mehr Verstärkung vor dem Wandler). Ob daraus draußen **mehr C/N0 je Satellit** wird oder ob starke Signale übersteuern, entscheidet erst die Messung unter freiem Himmel: `--gps lna normal`, dann `--setGPSDEBUG 2` (GSV-Werte) beziehungsweise die Balkenfarben der GPS-Kachel gegen `--gps lna low` am selben Ort, jeweils ein paar Minuten, dazu `--gps rf 30` für den AGC-Verlauf.

Bewusst nicht gemacht: OTP-Programmierung (unumkehrbar, IM Tab. 3), GLONASS zusätzlich (mehr Strom, keine bessere Empfindlichkeit), Antennenumbau. Die Farbschwellen der Satellitenbalken (grün ab 45 dB-Hz) werden erst nach der Messung neu bewertet — die u-blox-Doku nennt für eine Chipantenne am Körper 35–40 dB-Hz bereits als gut.

29 Absturzspuren: Reset-Journal, Brotkrumen und der Panic-Fanghaken (1.0.4.105–.113)

29.1 Das Problem: die Uhr stürzt nachts ab, und niemand war dabei

Auf dem Tisch am Kabel sieht man jeden Absturz auf der Konsole. Nachts ohne Kabel, am Handgelenk oder in der Tasche sieht man nichts — nur, dass die Uhr irgendwann neu gestartet ist. Genau so war es in der Messnacht 12./13.09.: ein Panic um 22:04 Ortszeit, exakt beim Beginn der ersten Nachtschlaf-Phase, und keine Spur außer einem Zähler, der um eins höher stand.

`CR_Crash` sammelt deshalb drei Spuren, alle im RTC-RAM (`RTC_NOINIT_ATTR`), das einen Neustart überlebt, nicht aber ein stromloses Aus:

Spur	seit	Inhalt	wo lesbar
Watchdog-Snapshot	1.0.3.x	Tasknamen beider Kerne, Laufzeit, 12 Rücksprungadressen aus dem TWDT-ISR	Boot-Log `[CRASH] TWDT-Vorfall`, NVS `crBT`
Reset-Journal	1.0.4.105	eine Zeile je Neustart: Grund, Laufzeit, Uhrzeit, Akku — aus einem 5-s-Lebenszeichen des vorigen Laufs	`--info` „Reset-Journal 1–5", NVS-Ring
Brotkrume	1.0.4.113	letzter benannter Arbeitsschritt (`CR_Crash_Mark()`), 23 Zeichen	am Ende der Journal-Zeile: „bei: Schlaf: light_sleep vor 3 s"
Panic-Snapshot	1.0.4.113	Grund (StoreProhibited, LoadProhibited, abort …), Core, PC, 12 Rücksprungadressen	Boot-Log `[CRASH] Panic-Vorfall`, `--info` „Letzter Panic", NVS `crPanic`

29.2 Brotkrumen: wer zuletzt gesprochen hat

`CR_Crash_Mark("Schlaf: light_sleep")` kopiert den Text und `millis()` ins RTC-RAM — kein Heap, keine Sperre, ein paar Dutzend Takte. Jede Marke überschreibt die vorige; sie ist bewusst kein Ringpuffer, sondern beantwortet genau eine Frage: **was war der letzte Schritt vor dem Ende?**

Die Marken sitzen dort, wo die Nacht gefährlich ist (`CR_Sleep.cpp`): Phase beginnt, WLAN pausieren, BLE pausieren, Phase läuft, Wecker scharf, light_sleep, aufgewacht, Wecker aus, Phase endet, WLAN zurück, wach. Nur bei Panic oder Watchdog wird die Marke ins Journal geschrieben; nach einem gewollten Neustart hätte sie keine Aussage. Der Abstand „vor n s" kommt aus dem Lebenszeichen (alle 5 s) und ist entsprechend grob.

29.3 Der Panic-Fanghaken und die Linker-Falle

Der Arduino-Core wickelt den IDF-Panic-Handler (`esp32-hal-misc.c`: `__wrap_esp_panic_handler`) und bietet `set_arduino_panic_handler()`: die Rückruffunktion bekommt Grund, Core, PC und einen schon gelaufenen Backtrace (bis 60 Adressen), **bevor** die IDF den Registerdump druckt und neu startet. `CR_Crash` kopiert davon 12 Adressen ins RTC-RAM; der nächste Boot baut die Zeile.

Die Falle: Ein Linker-Wrap wirkt nur mit `-Wl,--wrap=esp_panic_handler`. Das pioarduino-Framework 3.3.11 setzt in `flags/ld_flags` nur die Wraps für `log_printf` und `longjmp`. Ohne die Zeile in `platformio.ini` liegt `__wrap_esp_panic_handler` zwar im Abbild, wird aber nie gerufen — `set_arduino_panic_handler()` ist dann ein stilles Nichts. Genau so war der erste Versuch am 13.09.: `--crashtest panic` lieferte den Guru-Meditation-Dump auf der Konsole, das Journal die Brotkrume, aber keinen Panic-Snapshot. Erst mit dem Wrap:

```
Reset-Journal 5: Panic nach 0h00m, 13.09. 07:01 UTC, 3800 mV, bei: Test: crashtest vor 0 s
Letzter Panic: StoreProhibited Core0 PC=0x420AC383 uptime=43715ms BT=0x420AC380 0x420AC3FE 0x420AC51A ...
```

29.4 Adressen in Zeilen verwandeln

```
python3 tools/backtrace.py "Letzter Panic: StoreProhibited Core0 PC=0x420AC383 ... BT=0x420AC380 0x420AC3FE"
0x420ac383: Parser(char const*) at SMashCom42/src/WZ_Parser.cpp:2378
0x420ac3fe: cr_einen_befehl_ausfuehren(char const*) at SMashCom42/src/WZ_Parser.cpp:582
```

Das Werkzeug nimmt `firmware.elf` aus dem Build-Ordner und `xtensa-esp32s3-elf-addr2line` aus der pioarduino-Toolchain. ⚠ Das ELF muss zum Abbild auf der Uhr gehören (gleiche CR_VERSION); nach einem neuen Build zeigen die alten Adressen auf beliebige Zeilen. Rücksprungadressen der Form `0x8xxxxxxx` (Xtensa-Fensterbits, so stehen sie im Watchdog-Snapshot) rechnet das Werkzeug auf `0x4xxxxxxx` um.

`--crashtest panic` löst absichtlich einen Schreibzugriff auf Adresse 0 aus und ist der Selbsttest der ganzen Kette — die Regel aus dem Zettel „Eine Prüfung, die nur bestehen kann, prüft nichts".

29.5 Der Loop-Wächter (1.0.4.116)

Frage Christian (13.09.): „Hat die Uhr einen Watchdog?" Drei: den Interrupt-Watchdog und den RTC-Watchdog der IDF, und den Task-Watchdog, den `CR_GUI` seit D-03 nur auf den Render-Task setzt (5 s, `trigger_panic`). Der Hauptlauf hatte keinen: hängt `loop()`, dreht der Sekundenzeiger auf Core 1 weiter, aber LoRa, Meldungen, Konsole und WebUI stehen.

Der Task-Watchdog hat **eine** Frist für alle Teilnehmer, und 5 s sind für den Hauptlauf zu kurz (GPS-Kaltstart 1,6 s, OTA-Prüfung bis 2 min, Light Sleep 60 s). Deshalb ein eigener Wächter in `CR_Crash`: `CR_Crash_Loop()` stempelt bei jedem Durchlauf ein Lebenszeichen, ein `esp_timer` (Timer-Task, Core 0, hohe Priorität, läuft auch wenn loop() rechnet statt wartet) prüft es jede Sekunde. Bleibt es 30 s aus, landen Dauer und die letzte Brotkrume im RTC-RAM, dann `abort()`. Der Panic-Snapshot fängt den Rest, der nächste Boot meldet:

```
[CRASH] Loop-Waechter: Hauptlauf hing 30 s, letzter Schritt: Test: crashtest loop
Reset-Journal 5: Panic nach 0h00m, 13.09. 08:39 UTC, 3941 mV, bei: Test: crashtest loop vor 0 s
```

Damit die Brotkrume etwas sagt, setzt `loop()` vor jedem Modul-Loop eine Marke (`CR_LOOPMARK`, 22 Stück: Power, Sleep, IMU, Audio, NTP, Touch, ModuleToggle, WiFi, MQTT, OTA, WebUI, Wetter, GpsAno, Hotspot, GPS, LoRaCtl, LoRa, RxDiag, BLE, BlePhone, SwipeDiag, BattLog). Kosten: ein memcpy von 24 Byte je Marke, bei 240 Hz nicht messbar.

Gewollte Blockaden halten den Wächter an (`CR_Crash_LoopWdtPause`, verschachtelt zählend): der Schlafzyklus in `CR_Sleep` (bis 60 s, und der esp_timer steht im Light Sleep ohnehin), der OTA-Upload samt ECDSA-Prüfung und das URL-Update. Der Wächter startet erst mit dem ersten `loop()`-Durchlauf, weil `setup()` bis zu 26 s dauern darf. Selbsttest: `--crashtest loop` blockiert den Hauptlauf 40 s.

Seit 1.0.4.132 stempelt zusätzlich `CR_Crash_LoopBeat()` das Lebenszeichen neu, ohne (wie `CR_Crash_LoopWdtPause()`) eine ganze Blockade anzuhalten: `CR_WebUI.cpp` ruft sie in jeder dynamischen Tabellenzeile (`cr_webui_send_dyn()`, beide Überladungen) und am Seitenende (`cr_webui_send_foot()`) auf. Beleg 13.09. 19:15: die 91-s-Akkuseite (.129) löste den 30-s-Wächter mit der Brotkrume „loop: WebUI" aus — am Handy-Hotspot dauern Seiten bis 60 s, jede so lange Seite hätte die Uhr sonst mitten im Aufbau neu gestartet.

29.6 Nachtschlaf: LoRa-Warten statt Phasenende (1.0.4.132)

Drei Nächte im Vergleich (Akkuverbrauch ohne Kabel, Nachtschlaf-Automat `CR_Sleep`):

Nacht	mV/h	Nachtschlaf	Bemerkung
11./12.09.	43,8	aus	Referenz ohne Nachtschlaf-Automat
12./13.09.	24–30	an	Panic beim ersten Phasenbeginn (siehe §29.1)
13./14.09.	18,9	an	bislang bester Wert, aber nur 71 % Schlafanteil

Die Nacht 13./14.09. lief über 204 Phasen mit 1247 Zyklen und 1128 GPIO-Weckern, aber nur 71 % Schlafanteil (13520 s Schlaf). Fast jede Phase endete an der Hinderung „LoRa hat zu tun" (1231 LoRa-RX in der Nacht, Relay-/ACK-Wünsche) — jedes Phasenende rief `WZ_WiFi_SleepPause(false)` (WLAN zurück), die nächste Phase pausierte es Sekunden später sofort wieder. Reiner Leerlauf: der Gateway-Uplink stand die ganze Nacht ohnehin still.

`cr_sleep_bedingungen()` merkt sich seither in `s_loraGrund`, ob der verletzte Grund einer der beiden LoRa-Gründe war (`WZ_LoRa_IsIdle()` false oder DIO1 HIGH). `cr_sleep_automat()` beendet die laufende Phase bei `s_loraGrund` NICHT mehr sofort, sondern bleibt wach (WLAN bleibt aus, `WZ_LoRa_Loop()` arbeitet normal weiter) und zählt die Wartestrecke (`nachtLoraWarten`, je Wartestrecke einmal, nicht je Loop-Durchlauf). Hängt LoRa länger als `CR_SLEEP_LORA_WARTE_MAX_MS` (2 min) am Stück ohne Idle, wird die Phase doch beendet (Hinderung „LoRa haengt") — Sicherung gegen einen echten Hänger. Journal- und Statuszeilen (`cr_sleep_phase_beenden()`, `CR_Sleep_PrintStatus()`) nennen die Wartestrecken seither mit.

Entscheidung E-48 (siehe `doku/Entscheidungen.md`): Nachtschlaf beendet die Phase bei LoRa-Arbeit nicht mehr; der Gateway-Uplink ruht nachts ohnehin — das war schon vor .132 faktisch so.

Anhang A — Stromschienen

Eine Seite zum Ausdrucken. Die Begründungen stehen in Kapitel 5 und 6.

A.1 Geschaltete Schienen des AXP2101

Schiene	Versorgt	Spannung	Eingeschaltet in	Abgeschaltet in
BLDO1	GNSS-Modul (GPS)	3300 mV	`WZ_GPS.cpp:527–528`	`CR_ModuleToggle.cpp:83`
BLDO2	DRV2605 Haptik-Treiber (Vibration)	3300 mV	`CR_Power.cpp:96–97`	— (dauerhaft an)
ALDO4	SX1262 Funkmodul (LoRa)	3300 mV	`WZ_LoRa.cpp:97–98`	`CR_LoRaCtl.cpp:203`
PWM Pin 45	Hintergrundbeleuchtung	5000 Hz, 8 Bit	`CR_Power_Init()`	—

⚠️ **Die Schiene muss eingeschaltet werden.** Beispielcode aus dem Netz beginnt oft erst beim Funkbaustein — das Modul bleibt dann stromlos, und `radio.begin()` liefert nur einen nichtssagenden Fehlercode. **Richtigstellung 02.09.2026:** Die frühere Angabe, beim normalen T-Watch-S3 hänge das Funkmodul an ALDO3, ist falsch. Beide Modelle nutzen ALDO4; belegt in LilyGos Code für das Grundmodell (`documents_original/src_Lib/LilyGoWatchS3.cpp:414–418, 463–464`). Der einzige Schienen-Unterschied ist BLDO1: beim Grundmodell unbenutzt, beim Plus das GNSS. Siehe Kapitel 5.2.

⚠️ **Der Vibrationsmotor hängt dauerhaft an BLDO2.** Ob sich ein Abschalten lohnt, wurde bislang **nicht gemessen**. Das ist eine offene Frage, keine getroffene Entscheidung.

A.2 Die drei Regeln

> **1. Ein Modul abzuschalten heißt: die Schiene kappen.** Ein `en_XXX = false` bringt die Software zum Schweigen, nicht die Hardware. Bis zum 3. August 2026 zog das „abgeschaltete" GNSS-Modul unverändert seine ~30 mA — der größte Einzelposten der Uhr.

> **2. Nach dem Kappen ist beim Einschalten die vollständige Startroutine zu durchlaufen.** Ein Bauteil ohne Strom kommt nicht in dem Zustand zurück, in dem es war, sondern in dem, in dem es das Werk verlassen hat. Deshalb ruft der Einschaltweg `WZ_GPS_Init()` bzw. `WZ_LoRa_Init()` und nicht nur `enableBLDO1()`.

> **3. Die startende Routine versorgt das Modul auch.** Es gibt bewusst keinen zentralen Ort, an dem alle Schienen hochgefahren werden — sonst wäre der zweite Einschaltvorgang ein anderer Ablauf als der erste.

A.3 Wachposten im Startprotokoll

Jede Schaltung liest ihren Zustand aus dem Regler zurück und protokolliert ihn:

```
[PWR] BLDO1 (GPS-GNSS): eingeschaltet
[PWR] BLDO2 (DRV2605/Vibration): eingeschaltet
[PWR] ALDO4 (LoRa SX1262): eingeschaltet
```

Steht dort `FEHLER`, hat der Regler den Befehl nicht angenommen — alles, was danach an diesem Bauteil scheitert, ist Folgefehler und **sieht dabei aus wie ein Treiberproblem**.

Beim Abschalten dasselbe Muster (`CR_ModuleToggle.cpp:84–85`):

```
[MODTOGGLE] GPS aus, BLDO1 abgeschaltet     ← gut
[MODTOGGLE] GPS aus, BLDO1 NOCH AN (Fehler) ← Abschaltbefehl kam nicht an
```

Ohne diese Rückmeldung tarnt sich ein nicht angekommener Abschaltbefehl als unerklärlich kurze Akkulaufzeit.

A.4 Diagnose-Merksatz

Ein stromloses Bauteil meldet sich nicht als „stromlos", sondern als „antwortet nicht". Diese beiden Zustände sind von der Software aus nicht unterscheidbar. Bei „Bauteil antwortet nicht" sind deshalb zuerst Schiene und Pin zu prüfen (Anhang B), erst danach der Treiber.

Anhang B — Pinbelegung T-Watch-S3-Plus

Vollständig aus `variants/t-watch-S3-plus/pins_arduino.h`. Die Fallgeschichten dazu stehen in Kapitel 10.

B.1 Display (ST7789V3, 240 × 240)

Pin	Signal	Definition
13	MOSI	`BOARD_TFT_MOSI`
18	SCLK	`BOARD_TFT_SCLK`
12	CS	`BOARD_TFT_CS`
38	DC	`BOARD_TFT_DC`
45	Hintergrundbeleuchtung (PWM)	`BOARD_TFT_BL`
−1	MISO — **nicht vorhanden**	`BOARD_TFT_MISO`
−1	Reset — **nicht vorhanden**	`BOARD_TFT_RST`

Peripherie: **SPI3** (`-D USE_HSPI_PORT` , `platformio.ini:42`). Farbeinstellungen siehe Kapitel 9.

B.2 Funkmodul (SX1262)

Pin	Signal	Definition
3	SCK	`BOARD_RADIO_SCK`
4	MISO	`BOARD_RADIO_MISO`
1	MOSI	`BOARD_RADIO_MOSI`
5	CS	`BOARD_RADIO_SS`
9	DIO1 (IRQ)	`BOARD_RADIO_DI01`
8	Reset	`BOARD_RADIO_RST`
7	BUSY	`BOARD_RADIO_BUSY`
6	DIO3	`BOARD_RADIO_DI03`

Peripherie: **SPI2** (`FSPI`) — ⚠️ **nicht** dieselbe wie das Display, siehe Kapitel 7. Die Pins sind gegen LilyGos Hardwaredoku geprüft (`pins_arduino.h:60-61`).

Funkparameter (`pins_arduino.h:88-112`): 433,175 MHz · BW 250 kHz · SF 11 · CR 6 · **Sync Word** `0x2b` · **Präambel 8**. APRS-Frequenz 433,775 MHz.

B.3 Touch (FT6336U)

Pin	Signal	Definition
39	SDA (Wire1)	`BOARD_TOUCH_SDA`
40	SCL (Wire1)	`BOARD_TOUCH_SCL`
16	INT	`BOARD_TOUCH_INT`

Kein Reset-Pin — `WZ_Touch.cpp:304` übergibt bewusst `-1` .

B.4 I²C-Hauptbus (AXP2101, PCF8563, BMA423, DRV2605L)

Pin	Signal	Definition
10	SDA	`BOARD_I2C_SDA`
11	SCL	`BOARD_I2C_SCL`
21	PMU-Interrupt	`BOARD_PMU_INT`
17	RTC-Interrupt	`BOARD_RTC_INT_PIN`
14	BMA423-Interrupt	`BOARD_BMA423_INT1`

B.5 GPS (u-blox GNSS)

Pin	Signal	Definition
42	TX	`GPS_TX_PIN`
41	RX	`GPS_RX_PIN`

Feste Baudrate **38400** (`GPS_FIXED_BAUD`), Versorgung über BLDO1. ⚠️ Hier standen bis zum 30. Juli 2026 die Pins **44/43** — siehe B.7 und Kapitel 10.1.

B.6 Nicht übersetzte Bauteile

Diese Blöcke sind auskommentiert; die Pins bleiben dennoch physisch verdrahtet.

Pin	Signal	Definition	Block
47	Mikrofon-Daten (PDM)	`BOARD_MIC_DATA`	`HAS_MIC` (aus)
44	**Mikrofon-Takt (PDM)**	`BOARD_MIC_CLOCK`	`HAS_MIC` (aus)
48 / 15 / 46	I²S BCK / WS / DOUT	`BOARD_DAC_IIS_*`	`HAS_AUDIO` (aus)
2	Infrarot-Sender	`BOARD_IR_PIN`	—

B.7 ⚠️ Doppelt belegte und irreführende Namen

Die Tabelle, die beim Fehlersuchen am meisten hilft.

Pin	Erster Name	Zweiter Name	Es gilt
44	`BOARD_MIC_CLOCK`	früher `GPS_RX_PIN`	Mikrofon-Takt — **kostete einen halben Tag**
39	`BOARD_TOUCH_SDA`	`WS_RTC_INT`	Touch-SDA
10	`BOARD_I2C_SDA`	`WS_RTC_SCL` ⚠️	I²C-**SDA** (der `WS_` -Name ist vertauscht)
11	`BOARD_I2C_SCL`	`WS_RTC_SDA` ⚠️	I²C-**SCL** (der `WS_` -Name ist vertauscht)
40	`BOARD_TOUCH_SCL`	`WS_SYS_OUT` (auskommentiert)	Touch-SCL

Die `WS_` -Definitionen stammen vom **Waveshare**-Board und beschreiben nicht die T-Watch. Aus dieser Gruppe wird ausschließlich `WS_RTC_ADDRESS` verwendet (`CR_RTC.cpp:31`), die Pins kommen dort korrekt aus `BOARD_I2C_SDA` / `SCL` .

> **Vor der Verwendung eines Pins im Variant-Header nach dem Zahlenwert suchen, nicht nach dem Namen.** Ein `#define` abzuschalten trennt keine Leiterbahn.

B.8 Die Hardwarekennung steht *nicht* im Variant-Header

Sie steht ausschließlich in `CR_MeshCom.h` als `CR_MESH_SOURCE_HW` :

Wert	Herkunft
61 (0x3D)	Am 5. August 2026 von Kurt (OE1KBC) für die T-Watch-S3-Plus zugeteilt, im MeshCom-Projekt als `#define T_WATCH_S3 61` geführt

Damit ist die Uhr im Netz als eigenes Gerät kenntlich und fällt nicht mehr unter die 0 („nicht näher bezeichnet", die Belegung für Boards in Entwicklung). Zum Vergleich: 10 = HELTEC_V2_1, 42 = HELTEC_STICK_V3.

Der Wert steht **nur an dieser einen Stelle**: `CR_MESH_LAST_HW` leitet sich daraus ab (`0x80 | 61` = `0xBD`), und jeder Pakettyp mit Schwanz holt sich beide über `cr_mesh_append_tail()`. Die Quittung trägt keinen Schwanz und daher auch keine Kennung.

⚠️ **Nicht zu verwechseln mit `HARDWARE_ID 39`** im Variant-Header. Die beiden Zahlen sehen gleichartig aus, meinen aber Verschiedenes:

	Wert	Bedeutung
`CR_MESH_SOURCE_HW` (`CR_MeshCom.h`)	**61**	die **zugeteilte** Kennung dieser Uhr — **das geht über Funk hinaus**
`HARDWARE_ID` (`pins_arduino.h`)	**39**	`EBYTE_E22` — die Kennung des **E22-Funkmoduls**, für Eigenbauten in Erprobung

Die Zahlen sind keine freie Wahl: Sie stehen in der MeshCom-Quelle (`configuration_global.h`), wo jedes bekannte Gerät seine Nummer hat — `TBEAM 4`, `HELTEC_STICK_V3 42`, `EBYTE_E22 39`, und seit dem 5. August 2026 `T_WATCH_S3 61`.

⚠️ **Die allgemeine Entwicklungskennung ist die `0`, nicht die 39.** Die 39 bezeichnet ein bestimmtes Funkmodul.

> 🔴 **`HARDWARE_ID 39` darf nicht entfernt werden**, obwohl es projektweit keinen Leser hat. Genau das ist am 6. August 2026 geschehen — mit der Begründung, es sei tot. Wolfgang (OE3WAS) dazu unmissverständlich: *„NEIN, das ist Absicht zum Entwickeln von neuer HW."* Sie ist die Kennung, unter der ein E22-bestücktes Eigenbau-Board im Netz auftritt, solange es erprobt wird. Sie wurde wiederhergestellt.
>
> **Die Lehre ist unbequemer als die ursprüngliche:** *Kein Leser zu haben heißt nicht, entbehrlich zu sein.* Der Leser kann in der Zukunft liegen. Wer eine Definition entfernt, weil die Suche nur einen Treffer bringt, entfernt womöglich eine Vorbereitung — und der Einzige, der das weiß, ist der, der sie angelegt hat. **Im Zweifel fragen, nicht aufräumen.**

Anhang C — Was bereits versucht und verworfen wurde

Dieser Anhang ist der Kern des Leitprinzips dieses Buches: Ein Techniker, der das Projekt übernimmt, kann sich das *Was* aus dem Quelltext holen. Was er nicht herausfinden kann, ist, **welche naheliegenden Wege bereits begangen und verworfen wurden.**

Jede Zeile hier hat Arbeitszeit gekostet. Wer einen dieser Wege erneut einschlägt, bekommt dasselbe Ergebnis.

C.1 Hardware und Treiber

Versucht	Was geschah	Was stattdessen gilt
Funkmodul auf `HSPI` legen (`radioSPI(HSPI)`)	`HSPI` = SPI3 = **die Peripherie des Displays**. 3 Watchdog-Neustarts in ~12 Starts, jeweils direkt nach dem Einschalten von ALDO4	`FSPI` (SPI2) — Kap. 7
GPS an Pin 44/43	Pin 44 ist die **Mikrofon-Taktleitung**. Die Baudratenerkennung maß Rauschen: „50 Flanken", Fantasiewert 333333 Baud	Pins 42/41 — Kap. 10.1
GPS-Baudrate automatisch erkennen	9 von 10 Boots richtig, der zehnte scheiterte vollständig	`GPS_FIXED_BAUD 38400` , danach 10/10 — Kap. 10.2
Modul über `en_XXX = false` abschalten	Die Software schwieg, das Bauteil lief weiter (~30 mA). ⚠️ **Jede Sparmessung vor dem 3.8.2026 ist damit wertlos**	Schiene kappen — Kap. 6
Annahme, die Versorgung des Funkmoduls stehe schon	Beispielcode beginnt oft erst beim Funkbaustein. Modul bleibt stromlos, `radio.begin()` liefert einen nichtssagenden Fehler. ⚠️ Die frühere Erklärung „beim Grundmodell ist es ALDO3" war falsch (Richtigstellung 02.09.2026) — beide Modelle nutzen ALDO4	Schiene selbst einschalten und zurücklesen — Kap. 5.2
MeshCom-Präambel 32	Empfang lief trotzdem (der SX1262 braucht nur wenige Symbole) — aber **jede eigene Aussendung war rund 197 ms zu lang**	Präambel 8 — Kap. 21

C.2 Display und Farben

Versucht	Was geschah	Was stattdessen gilt
`TFT_RGB_ORDER=1` setzen	Bedeutet in TFT_eSPI **RGB**; das Panel braucht BGR. Reiner R/B-Tausch	`TFT_RGB_ORDER=0` — Kap. 9.4
`invertDisplay(COLOR_INV)`	Schaltete das `INVON` der ST7789-Startsequenz wieder **ab** → Negativfarben	`invertDisplay(!COLOR_INV)` — Kap. 9.3
Den Arduino_GFX-Altzustand als Farbreferenz nehmen	Der Altzustand war **selbst falsch** — der R/B-Tausch bestand die ganze Zeit, fiel in der blau/grau-lastigen Oberfläche nur nie auf	5-Farben-Boot-Test — Kap. 9.2
Die Farbsemantik aus der Recherche übernehmen	`01-RESEARCH.md` gab sie genau **falsch herum** wieder	Auf der Hardware messen — Kap. 9

C.3 Touchcontroller FT6336U

Versucht	Was geschah	Was stattdessen gilt
Gesten über SensorLibs `getGesture()`	Liest `GEST_ID` (0x01) — auf diesem Board **dauerhaft 0x00**	Special Gesture Mode `0xD0` / `0xD3` — Kap. 8.1
Ohne die Auflösung `0x98` – `0x9B` arbeiten	**Komplette Y-Achse tot** (X ging weiter — daher die verwirrende Asymmetrie). 0 von 6 Wischen	Vier Bytes schreiben, Engine vorher aus — Kap. 8.2
`0xD3` aktiv löschen (nach dem Lesen **oder** beim Aufsetzen)	Zustandsmaschine gestört, Vertikale brach auf 0 von 9 ein	Nie beschreiben, Flankenerkennung genügt — Kap. 8.5
Flankenspeicher beim Loslassen zurücksetzen	`0xD3` ist gelatcht und hält einen alten Wert → **falsche** Geste (Runter-Wisch meldete Hoch)	Nicht zurücksetzen — Kap. 8.5
`0xD1` mit `0x1F` beschreiben	Löscht die „keep"-Bits 7 und 6 → Gesten-Engine **komplett tot**, überlebt den ESP32-Reset	Werkswert `0xFF` — Kap. 8.5
Gestenerkennung am Arduino- `loop()` takten	`loop()` lief damals mit **2 Hz** = 60× zu langsam. ~65 % sporadische Gesten	Eigener Task, 8 ms — Kap. 8.4

C.4 Nebenläufigkeit und Bedienung

Versucht	Was geschah	Was stattdessen gilt
`attachInterrupt()` für den LoRa-Empfang	Der **erste** `attachInterrupt()` des Programms installiert den GPIO-ISR-Service über den ipc1-Task, dessen Stack mit 1024 Byte sehr knapp ist → *Stack canary watchpoint triggered*, Panic in etwa jedem zweiten Boot	DIO1 **pollen** — `WZ_LoRa.cpp:64–86`
`lv_scr_load_anim()` unter `lv_lock()` im Main-Loop	Drei Watchdog-Neustarts	Anforderungs-Flag, der GUI-Task erledigt es — Kap. 13
Den Main-Loop ungebremst laufen lassen	1000–4000 Hz kosteten den Render-Task **126 Aussetzer in 29 s**	3-ms-Bremse — Kap. 15
Auf einer Nebenseite waagrecht `CR_SCREEN_NONE` eintragen	Sackgasse: ⚠️ **Ein wirkungsloser Wisch ist von einem hängenden Gerät nicht zu unterscheiden.** Zweimal binnen zweier Tage als Fehler gemeldet (Meldungsblätter, Kachelseite 2)	Nachbarn der Hauptseite spiegeln — Kap. 17

C.5 Angenommen, nie gemessen — und falsch

Die lehrreichste Gruppe. Diese Sätze standen über Monate in der Projektdokumentation und haben die Fehlersuche aktiv in die Irre geführt.

Die Annahme	Die Wirklichkeit	Was sie widerlegt hätte
„Main- und Render-Task liegen auf getrennten Kernen"	Sie lagen auf **demselben** Kern (`ARDUINO_RUNNING_CORE=1`)	Drei Zeilen `xPortGetCoreID()`
„Die Renderlast bremst den Main-Loop"	Widerlegt. Es waren **drei** Fehler, der erste verdeckte die beiden anderen; 1 Hz → 223 Hz	Kap. 14
„Das Funkmodul hat einen eigenen SPI-Bus" (Quelltextkommentar)	`HSPI` und `USE_HSPI_PORT` führen beide auf SPI3	Ein Blick in `esp32-hal-spi.h:36`
„GPS ist aus, also zieht es keinen Strom"	Die Schiene lief weiter	Kap. 6
„Die Prozentanzeige zeigt den Ladezustand"	Dieselbe Spannung ergab **20 % beim Laden und 50 % beim Entladen**	Kap. 24

> **Die gemeinsame Wurzel:** Nicht eine dieser Annahmen war unplausibel. Jede war nur nie nachgemessen worden. Deshalb gilt in diesem Buch die Belegpflicht — und deshalb steht überall dort, wo etwas *nicht* gemessen wurde, ausdrücklich dabei, dass es eine Annahme ist.

C.6 Was hier bewusst nicht steht

Zwei Entscheidungen könnten in dieser Liste vermutet werden und gehören nicht hierher:

- **Die von Hand geschriebene Bedienoberfläche** ist kein verworfener Weg, sondern ein gewählter — begründet mit der feinen Steuerbarkeit im Quelltext (Kapitel 12b).
- **Die feste Baudrate** hat die Erkennung nicht ersetzt, weil diese schlecht war, sondern weil Determinismus beim Start höher wiegt als ein Zeitgewinn, den es ohnehin nicht gab (Kapitel 10.2).

Anhang D — Quellenverzeichnis

Wo das Wissen dieses Buches herkommt — und wo es beim Weiterarbeiten nachzuschlagen ist.

D.1 Kapitel → Belegstellen

Kapitel	Wesentliche Quellen
1 Der Kern	`WZ_LoRa.cpp`, `CR_LoRaCtl.cpp`, `pins_arduino.h`, `CLAUDE.md`, `PROJECT.md`
2b Bauen und Flashen	`platformio.ini`, `gen4esp32_8MBapp_8MBota.csv`, `arduino.py`
5 + 6 Stromschienen	`WZ_GPS.cpp`, `WZ_LoRa.cpp`, `CR_Power.cpp`, `CR_ModuleToggle.cpp`, `pins_arduino.h`
7 SPI	`WZ_LoRa.cpp:31-49`, `esp32-hal-spi.h:36`, `TFT_eSPI_ESP32_S3.h:73`, `platformio.ini:42`, `LilyGoWatch-S3.json`
8 Touchcontroller	`CR_Touch.h`, `CR_Touch.cpp`, **`documents_original/FT6336U_reg.pdf`**
9 Panel	`platformio.ini:104`, `main.cpp:207/473`, `pins_arduino.h:32`, `.planning/phases/01-…/01-01-SUMMARY.md`
10 Pins	`pins_arduino.h`, `CR_RTC.cpp:31`, `WZ_Touch.cpp:304`, LilyGos `lilygo-t-watch-s3-plus.md`
11 Zwei Kerne	`LilyGoWatch-S3.json`, `CR_GUI.cpp`
12b Bedienoberfläche	`CR_EEZShim.h`, `CR_Watchface.cpp`, `main.cpp`, `WZ_Touch.cpp`
13 Main-Loop-Verbote	`CR_ModuleToggle.cpp`, `CR_GUI.cpp` (Muster, keine Zeilenverweise)
14 Fall REND-03	Debug-Sitzung `rend03-loop-1hz`, `Timeout.cpp`
16 Bedienung/Abtastrate	`CR_Gesture.cpp`, `CR_SliderGuard.cpp`, `CR_MsgScreen.cpp`, `CR_QuickReply.cpp`
17 Schirm-Topologie	`CR_ScreenMgr.h`, `CR_ModuleScreens.cpp`, `CR_Watchface.cpp`, `tools/handbuch.py`
18–21 MeshCom	`CR_MeshCom.cpp/.h`, `WZ_LoRa.cpp`, **MeshCom-Quelle** (`aprs_functions.cpp`, `lora_functions.cpp`), `WZ_UDP.cpp`
22 NVS	`prefs.cpp`, `globals.h`
24 Akku	`CR_BattLog.cpp`, `CR_Power.cpp`, `main.cpp`, Akku-Logbuch
A Stromschienen	wie Kapitel 5/6
B Pinbelegung	`pins_arduino.h` vollständig, `CR_MeshCom.h`, `CR_RTC.cpp`
C Sackgassen	Querschnitt aller Kapitel

D.2 Quelldateien → wo sie erklärt werden

Der umgekehrte Weg: Wer eine Datei vor sich hat und wissen will, warum sie so aussieht.

Datei	Erklärt in
`variants/t-watch-S3-plus/pins_arduino.h`	Kap. 5, 10 · Anhang A, B
`src/WZ_LoRa.cpp`	Kap. 7 (SPI), 20, 21 · Kopfkommentar `:64-86` (ipc1-Panic)
`src/CR_Touch.cpp/.h`	Kap. 8 vollständig
`src/CR_Power.cpp` · `src/CR_ModuleToggle.cpp`	Kap. 5, 6, 13
`src/CR_ScreenMgr.h` · `src/CR_ModuleScreens.cpp`	Kap. 17
`src/CR_MeshCom.cpp/.h`	Kap. 18–20
`src/prefs.cpp`	Kap. 22
`src/globals.h`	Kap. 3 (Ebenenmodell), 4 (Versionsnummern), 22
`platformio.ini`	Kap. 2b, 7, 9
`boards/LilyGoWatch-S3.json`	Kap. 7, 11 (`ARDUINO_RUNNING_CORE`)
`src/main.cpp`	Kap. 9, 12b, 13, 14, 24

D.3 Fremddokumentation

Dokument	Wofür unentbehrlich
`documents_original/FT6336U_reg.pdf`	⚠️ Die einzige Quelle für den Special Gesture Mode (`0xD0` – `0xD8`). Weder Datenblatt noch CTPM Application Note führen ihn
LilyGo `docs/hardware/lilygo-t-watch-s3-plus.md`	Funkpins und Stromschienen des **Plus**-Modells — unterscheidet sich vom normalen T-Watch-S3
MeshCom-Firmware — liegt unter `reference/meshcom-firmware/`	Paketaufbau und Sendefallstricke (`src/aprs_functions.cpp` , `src/lora_functions.cpp`), BLE-Dienst (`src/esp32/esp32_main.cpp:1587-1644`). ⚠️ Raten brachte 80 %, und 80 % heißt hier: kommt nicht an
`esp32-hal-spi.h`	Zuordnung `FSPI` →Bus 0/SPI2, `HSPI` →Bus 1/SPI3
TFT_eSPI `TFT_eSPI_ESP32_S3.h`	`USE_HSPI_PORT` → `SPI_PORT 3`

⚠️ **Zur Auflösungs-Abhängigkeit des FT6336U (Kap. 8.2) gibt es überhaupt keine Quelle.** Sie steht in keinem der vier FocalTech-Dokumente und war ausschließlich durch Messen zu finden.

⚠️ Fremde Quellen gehören ins Arbeitsverzeichnis

Bis zum 6. August 2026 lag die MeshCom-Firmware ausschließlich auf einem externen Sicherungslaufwerk. Mehrere Belege dieses Buches verwiesen damit auf etwas, das nur zufällig angesteckt war.

> **Ein Beleg, den man nicht aufschlagen kann, ist keiner.** Ein Verweis auf ein fremdes Laufwerk ist auf jedem anderen Rechner und zu jedem späteren Zeitpunkt wertlos — er sieht aus wie ein Nachweis, ist aber keiner.

Seither liegt die Quelle unter `reference/meshcom-firmware/` (MIT-Lizenz; Herkunft, Umfang und Erneuerung sind in `reference/README.md` beschrieben). Mitkopiert wurden nur `src/` und `include/` — die Grafikbestände fremder Boards machten 97 % des Umfangs aus und haben hier keinen Wert.

⚖️ Vor dem Hinzufügen weiterer Fremdquellen dort gilt: **erst die Lizenz lesen, dann kopieren.** Dieses Projekt musste bereits einen GPL-3.0-Bestandteil wieder entfernen (`Arduino_DriveBus`).

D.4 Projektinterne Ablagen

Ort	Inhalt
`doku/Entscheidungen.md`	Warum sich die Uhr wo so verhält. ⚠️ **Zuerst dort nachsehen**, bevor ein Verhalten als Fehler gilt
`.planning/STATE.md`	Fortlaufender Arbeitsstand mit Messwerten
`.planning/quick/*/`	Einzelne Arbeitsschritte samt Plan und Ergebnis
`.planning/debug/*/`	Debug-Sitzungen — u. a. `rend03-loop-1hz` (Kap. 14)
`.planning/phases/*/`	Abnahmeprotokolle der Phasen, z. B. der 5-Farben-Farbtest (Kap. 9)
`.planning/security/`	Sicherheits-Audits

D.5 Messmittel

Wer eine Aussage dieses Buches nachprüfen will, braucht diese Werkzeuge — sie sind in Kapitel 27 beschrieben.

Messpunkt	Zeigt
`[LOOPHZ]`	Main-Loop-Takt (Kap. 14, 15)
`[CORE]` / `xPortGetCoreID()`	Tatsächliche Kernzuordnung (Kap. 11)
`GUI_LAG`	Aussetzer des Render-Tasks (Kap. 15)
`[SLGUARD]`	Schiebereglerschutz (Kap. 16)
Akku-Logbuch + `tools/battlog.py`	Verbrauch, Empfänge je Stunde, Fix-Anteil (Kap. 24)
5-Farben-Boot-Test	Farbpfad des Panels (Kap. 9.2)
LoRa-Selbsttest	Funkschicht (Kap. 21)

D.6 u-blox-Dokumente zum GNSS-Modul (Kap. 28)

Liegen als PDF unter `doku/hardware/mia-m10q/` (abgelegt 27.08.2026):

Dokument	Nummer	Wofür
MIA-M10Q Data Sheet	UBX-22015849 R08 (30.01.2026)	TTFF-Tabellen 2/3, Empfindlichkeiten, V_BCKP-Ströme
MIA-M10Q Integration Manual	UBX-21028173 R05 (30.01.2026)	Startarten 3.4, Zeit-Aiding 3.10.2, AssistNow 3.12, Backup/RTC 4.1–4.2
MIA-M10Q Integration Manual	UBX-21028173 R03 (19.06.2023)	ältere Ausgabe, nur zum Vergleich
u-blox M10 SPG 5.10 Interface Description	UBX-21035062 R03	UBX-Nachrichten und Konfigurationsschlüssel (online: content.u-blox.com)

D.7 Hinweis zu den Zeilenverweisen

Zeilenangaben in diesem Buch geben den Stand zum Zeitpunkt des Schreibens wieder. **Sie wandern**, Namensverweise nicht — ein Beispiel steht in Kapitel 7.4, wo ein Quelltextkommentar auf `pins_arduino.h:189-193` zeigt, während die Definitionen inzwischen in `:72-75` stehen.

> Wer eine Stelle nicht findet: nach dem **Bezeichner** suchen (`BOARD_RADIO_SCK` , `CR_MESH_LAST_HW`), nicht nach der Zeilennummer.

Abgleiche mit der MeshCom-Firmware

Jeder Stand der Vorlage (icssw-org/MeshCom-Firmware) wird gegen die Uhr abgeglichen: Was ändert sich auf der Luft und an der App, was betrifft die Uhr, was ist umgesetzt. Die Papiere liegen neben diesem Buch in `doku/` :

Papier	Vorlage	Umgesetzt
`MeshCom-Abgleich-v4.35t.md`	v4.35s → v4.35t	Quittungen, 0x41-Rahmen
`MeshCom-Abgleich-v4.35v.md`	v4.35t → v4.35v	PN-Wiederholungen (1.0.5.16), App-Telegramme, Rufzeichen, Hop-Grenze (1.0.5.25)
`MeshCom-Abgleich-v4.40a.md`	v4.35v → v4.40a	Briefkasten-Telegramme (1.0.5.24); Umstellung auf 4.40A (1.0.5.31)